



Welcome to our comprehensive course on creating a roguelike game using the Godot engine. Your instructor for this journey is Daniel Buckley. Throughout this course, we will explore the development of an action-packed roguelike game, similar to popular titles you may have already enjoyed. Our game will feature a player character navigating through procedurally generated rooms, battling enemies, and confronting bosses in a dynamic environment.

## Course Overview

In this course, we will cover a wide array of topics essential for creating a compelling roguelike game. These topics include:

- Procedurally generated room layouts
- Enemy and boss design
- Setting up a mini-map
- Implementing sounds and damage visuals
- Creating items for the player to collect
- Designing doors that open and close based on room state

## Prerequisites

While the course is designed to be accessible, an intermediate understanding of Godot and GDScript is recommended. If you are new to Godot, some parts of the course may be challenging due to the complexity and amount of code involved. However, we will proceed step-by-step, ensuring that each concept is clearly explained.

## Learning with Zenva

Zenva is an online learning academy with over 1 million students. We offer a diverse range of courses suitable for beginners and those looking to expand their knowledge. Our courses are flexible, allowing you to learn in various ways:

1. Watch online video tutorials
2. Read summarized notes
3. Follow along with the instructor using provided course files

## Getting Started

Now that you have an overview of what to expect, let's dive into the first lesson and begin our journey into creating an exciting roguelike game with Godot.

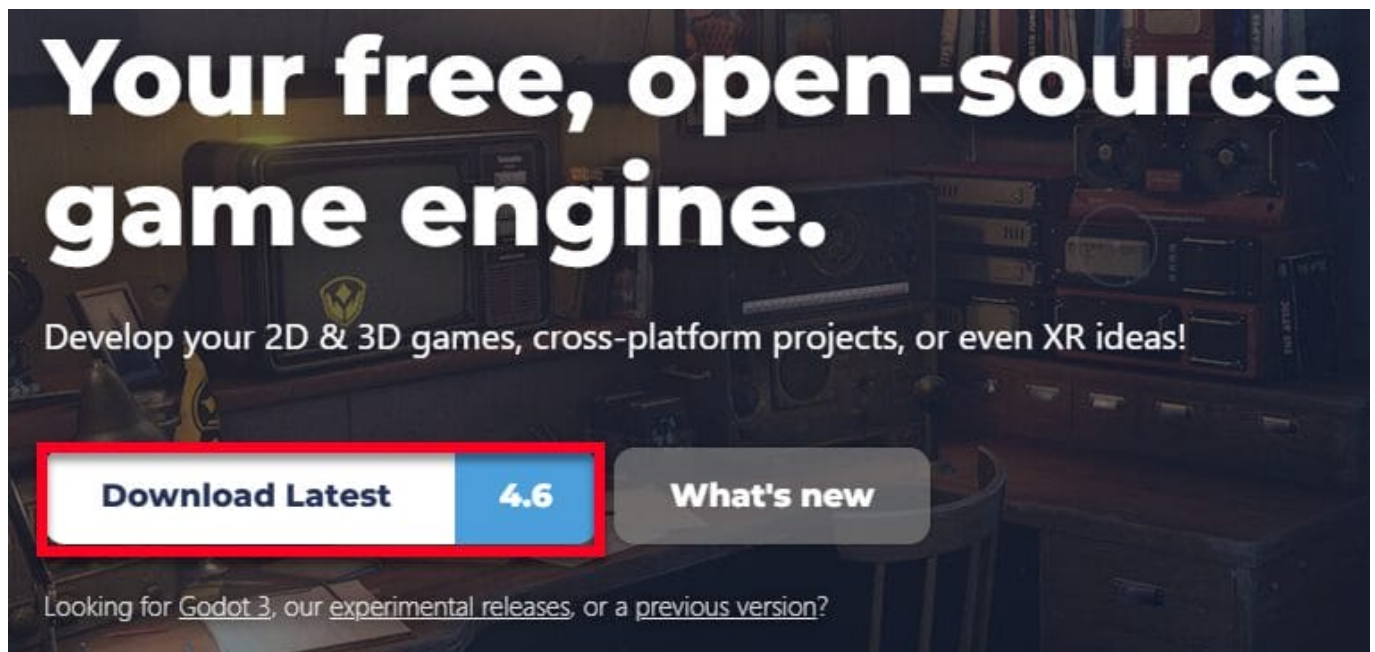
## Course Updated to Godot 4.6

We've updated the project files to Godot version 4.6 for this course – the latest stable release.

Please make sure NOT to use newer versions of the software than what we recommend, as the course material provided might not work as expected.

### How to Install Version 4.6

You can download the most recent version of Godot by heading to the Godot website (<https://godotengine.org/>) and clicking the “Download Latest” button on the front page. This will automatically take you to the download page for your operating system.



Once on the download page, just click the option for **Godot 4.6**. This will download the Godot engine to your local computer. From there, unzip the file and simply click on the application launcher – no further installation steps are required!



# Download Godot 4 for Windows



Godot Engine

4.6

x86\_64 - 26 January 2026



Godot Engine – .NET

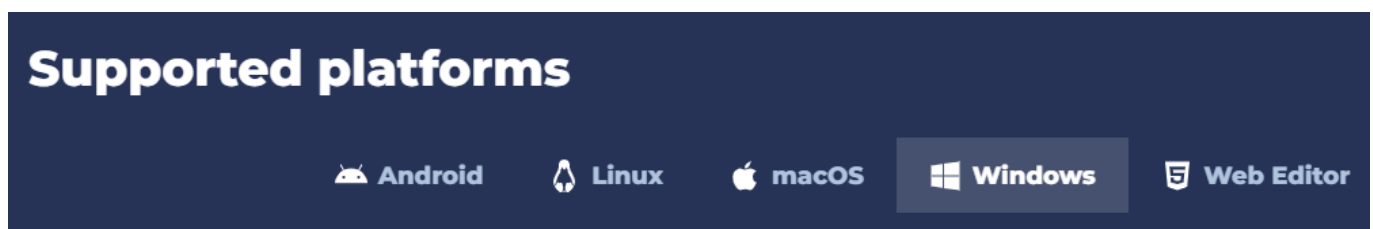
4.6

x86\_64 - C# support - 26 January 2026

Looking for [Godot 3](#), our [experimental releases](#), or a [previous version](#)?



If Godot fails to automatically select the correct operating system for you, you can scroll down to the “Supported platforms” section on the download page to manually select the download version you would like to install:





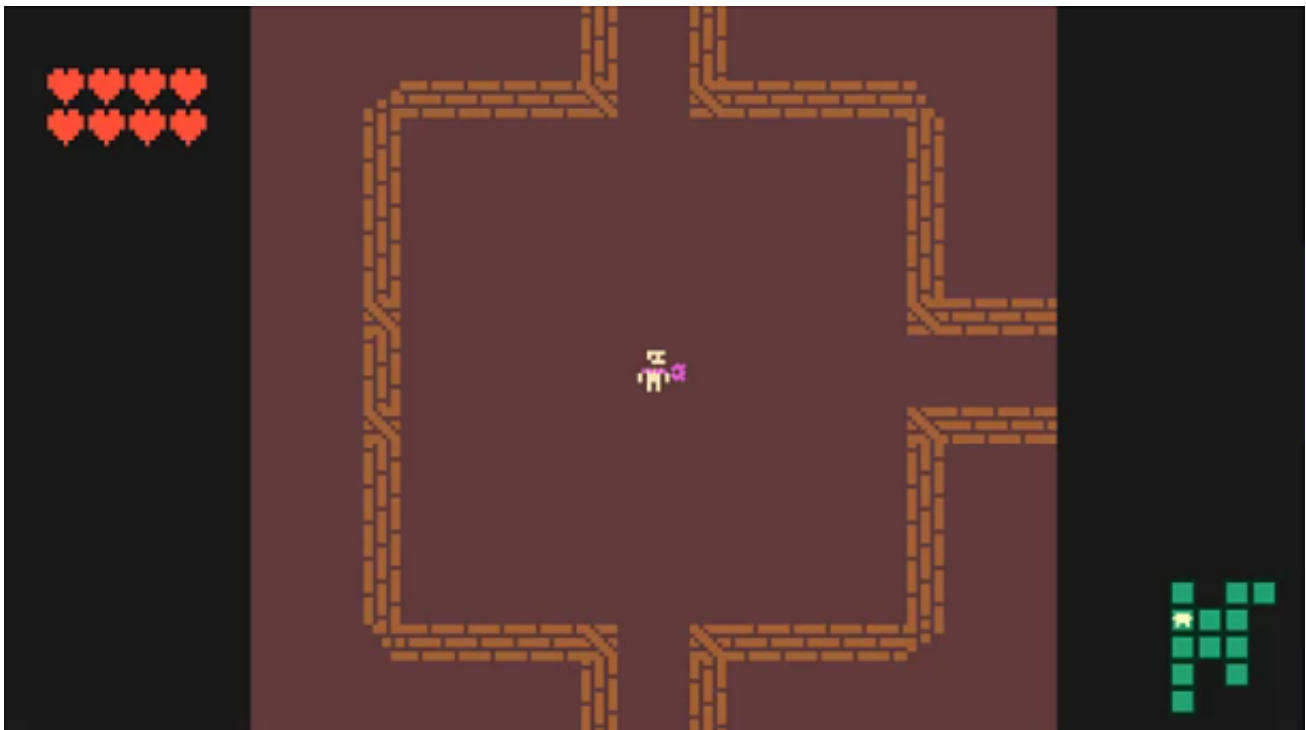


Before we dive into the development process of our roguelike game, let's take a moment to understand what we will be building. Our goal is to create a game similar to popular titles like *Binding of Isaac* or *Enter the Gungeon*. These are run-and-gun shooters where the player enters a room, defeats all enemies to proceed to the next room, collects items, and faces a boss in one of the rooms to advance to the next level.

## Game Overview

The game will feature:

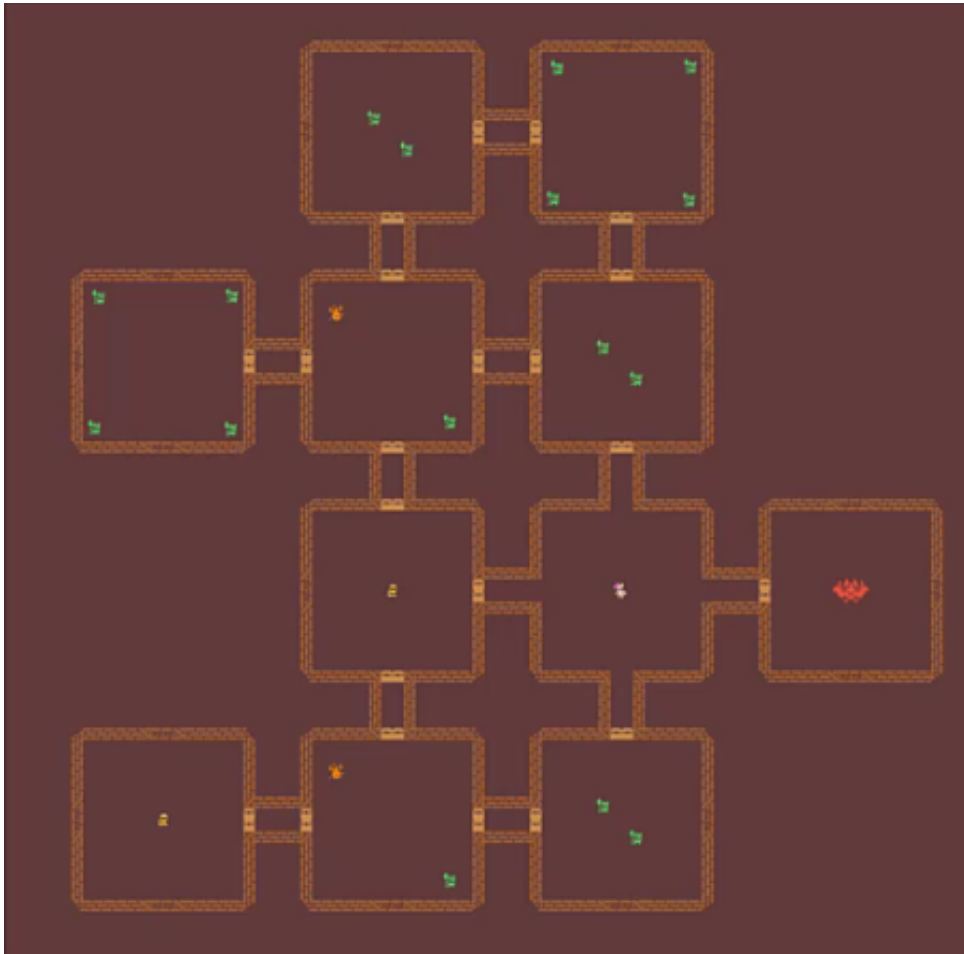
- Procedurally generated rooms with different types and connecting corridors.
- Items to collect and enhance the player's abilities.
- Enemies that activate when the player enters their room.
- A mini-map to display the room layout and player's position.
- A health system displayed as hearts.
- A menu system to allow players to start or quit the game.



## Core Gameplay Mechanics

The main focus of this course will be on room generation. Here's how it will work:

1. At the start of the game, a room layout will be procedurally generated.
2. Each room will have doors that open or close based on the presence of enemies.
3. Doors will close when the player enters a room with enemies and will only open once all enemies are defeated.



## Player Controls

The player will control their character using the following controls:

- **Movement:** W, A, S, and D keys
- **Aiming:** Mouse
- **Shooting:** Left mouse button

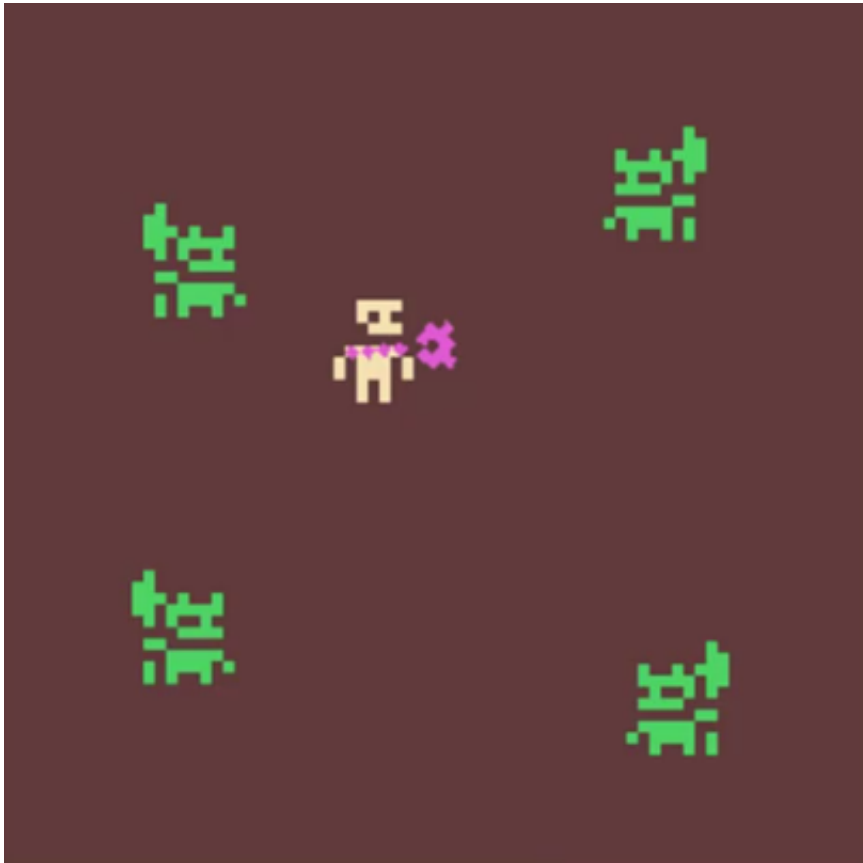
The player will shoot projectiles that deal damage to enemies upon impact.



## Enemy Behavior

Enemies will be designed with the following behavior:

- Enemies activate only when the player enters their room.
- Enemies chase the player and deal damage when in range.
- Enemies can be defeated by the player's projectiles.
- When all enemies in a room are defeated, the doors will open.



## User Interface

The game will include the following UI elements:

- **Mini-map:** Displays the room layout and player's position.



- **Health System:** Displays player's health as full or empty hearts.



- **Menu System:** Allows players to start or quit the game.





# GODOT ROGUELIKE

Play

Quit

Now that we have a clear understanding of what we will be creating, let's get started with the project.



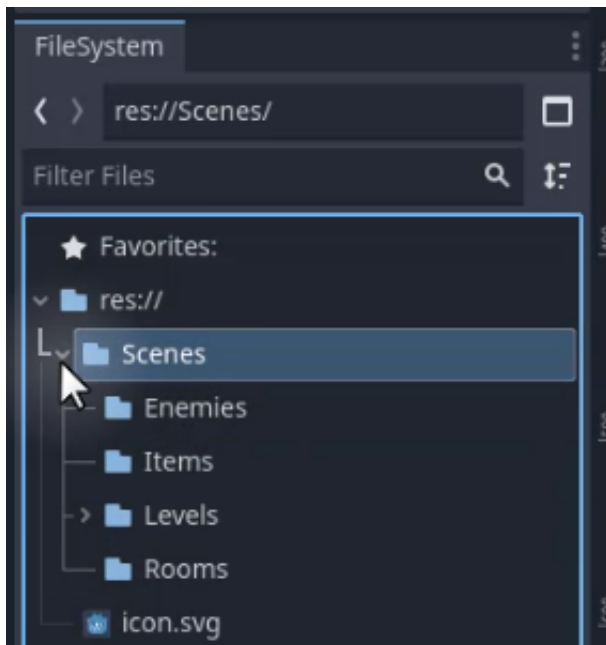
In this lesson, we will walk through the initial steps of creating a new project, organizing the file system structure, and configuring basic settings for our roguelike game.

To begin, we will create a brand new Godot project. Initially, the project will be empty, providing a clean slate for our development.

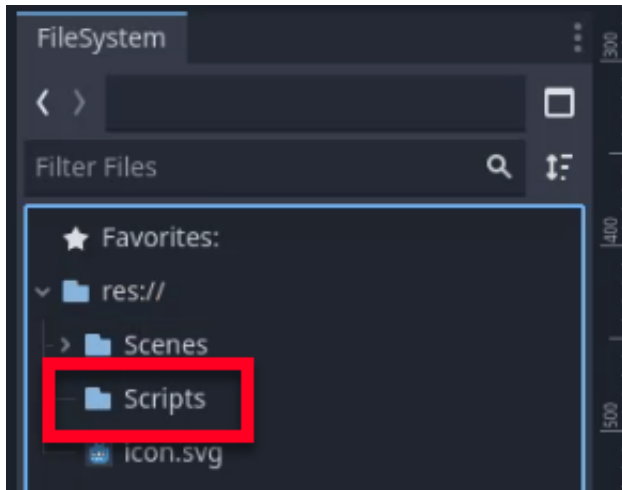
## Setting Up the File System Structure

Organizing your project's file system structure is crucial for maintaining clarity and efficiency. Follow these steps to set up your folders:

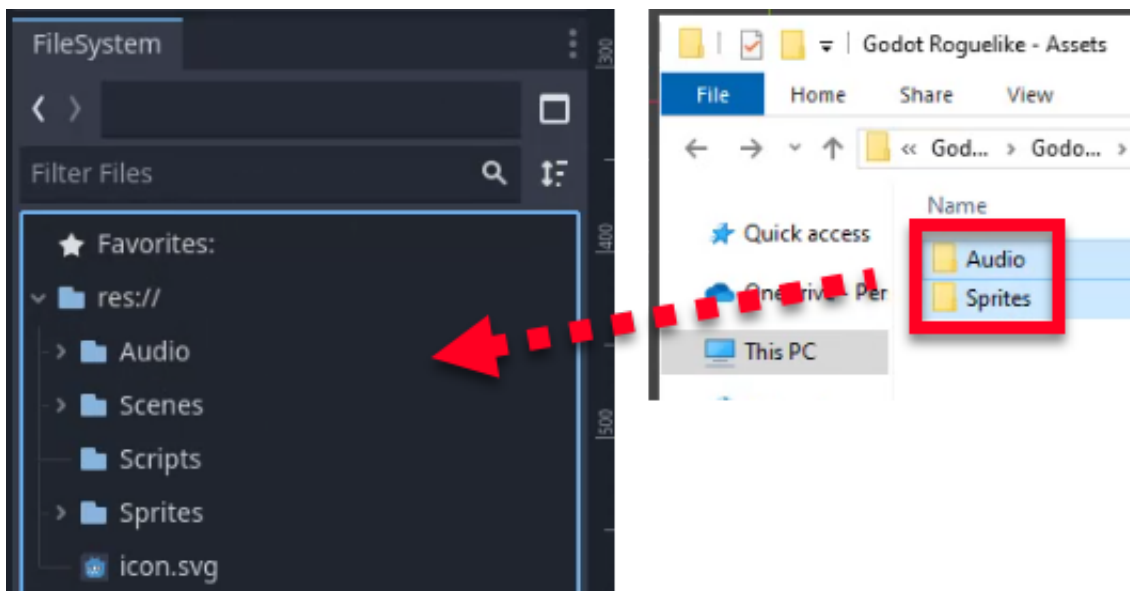
- Create a new folder named Scenes. This folder will store all the scenes you create.
- Within the Scenes folder, create a subfolder named Levels. This will hold the different levels of your game.
- Create additional folders within the Scenes folder:
  - Items: For storing item scenes.
  - Rooms: For storing different room types.
  - Enemies: For storing enemy scenes.



- Create a Scripts folder to store all your scripts.



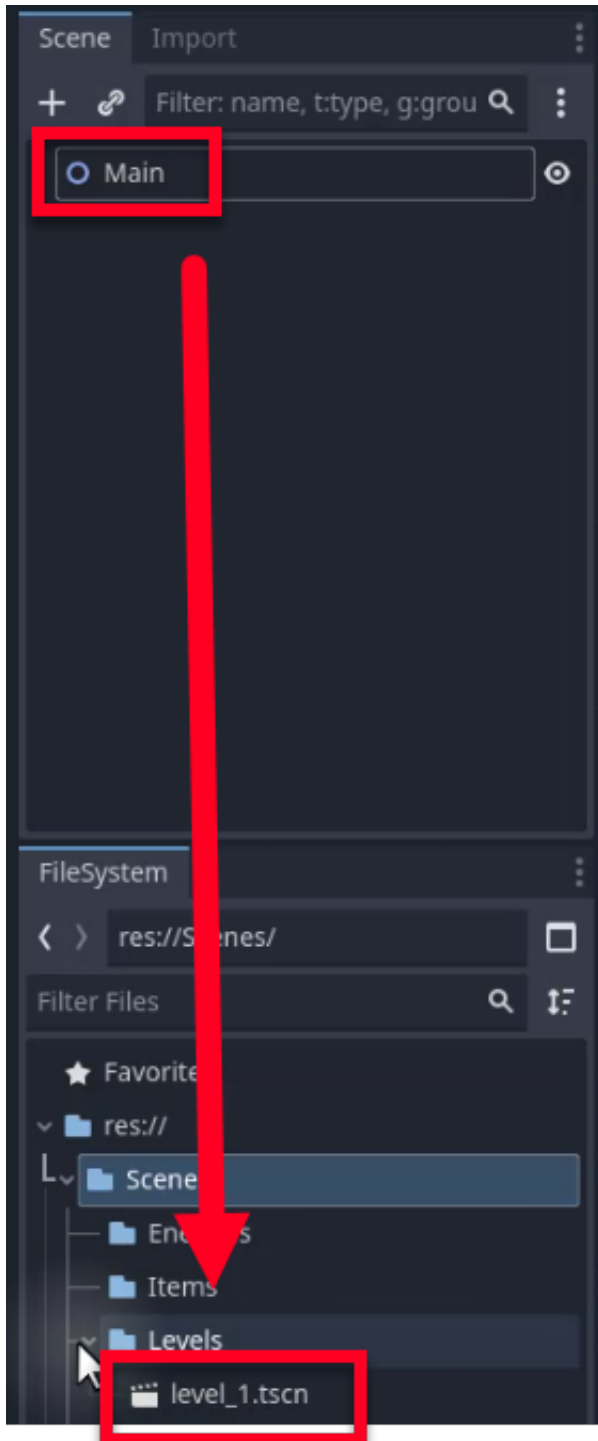
- Download and extract the course files, which include Audio and Sprites folders. Drag these folders into your project's FileSystem.



## Creating the First Scene

Let's create our first scene, which will serve as the main level where the player can walk around:

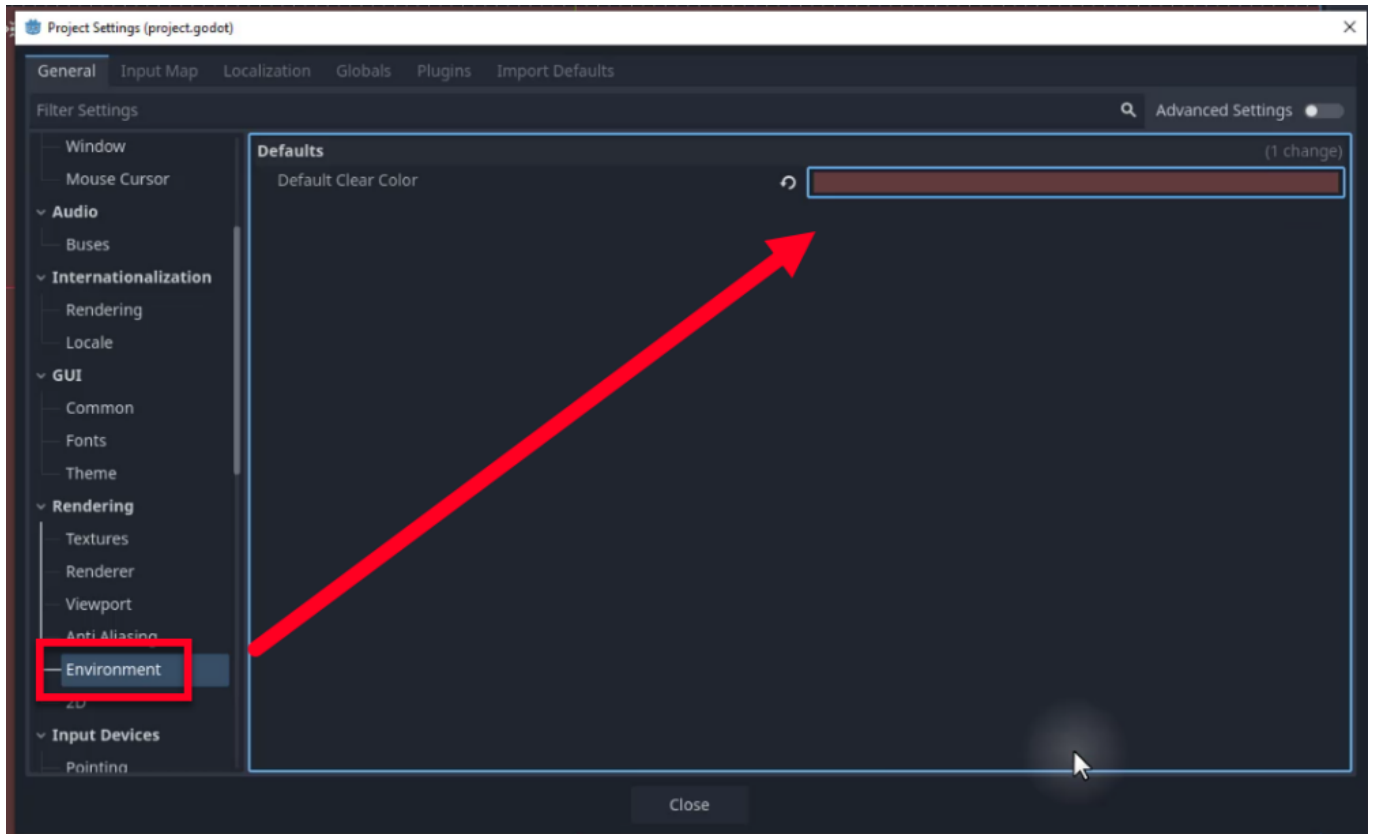
- Create a new 2D scene and set the root node to be called Main.
- Save this scene in the Levels folder as level\_1.



## Changing the Background Color

For our game, we want a specific background color that complements our tileset. Follow these steps to change the background color:

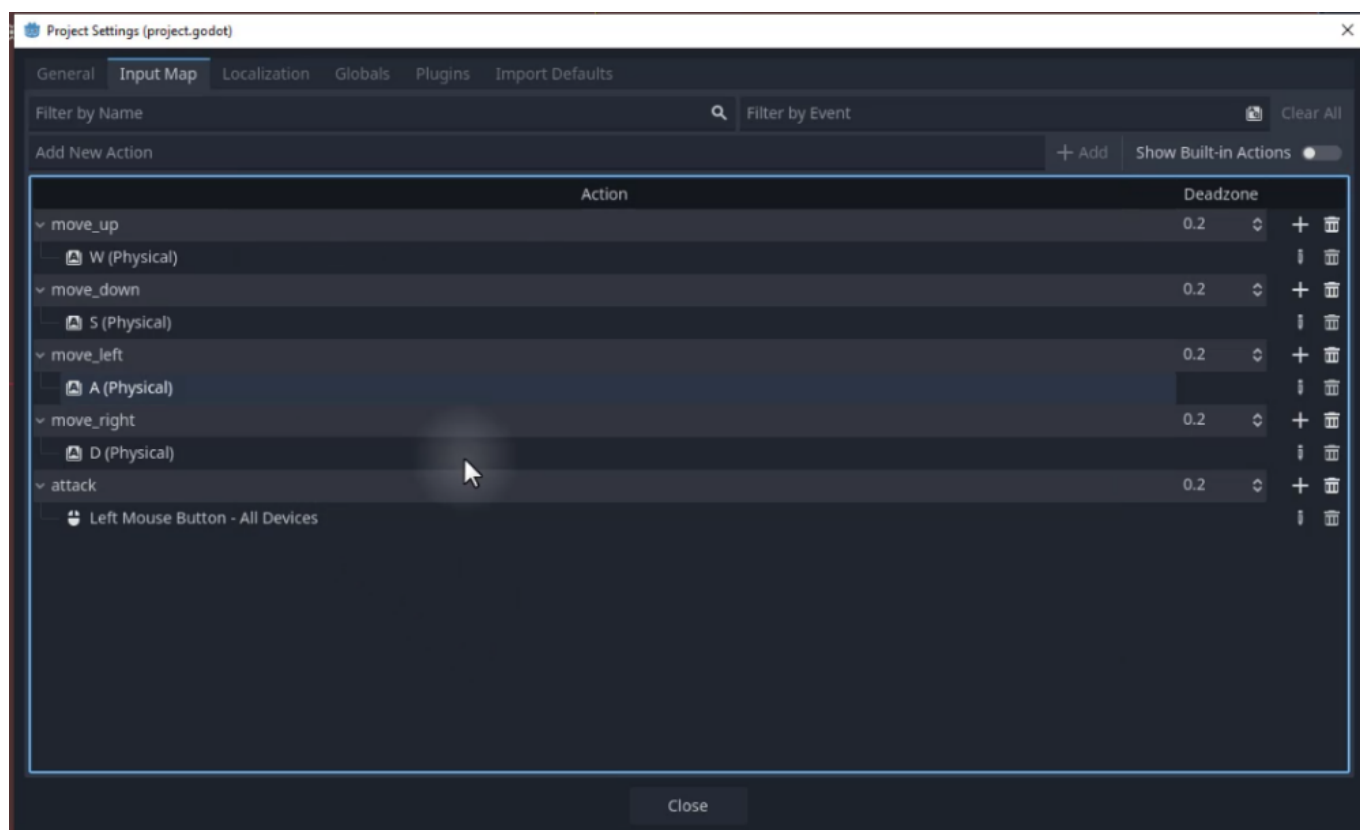
- Go to Project -> Project Settings.
- Navigate to the Rendering section and find the Environment option.
- Open the Default Clear Color and modify it to your desired color. For this guide, we recommend a brownish-red color that matches the provided tileset (#613c3c).



## Setting Up Inputs

Before setting up the player controller, we need to define the inputs for movement and attacking. Follow these steps:

- Go to Project -> Project Settings and navigate to the Input Map section.
- Define the following actions:
  - move\_up: Assign the W key.
  - move\_down: Assign the S key.
  - move\_left: Assign the A key.
  - move\_right: Assign the D key.
  - attack: Assign the left mouse button.



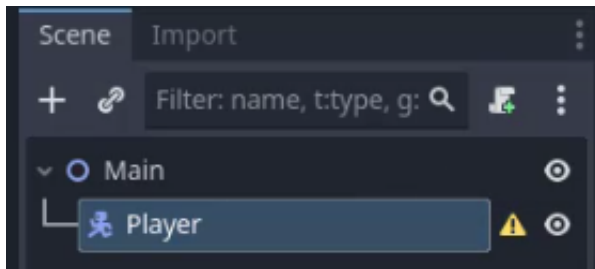
With these steps completed, you are now ready to begin setting up your player in the next lesson.

In this lesson, we will set up our player character and getting all the necessary nodes in place.

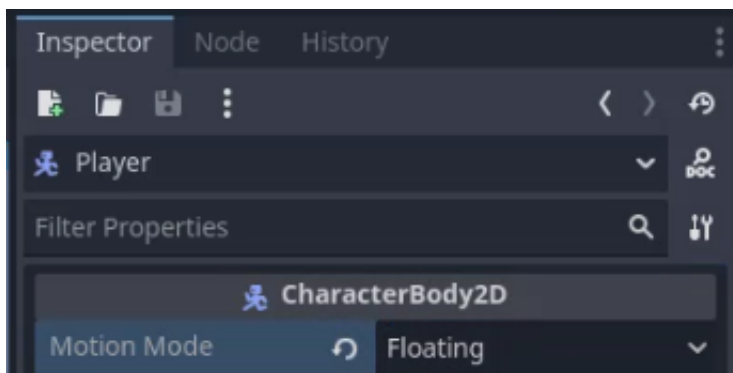
## Creating the Player Node

To begin, we need to create the root node for our player. This node will be of type `CharacterBody2D`. Follow these steps:

- Create a new node of type `CharacterBody2D`.
- Rename this node to `Player`.



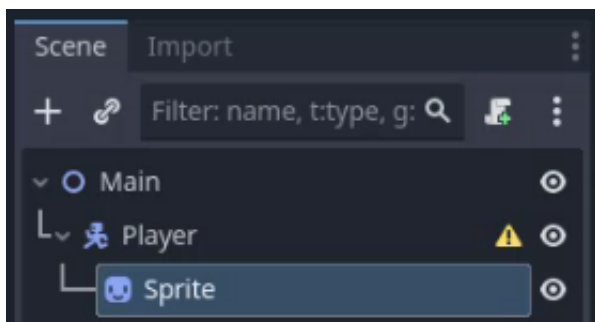
- In the Inspector, change the Motion Mode to Floating. This ensures that the calculations for movement are exact.



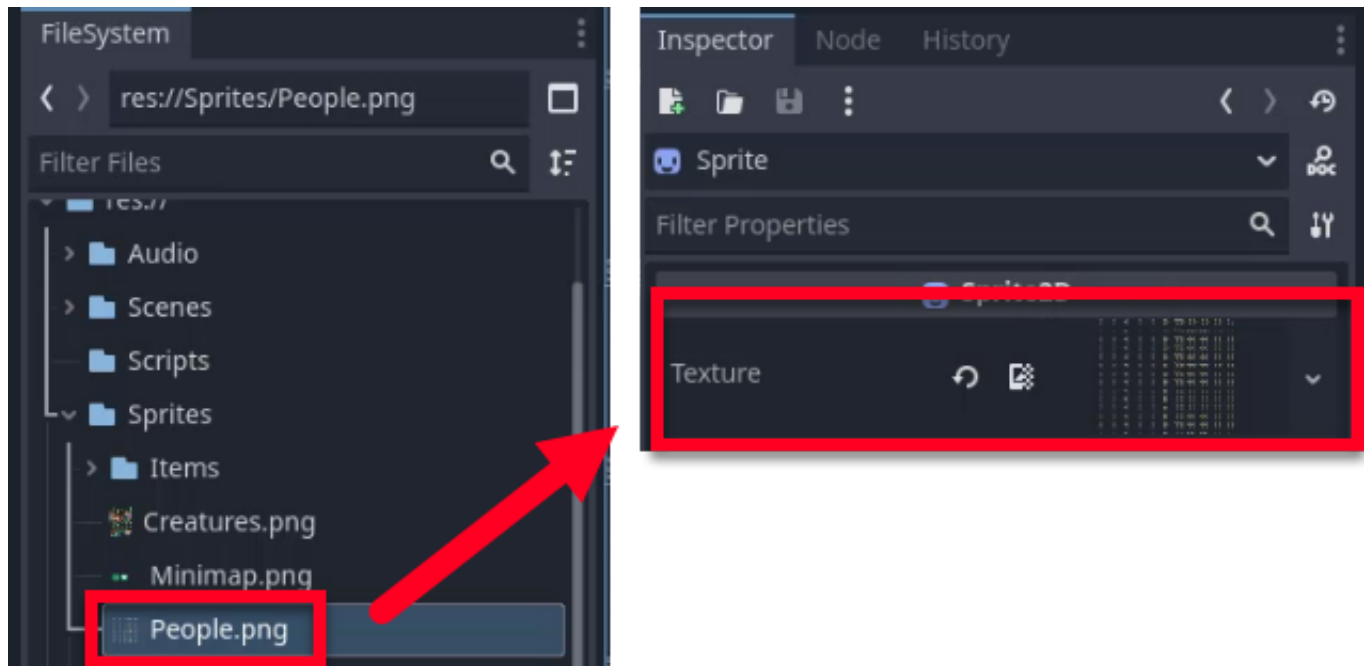
## Adding the Player Sprite

Next, we will add a sprite to represent our player visually. Since the collider size depends on the sprite, we will add the collider afterward.

- Create a child node of type `Sprite2D` under the `Player` node. We will rename it to "Sprite" for simplicity.



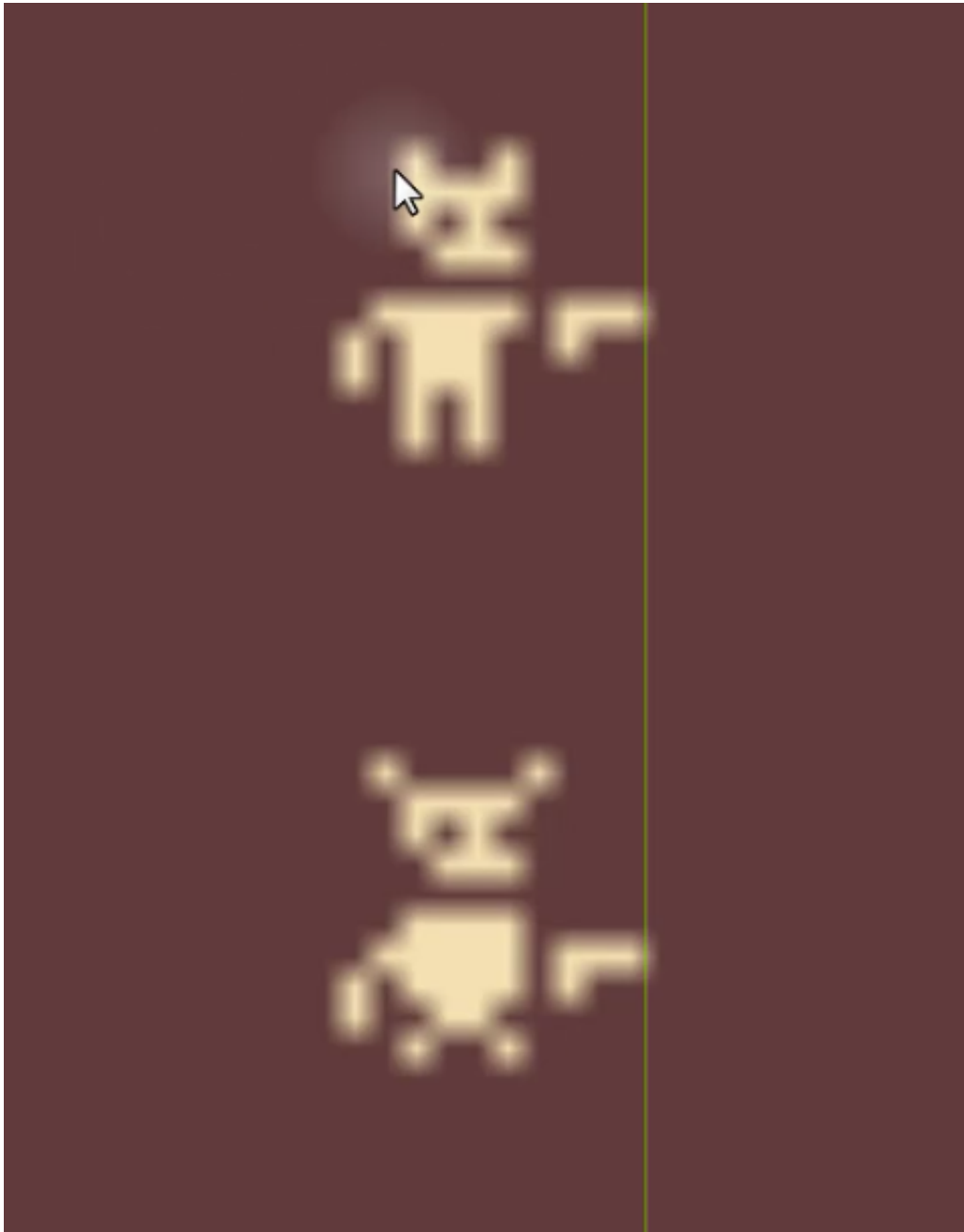
- In the texture field for the sprite, navigate to your Sprites folder, find the appropriate sprite (e.g., under `people.png`), and drag it into the texture field.



### Adjusting Texture Settings

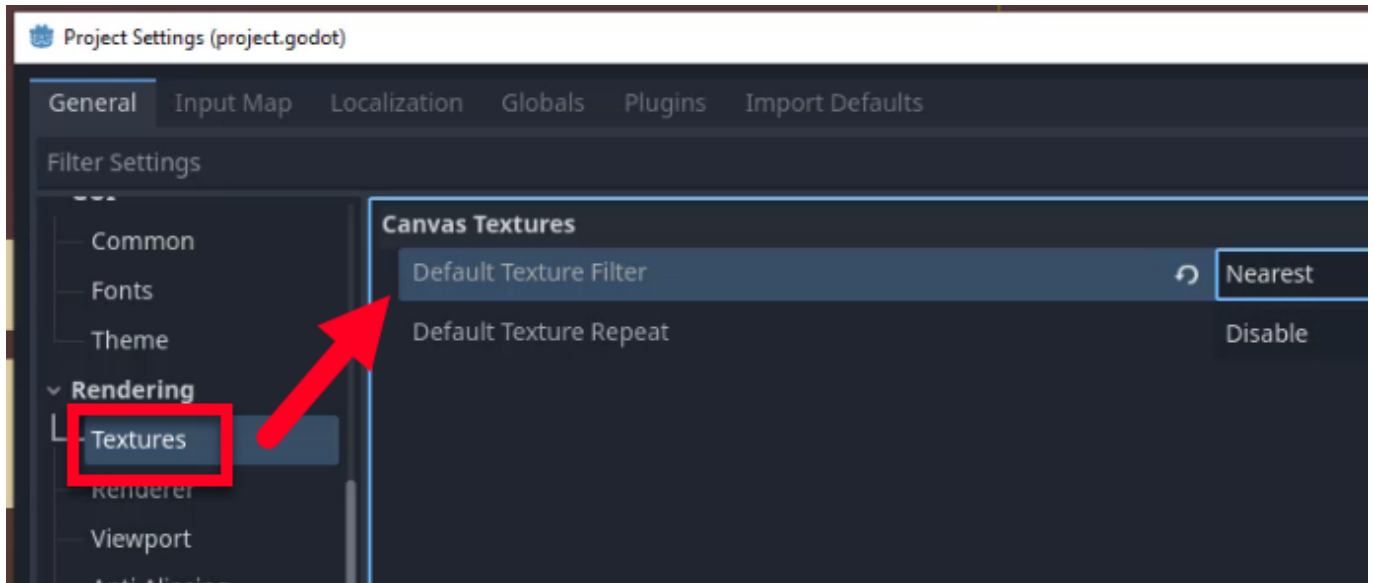
You might notice that the sprite appears blurry.





Let's fix that:

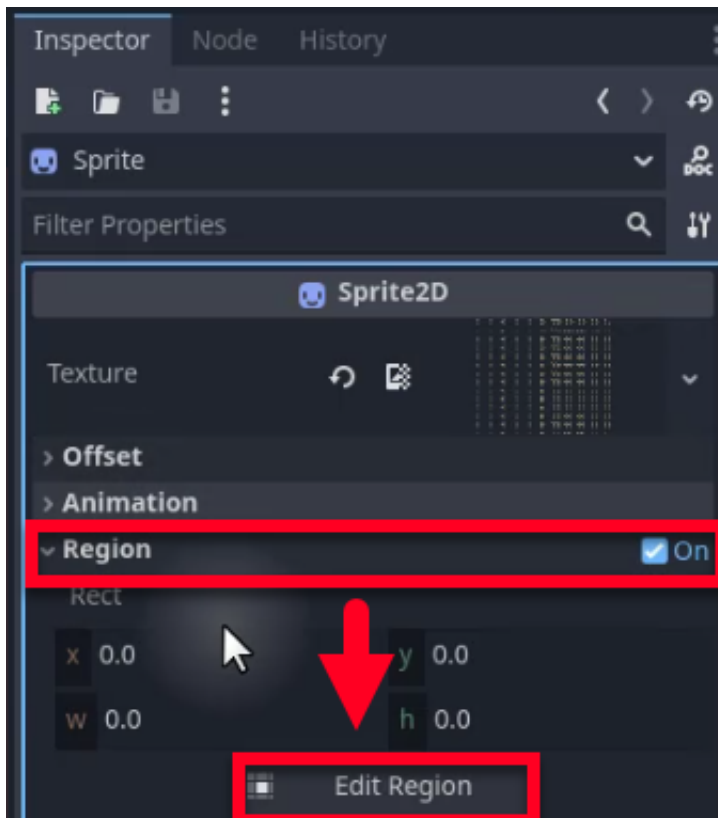
- Go to Project -> Project Settings -> General -> Rendering -> Textures.
- Change the Default Texture Filter from Linear to Nearest. This will make the sprites crisp.



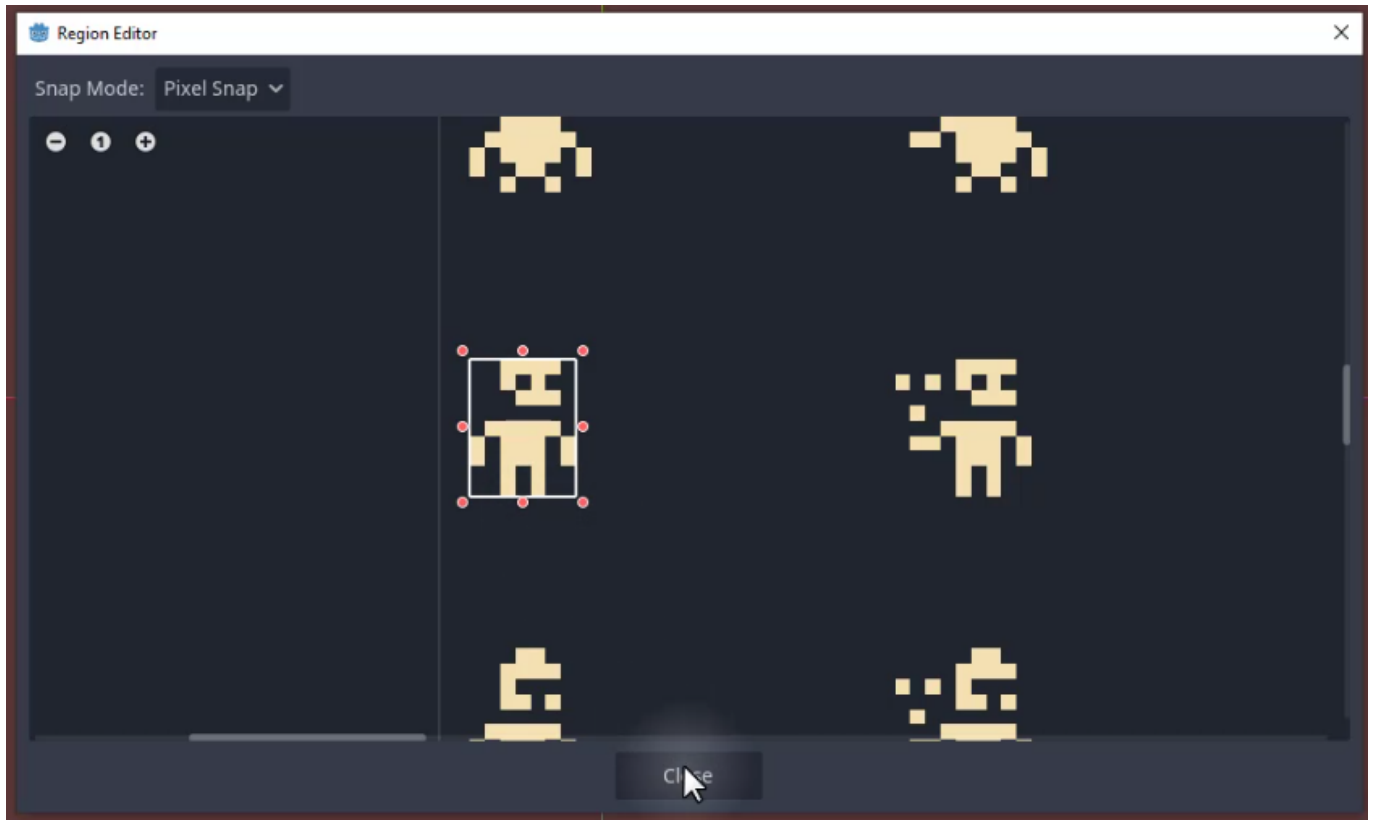
### Selecting a Region of the Sprite

If your sprite sheet contains multiple characters, you can select a specific region:

- Enable the Region option on the sprite and click Edit Region.



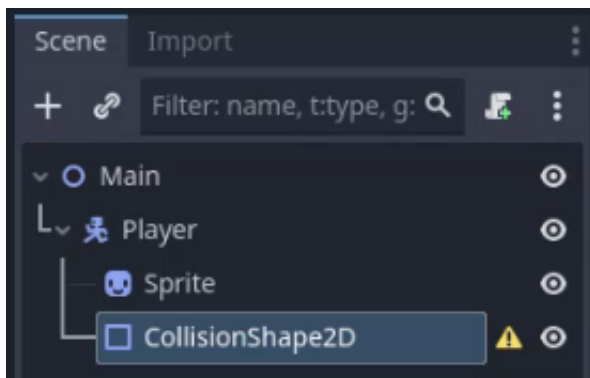
- Select the part of the sprite sheet that you want to use. You can change the Snap Mode to Pixel Snap for precise selection.



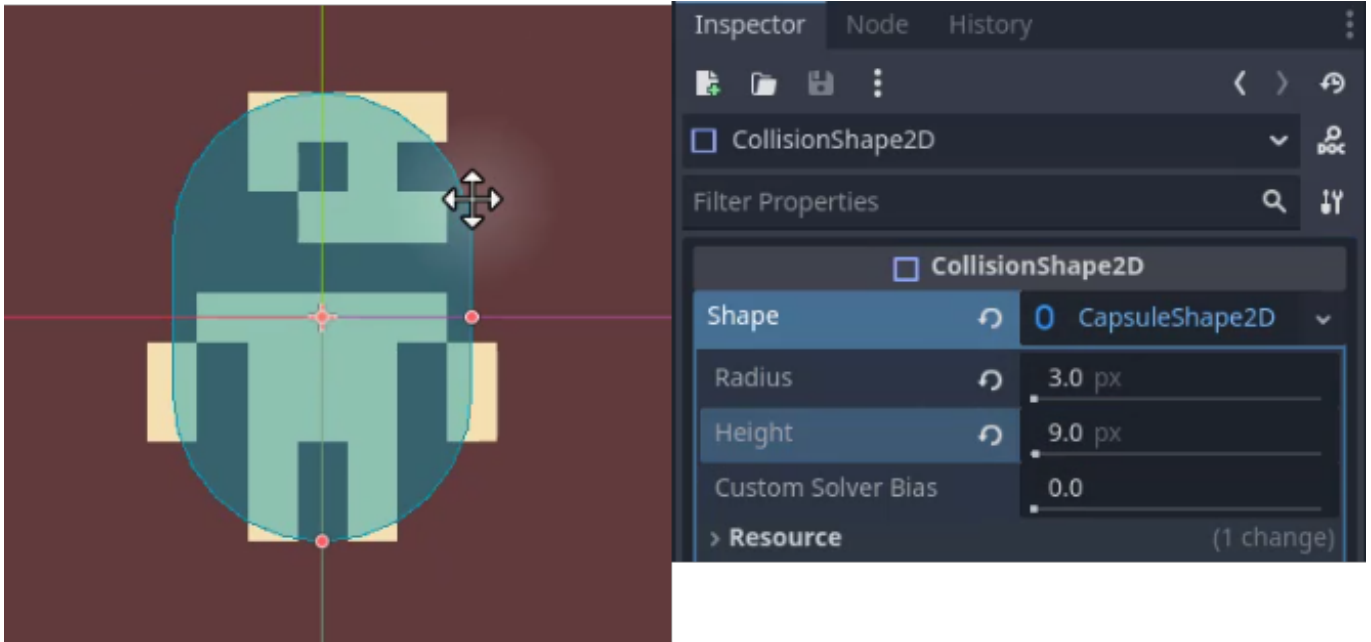
## Adding a Collision Shape

Now, let's add a collision shape to our player:

- Add a child node of type CollisionShape2D under the Player node.



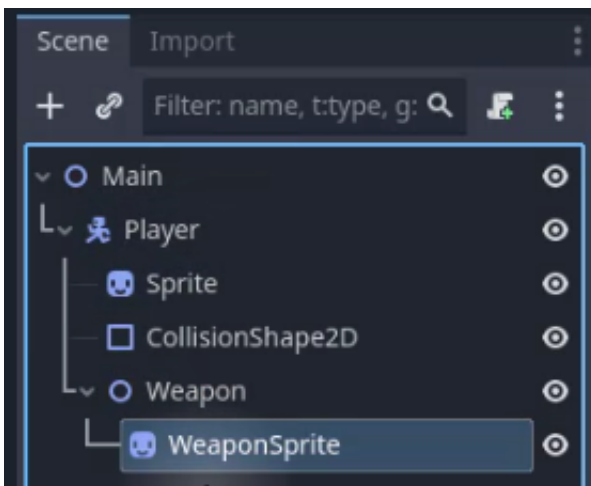
- Set the shape to CapsuleShape2D and adjust the radius and height to match your sprite. We found a radius of 3px and a height of 9px works for our sprite.



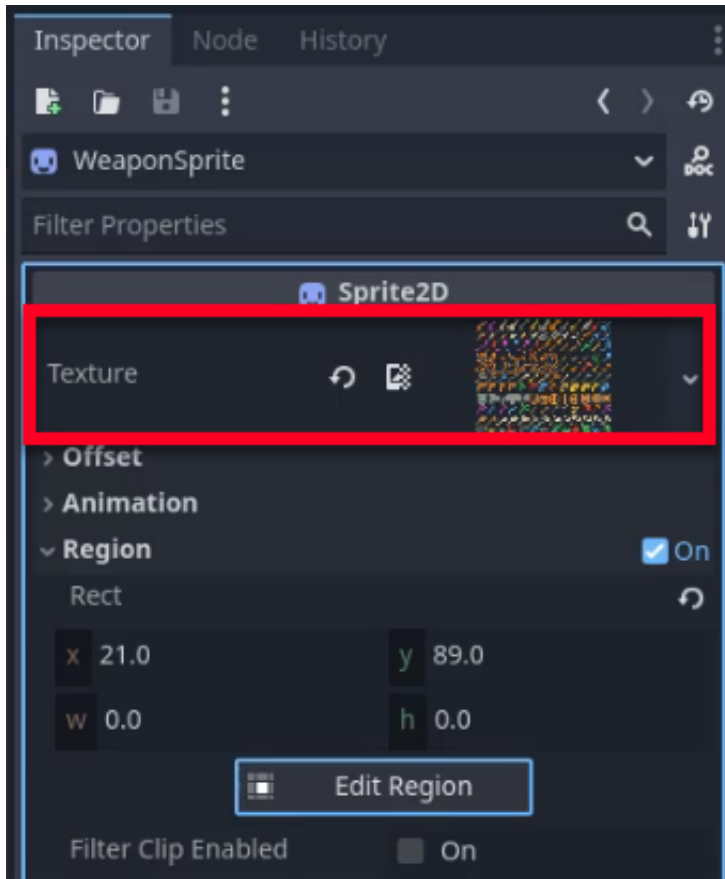
## Setting Up the Weapon

Our player will have a weapon that rotates towards the mouse cursor. Follow these steps to set it up:

- Create a new node of type Node2D under the Player node and rename it to Weapon.
- Add a child node of type Sprite under the Weapon node and rename it to WeaponSprite.



- In the texture field for WeaponSprite, drag the appropriate weapon sprite (e.g., weapons.png).

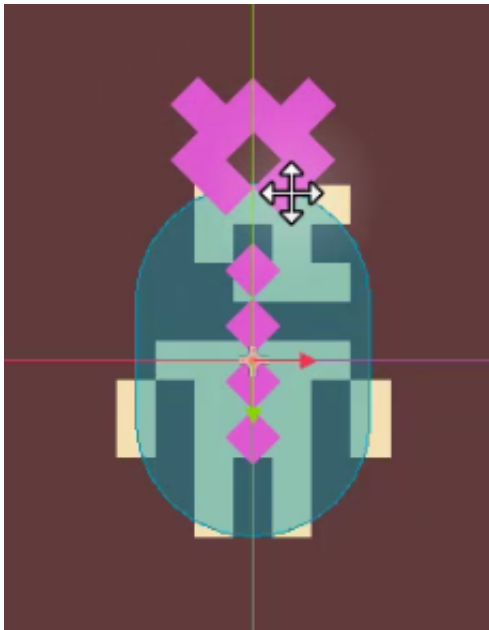


- Select a region of the weapon sprite sheet that you want to use (e.g., a purple wand).





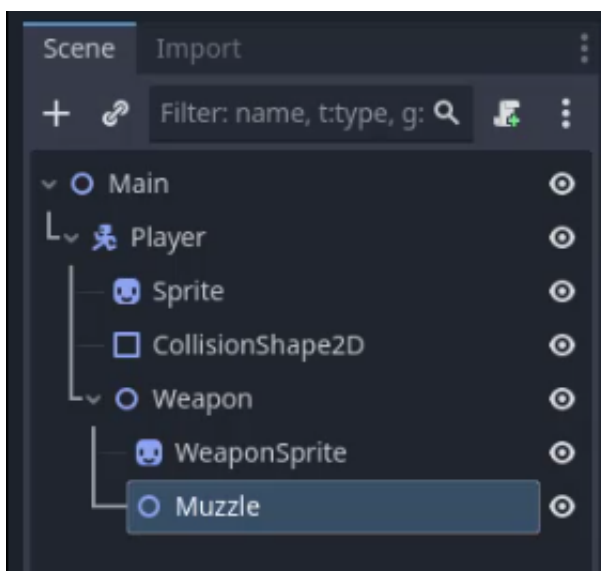
- Rotate the weapon sprite so that it points upwards. You can use the rotate tool and hold Control to snap to precise angles.
- Adjust the position of the weapon sprite so that it looks like the player is holding it.



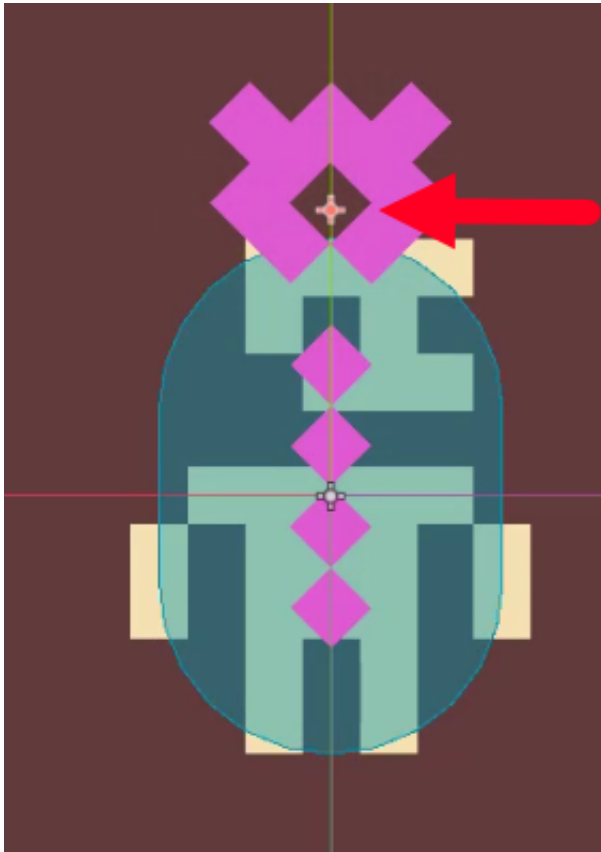
### Adding a Muzzle Point

The muzzle point is where projectiles will spawn. Follow these steps to add it:

- Add a child node of type Node2D under the Weapon node and rename it to Muzzle.



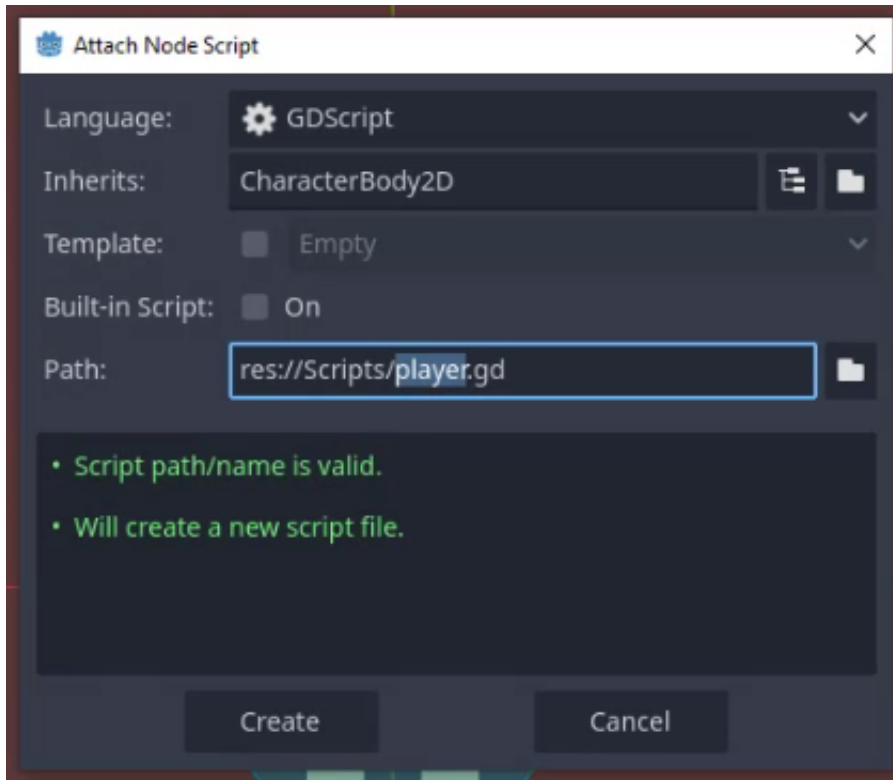
- Position the Muzzle node at the top of the weapon where you want projectiles to spawn.



### Creating the Player Script

Finally, let's create the script that will control our player:

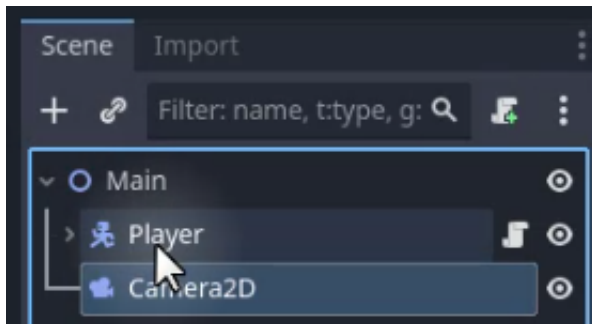
1. Select the Player node.
2. In the script field, create a new script and name it Player.
3. Save the script in the Scripts folder.



## Setting Up the Camera

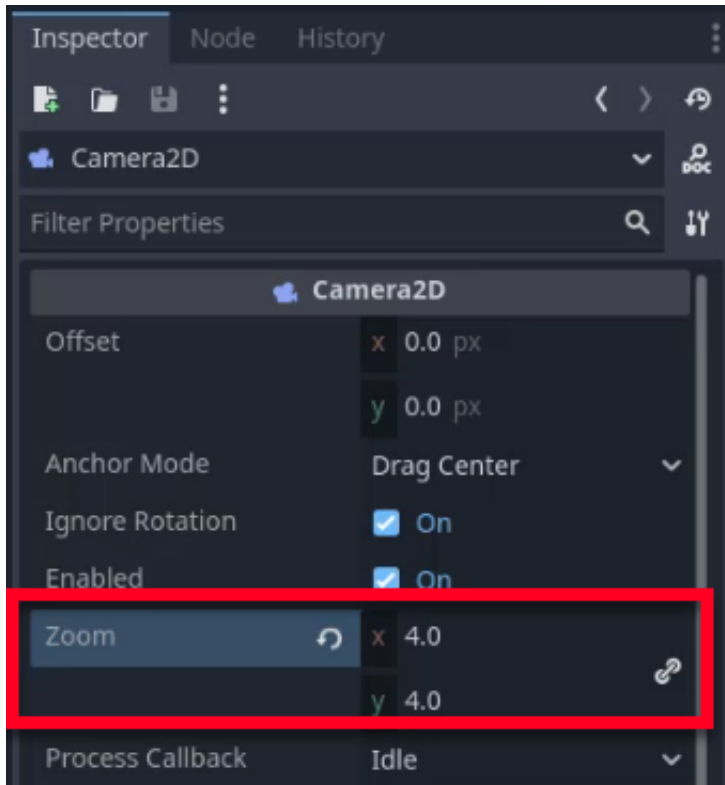
To see our player in action, we need to set up a camera:

- Create a new node of type Camera2D (not as a child of the Player node but in the Main scene).

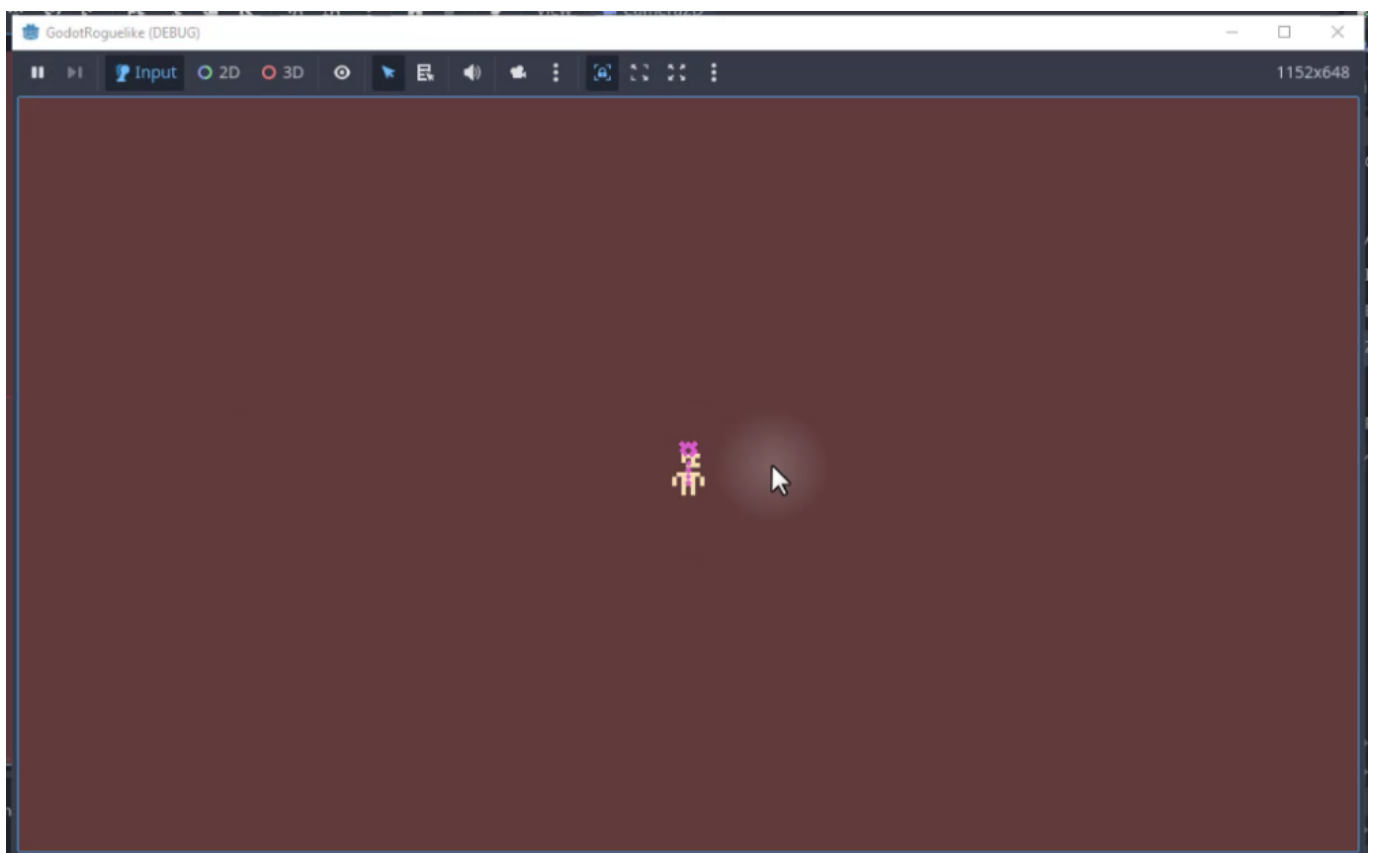


- Set the zoom level to 4 so that each pixel in the game is represented by 4 pixels in the game window.





That's it for this lesson! You should now be able to see our player character in the game.



In the next lesson, we will implement the movement and aiming functionality for our player.



In this lesson, we will focus on setting up the player script. This script will handle the player's movement and aiming mechanics. We will define variables, set up the necessary nodes, and implement the logic for moving the player and rotating the weapon towards the mouse cursor.

## Defining Variables

First, let's define the variables we need in our player script:

- `move_speed`: This will determine how fast the player moves. We will make it an export variable so we can adjust it in the inspector.
- `sprite`: This will reference the player's sprite node.
- `weapon_origin`: This will reference the node around which the weapon will rotate.
- `muzzle`: This will reference the node that represents the muzzle of the weapon, from where the projectiles will be fired.

Here is how you can define these variables in the script:

```
extends CharacterBody2D

@export var move_speed : float = 50

@onready var sprite : Sprite2D = $Sprite
@onready var weapon_origin : Node2D = $Weapon
@onready var muzzle : Node2D = $Weapon/Muzzle
```

We will use the `@onready` keyword to ensure that the nodes are initialized correctly when they're loaded into the scene tree.

## Implementing Player Movement

To move the player, we will use the `_physics_process` function. This function is called every frame and is suitable for handling physics-based movements.

Here are the steps to implement player movement:

1. Get the move input from the player using the `Input.get_vector` function.
2. Assign the move input times the player speed to the velocity of the player.
3. Call the `move_and_slide` function to apply the velocity to the player and check for collisions.

Here is the code to implement player movement:

```
func _physics_process(delta):
    var move_input : Vector2 = Input.get_vector("move_left", "move_right", "move_up", "move_down")
    velocity = move_input * move_speed
    move_and_slide()
```

## Implementing Weapon Aiming



To make the weapon rotate towards the mouse cursor, we will use the `_process` function. This function is called every frame and is suitable for handling non-physics-based updates.

Here are the steps to implement weapon aiming:

1. Get the mouse position using the `get_global_mouse_position` function.
2. Calculate the direction from the player to the mouse position.
3. Normalize the direction vector.
4. Set the rotation of the weapon origin based on the direction vector.
5. Convert the rotation from radians to degrees using the `rad_to_deg` function.
6. Adjust the rotation by adding 90 degrees to point the weapon in the correct direction.

Here is the code to implement weapon aiming:

```
func _process(delta):
    var mouse_pos : Vector2 = get_global_mouse_position()
    var mouse_dir : Vector2 = (mouse_pos - global_position).normalized()
    weapon_origin.rotation_degrees = rad_to_deg(mouse_dir.angle()) + 90
```

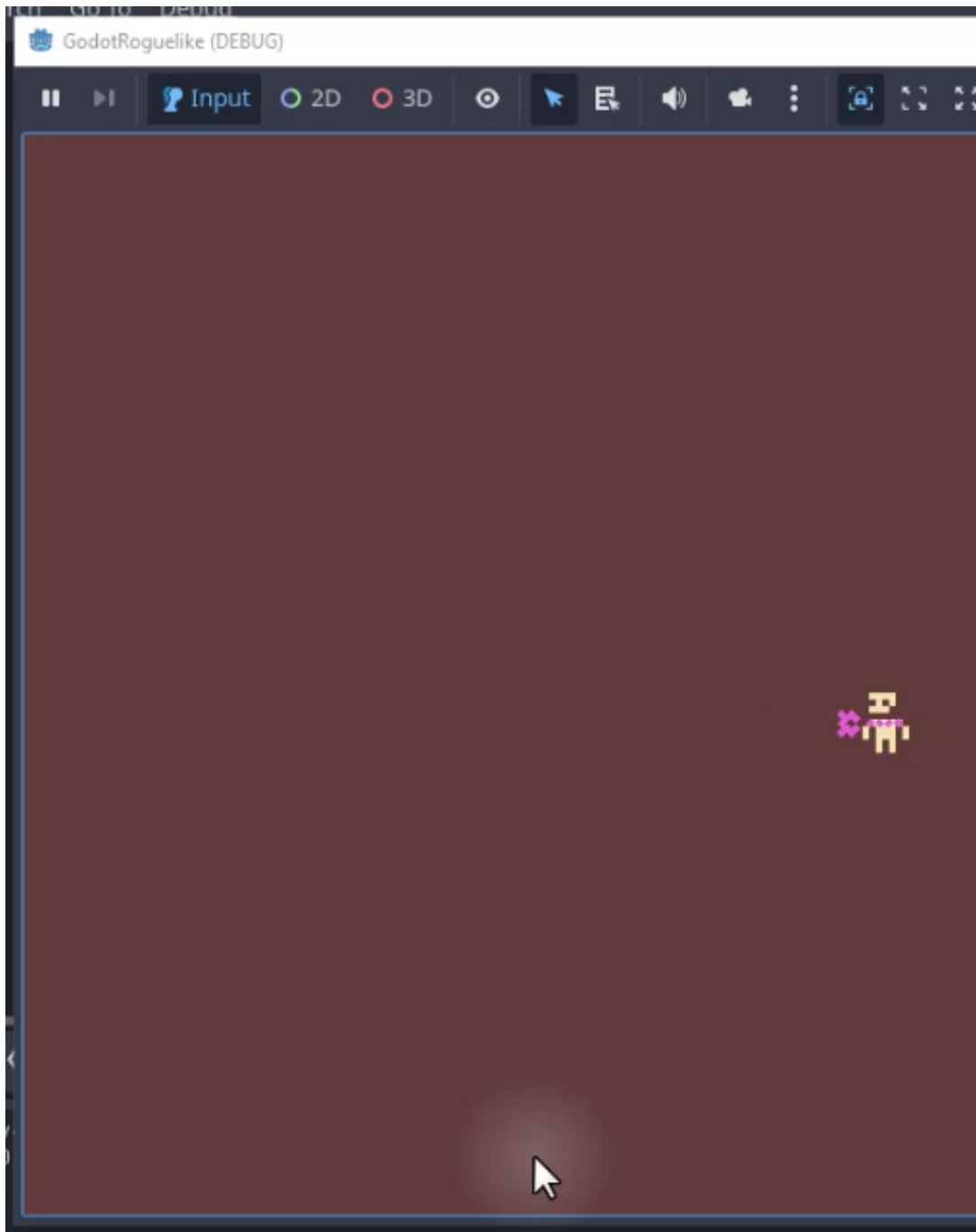
## Flipping the Sprite

To make the player sprite face the direction of the mouse, we will flip the sprite horizontally based on the mouse direction.

Here is the code to flip the sprite:

```
sprite.flip_h = mouse_dir.x < 0
```

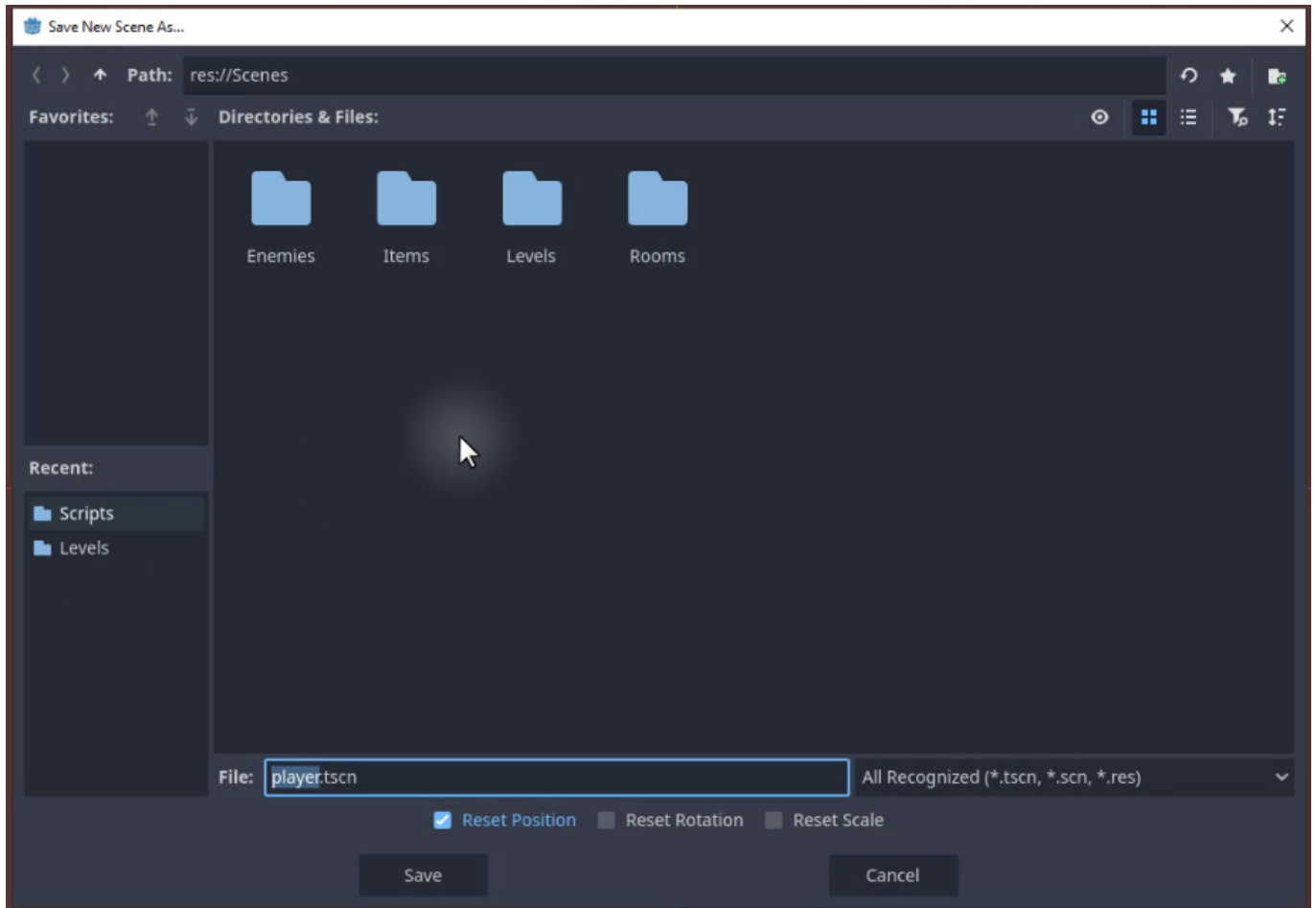
Now you should have a player who can move, flip, and have the weapon's rotation follow the mouse.



### **Saving the Player Scene**

Finally, save the player node as a scene. This will allow us to reuse the player node in other scenes.

1. Go to the Scene panel.
2. Select the Player node.
3. Click on the “Save Branch as Scene” button (the floppy disk icon).
4. Save the scene as player.tscn in the Scenes folder.



By following these steps, you have successfully set up the player's movement and aiming mechanics. In the next lesson, we will focus on setting up the room creation and generation for our game.

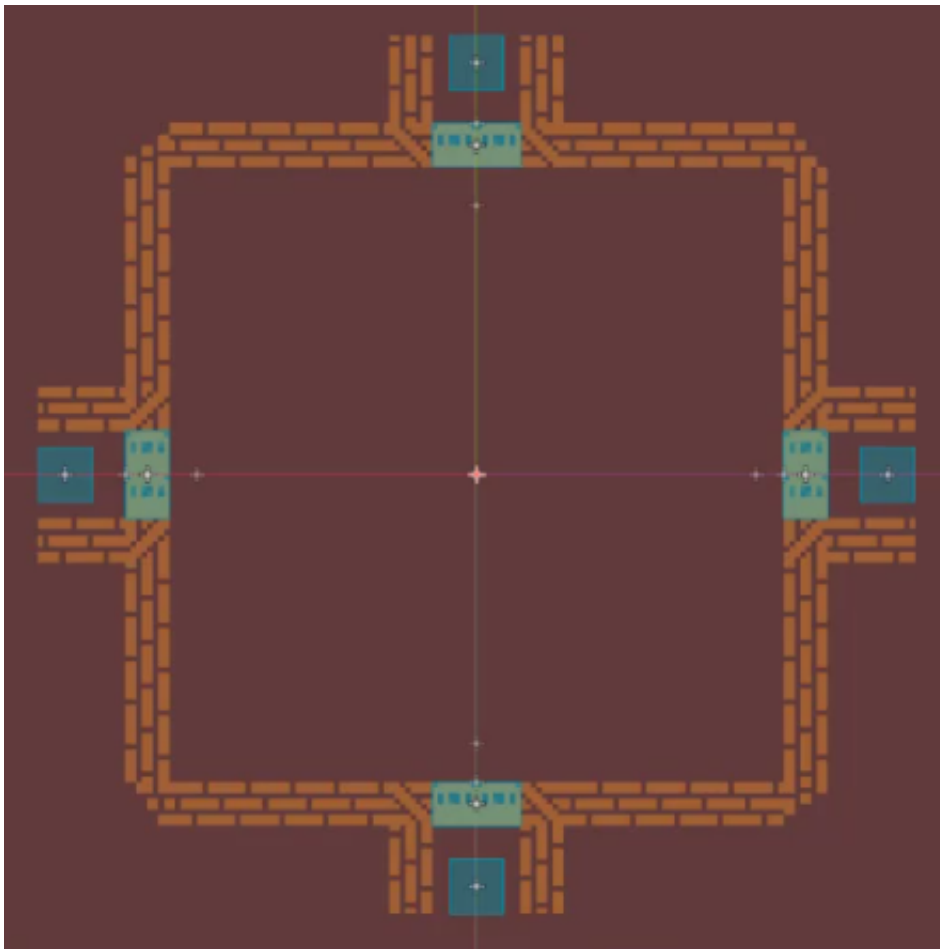


In this lesson, we will be creating our base room template. This template will serve as the foundation for all the rooms in our game. Each room will be a square, connected to others via tunnels and doors. Let's get started!

## Understanding the Room Structure

Before we dive into the creation process, let's understand the basic structure of our room:

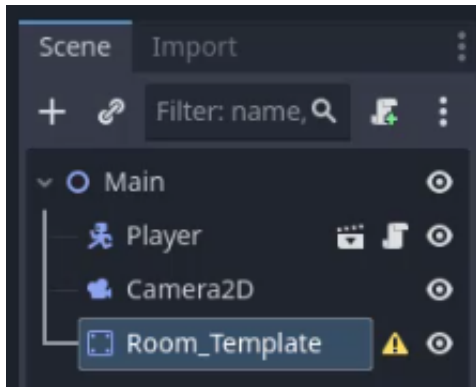
- The room is a square with four sides.
- Each side can have either a door or a barrier.
- A barrier is a flat wall, while a door is an opening that the player can pass through.
- Doors can be opened or closed, depending on the presence of enemies in the room.



Follow these steps to create the room template:

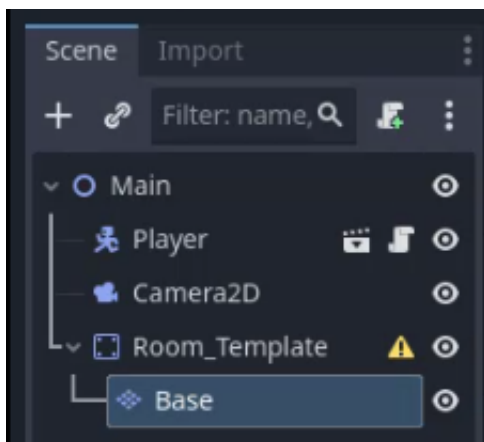
### Step 1: Setting Up the Room Node

1. Create a new StaticBody2D node and rename it to Room\_Template.
2. This node will serve as the base for all rooms in our game.

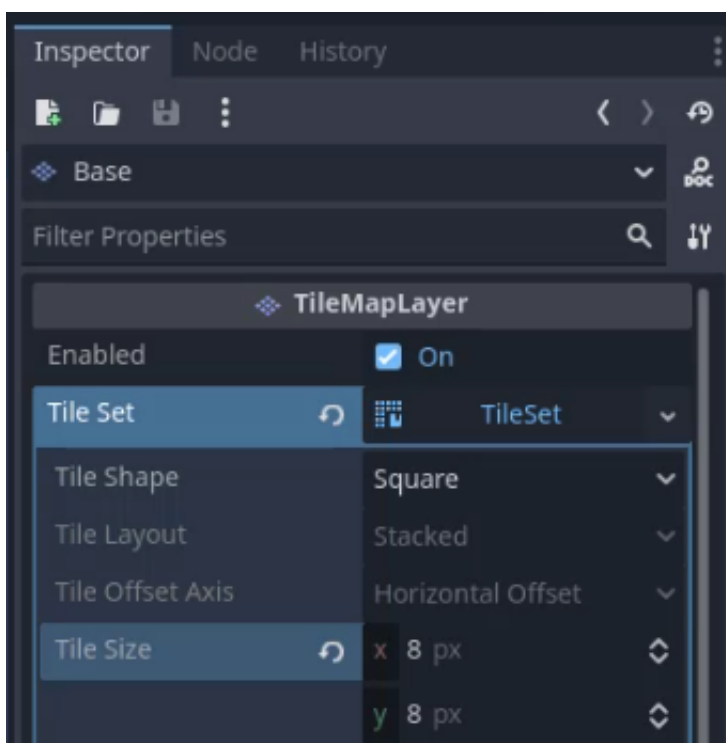


## Step 2: Adding the TileMap Layer

- Create a new TileMapLayer node and name it Base.

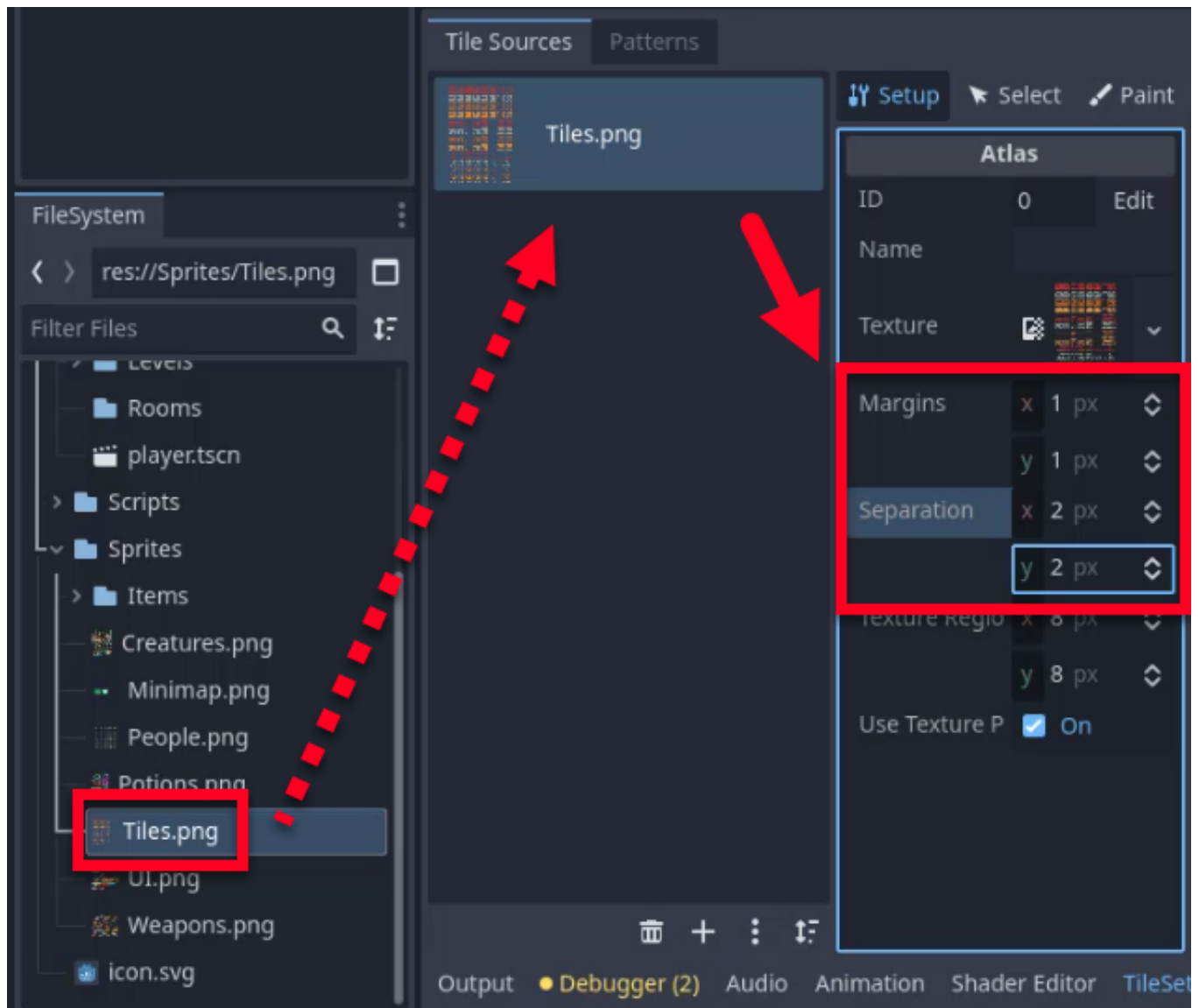


- In the inspector, create a new TileSet and set the tile size to 8x8 pixels.





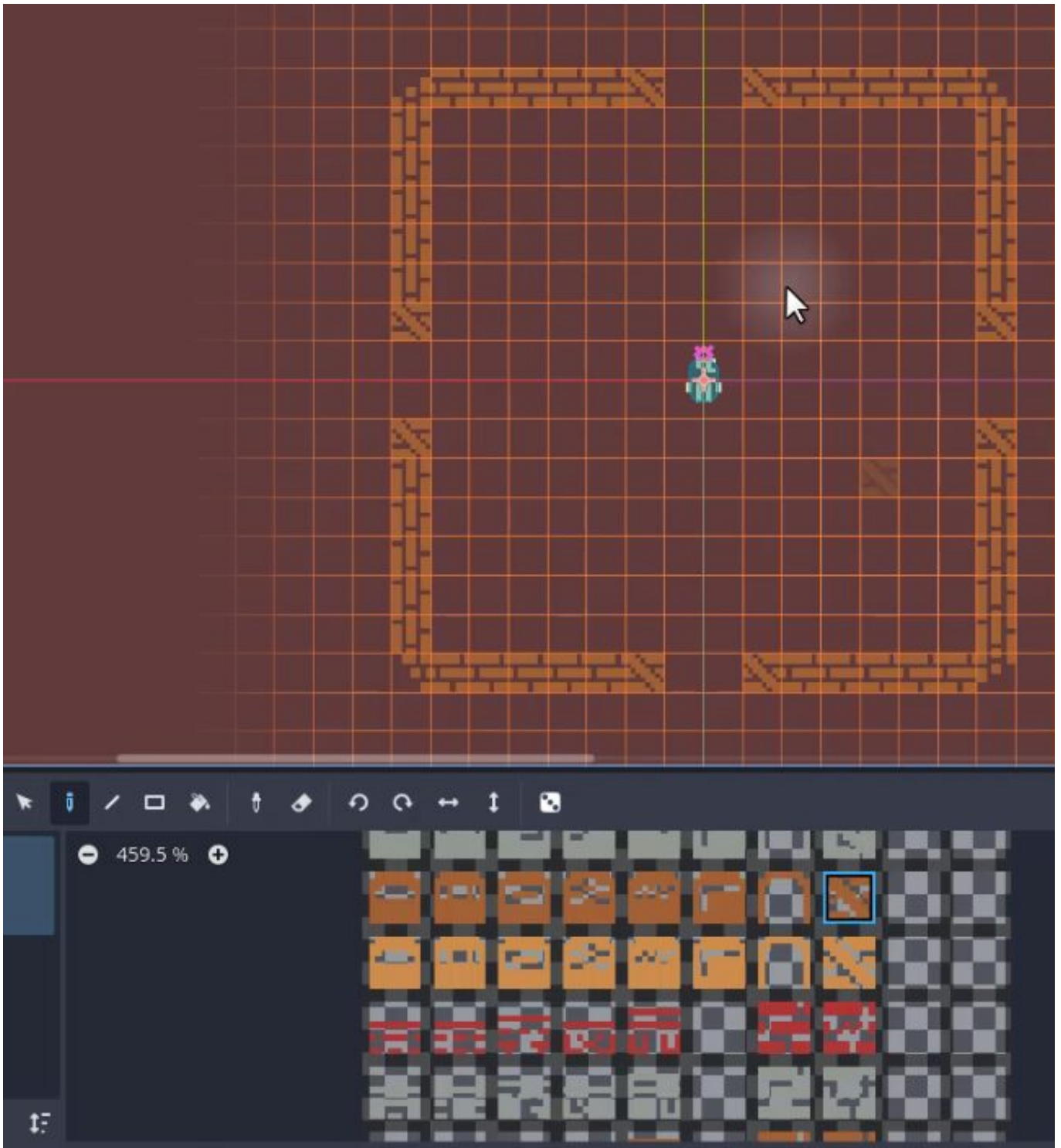
- Drag your tileset image (e.g., Tiles.png) into the tile set.
- Adjust the margin and separation settings to ensure the tiles are properly spaced. For us, the Margins are 1px on the X and Y, and the separation is 2px on the X and Y.



### Step 3: Drawing the Room Corners

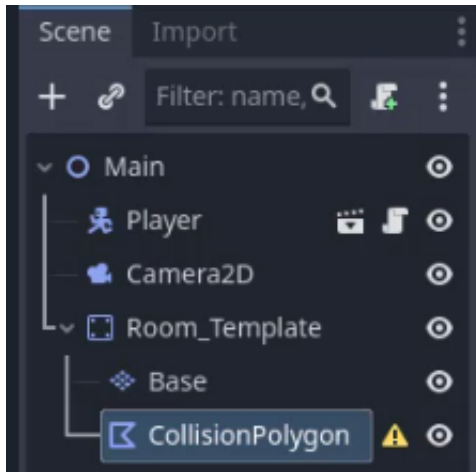
1. Using the TileMap, draw the corners of the room.
2. Remember, we will add doors and barriers as separate nodes later.



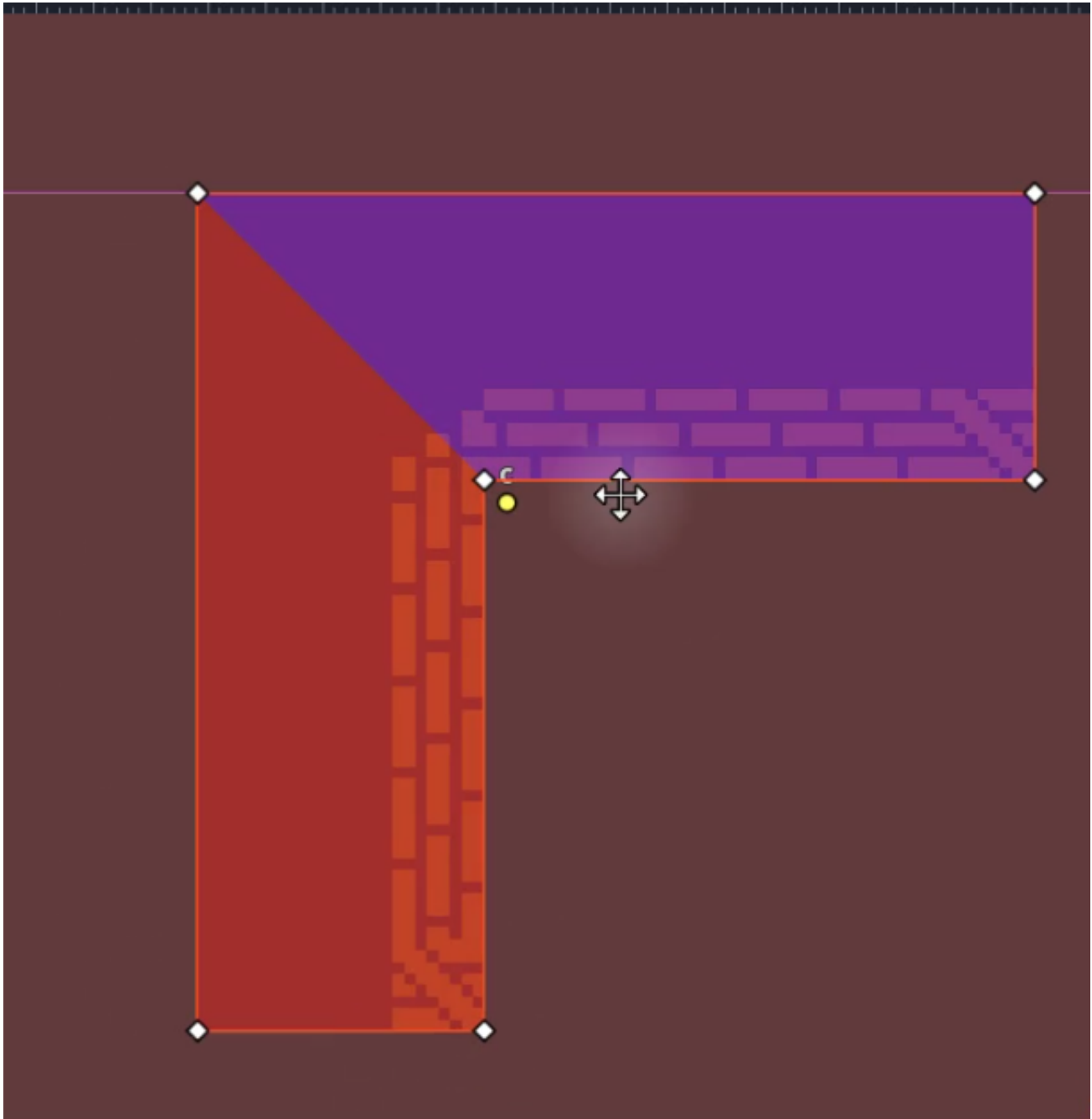


#### Step 4: Adding Wall Colliders

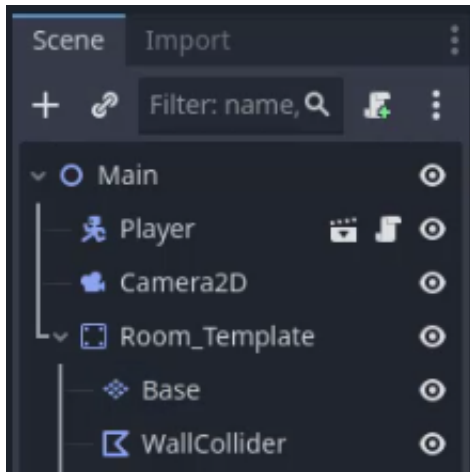
- Add a CollisionPolygon2D as a child node to create wall colliders.



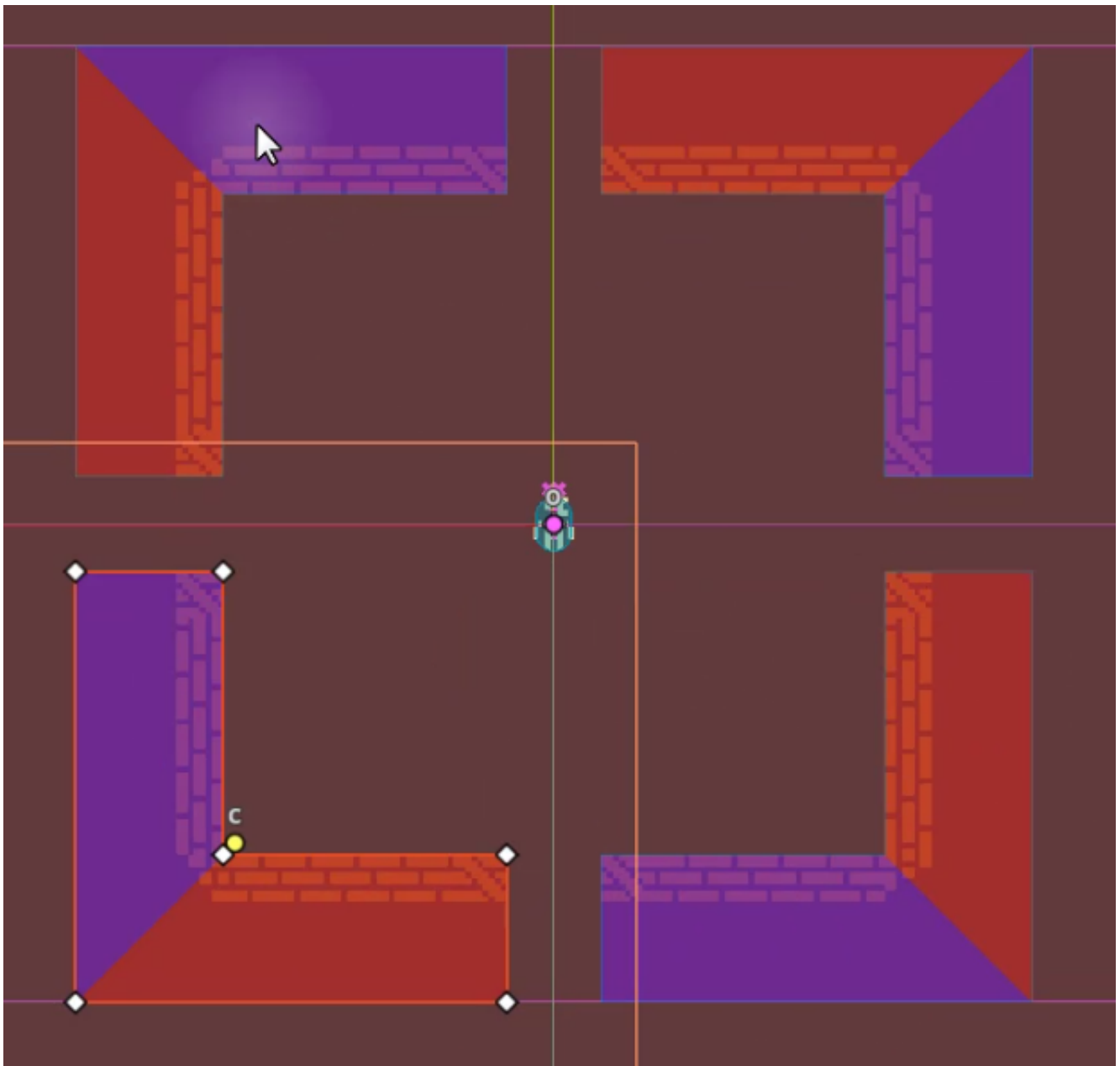
- Define the custom shape for the collider by clicking and dragging points.



- Rename the node to “WallCollider”.



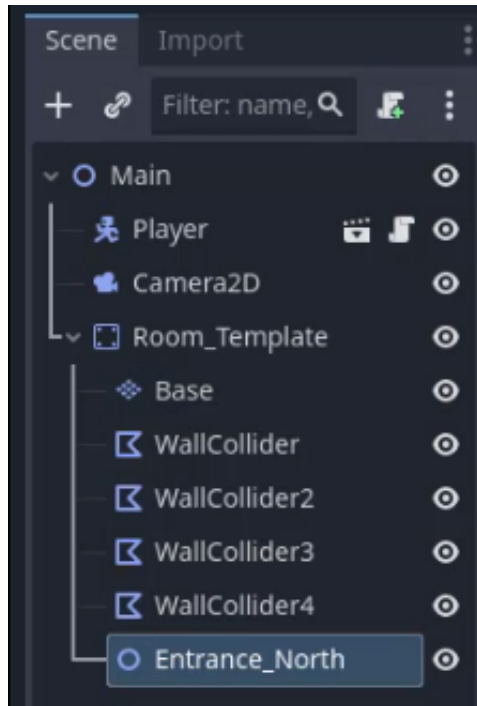
- Duplicate the node and rotate the collider to cover all four walls.





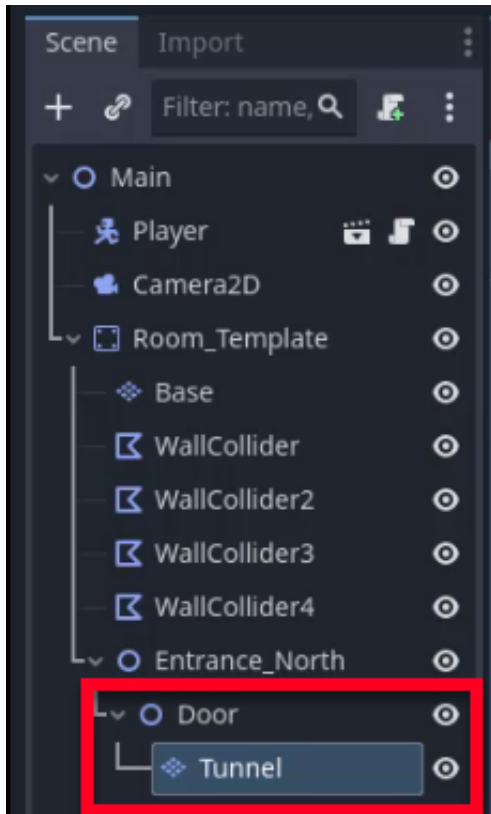
## Step 5: Creating Entrances

1. Create a new Node2D and name it Entrance\_North.
2. This node will represent the northern entrance of the room.

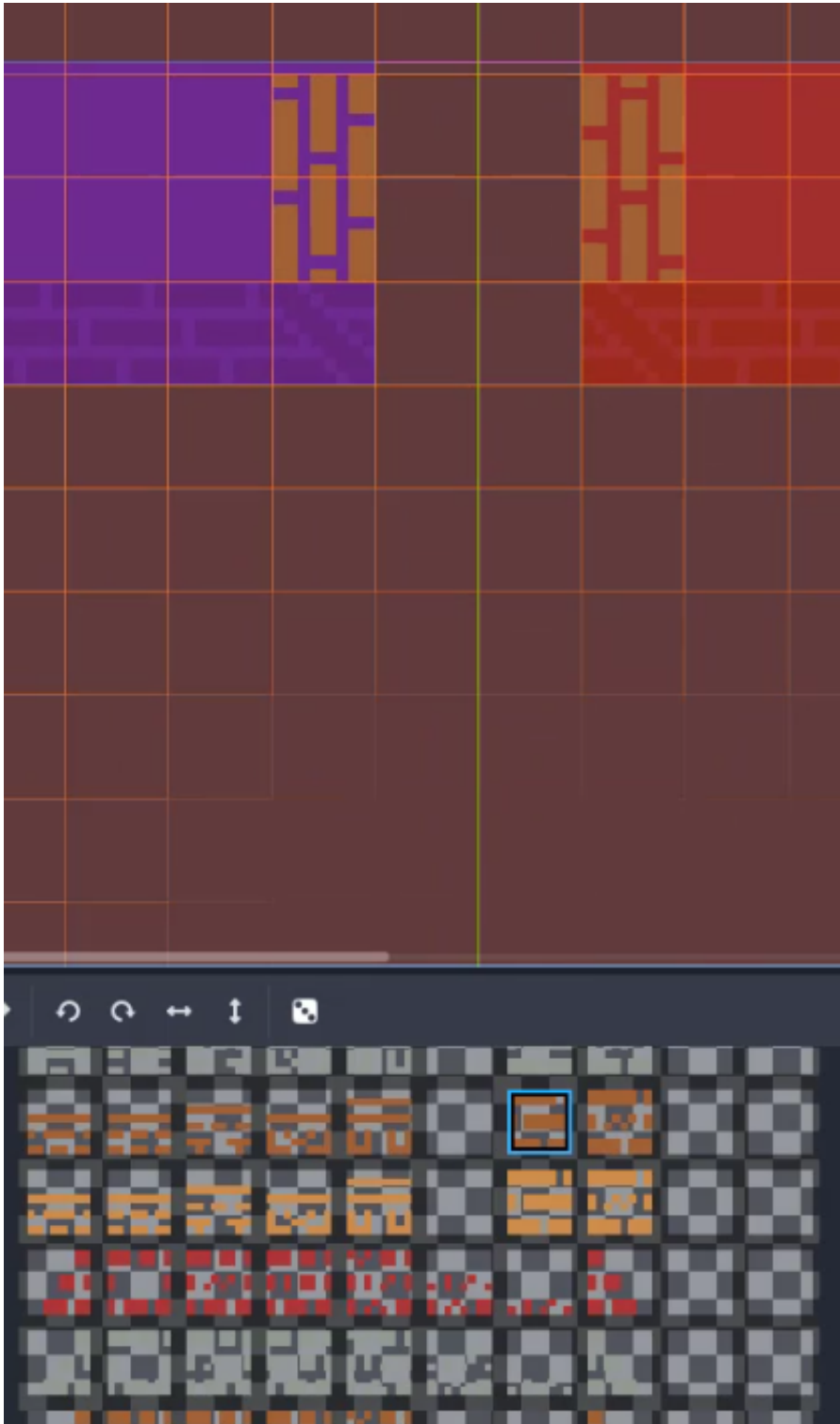


## Step 6: Adding Doors and Tunnels

- Add a Node2D as a child of Entrance\_North and name it Door.
- Create a TileMap layer as a child of the Door node and name it Tunnel.

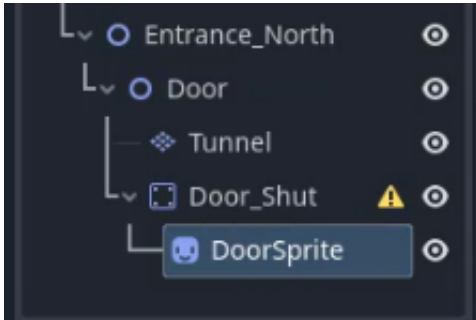


- Draw the tunnel tiles to create the appearance of a tunnel leading out of the room.

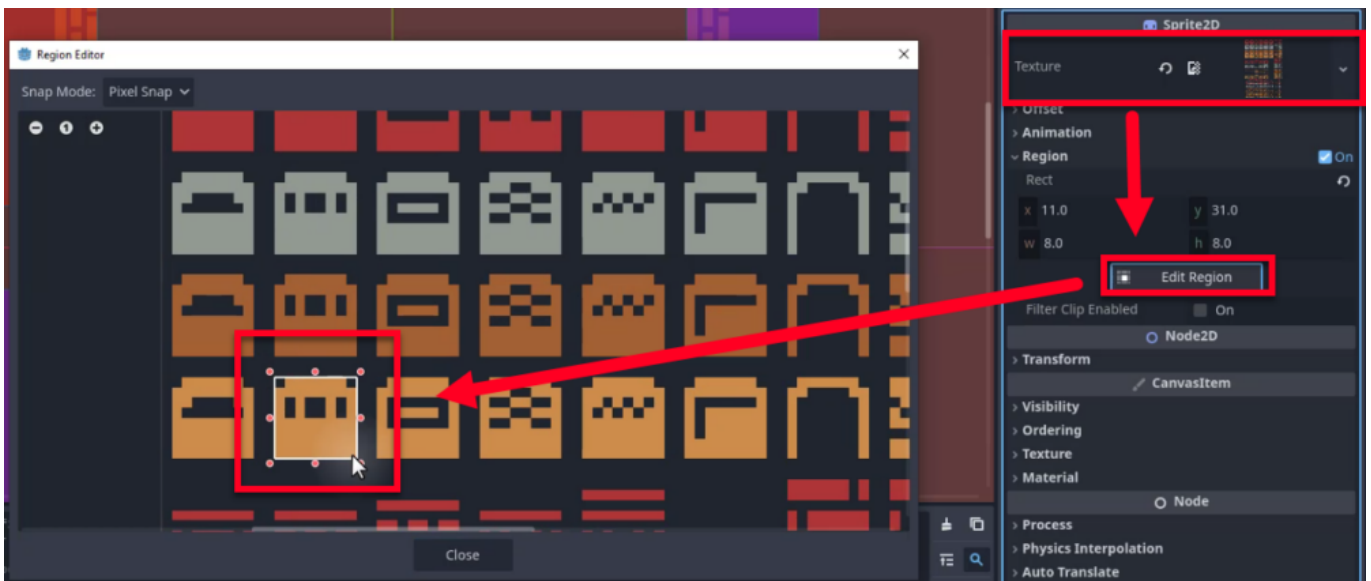


### Step 7: Adding Door Sprites and Collisions

- Add a StaticBody2D as a child of the Door node and name it Door\_Shut.
- Add a Sprite2D as a child of Door\_Shut . Rename this to “DoorSprite”



- With DoorSprite selected, add the Tiles.png to the Texture property in the Inspector. Then, click Edit Region and select an appropriate sprite.

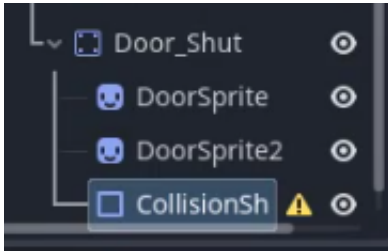


- Duplicate the DoorSprite as our tunnel will need to be two double-doors to fit the space.

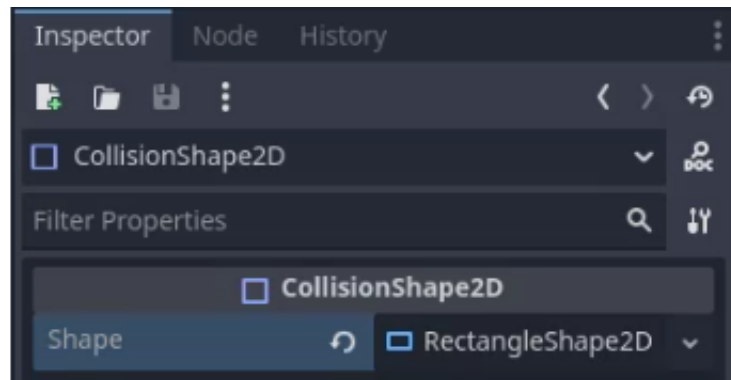
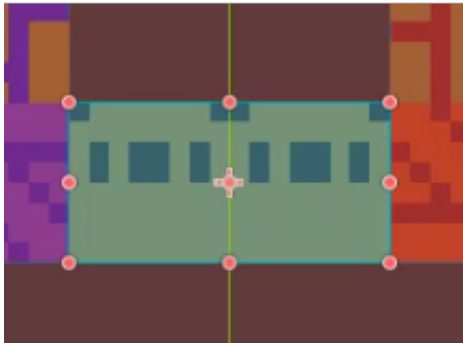


- As a child of Door\_Shut, add a CollisionShape2D to define the collision area for the door.



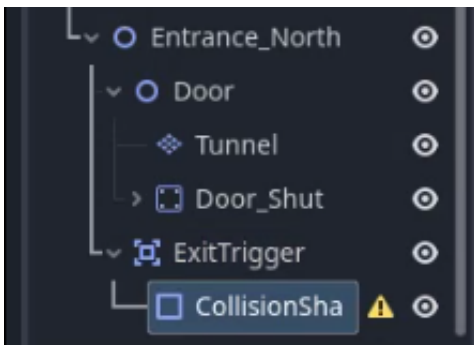


- In the Inspector, select RectangleShape2D for the shape and resize it to fit the door.

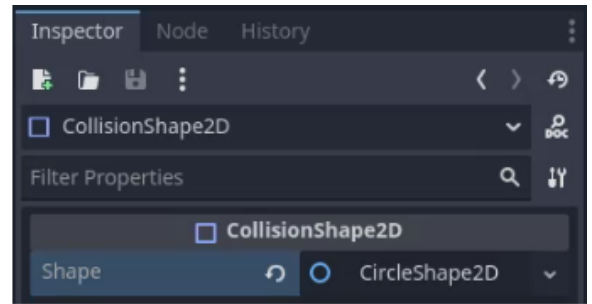
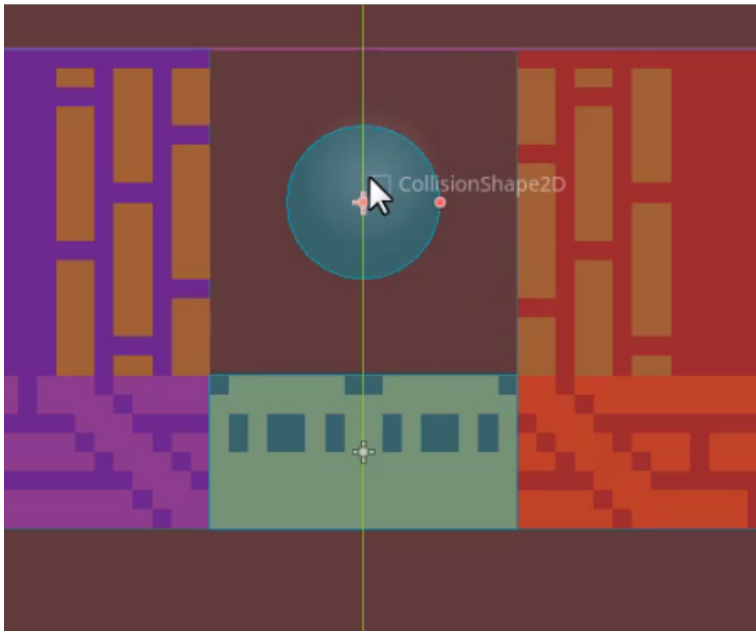


### Step 8: Adding Exit Triggers and Player Spawn

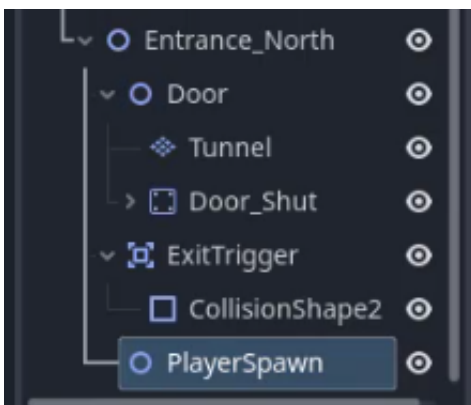
- Add an Area2D as a child of Entrance\_North and name it ExitTrigger.
- Attach a CollisionShape2D to detect when the player walks through the door.



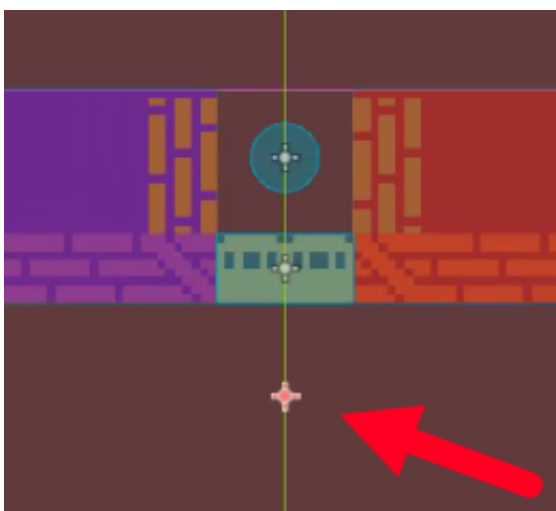
- Make the collision shape a circle. Resize and place it within the tunnel section.



- Add a Node2D as a child of Entrance\_North and name it PlayerSpawn.



- This node will define where the player spawns when entering the room from this direction. Place it somewhere that makes sense by the doors.

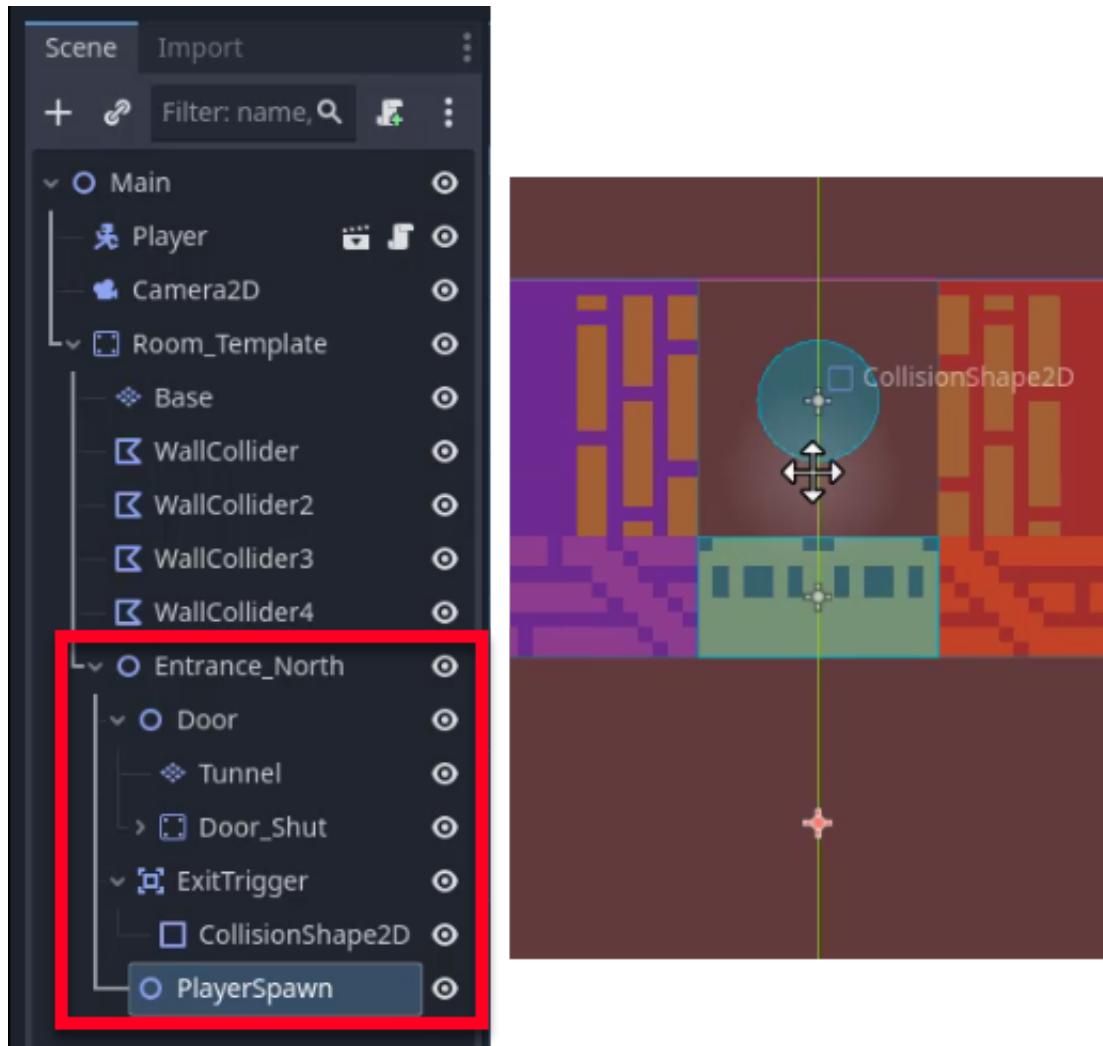


In the next lesson, we will finish our room template by completing the other entrances.

In this lesson, we will continue working on our room template, focusing on setting up the entrances for each direction: north, east, west, and south. By the end of this lesson, you will have a room template with functional entrances that can be used to generate procedural levels.

## Setting Up the North Entrance

We left off last lesson by setting up the north entrance.



A complete entrance consists of several components:

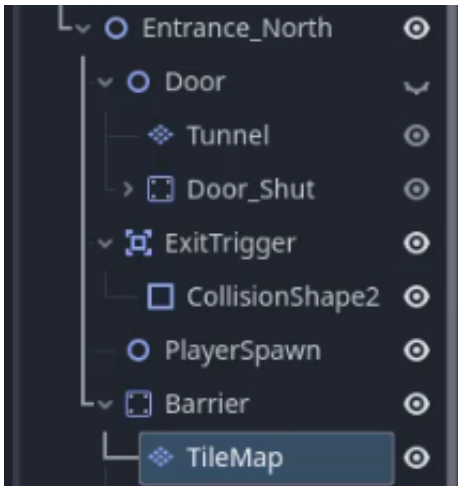
- **Door:** A node containing a tunnel and a door shut visual with a collider.
- **Barrier:** A static body that blocks the entrance when there is no room above.

If there is no room above the north entrance, the door will be disabled, and the barrier will be enabled. The barrier is created using a `StaticBody2D` node and a `TileMapLayer` for the visual representation.

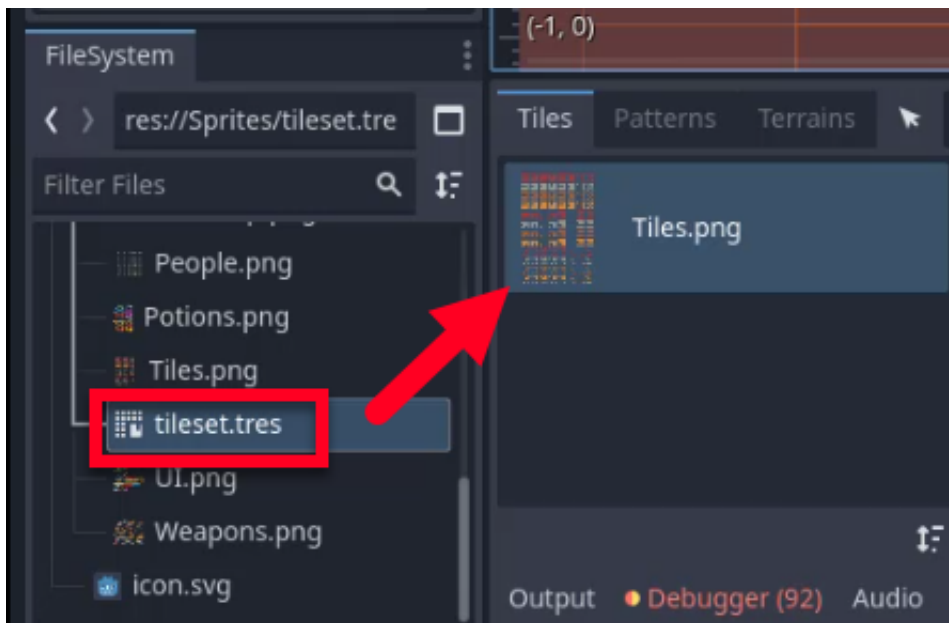
## Creating the Barrier

We have the door in place, so let's work on creating the barrier.

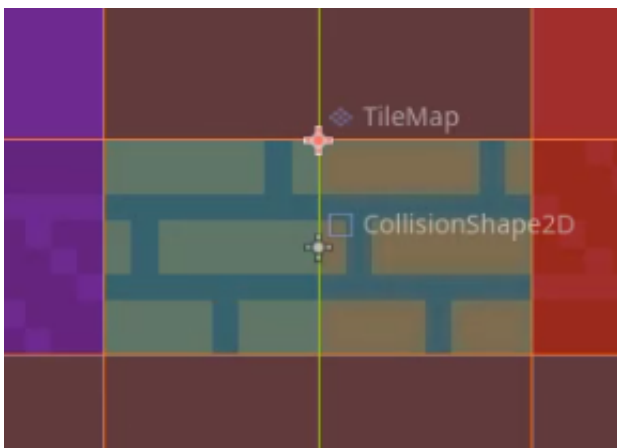
- Create a new `StaticBody2D` node and rename it to `Barrier`.
- Add a `TileMapLayer` as a child of the `Barrier` node.



- Drag your tile set into the TileMapLayer.



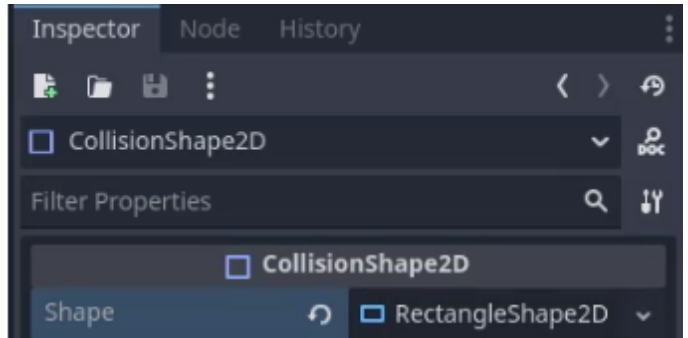
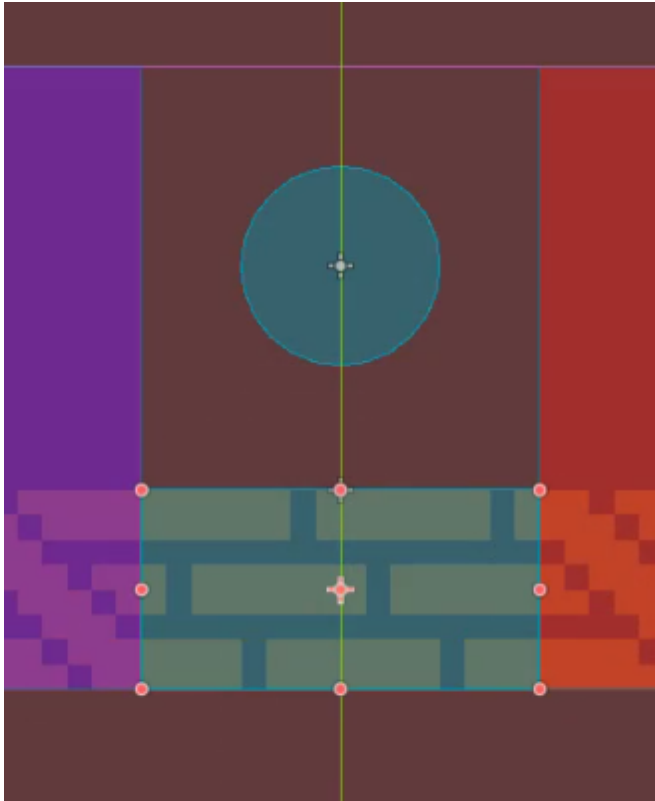
- Design the barrier using the tiles, typically two blocks to represent the barrier.



- Add a CollisionShape2D to the Barrier node.



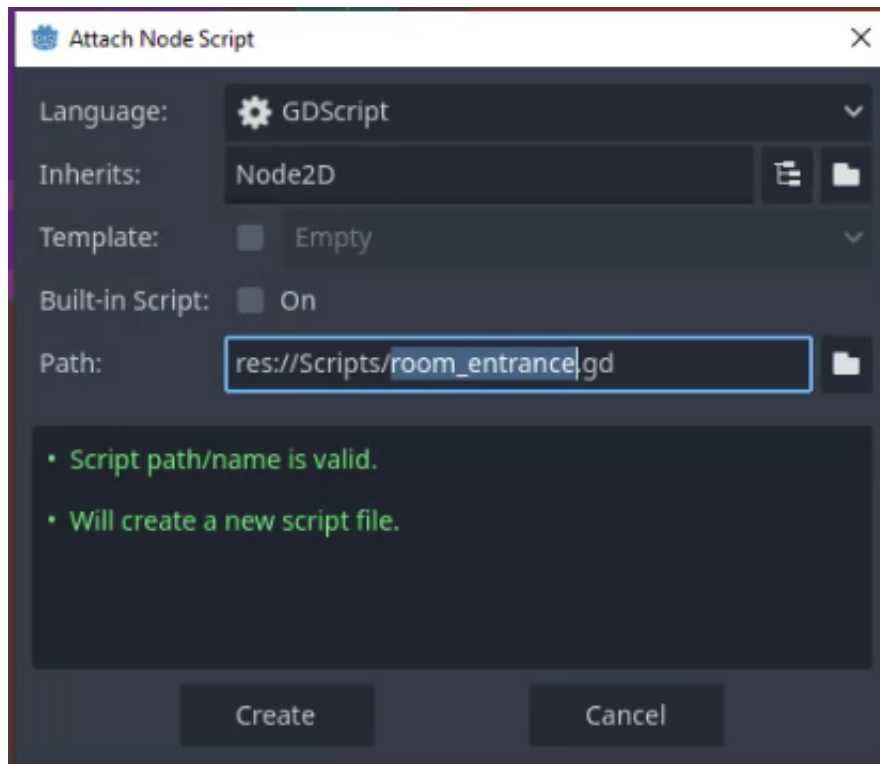
- Set the shape to a rectangle in the Inspector and fit it to the bounds of the barrier.



### Adding Scripts to Manage Entrances

Each entrance will have its own script to manage its individual components. Let's create a script for the north entrance:

1. Select the Entrance\_North node.
2. Add a new script to it and name it room\_entrance.gd.
3. Save the script in the Scripts folder.

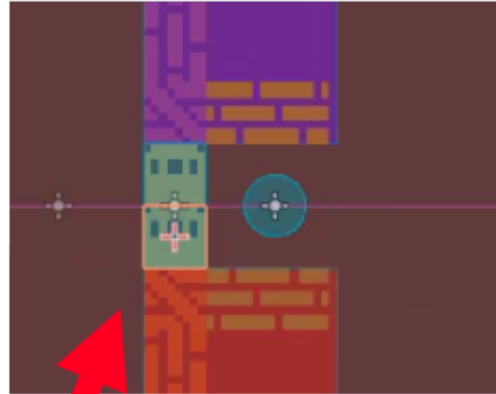
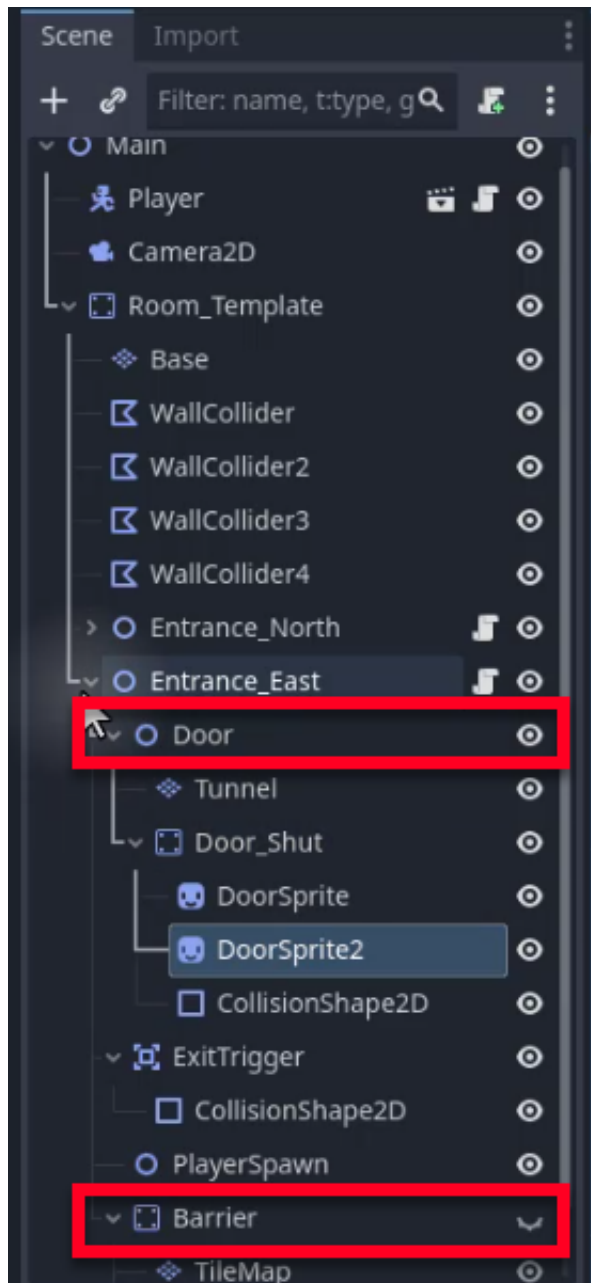


We will not edit this script just yet, but having it ready will be useful for future lessons.

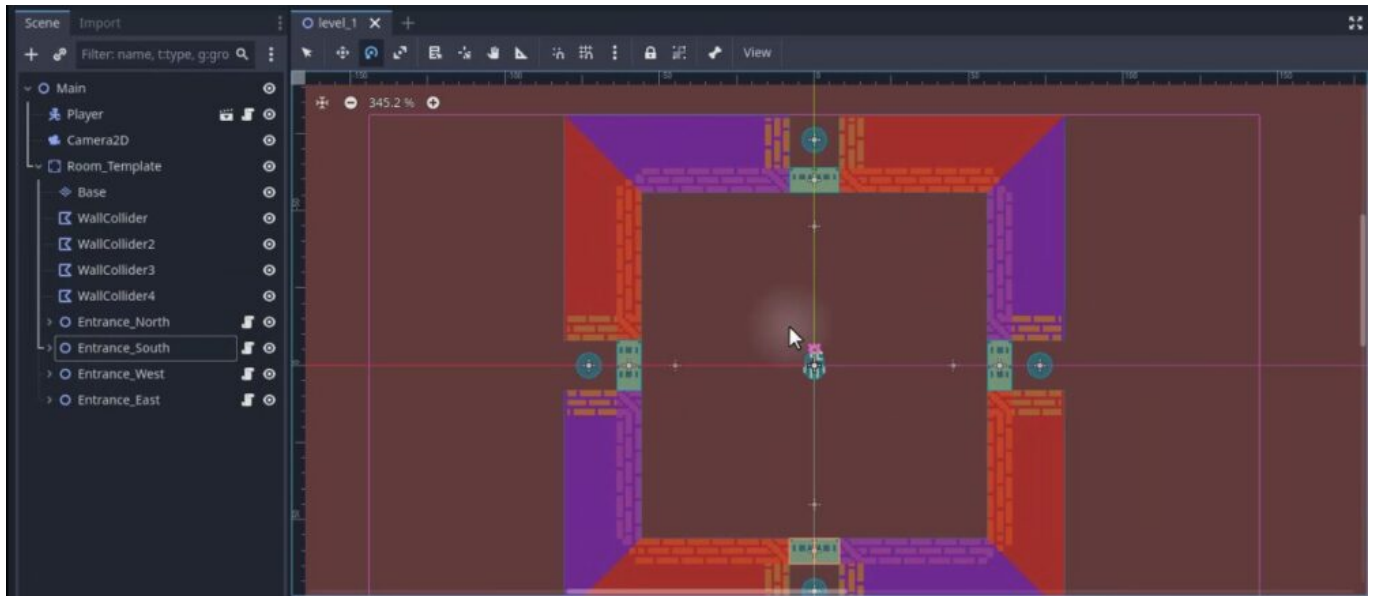
### Duplicating Entrances for Other Directions

To create the entrances for the east, west, and south directions, we will duplicate the north entrance and rotate it accordingly:

- Select the Entrance\_North node and duplicate it (Ctrl + D).
- Rename the duplicated node to Entrance\_East and rotate it 90 degrees clockwise.
- Enable the door and disable the barrier for the east entrance.
- Rotate the door sprites to face the correct direction.



- Repeat the process for the west and south entrances, rotating them 90 degrees counterclockwise and 180 degrees, respectively.

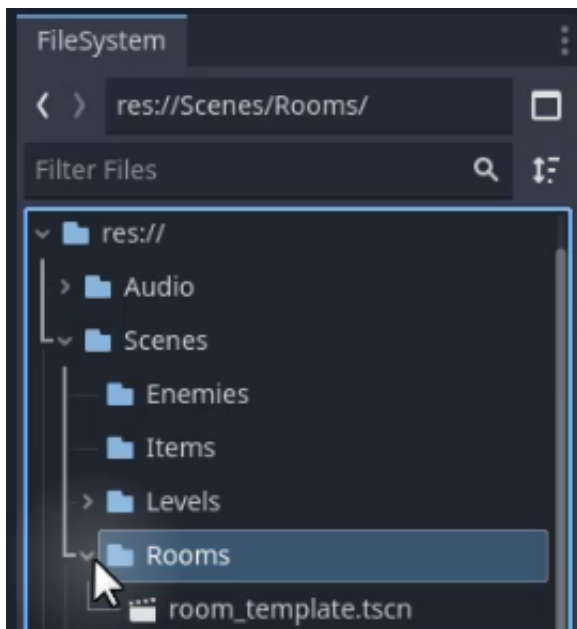


Ensure that the door sprites are correctly oriented for each entrance.

### Saving the Room Template

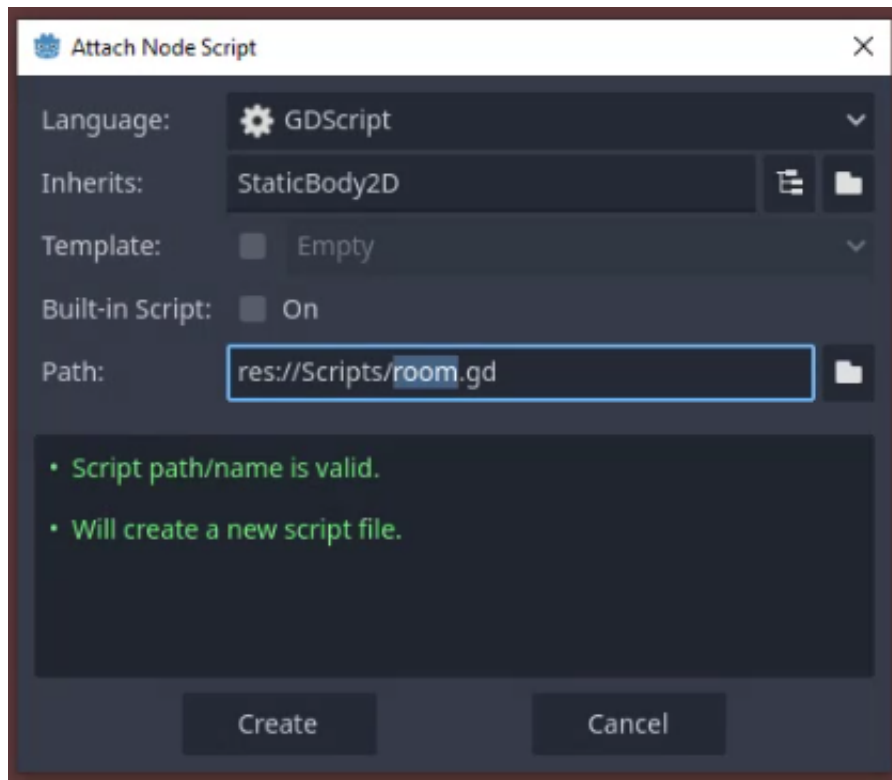
Once all entrances are set up, save the room template as a scene:

1. Go to the Scenes folder and then to the Rooms folder.
2. Drag the room template into the Rooms folder and save it.



Additionally, attach a new script to the root node of the room template and name it room.gd.



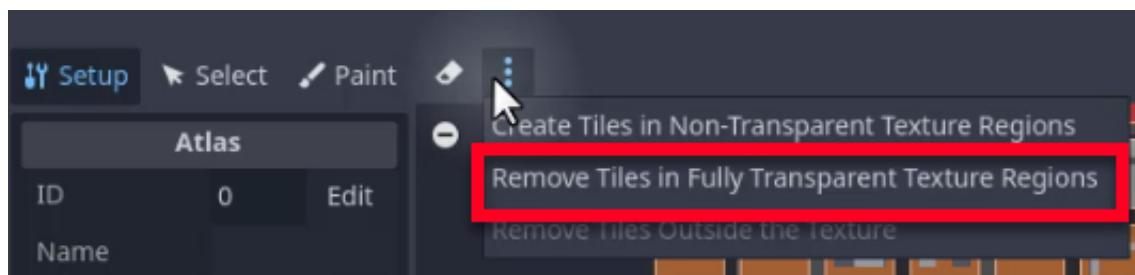


This script will be used to define the room's behavior during procedural generation. For now, all we want to do is add a class name to this script:

```
class_name Room
extends StaticBody2D
```

### Fixing Debugger Errors

If you encounter debugger errors related to the tile set, you can fix them by removing tiles in fully transparent regions. Open the tile set and click on the three arrows to access this option.

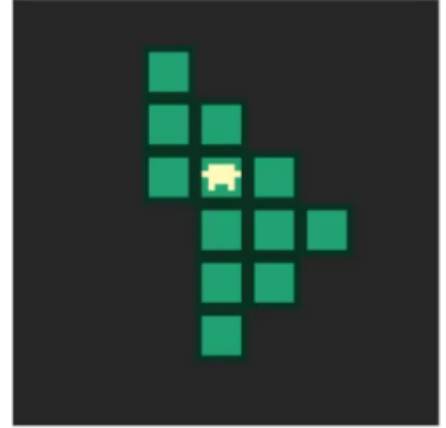
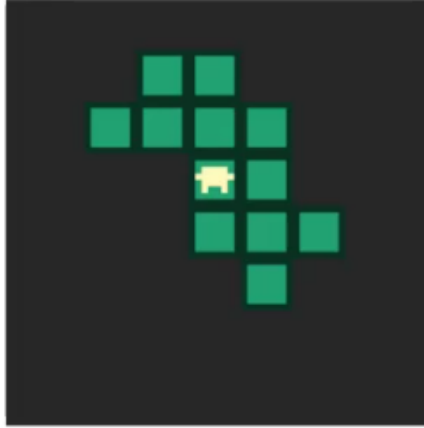
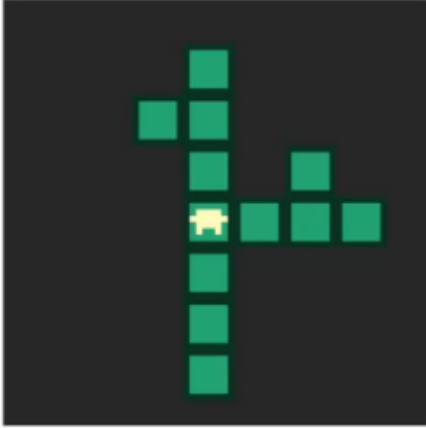


### Preparing for Procedural Generation

With the room template complete, we are now ready to begin working on the procedural generation of rooms. In the next lesson, we will delete the room template from the scene and start implementing the procedural generation logic.



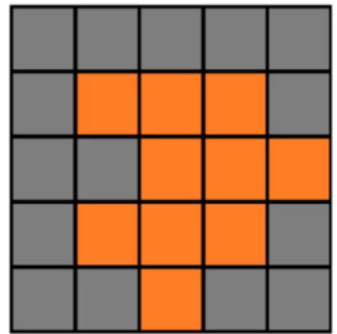
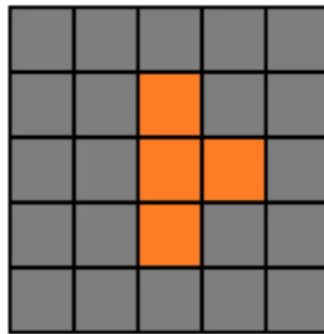
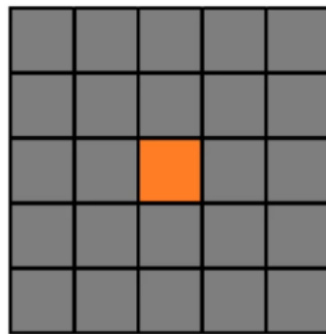
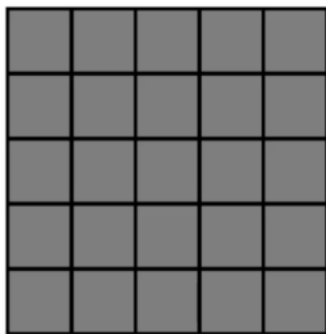
In this lesson, we will begin setting up room generation for our game. This system will be one of the most significant parts of our game, allowing us to procedurally generate levels. This involves placing rooms, connecting them, and ensuring that the game can be played multiple times with different layouts. Our objective is to generate a random layout of rooms, ensure they are all connected, and have the starting room in the middle.



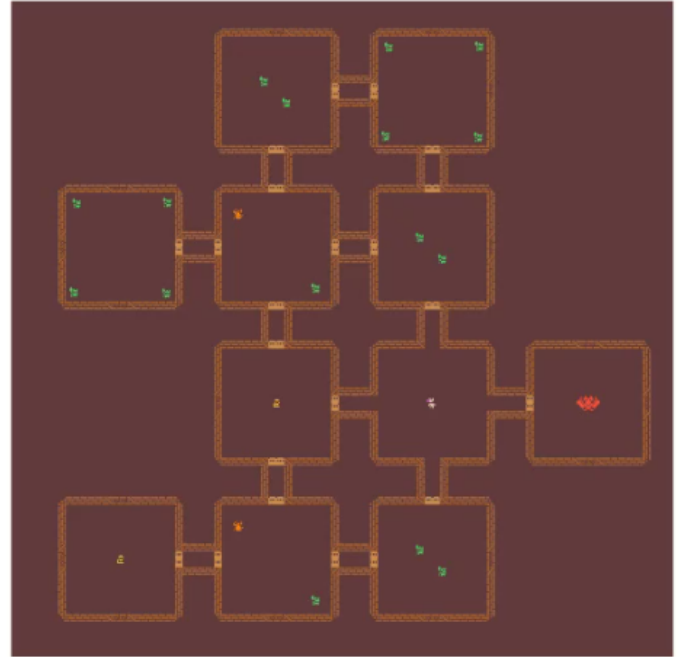
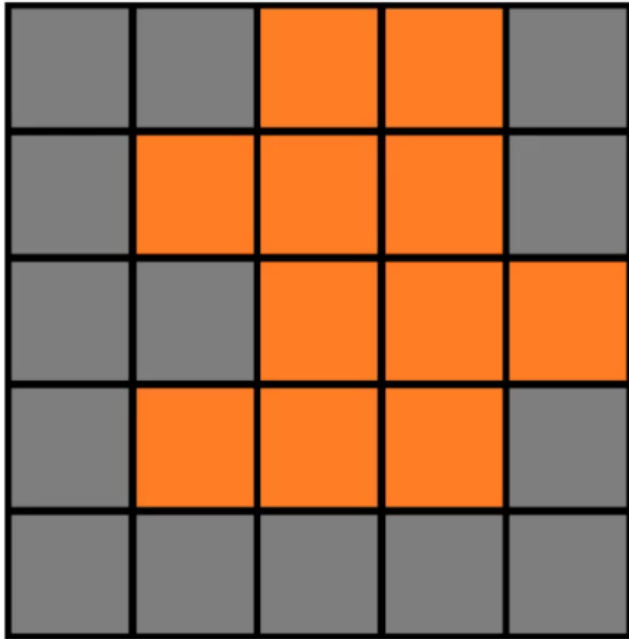
This process will be broken down into three steps:

### Steps for Room Generation

- **Block out room positions:** Determine where rooms will be placed.

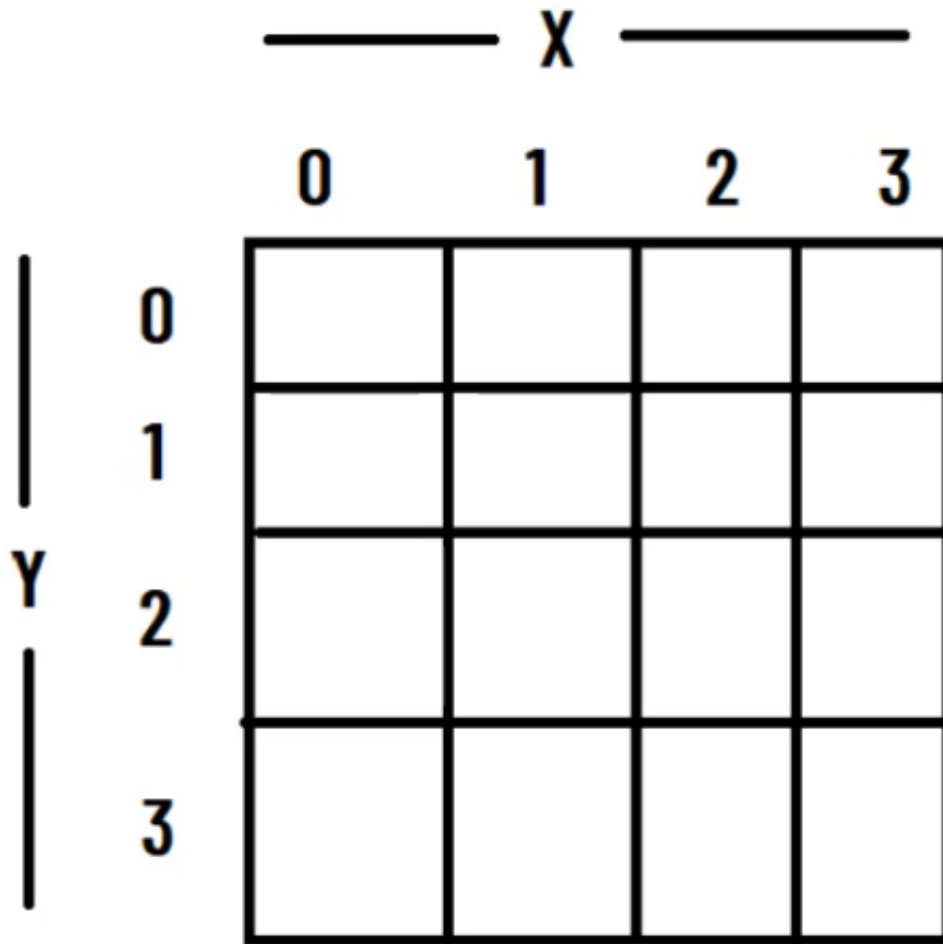


- **Instantiate room scenes:** Spawn the room scenes based on the blocked-out positions.
- **Initialize rooms:** Inform each room about its neighbors and add doors accordingly.



## Blocking Out Room Positions

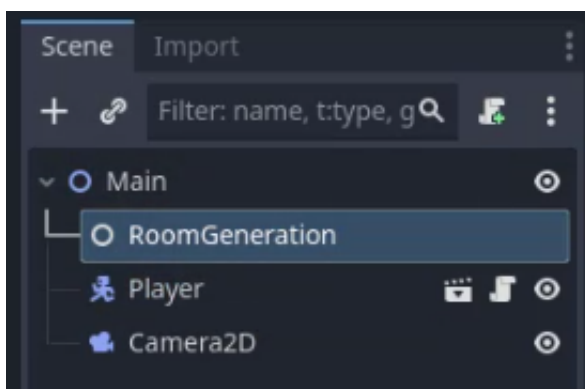
To block out room positions, we will use a 2D array. A 2D array is a data structure that holds multiple values, each accessed by two indices (x and y), similar to a grid. In Godot, we don't have built-in 2D arrays, but we can create our own. Our 2D array will use boolean values to indicate the presence (true) or absence (false) of a room.



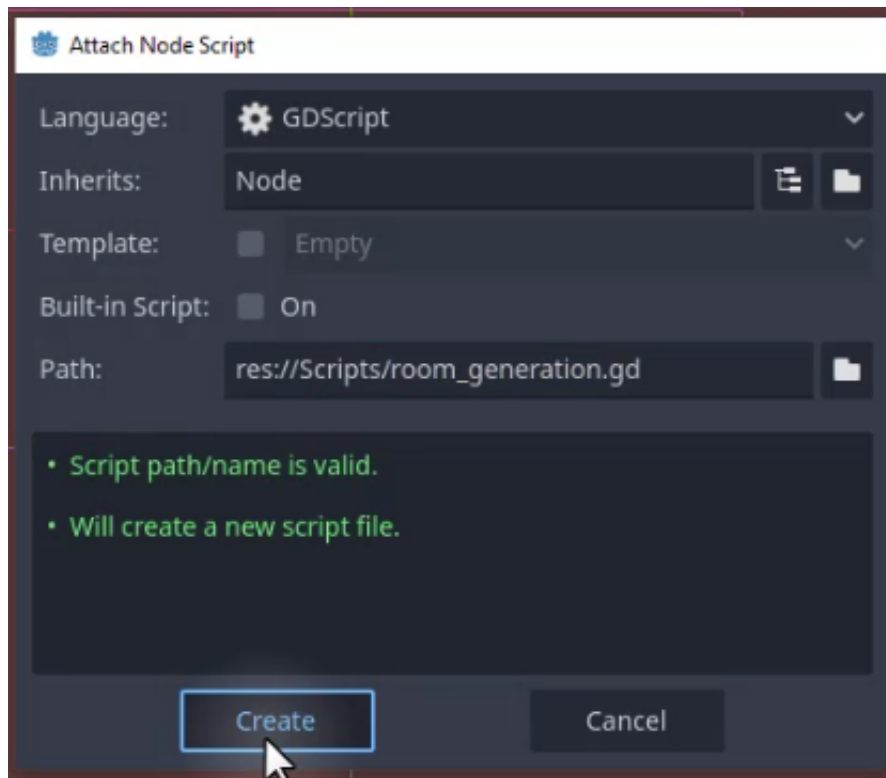
## Setting Up the Room Generation Node

Let's start by creating a new node in Godot:

- In the Main scene, create a new node and name it RoomGeneration. Also, remove the Room Template scene if you still have it in the tree. We will not be needing this now.



- Attach a script to this node and name it room\_generation.gd. Store this script inside the Scripts folder.



## Defining Variables

In the `room_generation.gd` script, we will define several variables:

```
class_name RoomGeneration
extends Node

@export var map_size : int = 7
@export var rooms_to_generate : int = 12
var room_count : int = 0
var map : Array[bool]
var rooms : Array[Room]
var room_pos_offset : float = 160

var first_room_x : int = 3
var first_room_y : int = 3
var first_room : Room

@export var player : CharacterBody2D
```

- `map_size`: Defines the width and height of our 2D map.
- `rooms_to_generate`: The number of rooms we aim to generate.
- `room_count`: Keeps track of the current number of rooms generated.
- `map`: A 2D array to define the presence of rooms.
- `rooms`: An array to keep track of instantiated rooms.
- `room_pos_offset`: The gap between rooms.
- `first_room_x` and `first_room_y`: The starting position of the first room.
- `first_room`: Keeps track of the first room once instantiated.
- `player`: An export variable for the player character.



## Creating Functions

Next, we will create the necessary functions, though we won't fill any out just yet:

```
func _ready():  
    pass
```

```
func _generate():  
    pass
```

```
func _check_room(x : int, y : int, desired_direction : Vector2, is_first_room : bool  
= false):  
    pass
```

```
func _instantiate_rooms():  
    pass
```

- `_ready`: Initializes the generation process.
- `_generate`: Sets up the map and starts the room generation.
- `_check_room`: Recursively checks and sets room positions.
- `_instantiate_rooms`: Instantiates the rooms.

## First Room Scene

Later on, our system will be able to select a random type of room from an Array. However, our first room will always be our empty room template. In order to use it, we need a reference to that scene. As such, we'll store a variable for it at the top of our script. This will also be useful for testing as we develop our base generation script.

```
var room_scene : PackedScene = preload("res://Scenes/Rooms/room_template.tscn")
```

## Generation

Now that we have all our variables and functions set up, let's start working on the actual generation part. First, in our **ready** function, we want to call our **generate** function, as this sets up the map and starts the generation.

```
func _ready ():  
    _generate()
```

Next, we'll work a bit on our **generate function**. We begin by making sure our room count is set to 0, as we haven't generated any rooms yet. We then define the size of the map array, which in this case is a 7 by 7 grid defined by our **map\_size** variable.

Next, we want to call our **\_check\_room** function, sending over the position of our first room coordinates so that our generation will start at the center of our grid. Finally, we can call **instantiate\_rooms**, as **\_check\_room** will handle the main portion of our generation and fill out the Array.

```
func _generate ():
```



```
room_count = 0
map.resize(map_size * map_size)

# start the check room chain at the center of the grid
_check_room(first_room_x, first_room_y, Vector2.ZERO, true)

# once we've defined our map, spawn in the room scenes
_instantiate_rooms()
```

In the next lesson, we will dive deeper into the `_check_room` function and understand how it recursively generates the room layout.



In this lesson, we will delve into the process of generating rooms for our game level. We will focus on the `check_room` function, which is crucial for determining the placement and expansion of rooms within our map grid. Let's break down the steps involved in this process.

## Overview

First, let's recall what we have done so far. Inside the `_ready` function, we call the `generate` function. The `generate` function initializes our room count to zero and sets up our map size to be a  $7 \times 7$  grid. It then calls the `check_room` function to start the room generation process.

The `check_room` function is responsible for determining whether a room can be placed at a specific coordinate and in which directions it can expand. Here's a step-by-step breakdown of what this function does:

## Setting Up the Room

When we call `check_room`, we pass in the `x` and `y` coordinates relative to the map array, a desired direction for expansion, and a boolean flag `is_first_room` which is set to true for the first room.

```
func _check_room(x: int, y: int, desired_direction: Vector2, is_first_room: bool = false):
```

## Conditions for Returning Early

The function first checks several conditions to determine if it should return early without creating a room:

1. **Room Count Exceeded:** If the room count is greater than or equal to the maximum number of rooms to generate, the function returns early.

```
if room_count >= rooms_to_generate:
    return
```

2. **Out of Bounds:** If the coordinates are outside the map bounds, the function returns early.

```
if x < 0 or x > map_size - 1 or y < 0 or y > map_size - 1:
    return
```

3. **Room Already Exists:** If a room already exists at the given coordinates, the function returns early. In order to simplify this, we will make a helper function. This helper function takes an `x` and `y` value, and returns true or false based on whether the coordinate exists within the map array.

```
func _get_map(x : int, y : int) -> bool:
    return map[x + y * map_size]
```

Back in the `_check_room` function, we can then call this function and return early to perform this check.





```
if _get_map(x, y):  
    return
```

## Creating the Room

If all the above conditions are passed, the function proceeds to create the room:

First, it will increment the room count.

```
room_count += 1
```

Next, it will set the room in the map array using the `_set_map` function. This is another helper function we'll create. This function will take an `x` and `y` value, as well as a boolean value as to whether or not that room exists. It then sets the map array with that value.

```
func _set_map (x : int, y : int, value : bool):  
    map[x + y * map_size] = value
```

Back in the **`_check_room` function**, we can then call this helper function to set up the map array.

```
_set_map(x, y, true)
```

## Deciding Expansion Directions

The function then decides in which directions the room can expand. It uses a random number and a ternary operator to determine the probability of expanding in each direction based on the desired direction. We'll set up the North direction first.

```
var go_north: bool = randf() > (0.2 if desired_direction == Vector2.UP else 0.8)
```

## Recursive Room Generation

Finally, the function calls itself recursively to continue generating rooms in the decided directions. In this case, we need to check if we can go north or if this is the first room.

```
if go_north or is_first_room:  
    _check_room(x, y - 1, Vector2.UP if is_first_room else desired_direction)
```

## Challenge

As a challenge, try to implement the variables and condition checks for expanding south, east, and west, similar to how we did for the north direction. This will help you understand the recursive nature of the room generation process better.

In the next lesson, we will finalize our **`_check_room`** function with the solution above and expand



our generation further.



In this lesson, we will continue building our room generation system. We will add more directions for room expansion and implement the instantiation of room scenes. By the end of this lesson, you will be able to generate a random room layout and visualize it in the game.

## Adding More Directions for Room Expansion

Previously, we only expanded rooms to the north. Now, we will add more directions: south, east, and west. This will make our room generation more dynamic and interesting.

First, let's define boolean variables for each direction to determine if a room should be generated in that direction. We'll use the `randf()` function to generate a random number between 0 and 1, and compare it to a threshold value to decide the direction.

Here's how you can add the variables for the south, east, and west directions:

```
var go_south = randf() > (0.2 if desired_direction == Vector2.DOWN else 0.8)
var go_east = randf() > (0.2 if desired_direction == Vector2.RIGHT else 0.8)
var go_west = randf() > (0.2 if desired_direction == Vector2.LEFT else 0.8)
```

## Expanding Rooms in All Directions

Next, we'll modify the `_check_room` function to include the new directions. We'll copy and paste the existing `if` statement for the north direction and modify it for the south, east, and west directions.

Here's the updated `_check_room` function:

```
func _check_room(x, y, desired_direction, is_first_room = false):
    # ... (existing code)

    if go_south or is_first_room:
        _check_room(x, y + 1, Vector2.DOWN if is_first_room else desired_direction)
    if go_east or is_first_room:
        _check_room(x + 1, y, Vector2.RIGHT if is_first_room else desired_direction)
    if go_west or is_first_room:
        _check_room(x - 1, y, Vector2.LEFT if is_first_room else desired_direction)
```

## Temporary Room Generation

Before we instantiate the rooms, you may wish to verify that our generation works so far. We can do this easily by creating a small `for` loop in our **ready** function to check our map array after we generate it. Using this loop, we can print a `#` to the console anytime we find a room.

```
# room layout preview
for x in range(map_size):
    var line : String
    for y in range(map_size):
        line += "# " if _get_map(x, y) else " "
    print(line)
```

If you run this several times, you should see room layouts printed to the console. You can run it



several times to verify the randomization elements too.



We don't need this in the game, so be sure to comment it out when you're through. However, this serves as an easy way to check the logic of our generation.

### Instantiating Room Scenes

Now that we have our room map generated, let's instantiate the room scenes to visualize them in the game. We'll modify the `_instantiate_rooms` function to loop through the map and instantiate room scenes at the corresponding positions.

To do so, we need to use a nested for loop to check for rooms at both the x and y values. If we can't find a room, we'll continue to the next loop. If one is found, we will add our first room scene to the scene tree, position it (considering the offset), and append it to our Array of rooms.

Here's how you can instantiate the room scenes:

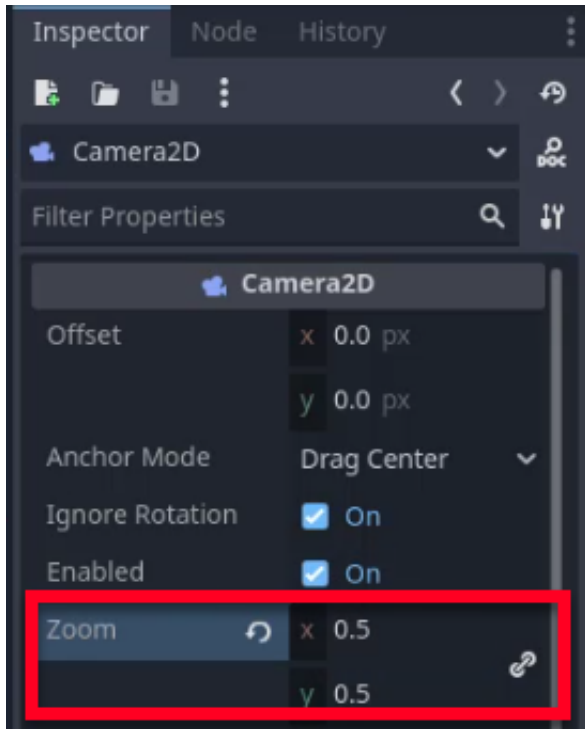
```
func _instantiate_rooms():
    for x in range(map_size):
        for y in range(map_size):
            if _get_map(x, y) == false:
                continue

            var room : Room = room_scene.instantiate()
            get_tree().root.add_child.call_deferred(room)
            rooms.append(room)

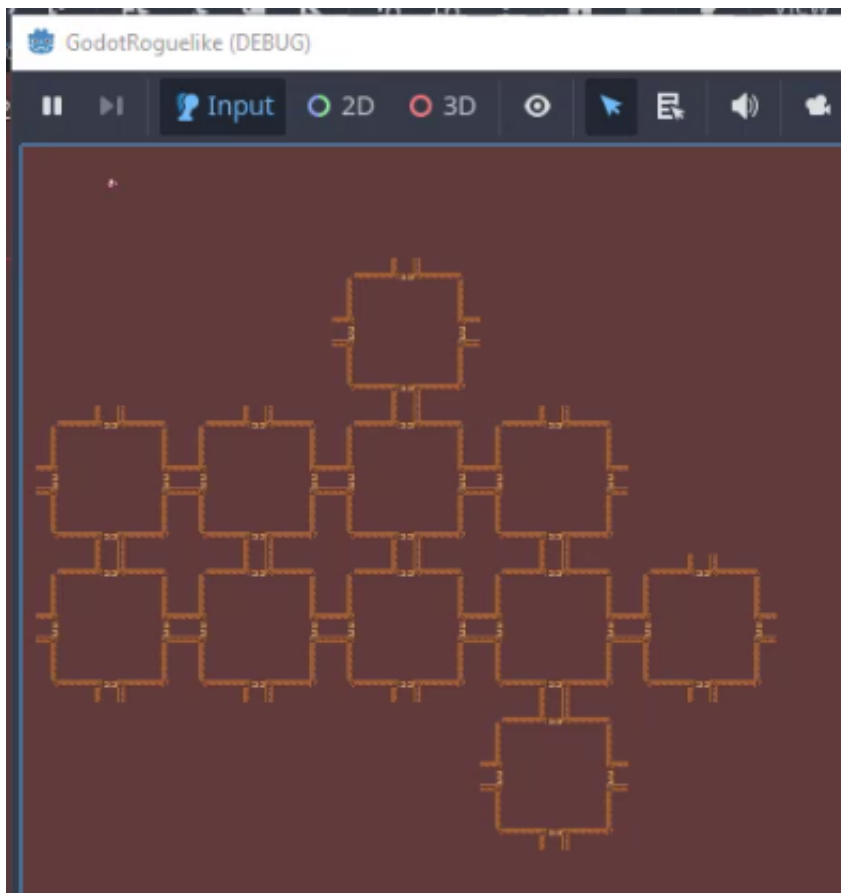
            room.global_position = Vector2(x, y) * room_pos_offset
```

### Adjusting the Camera

After instantiating the rooms, you might notice that the rooms are generated off-screen. To fix this, we'll adjust the camera's zoom level to 0.5 temporarily.



Now, if you run the game, you should see your map generate with the rooms in place (though the entrances are not properly configured yet).



Congratulations! You have successfully implemented room generation in all directions and instantiated room scenes. In the next lesson, we will focus on connecting rooms with doors and



managing room neighbors.



In this lesson, we will focus on setting up the scripts related to our rooms and room entrances. We will start with the RoomEntrance script, which is simpler and will help us understand the basic functionalities before moving on to the more complex Room script.

## Room Entrance Script

The RoomEntrance script will handle various functionalities such as opening and closing doors, toggling barriers, detecting when the player enters the room, and teleporting the player to the next room. Let's break down the steps to set up this script:

### Variables

We will start by defining the variables needed for our RoomEntrance script. The only exported variable will be the direction, which will use an enumerator.

First, we need to create an enumerator for the directions. This will be defined in the Room script to centralize the direction data. Here is how you can define the enumerator in the Room script:

```
enum Direction
{
    NORTH,
    SOUTH,
    EAST,
    WEST
}
```

Now, in the RoomEntrance script, we can define the direction variable using this enumerator:

```
@export var direction : Room.Direction = Room.Direction.NORTH
```

Next, we will define several onready variables to reference specific nodes:

- barrier of type StaticBody2D
- barrier\_collider of type CollisionShape2D
- door of type Node2D
- door\_shut of type StaticBody2D
- door\_shut\_collider of type CollisionShape2D
- player\_spawn of type Node2D
- exit\_trigger of type Area2D
- neighbor of type Room

Here is how you can define these variables in the RoomEntrance script:

```
@onready var barrier : StaticBody2D = $Barrier
@onready var barrier_collider : CollisionShape2D = $Barrier/CollisionShape2D
@onready var door : Node2D = $Door
@onready var door_shut : StaticBody2D = $Door/Door_Shut
@onready var door_shut_collider : CollisionShape2D = $Door/Door_Shut/CollisionShape2D
@onready var player_spawn : Node2D = $PlayerSpawn
```



```
@onready var exit_trigger : Area2D = $ExitTrigger  
var neighbor : Room
```

## Functions

Now, let's define the functions needed for our RoomEntrance script. We will start by outlining the functions and then fill them in later.

1. `_ready`
2. `set_neighbor`
3. `toggle_barrier`
4. `open_door`
5. `close_door`
6. `_on_body_entered_exit_trigger`

```
func _ready ():  
    pass
```

```
func set_neighbor (neighbor_room : Room):  
    pass
```

```
func toggle_barrier (toggle : bool):  
    pass
```

```
func open_door ():  
    pass
```

```
func close_door ():  
    pass
```

```
func _on_body_entered_exit_trigger (body):  
    pass
```

## Ready Function

In the `_ready` function, we will connect the `body_entered` signal of the `exit_trigger` to the `_on_body_entered_exit_trigger` function.

```
func _ready():  
    exit_trigger.body_entered.connect(_on_body_entered_exit_trigger)
```

## Get Neighbor Entry Direction Function

Below the functions we already made, let's make a new function call **`_get_neighbor_entry_direction`**. The `_get_neighbor_entry_direction` function will return the entry direction for the neighboring room based on the current direction of this entrance.

```
func _get_neighbor_entry_direction() -> Room.Direction:
```





```
if direction == Room.Direction.NORTH:
    return Room.Direction.SOUTH
elif direction == Room.Direction.SOUTH:
    return Room.Direction.NORTH
elif direction == Room.Direction.EAST:
    return Room.Direction.WEST
else:
    return Room.Direction.EAST
```

## Open Door Function

The `open_door` function will open the door by setting the visibility of `door_shut` to false and disabling its collider. We also need to return early if there is a barrier, as remember, barriers represent directions our player cannot go.

```
func open_door():
    if barrier.visible:
        return
    door_shut.visible = false
    door_shut_collider.disabled = true
```

## Close Door Function

The `close_door` function will close the door by setting the visibility of `door_shut` to true and enabling its collider. Like the open door function, we want to return early if it's a direction our player can't go (by checking if the barrier is visible).

```
func close_door():
    if barrier.visible:
        return
    door_shut.visible = true
    door_shut_collider.disabled = false
```

## Toggle Barrier Function

The `toggle_barrier` function will toggle the visibility and collision of the barrier based on the input boolean value.

```
func toggle_barrier(toggle : bool):
    barrier.visible = toggle
    barrier_collider.disabled = !toggle
    door.visible = !toggle
```

## Set Neighbor Function



The `set_neighbor` function will be called when we want to define a neighboring room for this entrance. It will set the `neighbor` variable and toggle the barrier to false.

```
func set_neighbor(neighbor_room : Room):  
    neighbor = neighbor_room  
    toggle_barrier(false)
```

Don't worry about our **`_on_body_entered_exit_trigger`** function at the moment. We will fill this out later. In the next lesson, we will focus on setting up the Room script, where we will manage room entrances and connect them to our room generation script.



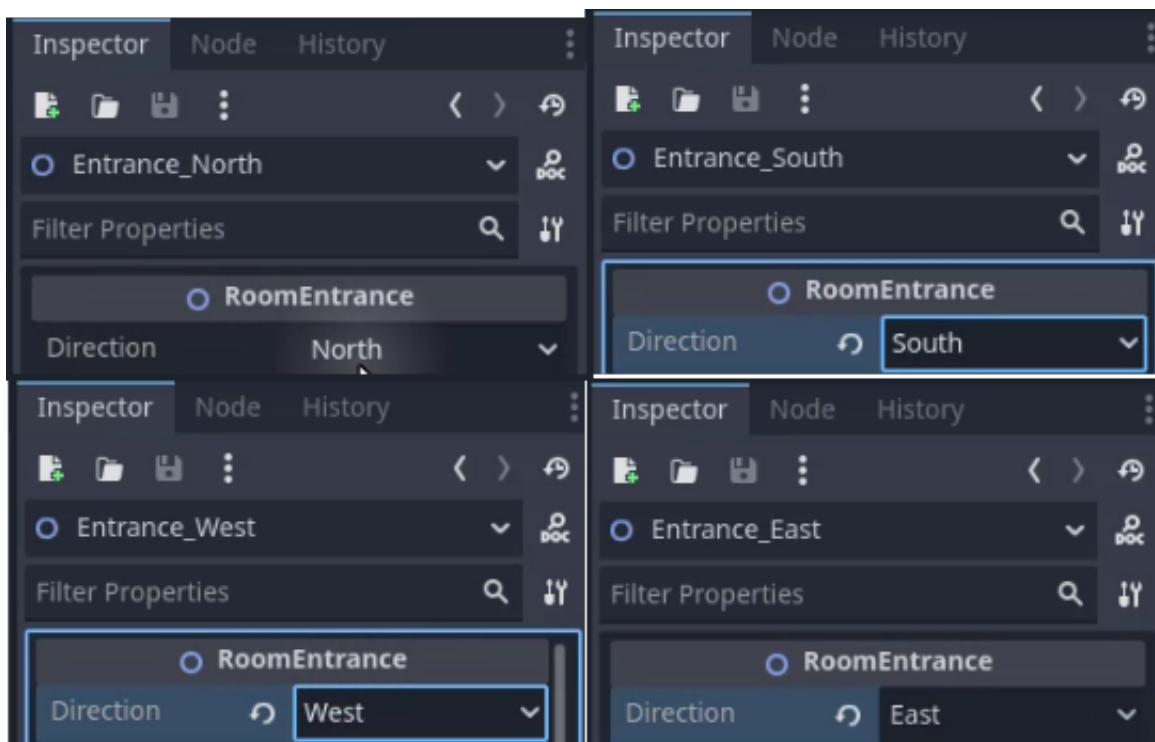
In this lesson, we will focus on setting up our room script. This script will manage the behavior of rooms in our game, including handling entrances, tracking enemies, and controlling door states. Let's dive right in.

## Setting Up Room Entrances

Before we start coding, let's ensure our room template scene is correctly set up. In our room template scene, we need to assign the direction property for each entrance:

- Set south to Direction.SOUTH.
- Set west to Direction.WEST.
- Set east to Direction.EAST.

Since North is the default value, this entrance should already be set.



## Creating the Room Script

Now, let's create the room script. This script will manage the room's behavior, including handling entrances, tracking enemies, and controlling door states.

### Variables

We will start by defining the variables needed for our room script:

```
class_name Room
extends StaticBody2D
```

```
enum Direction
{
    NORTH,
    SOUTH,
    EAST,
```



```
WEST
}
```

```
@export var doors_always_open : bool = false
```

```
@onready var entrance_north : RoomEntrance = $Entrance_North
```

```
@onready var entrance_south : RoomEntrance = $Entrance_South
```

```
@onready var entrance_east : RoomEntrance = $Entrance_East
```

```
@onready var entrance_west : RoomEntrance = $Entrance_West
```

```
var enemies_in_room : int
```

- doors\_always\_open: A boolean variable that determines if the doors should always be open.
- entrance\_north, entrance\_south, entrance\_east, entrance\_west: Variables that reference the room's entrances.
- enemies\_in\_room: An integer that keeps track of the number of enemies in the room.

## Functions

Next, we will define the functions that will handle the room's behavior. We won't fill them out yet, just define them. We'll dive into each function as we create it.

```
func _ready ():
    pass
```

```
func initialize ():
    pass
```

```
func set_neighbor (neighbor_direction : Direction, neighbor_room : Room):
    pass
```

```
# called when the player enters the room
```

```
func player_enter (entry_direction : Direction, player : CharacterBody2D, first_room
: bool = false):
    pass
```

```
func _on_defeat_enemy (enemy):
    pass
```

```
func open_doors ():
    pass
```

```
func close_doors ():
    pass
```

**Note:** While we define an initialize function, we will not actually use it in this course. However, adding it does not affect the project.

## open\_doors and close\_doors Functions

Let's start with our easiest functions. The open\_doors and close\_doors functions control the state of the doors in the room. As such, all they need to do is open or close the door at each entrance (the functions we set up in the previous lesson).



```
func open_doors():
    entrance_north.open_door()
    entrance_south.open_door()
    entrance_east.open_door()
    entrance_west.open_door()

func close_doors():
    entrance_north.close_door()
    entrance_south.close_door()
    entrance_east.close_door()
    entrance_west.close_door()
```

## player\_enter Function

Next let's work on our `player_enter` function that is called when the player enters the room. It handles spawning the player at the correct entrance and managing the door states.

- The function first checks the entry direction and places the player at the correct entrance spawn position.
- We also check if the room is the first room. If it is, we move the player to the center of the room instead.
- Last, we check if there are enemies in the room and the doors aren't always set to open. If that's the case, we close all the doors. Otherwise, we open all the doors. This will prevent errors in case we accidentally leave any doors closed/open by mistake in our setup.

```
func player_enter(entry_direction : Direction, player : CharacterBody2D, first_room :
bool = false):
    if entry_direction == Direction.NORTH:
        player.global_position = entrance_north.player_spawn.global_position
    elif entry_direction == Direction.SOUTH:
        player.global_position = entrance_south.player_spawn.global_position
    elif entry_direction == Direction.EAST:
        player.global_position = entrance_east.player_spawn.global_position
    elif entry_direction == Direction.WEST:
        player.global_position = entrance_west.player_spawn.global_position

    if first_room:
        player.global_position = global_position

    if enemies_in_room > 0 and not doors_always_open:
        close_doors()
    else:
        open_doors()
```

## set\_neighbor Function

The `set_neighbor` function is called to set the neighboring rooms for each entrance. We do this by sending over the direction and room. We can then call the function from our Room Entrance script to set the neighbor.



```
func set_neighbor(neighbor_direction : Direction, neighbor_room : Room):
    if neighbor_direction == Direction.NORTH:
        entrance_north.set_neighbor(neighbor_room)
    elif neighbor_direction == Direction.SOUTH:
        entrance_south.set_neighbor(neighbor_room)
    elif neighbor_direction == Direction.EAST:
        entrance_east.set_neighbor(neighbor_room)
    elif neighbor_direction == Direction.WEST:
        entrance_west.set_neighbor(neighbor_room)
```

## Updating the Room Generation Script

Finally, let's update the room generation script to handle the instantiation of rooms and setting their neighbors.

In the `instantiate_rooms` function, we will check if the room is the first room and initialize it accordingly:

```
func _instantiate_rooms():
    for x in range(map_size):
        for y in range(map_size):
            if _get_map(x, y) == false:
                continue

            var room : Room
            var is_first_room : bool = first_room_x == x and first_room_y == y

            get_tree().root.add_child.call_deferred(room)
            rooms.append(room)

            room.global_position = Vector2(x, y) * room_pos_offset

            if is_first_room:
                first_room = room

            room.initialize()
```

## Setting Room Neighbors

To set the room neighbors, we will loop through each room and check for neighboring rooms in each direction.

```
for room in rooms:
```

We will handle this implementation next lesson.



In this lesson, we will continue building upon our room generation system by implementing a way to check for neighboring rooms and inform them of their neighbors. This will help us create a more interconnected and dynamic game environment.

## Setting Up the Map Position

To begin, we need to determine the map position for each room. This involves creating a function that will return the position of a room as a `Vector2i`. The reason we use `Vector2i` instead of `Vector2` is to avoid any floating-point errors that might occur during division.

First, let's define the function `get_map_index` that takes a room as an argument and returns its position on the map:

```
func _get_map_index(room: Room) -> Vector2i:
    return Vector2i(room.global_position / room_pos_offset)
```

Next, back in our **`_instantiate_rooms` function**, we will set up a loop and use this function to get the map position for each room. We then separate the x and y properties from the map position:

```
func _instantiate_rooms ():
    # rest of code

    # set room neighbors
    for room in rooms:
        var map_pos : Vector2 = _get_map_index(room)
        var x = map_pos.x
        var y = map_pos.y
```

## Creating the `get_room_from_map` Function

To retrieve a room from the map based on its x and y coordinates, we need to create the `get_room_from_map` function. This function will loop through all the rooms and return the room that matches the given coordinates:

```
func get_room_from_map(x: int, y: int) -> Room:
    for room in rooms:
        var pos = _get_map_index(room)
        if pos.x != x and pos.y != y:
            continue

    return room
return null
```

## Checking for Neighbors

Let's return back to our for loop from earlier. With the map position elements handled, we can now check if each room has neighbors to the north, south, east, or west. We will use conditional statements to check the boundaries of the map and determine if a neighboring room exists.

When setting the neighbors, we need to defer the `set_neighbor` call to ensure that all rooms are



properly initialized before establishing the connections. This can be done using the `call_deferred` method.

For the northern neighbor, we check if `y` is greater than 0:

```
if y > 0 and _get_map(x, y - 1):
    room.set_neighbor.call_deferred(Room.Direction.NORTH, get_room_from_map(x, y - 1))
)
```

For the southern neighbor, we check if `y` is less than `map_size - 1`:

```
if y < map_size - 1 and _get_map(x, y + 1):
    room.set_neighbor.call_deferred(Room.Direction.SOUTH, get_room_from_map(x, y + 1))
)
```

For the eastern neighbor, we check if `x` is less than `map_size - 1`:

```
if x < map_size - 1 and _get_map(x + 1, y):
    room.set_neighbor.call_deferred(Room.Direction.EAST, get_room_from_map(x + 1, y))
```

For the western neighbor, we check if `x` is greater than 0:

```
if x > 0 and _get_map(x - 1, y):
    room.set_neighbor.call_deferred(Room.Direction.WEST, get_room_from_map(x - 1, y))
```

## Handling Barriers and Doors

To ensure that rooms without neighbors have barriers instead of doors, we need to modify the `RoomEntrance` script. By default, all barriers should be enabled, and only when a neighbor is set should they be disabled. We can do this in the `_ready` function.

```
func _ready():
    exit_trigger.body_entered.connect(_on_body_entered_exit_trigger)
    toggle_barrier(true)
```

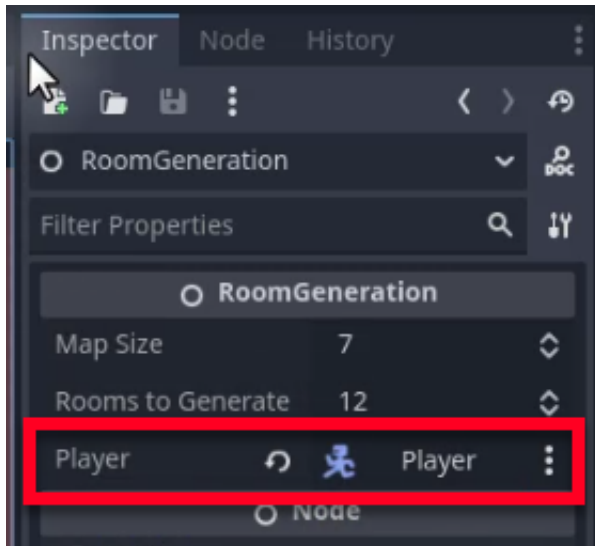
## Spawning the Player

Finally, we need to spawn the player in the first room. We do this by calling the `player_enter` function on the first room at the end of the `instantiate_rooms` function and passing the player and the entry direction:

```
first_room.player_enter.call_deferred(Room.Direction.NORTH, player, true)
```

Remember to assign the player to the `player` field in the `RoomGeneration` node in the Godot editor.

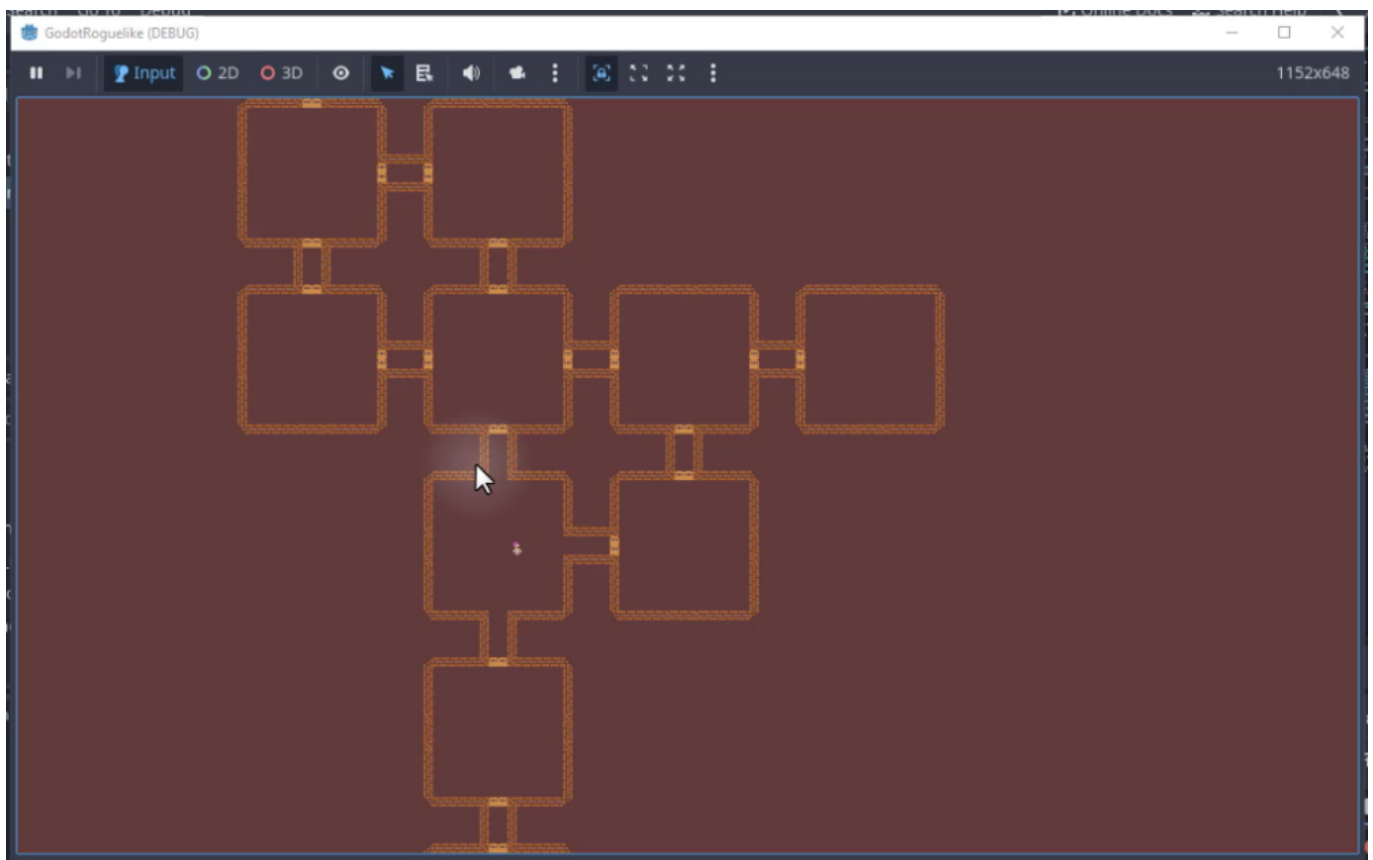




## Conclusion

In this lesson, we have implemented a system to check for neighboring rooms and inform them of their neighbors. We have also ensured that rooms without neighbors have barriers instead of doors and spawned the player in the first room.

If you test the game now, you should see the map appear as described and the player spawned in the first room.



In the next lesson, we will look at setting up the ability for the player to begin attacking.

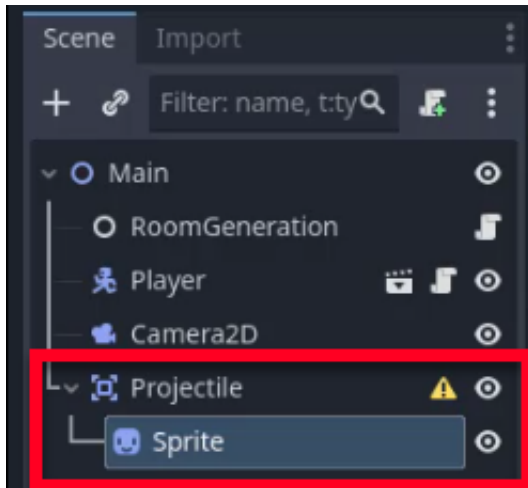


In this lesson, we will set up the player's shooting ability. This involves aiming with the mouse and left-clicking to spawn a projectile that moves in the direction of the mouse cursor. Let's dive into the steps to achieve this.

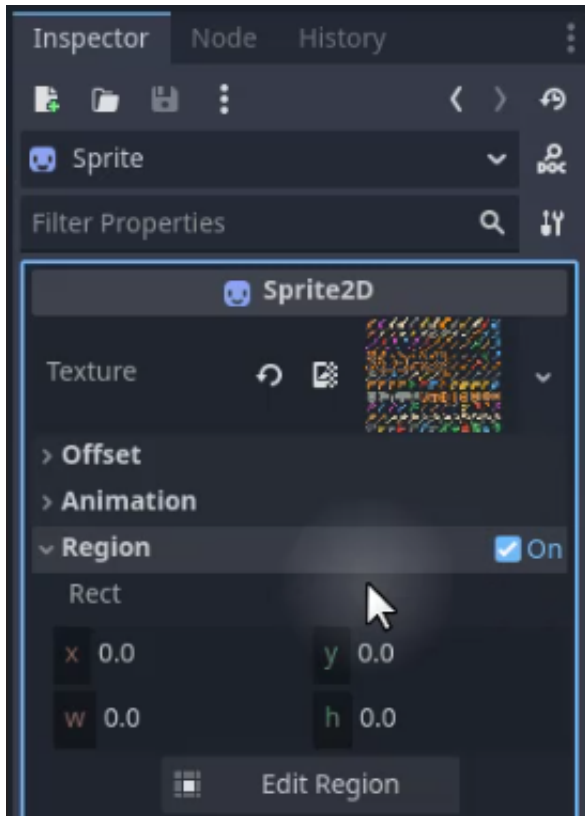
## Creating the Projectile Scene

To begin, we need to create the projectile itself. Follow these steps:

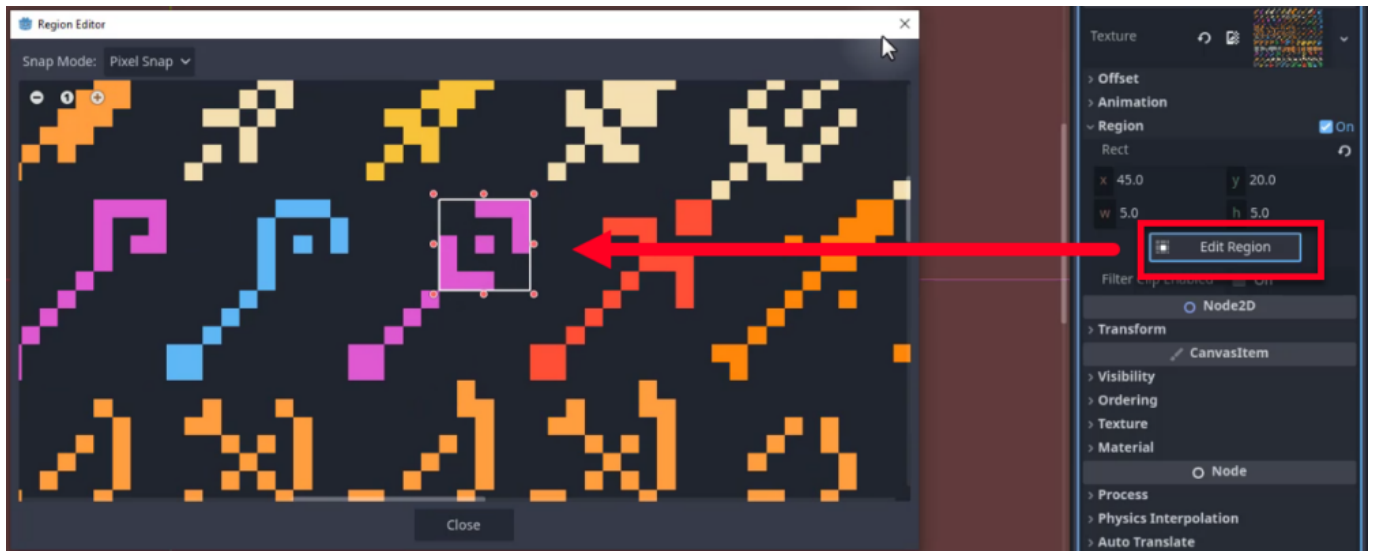
- Create a new scene of type Area2D.
- Rename the Area2D node to Projectile.
- Add a Sprite2D node as a child of Projectile. Rename it to "Sprite" for simplicity.



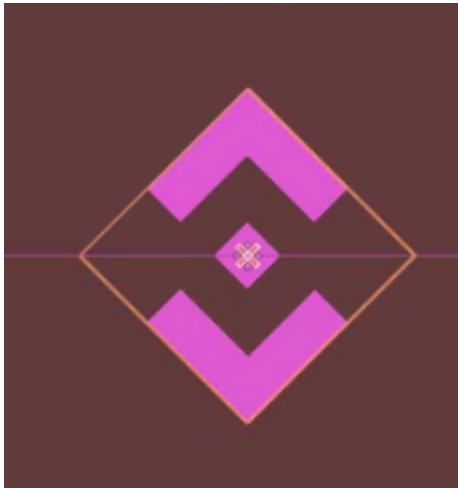
- Set the texture for the Sprite2D node:
  - Navigate to your sprites folder, find Weapons.png, and drag the desired texture into the Texture property.



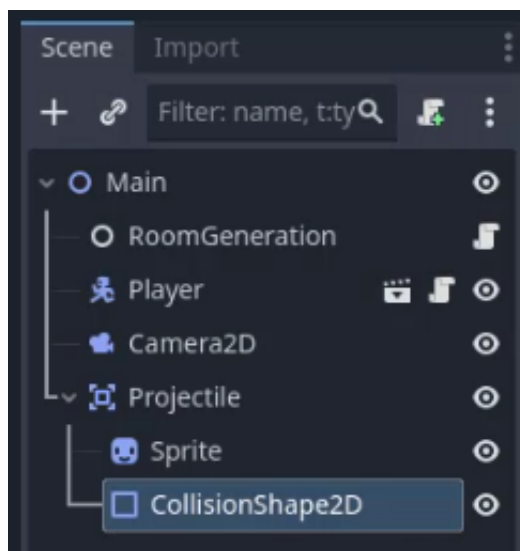
Enable the Region option and adjust the region to select a part of the texture that resembles a projectile.



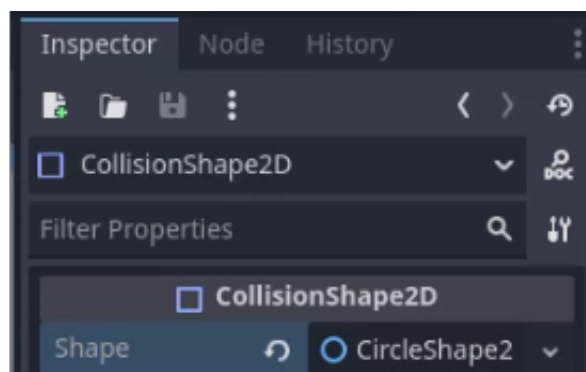
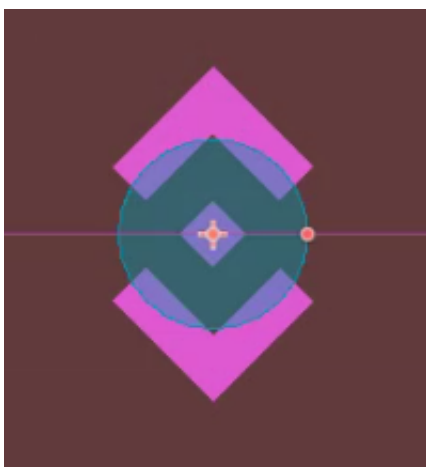
Orient the sprite so that it points upwards.



- Add a CollisionShape2D node as a child of Sprite2D.



- Set it to a simple circle shape, adjusting the size as needed.



Your projectile scene should now look something like this:

Projectile (Area2D)

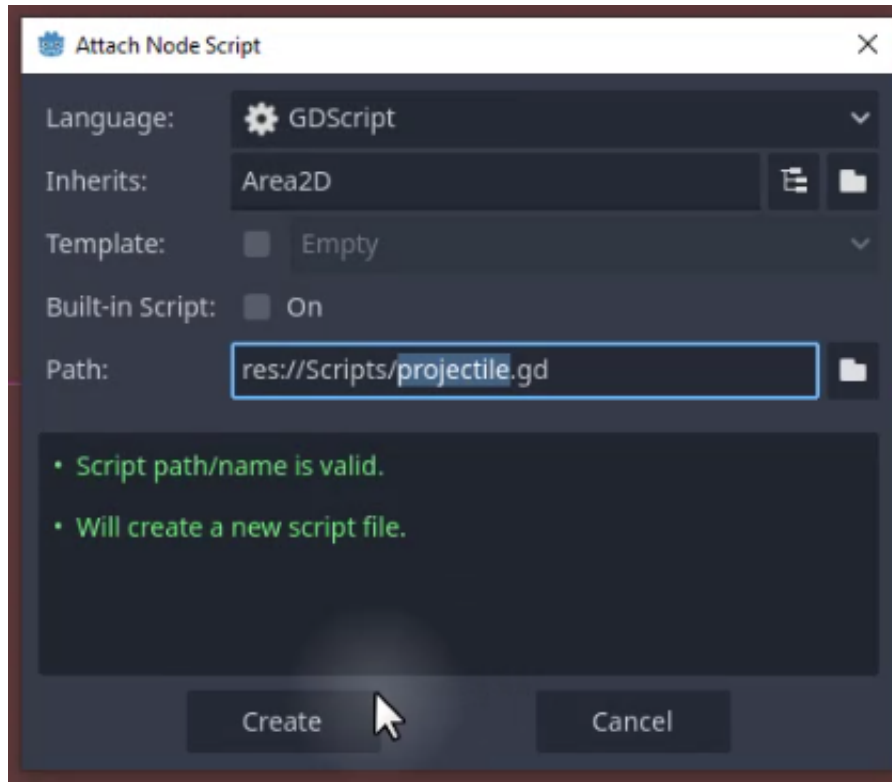


??? Sprite (Sprite2D)

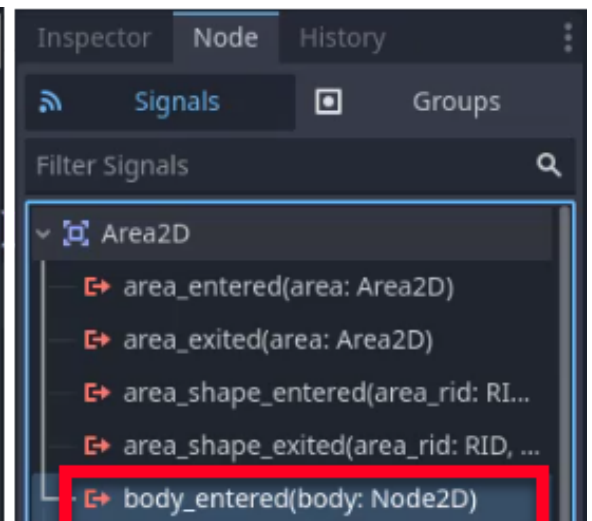
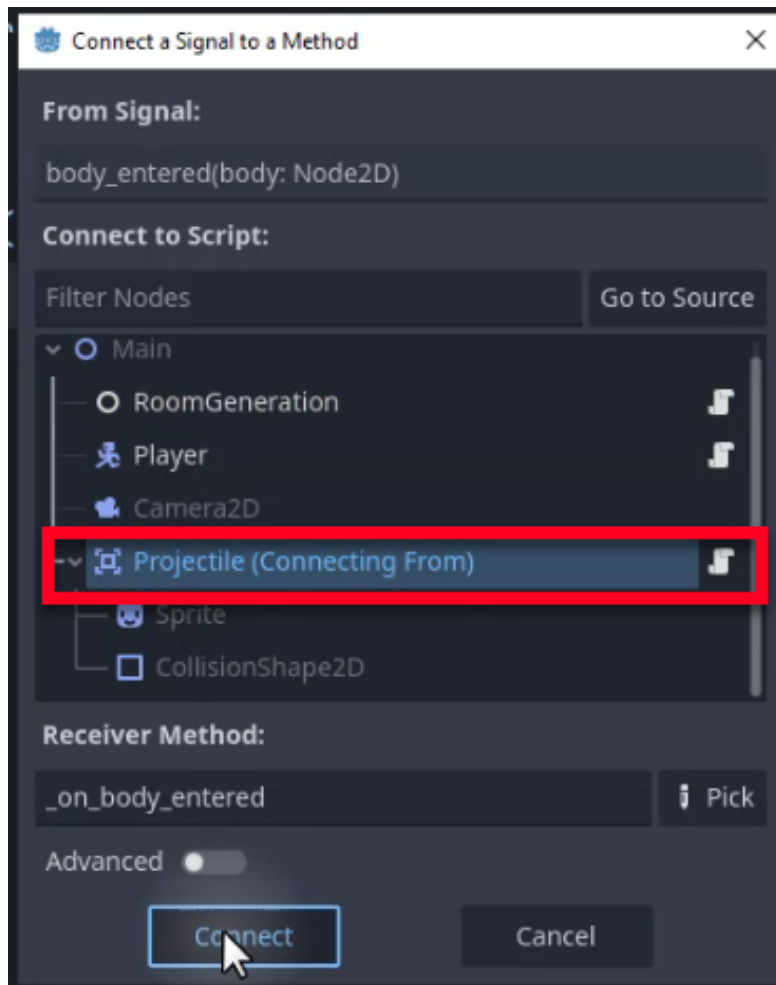
??? CollisionShape2D

## Adding the Projectile Script

Next, we need to add a script to the Projectile node to handle its movement and collision detection. Create a new script on the Projectile node and name it projectile.gd.



We also want to connect the `body_entered` signal to our script. With the Projectile root node select, go to the Node section in the Inspector and double-click the `body_entered` signal. Then connect it to the Projectile with the auto-created function.



Add the following code to the script:

```
extends Area2D

@export var base_speed : float = 100
var owner_character : CharacterBody2D

func _process(delta):
    translate(-transform.y * base_speed * delta)

func _on_body_entered(body):
    if body == owner_character:
        return

    if body.has_method("take_damage"):
        body.take_damage(1)

    queue_free()
```

This script does the following:

- **base\_speed:** Sets the speed at which the projectile moves.



- `owner_character`: Keeps track of the character that fired the projectile to prevent self-damage.
- `_process(delta)`: Moves the projectile along its local Y-axis at the specified speed.
- `_on_body_entered(body)`: Checks for collisions and calls the `take_damage` method on the collided body if it exists.

## Altering the Player

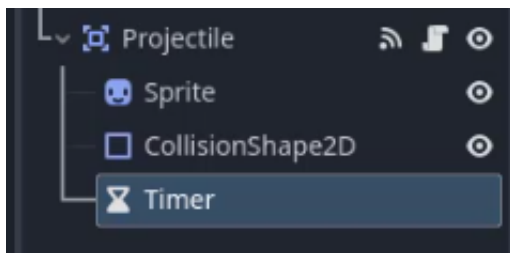
At current, our player has no way to take damage. As we're calling this non-existent function, in the **player** script we need to add a take damage function. We will fill this out later when we start working on our player health.

```
func take_damage (amount : int):  
    pass
```

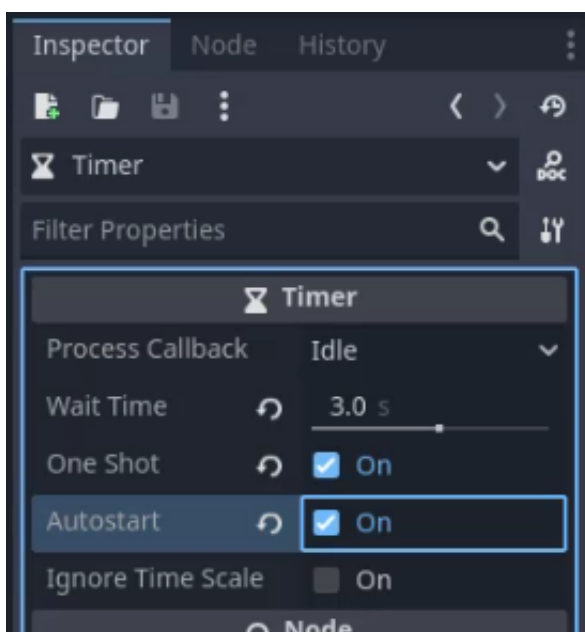
## Setting Up the Timer

To ensure the projectile does not exist indefinitely, we need to add a Timer node to it. Follow these steps:

- Add a Timer node as a child of Projectile.

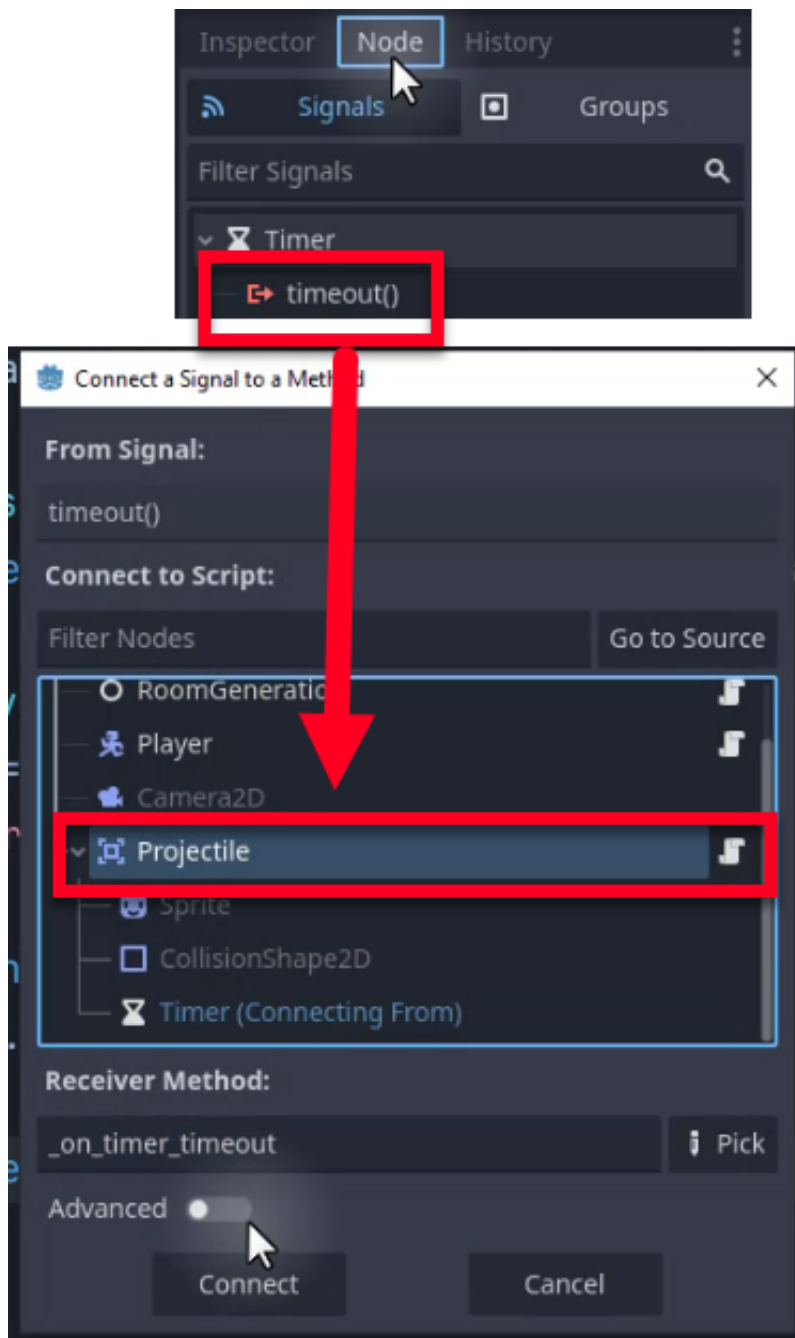


- Set the Wait Time property to 3 seconds.
- Enable Autostart and One Shot.





- Connect the timeout signal of the Timer node to the `_on_timer_timeout` function in the Projectile script.

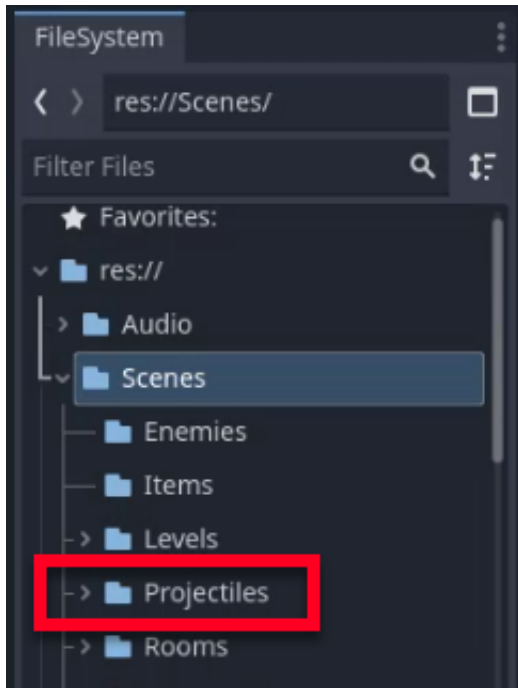


For functionality, we simply need to delete the projectile when the timer runs out.

```
func _on_timer_timeout():  
    queue_free()
```

Save the scene as `projectile.tscn` in the `Scenes/Projectiles` folder (you may need to create the `Projectiles` subfolder).





## Enabling Player Shooting

Now, let's enable the player to shoot these projectiles. Follow these steps:

1. Open the player script (player.gd).
2. Add the following variables at the top of the script:

```
@export var shoot_rate : float = 0.4
var last_shoot_time : float
var projectile_scene : PackedScene = preload("res://Scenes/Projectiles/projectile.tscn")
```

These variables do the following:

- `shoot_rate`: Sets the rate at which the player can shoot projectiles.
- `last_shoot_time`: Keeps track of the last time the player shot a projectile.
- `projectile_scene`: Preloads the projectile scene for instantiation.

Next, add the following code to the `_process` function to detect shoot inputs:

```
func _process(delta):
    var mouse_pos : Vector2 = get_global_mouse_position()
    var mouse_dir : Vector2 = (mouse_pos - global_position).normalized()
    weapon_origin.rotation_degrees = rad_to_deg(mouse_dir.angle()) + 90

    sprite.flip_h = mouse_dir.x < 0

    if Input.is_action_pressed("attack"):
        if Time.get_unix_time_from_system() - last_shoot_time > shoot_rate:
            _shoot()
```



This new code does checks if the attack action is pressed and, if enough time has passed since the last shot, calls the `_shoot` function.

Finally, add the `_shoot` function to the script:

```
func _shoot():
    last_shoot_time = Time.get_unix_time_from_system()

    var proj = projectile_scene.instantiate()
    get_tree().root.add_child(proj)
    proj.global_position = muzzle.global_position
    proj.rotation = weapon_origin.rotation
    proj.owner_character = self
```

This function does the following:

- Updates the `last_shoot_time`.
- Instantiates the projectile scene and adds it to the scene tree.
- Sets the projectile's position to the muzzle's position.
- Sets the projectile's rotation to the weapon's rotation.
- Sets the projectile's owner to the player to prevent self-damage.

With these steps completed, your player should now be able to shoot projectiles by left-clicking the mouse. The projectiles will move in the direction of the mouse cursor and will be destroyed upon collision or after a certain time.



In the next lesson, we will set up global signals to handle camera movement and introduce a new system to manage game logic more efficiently.

In this lesson, we will set up a global signals system. This system will allow us to manage and emit signals that can be accessed from anywhere in our project. This is particularly useful for events that multiple parts of the game need to be informed about, such as when a player enters a room, an enemy is defeated, or the player's health is updated.

## What Are Global Signals?

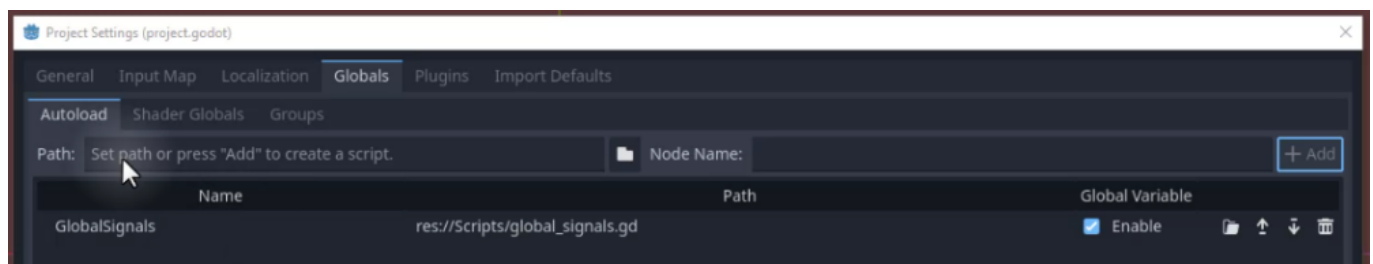
Global signals are a way to manage events in your game without creating complex dependencies between nodes. Instead of connecting signals directly to specific nodes, we use an autoload script that exists outside the scene structure. This script can emit signals that any part of the game can connect to and respond to.

## Setting Up Global Signals

To set up global signals, follow these steps:

1. Go to Project -> Project Settings -> AutoLoad.
2. Enter a node name, such as GlobalSignals, and click Add.
3. Create a new script for the autoload (global\_signals.gd). Save it in your scripts folder.

Your Project Settings should now include an entry for GlobalSignals with the path to your script. The global variable is enabled, meaning that all variables, functions, and signals in this script can be accessed from anywhere in your project.



## Creating a Signal

Let's create a signal that gets called whenever the player enters a room. This signal will inform other parts of the game, such as enemies and the camera, about the player's new location.

Open the global\_signals.gd script and add the following code:

```
extends Node

signal OnPlayerEnterRoom (room : Room)
```

This code defines a signal called OnPlayerEnterRoom that takes a Room object as a parameter.

## Emitting the Signal

Next, we need to emit this signal whenever the player enters a new room. Open your room script (e.g., room.gd) and find the player\_enter function. Add the following code to emit the signal:

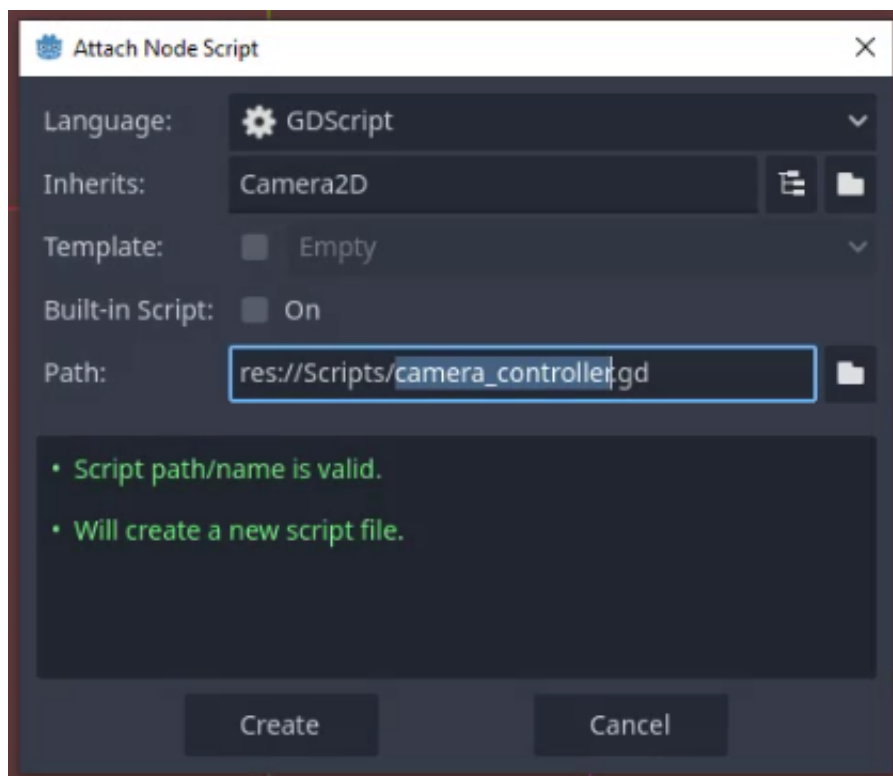
```
func player_enter(entry_direction : Direction, player : CharacterBody2D, first_room :
```



```
bool = false):  
    # ... Existing code ...  
  
    # If this is the first room, spawn the player in the middle  
    if first_room:  
        player.global_position = global_position  
  
    # Emit the global signal  
    GlobalSignals.OnPlayerEnterRoom.emit(self)  
  
    # Close the doors if there are enemies to defeat  
    if enemies_in_room > 0 and not doors_always_open:  
        close_doors()  
    else:  
        open_doors()
```

## Connecting to the Signal

Now that the signal is emitted, we can connect other parts of the game to respond to it. For example, let's make the camera snap to the player's new room. Create a new script for the camera (e.g., camera\_controller.gd).



With this script, we will add the following code:

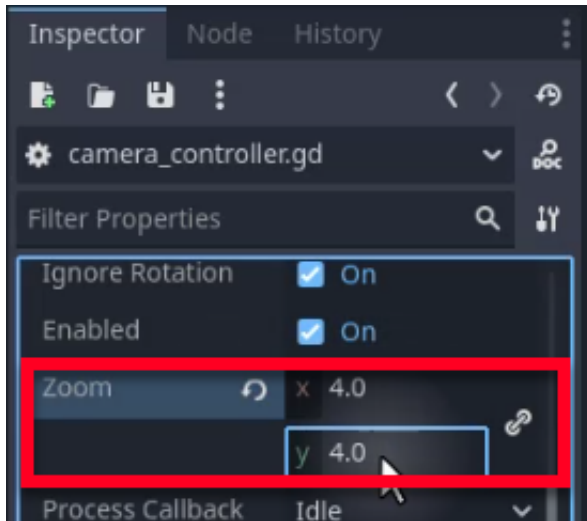
```
extends Camera2D  
  
func _ready():  
    GlobalSignals.OnPlayerEnterRoom.connect(_on_player_enter_room)
```



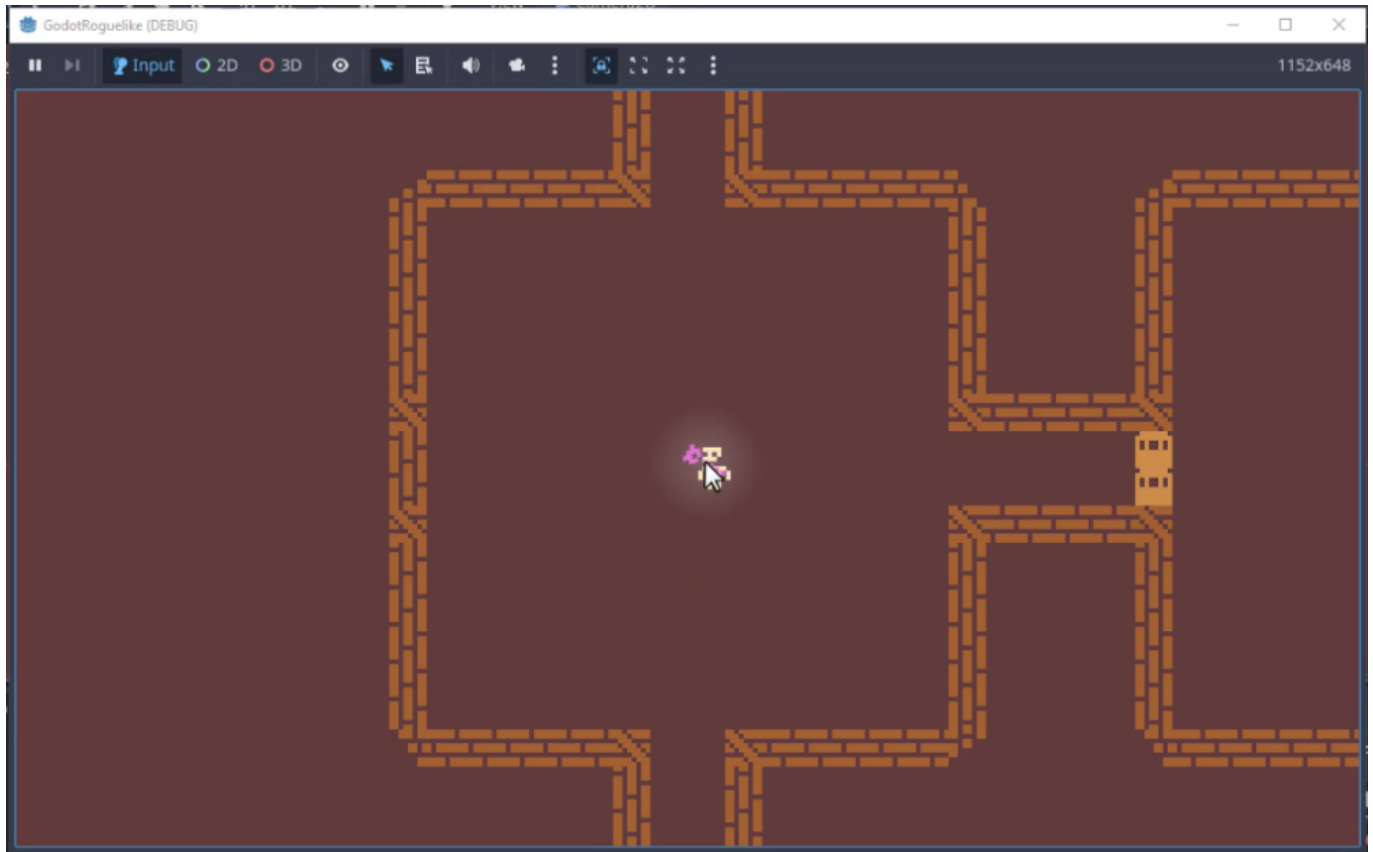
```
func _on_player_enter_room(room : Room):  
    global_position = room.global_position
```

This code connects the OnPlayerEnterRoom signal to the `_on_player_enter_room` function, which sets the camera's global position to the room's global position.

To test this, back in the editor, change the zoom level back to 4 in the Inspector for the camera.



When you load in the game, you will be able to see the player and room clearly, as the camera automatically snaps to that location.



### Benefits of Global Signals

- Reduces dependencies between nodes.
- Makes it easier to manage and emit signals from anywhere in the project.
- Simplifies the process of informing multiple parts of the game about important events.

That's it for this lesson on setting up a global signals system in Godot. In the next lesson, we will work on allowing the player to actually move between rooms.



In this lesson, we will focus on enabling the player to navigate between different rooms in our game. Currently, the player spawns in the first room and is unable to move to other rooms. Our goal is to allow the player to transition between rooms by walking through designated tunnels, which will teleport the player to the neighboring room.

To achieve room transitions, we will work with the RoomEntrance script. This script manages the entrances and exits of each room. Here's how we can set it up:

### Connecting the Exit Trigger

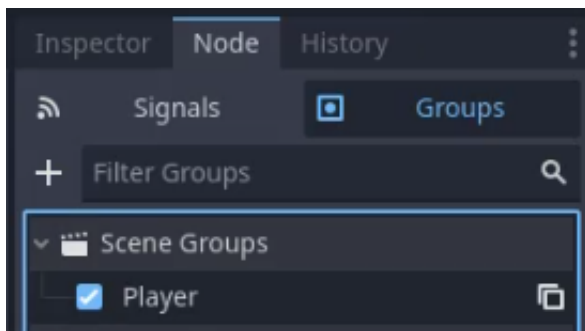
Each room entrance has an exit trigger at the end. When the player walks through this trigger, they should be teleported to the neighboring room. We have already connected the body\_entered signal of this exit trigger to the `_on_body_entered_exit_trigger` function in our RoomEntrance script.

```
func _ready():
    exit_trigger.body_entered.connect(_on_body_entered_exit_trigger)
    toggle_barrier(true)
```

### Checking for Player Entry

We need to ensure that only the player can trigger the room transition. To do this, we will check if the body that entered the trigger is in the "player" group. Here's how you can set this up:

- Open the player scene.
- Select the player root node.
- In the Node panel, go to the Groups section.
- Add the player to a group called "Player".



Now, in the RoomEntrance script, we will modify the `_on_body_entered_exit_trigger` function to check if the body is in the "player" group:

```
func _on_body_entered_exit_trigger(body):
    if body.is_in_group("Player"):
```

### Accessing the Neighbor Room

Once we confirm that the body is the player, we need to access the neighboring room and call its `player_enter` function. This function requires two parameters: the entry direction and the player body.



The entry direction can be obtained using the `_get_neighbor_entry_direction` function. Here's the updated function:

```
func _on_body_entered_exit_trigger(body):
    if body.is_in_group("player"):
        neighbor.player_enter(_get_neighbor_entry_direction(), body)
```

## Fixing Room Generation Errors

You might encounter an error where the player is teleported to a random room instead of the correct neighboring room. This issue is due to a mistake in the `RoomGeneration` script. Specifically, in the `get_room_from_map` function, we need to replace the AND operator with an OR operator.

Here's the corrected code snippet:

```
func get_room_from_map(x: int, y: int) -> Room:
    for room in rooms:
        var pos = _get_map_index(room)
        if pos.x != x or pos.y != y:
            continue
        return room
    return null
```

## Deferring Door Calls

Another issue you might face is that the doors do not open correctly when the player enters a new room. This is because all actions are happening on the same frame. To fix this, we need to defer the calls to open and close the doors.

In the `Room` script, modify the `open_doors` and `close_doors` functions to use `call_deferred`:

```
func open_doors():
    entrance_north.open_door.call_deferred()
    entrance_south.open_door.call_deferred()
    entrance_east.open_door.call_deferred()
    entrance_west.open_door.call_deferred()

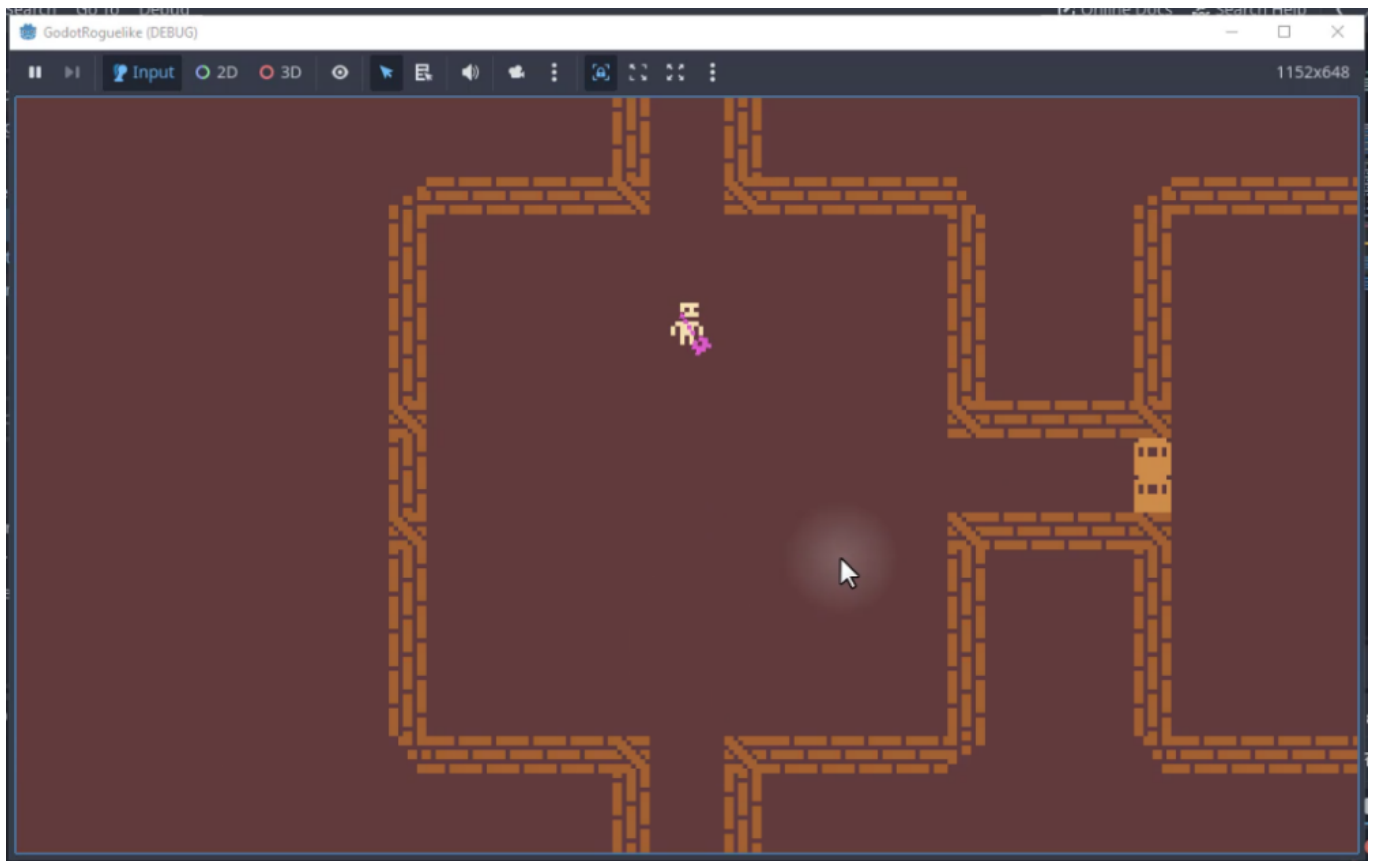
func close_doors():
    entrance_north.close_door.call_deferred()
    entrance_south.close_door.call_deferred()
    entrance_east.close_door.call_deferred()
    entrance_west.close_door.call_deferred()
```

## Testing the Implementation

After making these changes, you should be able to test the game and navigate between rooms smoothly. The doors should open correctly, and the player should be teleported to the neighboring



room when walking through the tunnels.



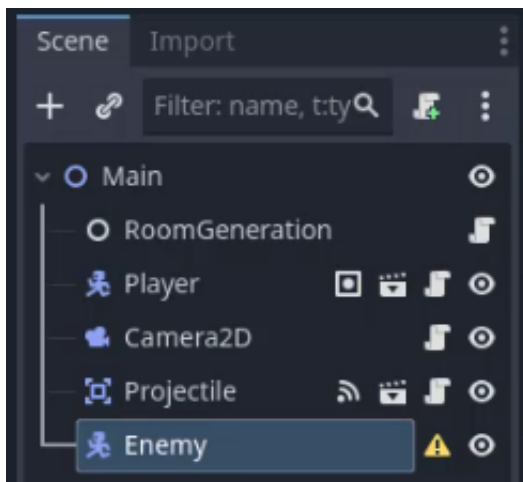
In the next lesson, we will introduce enemies to make the game more challenging.

In this lesson, we will focus on setting up enemies in our game. Enemies are essential for adding challenge and excitement to your game. We will create an enemy character, add necessary components, and write a script to control its behavior. Let's get started!

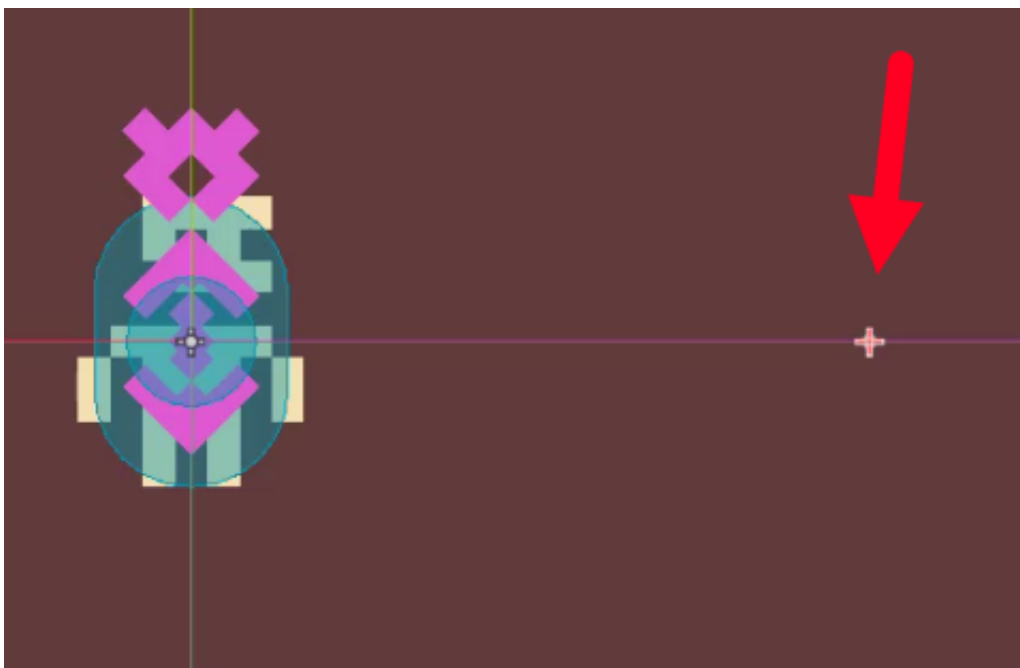
## Creating the Enemy Character

To begin, we need to create a new character for our enemy. Follow these steps:

- Create a new CharacterBody2D node and name it Enemy.

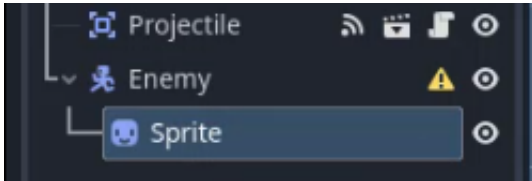


- Position the enemy character to the right side of the screen to keep it out of the way initially.

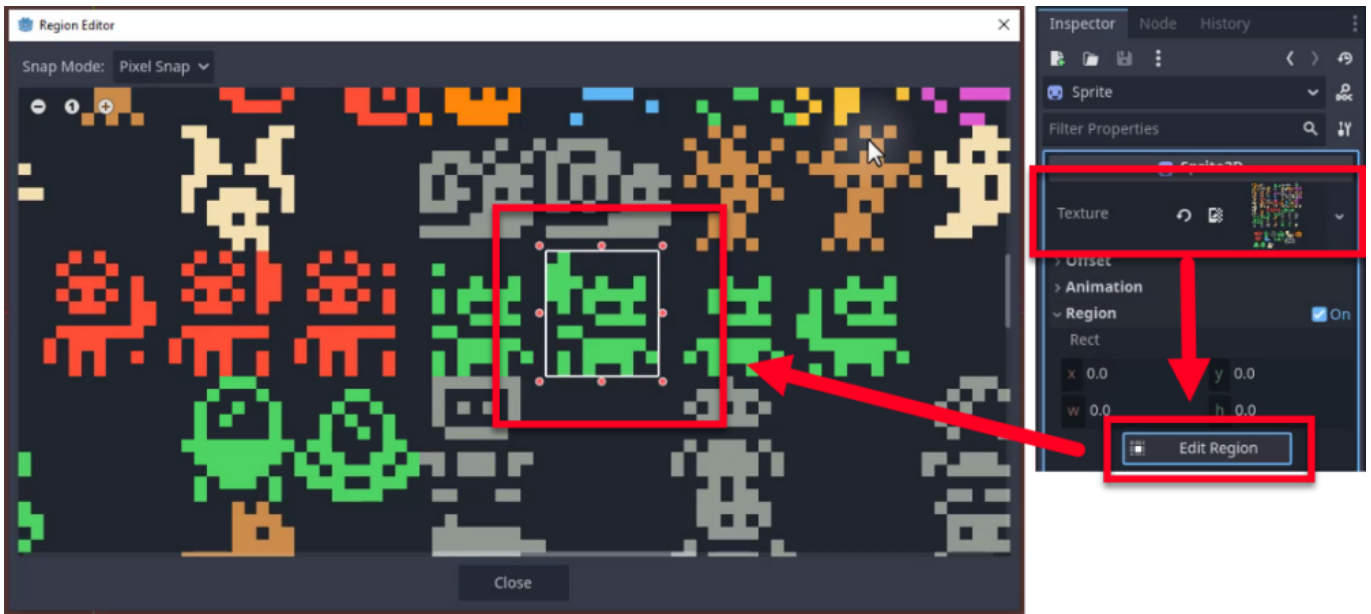


Next, we will add the necessary components to our enemy character:

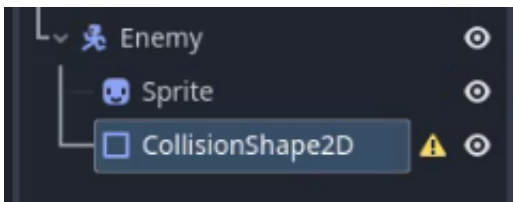
- Add a Sprite2D to the enemy. This will be used to display the enemy's texture. Like the player and projectile, we'll name this "Sprite" for simplicity's sake.



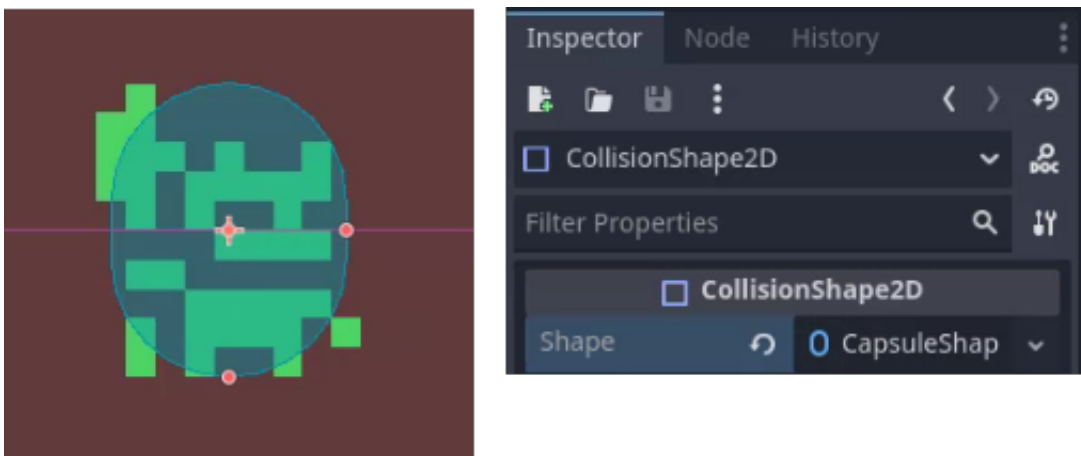
- Set the texture for the sprite. We will use "Creatures.png", enable region editing, and select an appropriate sprite.



- Add a CollisionShape2D to the enemy. This will define the collision area for the enemy.



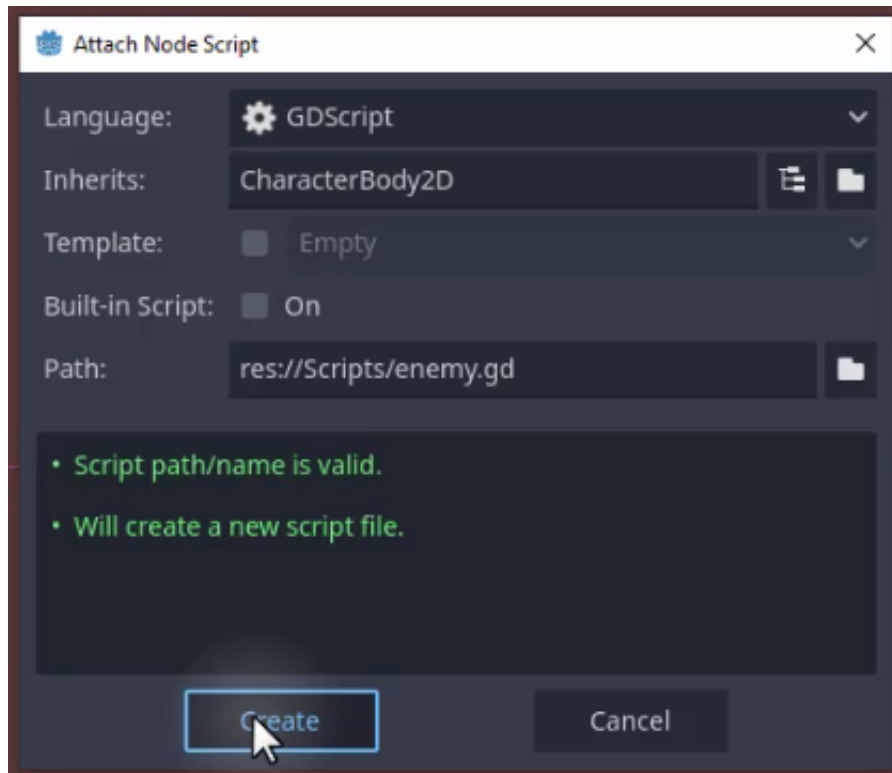
- Set the collision shape to a simple capsule shape and adjust its size to fit the enemy sprite.



## Adding the Enemy Script



Now, let's add a script to control the enemy's behavior. Select the enemy node and create a new script named enemy.gd.



Inside the script, we will define the enemy's properties and behaviors. Here is the initial setup for the enemy script:

```
extends CharacterBody2D

@export var cur_hp : int = 4
@export var max_hp : int = 4
@export var move_speed : float = 20

@export var attack_damage : int = 1
@export var attack_range : float = 10
@export var attack_rate : float = 0.5
var last_attack_time : float

@onready var sprite : Sprite2D = $Sprite

var room : Room
var is_active : bool = false
var player : CharacterBody2D

var player_direction : Vector2
var player_distance : float
```

These variables track the following:



- **cur\_hp** and **max\_hp**: These track values related to the enemy's health
- **move\_speed**: Determines how fast the enemy moves
- **attack\_damage**: Determines how much damage the enemy does per hit
- **attack\_range**: Determines how far away the enemy can be to attack
- **attack\_rate**: Determines how fast the enemy attacks
- **last\_attack\_time**: Similar to the player shooting, tracks the last time the enemy attacked
- **sprite**: Holds a reference to the Enemy sprite
- **room**: Enemies will know which room of our game they're in.
- **is\_active**: determines if the enemy can attack
- **player**: Holds a reference to the player
- **player\_direction**: Holds the value of the vector direction of the player
- **player\_distance**: Tracks how far away the player is from the enemy

## Defining Enemy Functions

We will define several functions to control the enemy's behavior. Here are the functions we will implement:

- **initialize**: Called when the enemy is spawned in a room.
- **on\_player\_enter\_room**: Activates the enemy when the player enters the room.
- **\_physics\_process**: Moves the enemy towards the player.
- **\_try\_attack**: Attempts to attack the player if within range.
- **take\_damage**: Reduces the enemy's health when damaged.
- **die**: Kills the enemy and emits a defeat signal.

```
func _ready ():  
    pass  
  
func initialize (in_room : Room):  
    pass  
  
func _on_player_enter_room (player_room : Room):  
    pass  
  
func _physics_process(delta):  
    pass  
  
func _try_attack ():  
    pass  
  
func take_damage (amount : int):  
    pass  
  
func die ():  
    pass
```

## Enemy Movement

To start, we will implement the `_physics_process` function to move the enemy towards the player. In this case, we first want to get the direction and distance to the player. We flip the enemy in that direction. Finally, we set the enemy's velocity to that direction and move them.

```
func _physics_process(delta):
```



```
player_direction = global_position.direction_to(player.global_position)
player_distance = global_position.distance_to(player.global_position)

sprite.flip_h = player_direction.x < 0

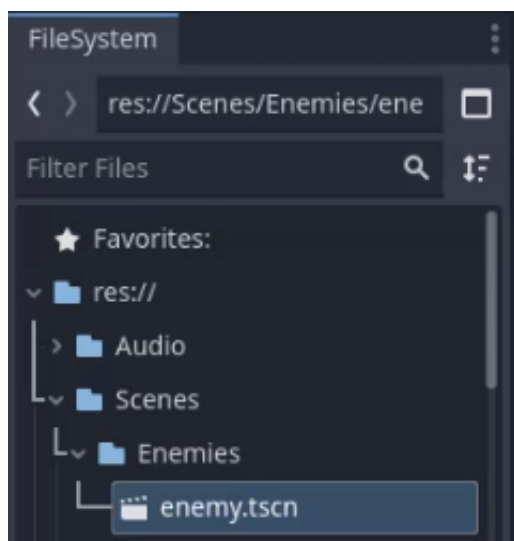
velocity = player_direction * move_speed
move_and_slide()
```

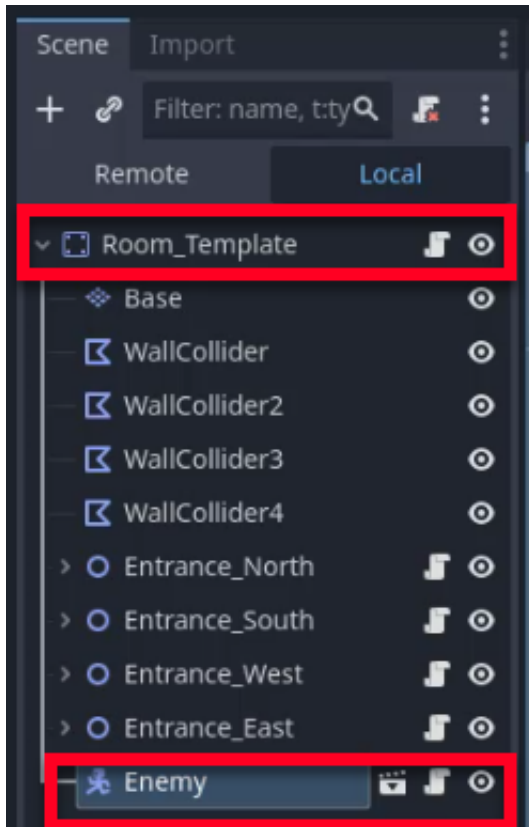
## Connecting the Player

To ensure the enemy can interact with the player, we need to connect the player node to the enemy script. In the `_ready` function, add the following code:

```
func _ready():
    player = get_tree().get_first_node_in_group("Player")
```

To test the enemy, save the enemy as a scene in the Scenes/Enemies folder and add it to a room.





Play the game and observe the enemy's behavior. You should see the enemy moving towards the player and attempting to attack when within range.



### Setting up the Layers and Masks

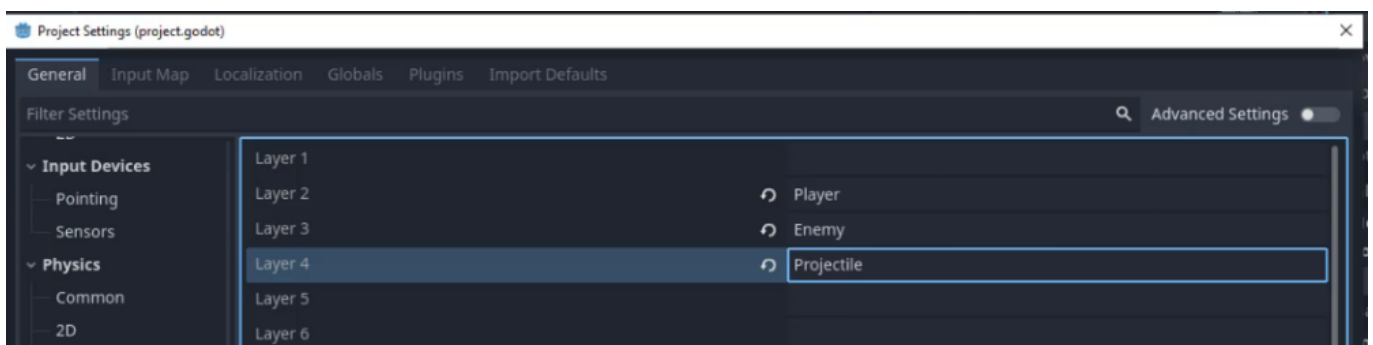
When the enemy and player share the same collision layer, their movements can interfere with each other and cause buggy interactions. To prevent this, start by opening the player scene, then navigate to the inspector and locate the collision settings. Here, you'll find both a layer and a mask. The layer determines which group the player belongs to, while the mask controls which other groups the player can interact with.

Assign specific roles to each collision layer by clicking on the three dots in the collision section and

selecting “Edit Layer Names.”



Name Layer 2 as “Player,” Layer 3 as “Enemy,” and Layer 4 as “Projectile,” while leaving Layer 1 as is or for barriers.



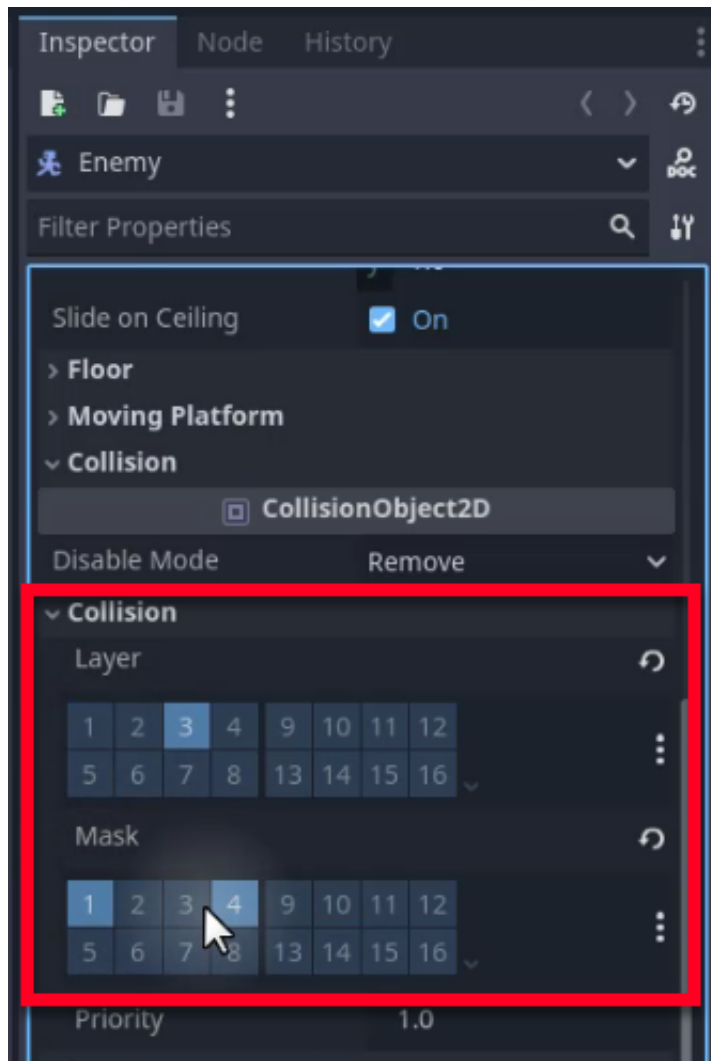




In the player's collision settings, set the layer to 2 (Player) by enabling only the checkbox for Layer 2. In the mask, enable Layer 1 (Barriers) and Layer 4 (Projectiles) so the player can interact with barriers and projectiles, but make sure Layers 2 and 3 (Player and Enemy) are disabled in the mask so the player doesn't interact with other players or enemies.



For the enemy, open the enemy scene, select the enemy object, and go to its collision settings. Set its Layer to 3 (Enemy) by enabling only the checkbox for Layer 3. In its mask, enable just Layer 1 (Barriers) and Layer 4 (Projectiles), so enemies do not interact or collide with players or other enemies.



You have successfully set up an enemy character in your game. In the next lesson, we will continue to refine the enemy's behavior.



In this lesson, we will continue setting up our enemy. Our goal is to make the enemy stop moving when it is within range of the player and start attacking. We will also ensure that the enemy only chases the player when the player is within the same room. Additionally, we will address some layer and mask issues to ensure proper interactions between the player, enemy, and projectiles.

## Stopping Enemy Movement Within Attack Range

To make the enemy stop moving when it is within attack range of the player, we need to modify the `_physics_process` function in the `enemy.gd` script. Here are the steps:

1. Open the `enemy.gd` script.
2. In the `_physics_process` function, add a condition to check if the player distance is less than the attack range before setting the enemy's velocity.
3. If the condition is true, call the `_try_attack` function and return to prevent further movement.

Here is the modified code snippet for the `_physics_process` function:

```
func _physics_process(delta):
    player_direction = global_position.direction_to(player.global_position)
    player_distance = global_position.distance_to(player.global_position)

    sprite.flip_h = player_direction.x < 0

    if player_distance < attack_range:
        _try_attack()
        return

    velocity = player_direction * move_speed
    move_and_slide()
```

## Adding a Condition to the Top of the Physics Process Function

We also need to add a condition at the top of the `_physics_process` function to ensure that the enemy only chases the player when the player is within the same room. Here are the steps:

1. At the top of the `_physics_process` function, add a condition to check if the enemy is not active or if the player does not exist.
2. If the condition is true, return to prevent any further execution of the function.

Here is the modified code snippet for the top of the `_physics_process` function:

```
func _physics_process(delta):
    if not is_active or not player:
        return

    # rest of code
```

## Implementing the Try Attack Function

Next, we need to implement the `_try_attack` function to handle the enemy's attack logic. Here are



the steps:

1. In the `_try_attack` function, check if enough time has elapsed since the last attack by comparing the current time with the last attack time plus the attack rate.
2. If not enough time has elapsed, return to prevent further execution.
3. If enough time has elapsed, update the last attack time to the current time.
4. Call the `take_damage` function on the player, passing the attack damage as an argument.

Here is the code snippet for the `_try_attack` function:

```
func _try_attack():  
    if Time.get_unix_time_from_system() - last_attack_time < attack_rate:  
        return  
  
    last_attack_time = Time.get_unix_time_from_system()  
  
    player.take_damage(attack_damage)
```

## Updating the Player Script

To ensure that the player can take damage from the enemy, we need to update the `take_damage` function in the `player.gd` script. Here are the steps:

1. Open the `player.gd` script.
2. In the `take_damage` function, add a print statement to verify that the function is being called.

Here is the modified code snippet for the `take_damage` function:

```
func take_damage(amount):  
    print("Take Damage")
```

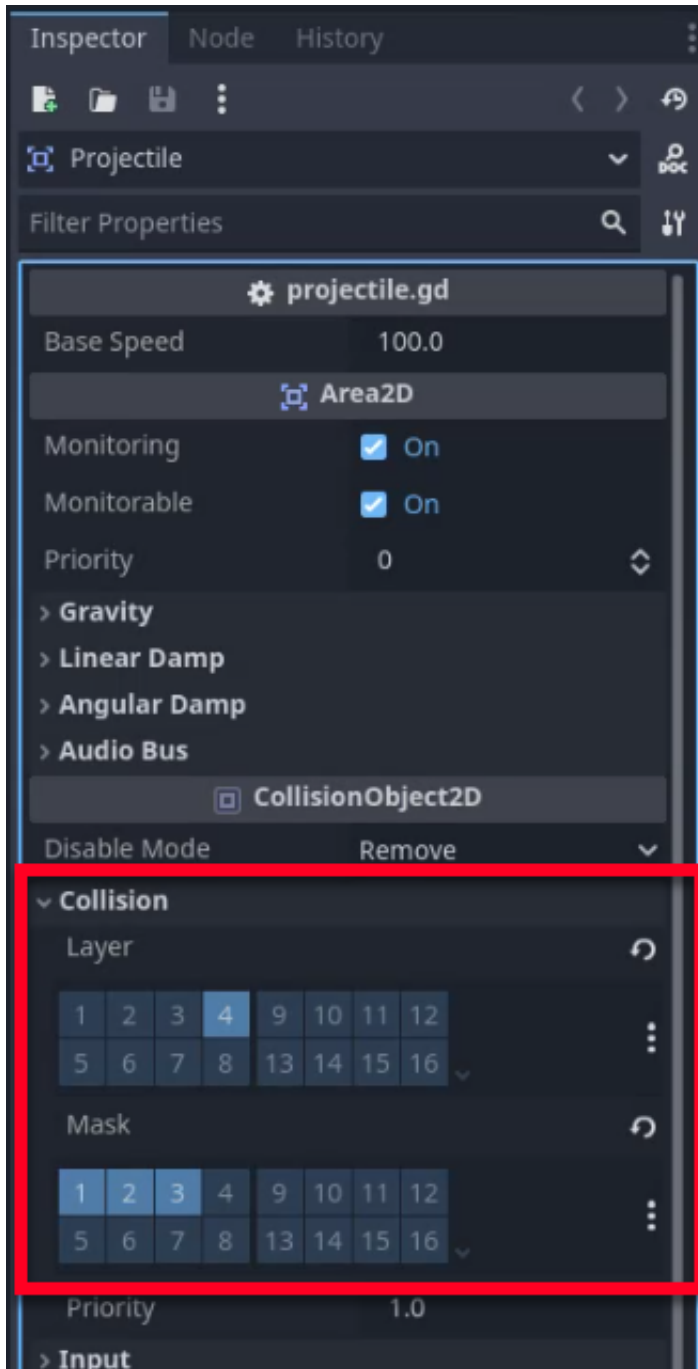
If you test now, you should see a message in the console if the enemy gets close enough to attack the player.



## Fixing Layer and Mask Issues

To ensure proper interactions between the player, enemy, and projectiles, we need to update the layer and mask settings for the projectile. Here are the steps:

1. Open the projectile scene (projectile.tscn).
2. Select the CollisionShape2D node.
3. In the Inspector, go to the Collision section and set the Mask to include Layers 1 (Barriers), 2 (Player), and 3 (Enemy).
4. Set the Layer to 4 (Projectile).



## Enemy Taking Damage

Next, let's work on the enemy's **take\_damage** function. This function simply needs to subtract the amount of damage from **cur\_hp**. We then check if **cur\_hp** is less than 0. Should that happen, we'll call the **die** function.

We can also set up the **die** function now, as all the enemy needs to do is be removed from the scene.

```
func take_damage (amount : int):  
    cur_hp -= amount  
  
    if cur_hp <= 0:
```



```
die()
```

```
func die():  
    queue_free()
```

Testing the game now should result in the enemy being able to be killed and removed from the room.



### Updating the Enemy Script for Room Interactions

Finally, we need to update the enemy.gd script to handle room interactions properly. Here are the steps:

1. In the enemy.gd script, put the initialization of the `is_active` variable to the `initialize` function.
2. In the `initialize` function, set the `room` variable to the current room.
3. In the `_on_player_enter_room` function, update the `is_active` variable based on whether the player's room is equal to the enemy's room.
4. Connect the `_on_player_enter_room` function to the global signal `OnPlayerEnterRoom`.

Here is the modified code snippets for the enemy.gd script:

```
func initialize(in_room):  
    is_active = false  
    room = in_room
```

```
func _on_player_enter_room(player_room):  
    is_active = player_room == room
```

```
func _ready():  
    player = get_tree().get_first_node_in_group("Player")
```



```
GlobalSignals.OnPlayerEnterRoom.connect(_on_player_enter_room)
```

By following these steps, you will have successfully set up the enemy to stop moving within attack range, implemented the attack logic, updated the player script to handle damage, fixed layer and mask issues, and ensured proper room interactions. In the next lesson, we will properly set up our game to have enemies in the room.





In this lesson, we will set up our game so that when a player enters a room, the doors will shut if there are enemies present. The doors will only open again once all the enemies in the room have been defeated. This will add an exciting challenge to our game, making the player strategize to defeat enemies and progress through the rooms.

## Setting Up the Room Script

To achieve this functionality, we need to modify our room script. Here are the steps we will follow:

1. Count the number of enemies in the room.
2. Close the doors if there are enemies present (we already implemented this).
3. Open the doors once all enemies are defeated.

## Counting Enemies in the Room

First, we need to count the number of enemies in the room. We will do this in the `_ready` function of our room script. We will loop through each child node in the room and check if it is an enemy.

To identify enemy nodes, we will use a class name. Let's add a class name to our enemy script:

```
class_name Enemy
extends CharacterBody2D
```

Now, in our room script, we can loop through the children and count the enemies:

```
func _ready():
    for child in get_children():
        if child is Enemy:
            enemies_in_room += 1
            child.initialize(self)
```

## Opening the Doors After Defeating Enemies

To open the doors after all enemies are defeated, we need to create a signal that is emitted when an enemy is defeated. We will add this signal to our global signals script:

```
signal OnDefeatEnemy(enemy: Enemy)
```

Next, we will emit this signal in the `die` function of our enemy script:

```
func die():
    GlobalSignals.OnDefeatEnemy.emit(self)
    queue_free()
```



In our room script, we will connect to this signal and decrement the enemy count when an enemy is defeated. If the enemy count reaches zero, we will open the doors:

```
func _ready():
    GlobalSignals.OnDefeatEnemy.connect(_on_enemy_defeated)
    # ... Existing code ...

func _on_enemy_defeated(enemy: Enemy):
    if enemy.get_parent() == self:
        enemies_in_room -= 1

        if enemies_in_room <= 0:
            open_doors()
```

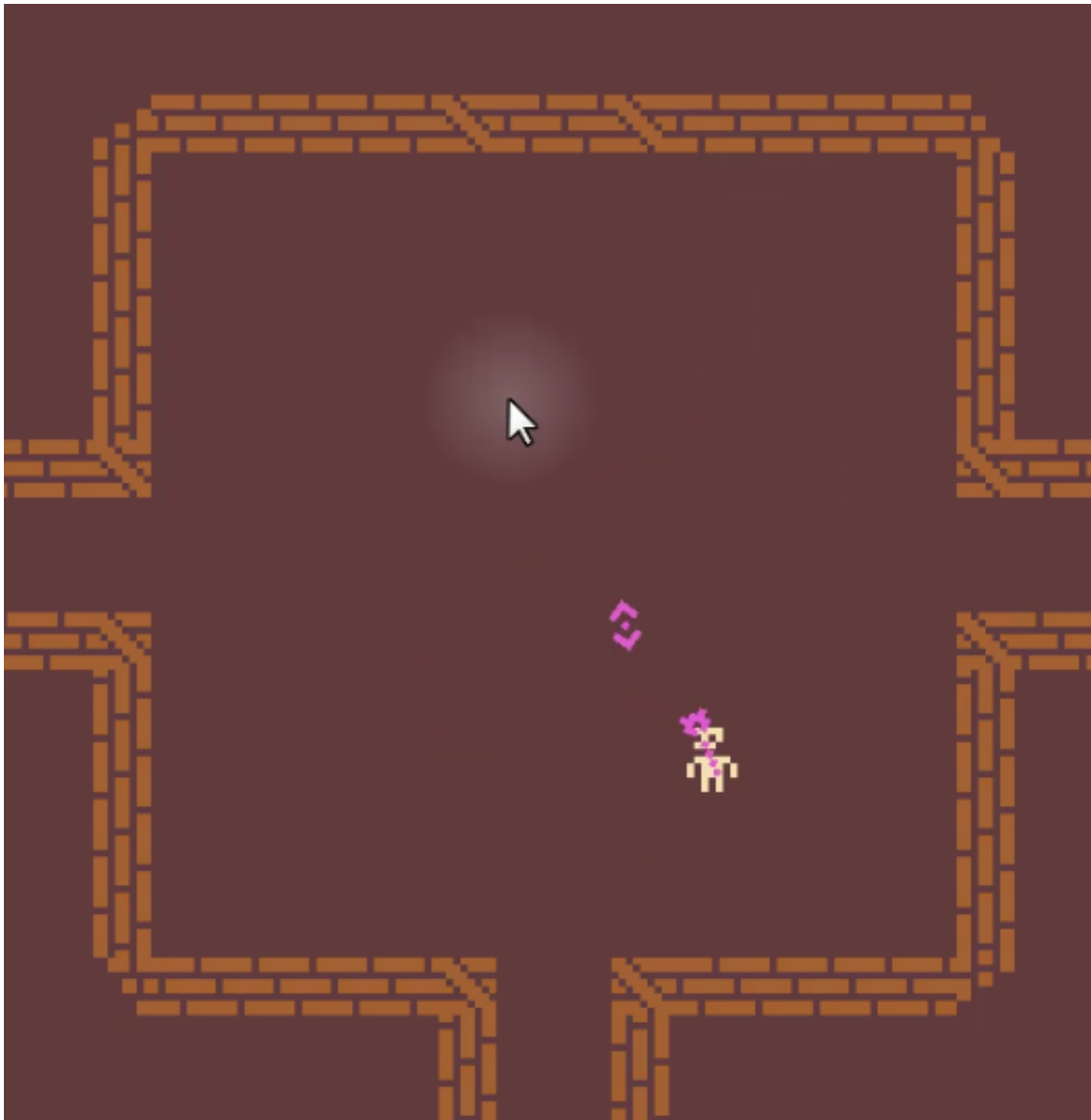
## Testing the Implementation

Finally, let's test our implementation. When the player enters a room with enemies, the doors should close.





Once all enemies are defeated, the doors should open, allowing the player to progress to the next room.



By following these steps, we have successfully set up our game so that the doors in a room will close if there are enemies present and will only open once all enemies are defeated. This adds an exciting challenge to our game and makes the player strategize to defeat enemies and progress through the rooms.

In the next lesson, we will explore how to create multiple different room types, such as rooms with one enemy, two enemies, empty rooms, item rooms, and boss rooms.

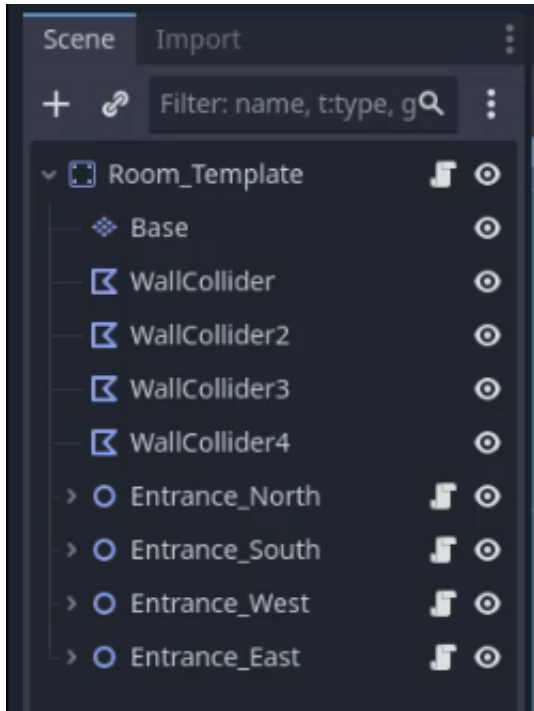


In this lesson, we will focus on setting up different room types in our game. By the end of this lesson, you will be able to create various room templates and integrate them into your game's room generation system. Let's get started!

## Clearing the Room Template

First, let's ensure our room template is clean and ready for inheritance:

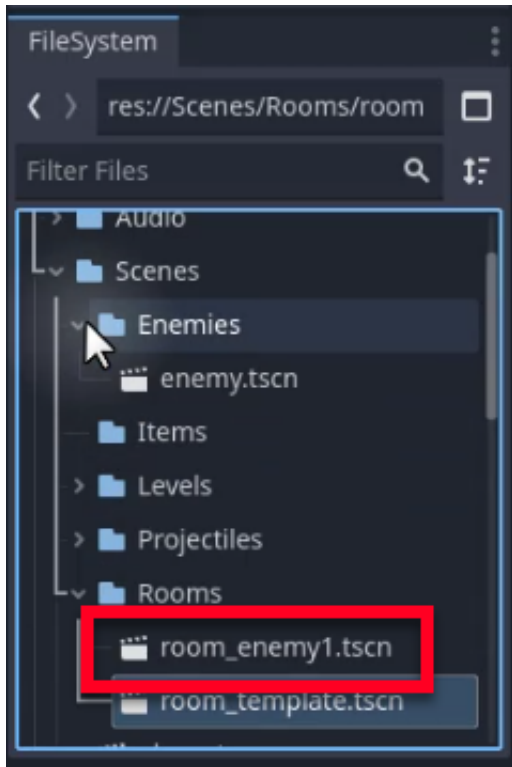
- Navigate to your room template scene.
- Delete any enemy scenes or other objects that should not be part of the template.



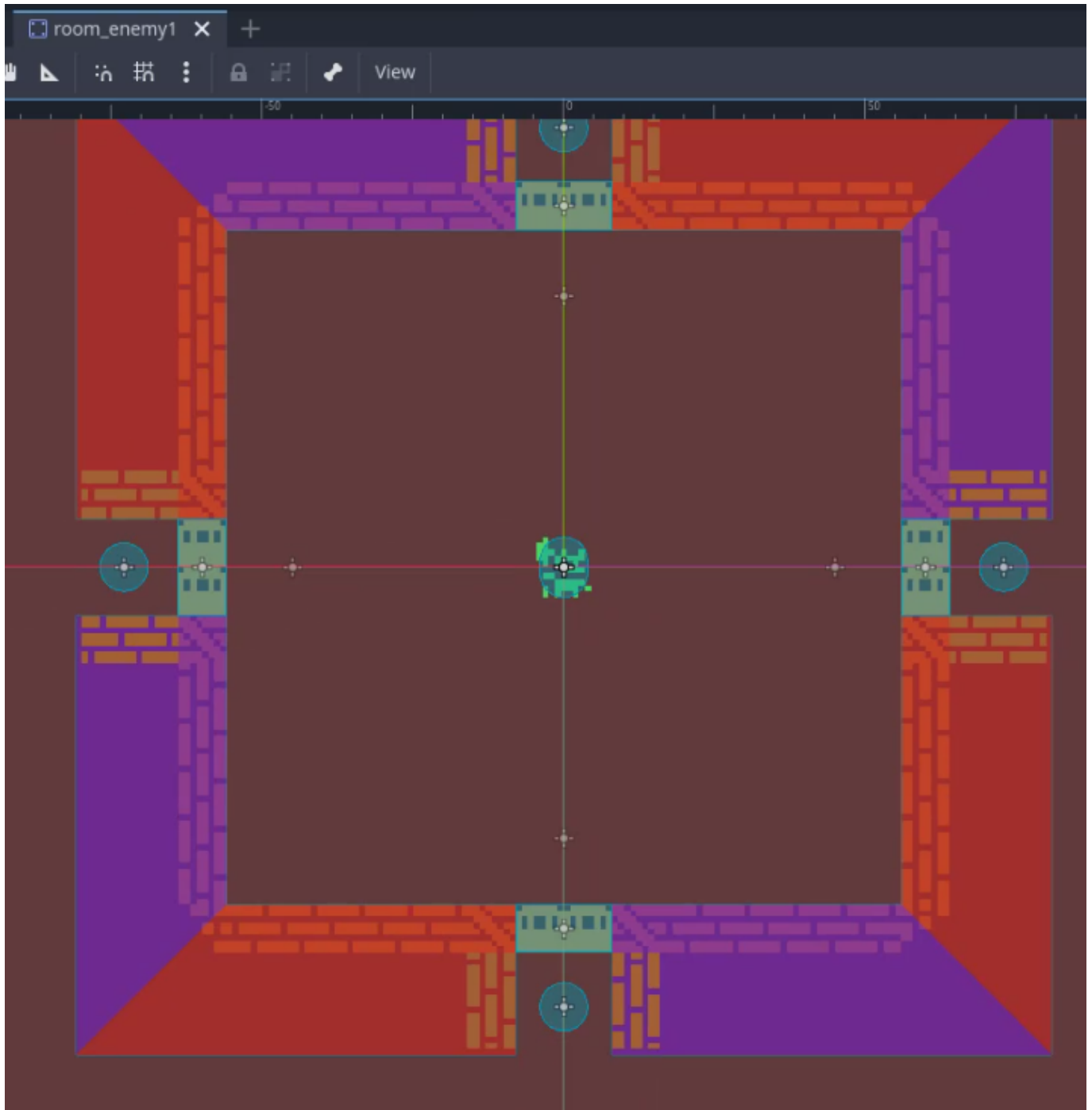
## Creating Inherited Room Scenes

Next, we will create new room scenes that inherit from the room template:

- Right-click on the room template scene and select New Inherited Scene.
- Save the new scene in the rooms folder. For this example, name it room\_enemy1.

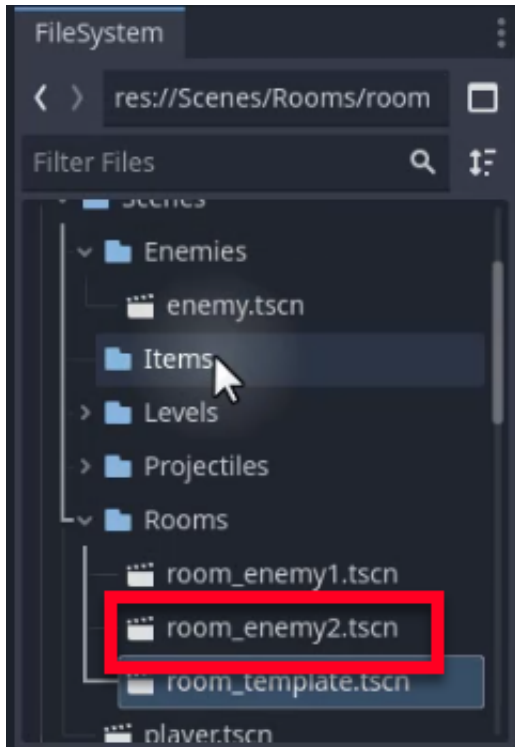


- Drag an enemy scene into the newly created room scene.

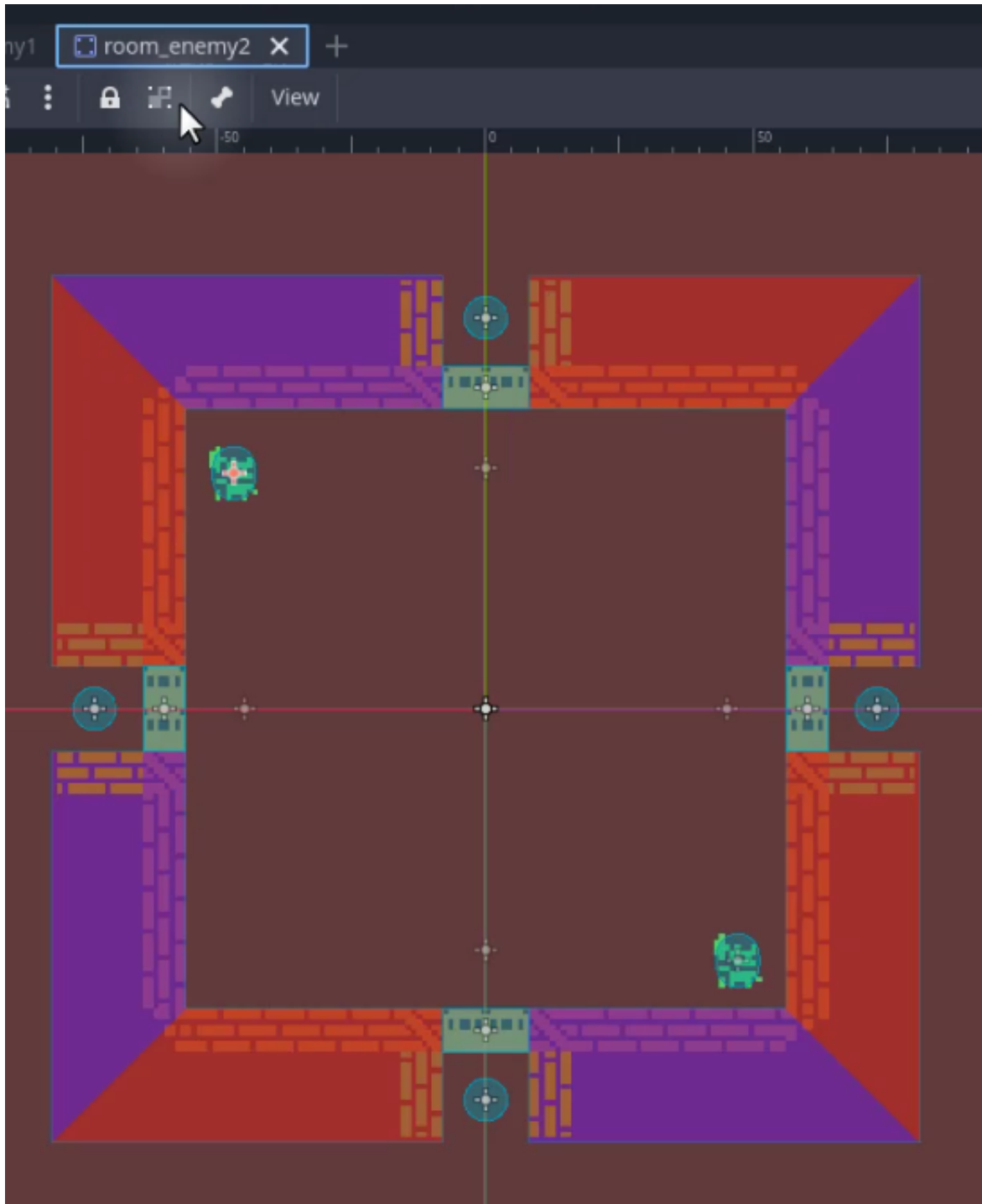


Repeat the process to create another room scene with two enemies:

- Right-click on the room template scene and select New Inherited Scene.
- Save the new scene as room\_enemy2.



- Add two enemies to this room scene, placing them in different corners.



## Updating the Room Generation Script

Now, let's update the room generation script to support multiple room types. First, open the room generation script and modify the script to include an array of room scenes and a variable for the first room scene:

```
extends Node
```

```
@export var first_room_scene : PackedScene  
@export var room_scenes : Array[PackedScene]
```





Next, update the `_instantiate_rooms` function to handle the new room types. In this case, we'll check if it is the first room and instantiate that scene. If it isn't, we'll select a random room from our `room_scenes` array:

```
func _instantiate_rooms():
    for x in range(map_size):
        for y in range(map_size):
            if not _get_map(x, y):
                continue

            var room : Room
            var is_first_room = first_room_x == x and first_room_y == y

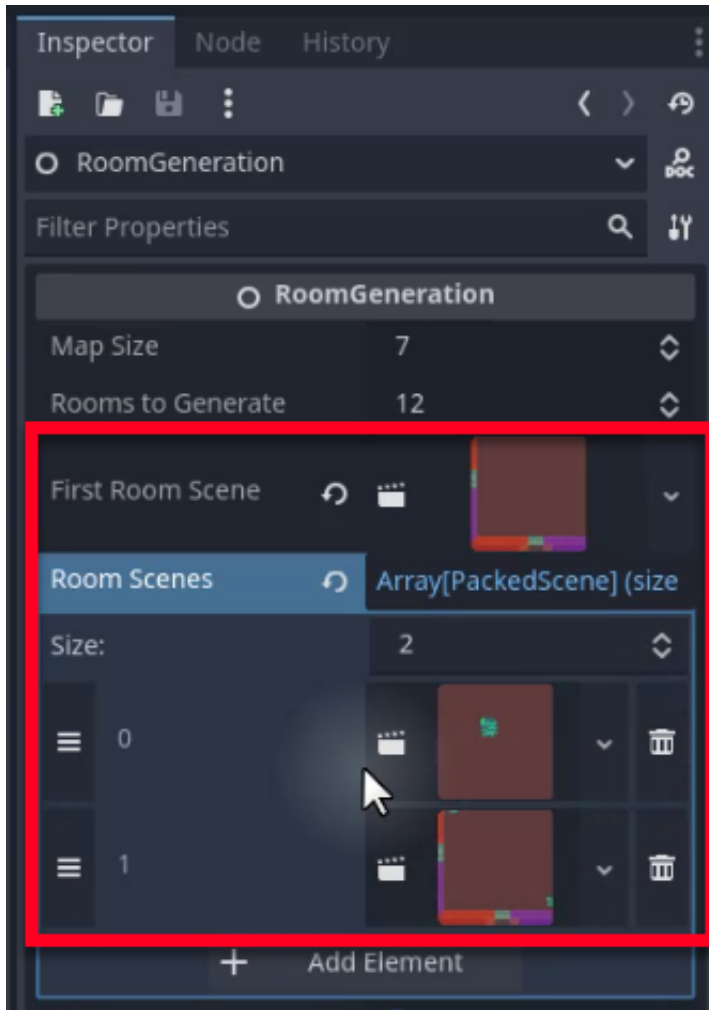
            if is_first_room:
                room = first_room_scene.instantiate()
            else:
                room = room_scenes[randi_range(0, len(room_scenes) - 1)].instantiate(
)

            get_tree().root.get_node("/root/Main").add_child(room)
            rooms.append(room)
            room.global_position = Vector2(x, y) * room_pos_offset

            if is_first_room:
                first_room = room

            room.initialize()
```

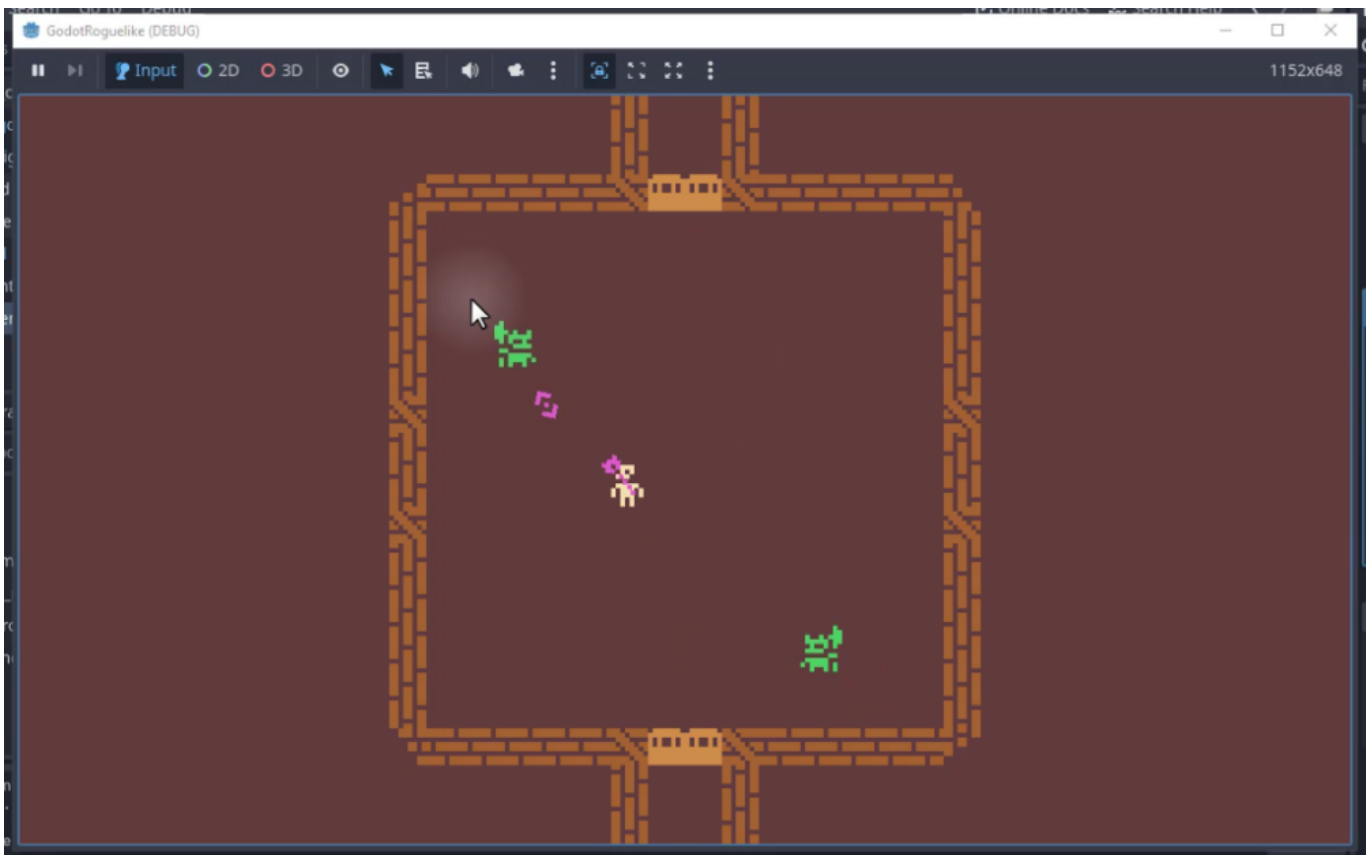
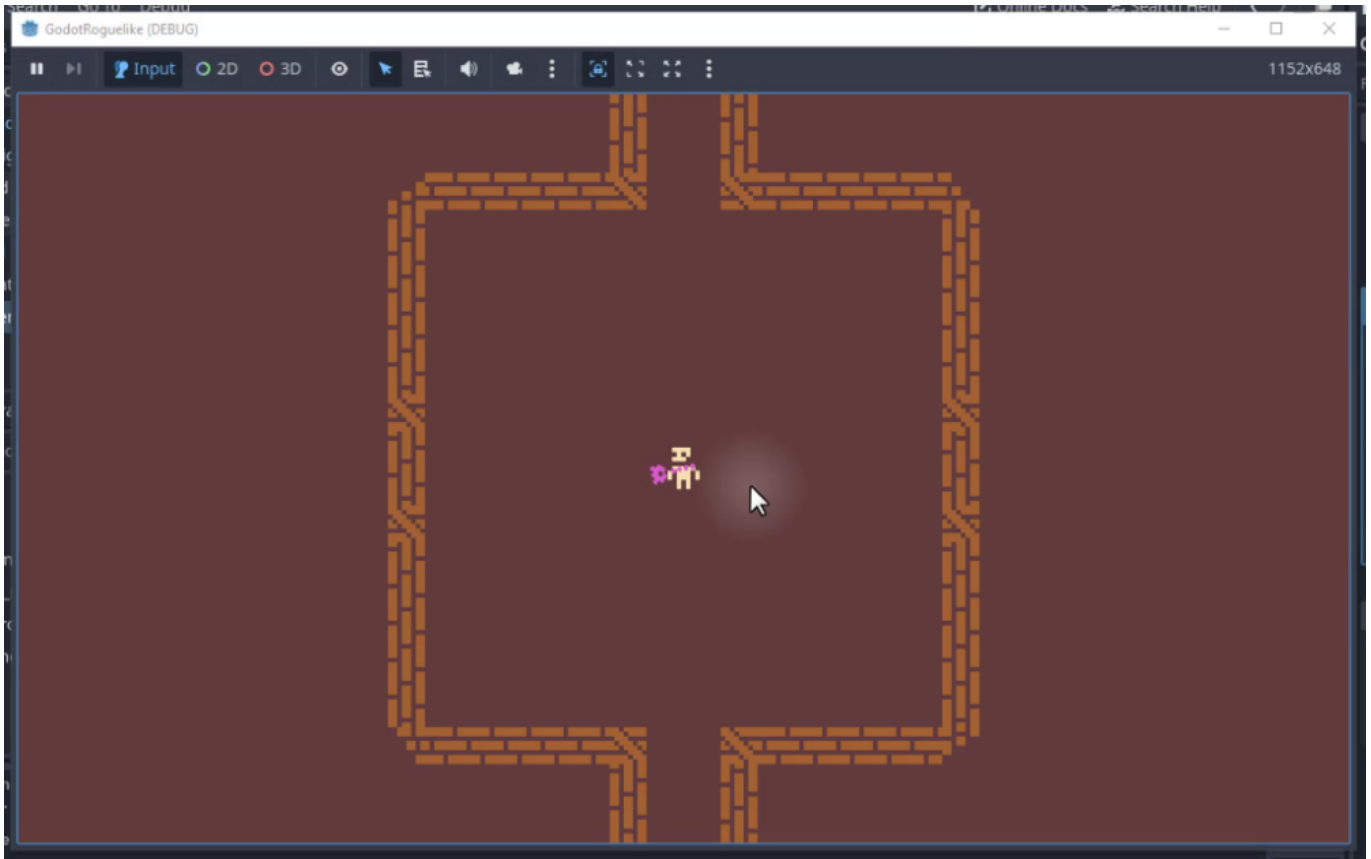
Back in the Editor, assign the blank room template to our First Room Scene, and our inherited scenes to our Room Scenes in the Inspector.



### Testing the Room Generation

Finally, test the room generation to ensure that the rooms are being created correctly:

- Run the game and observe the room generation.
- Verify that the first room is empty and subsequent rooms contain enemies as expected.



## Challenge



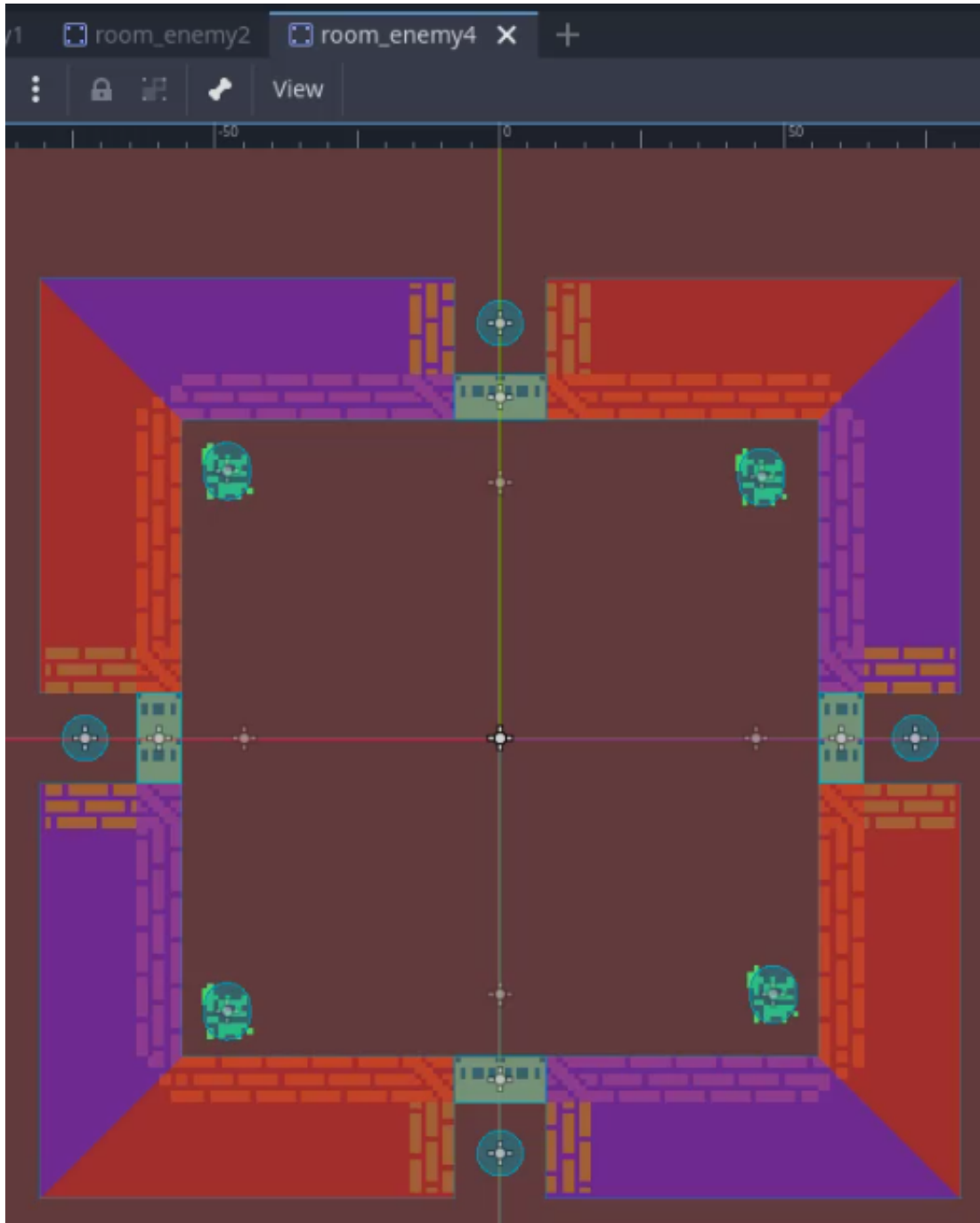
As a challenge, try creating a room with four enemies, one in each corner, and add it to the array. See if you can integrate this new room type into the game.

That's it for this lesson! You've learned how to set up different room types and integrate them into your game's room generation system. In the next lesson, we will go over our challenge and start working on items!

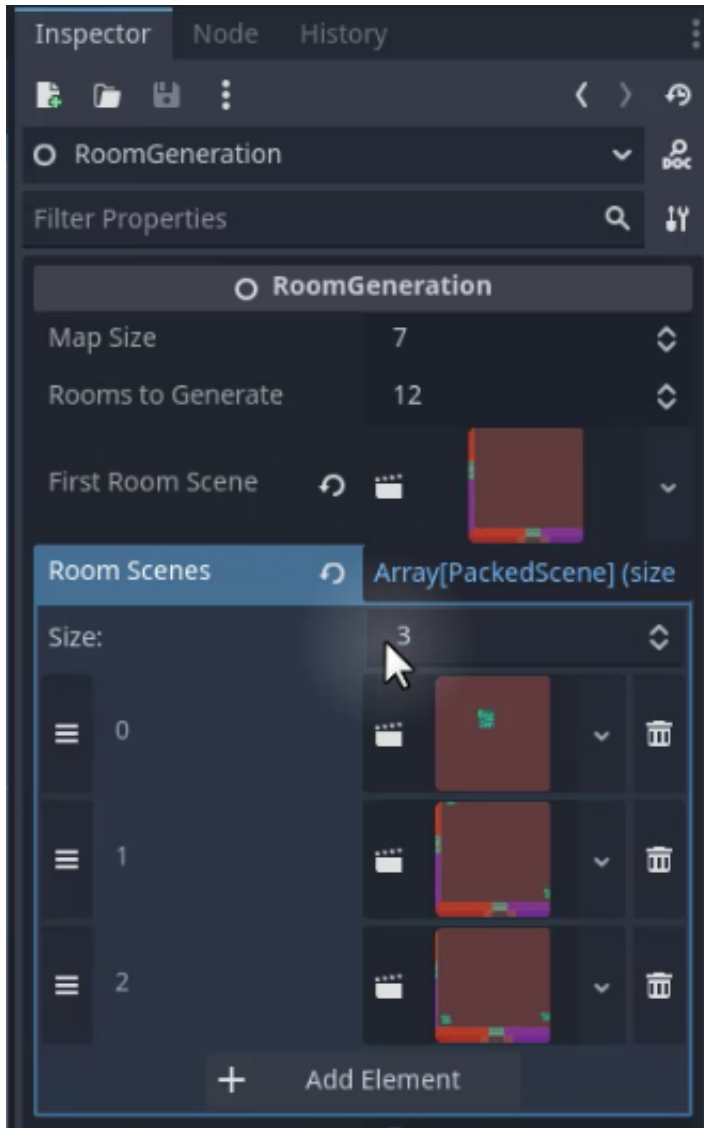
In this lesson, we will add collectible items to the game. These items will appear in special rooms and, when the player touches them, will permanently improve one of three attributes: health, movement speed, or firing rate. By the end of the lesson, you will have a reusable base scene for items and three concrete potions that the player can pick up.

### Four Enemy Room

Before we jump into our items, you should have created a new room scene from our challenge last lesson. This scene should inherit from the `room_template`. We named ours “`room_enemy4`” and placed 4 enemies in the room (one at each corner).



In order for it to be included in the game, you'll need to have selected the `RoomGeneration` node and assigned the new room to the `Room Scenes` array in the Inspector.

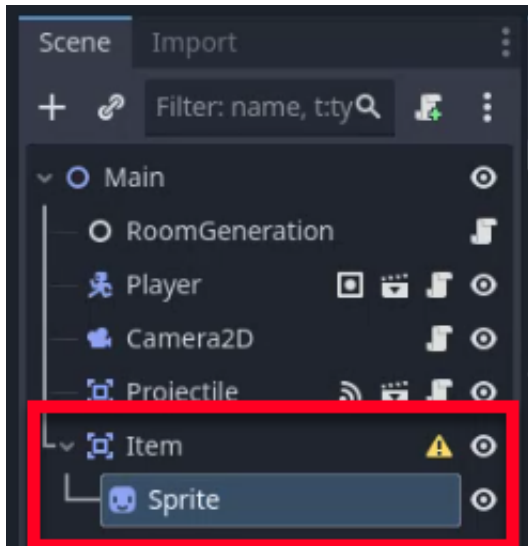


Now when you play the game, you should have a random chance of this room appearing.

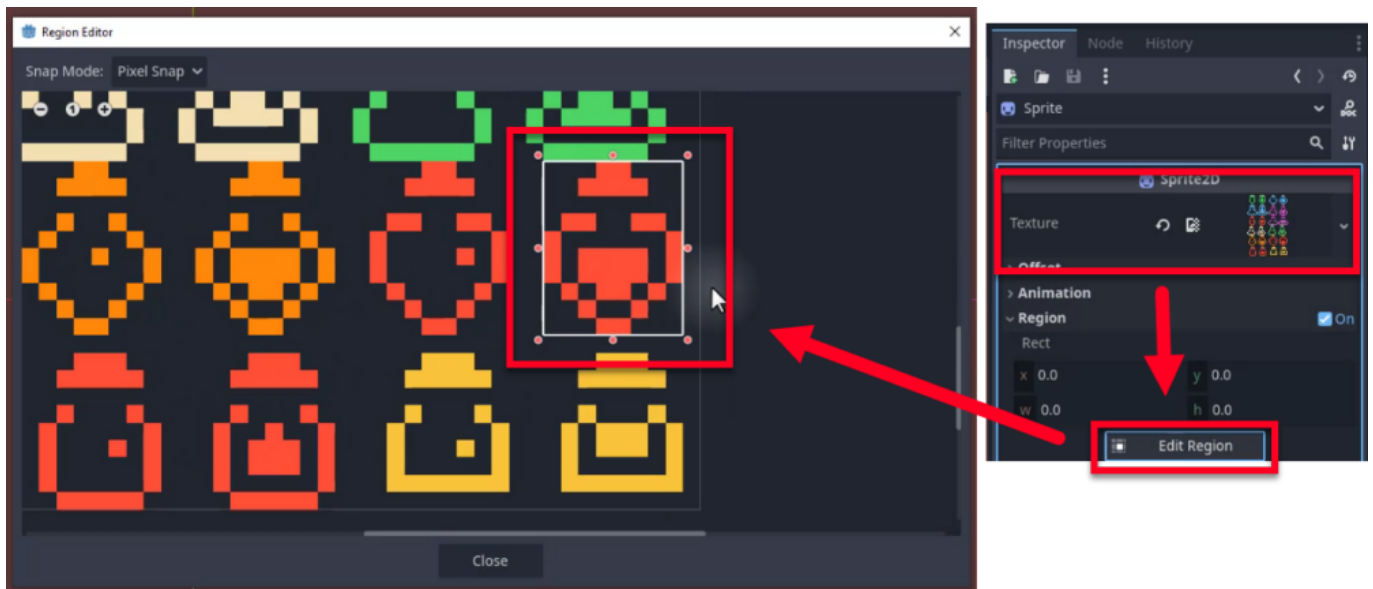
### Creating the Base Item Scene

Let's jump into our items now. We begin by creating a scene that every potion will inherit from.

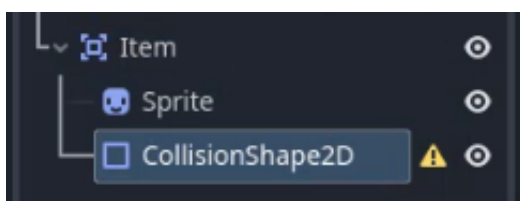
- In the **FileSystem** dock, right-click the **Scenes/Items** folder and choose **New Scene**.
- Add an **Area2D** node as the root and rename it **Item**.
- Save the scene immediately as **item.tscn** inside **Scenes/Items**.
- Add a **Sprite2D** child and rename it to "Sprite".



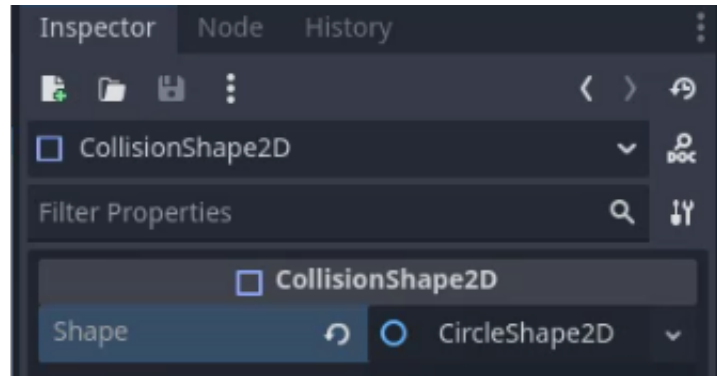
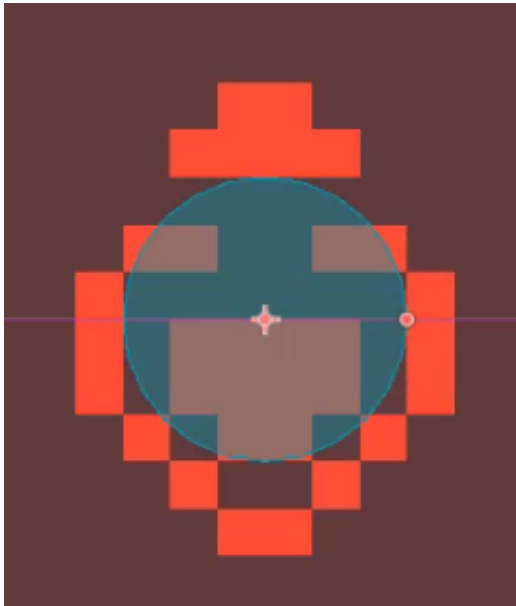
- Drag the texture **Potions.png** into its **Texture** property. Select any region you like; this will be overridden in inherited scenes.



- Add a **CollisionShape2D** child.

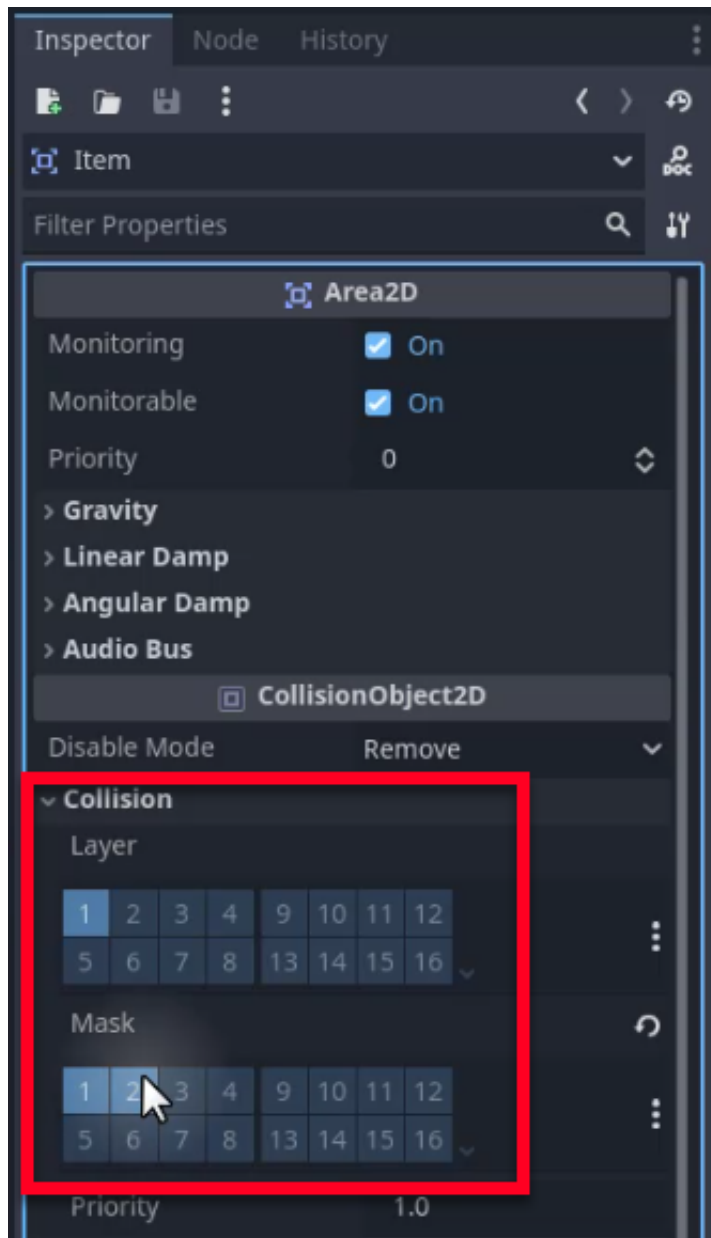


- Set its **Shape** to a **CircleShape2D**, and scale it so it encloses the sprite.

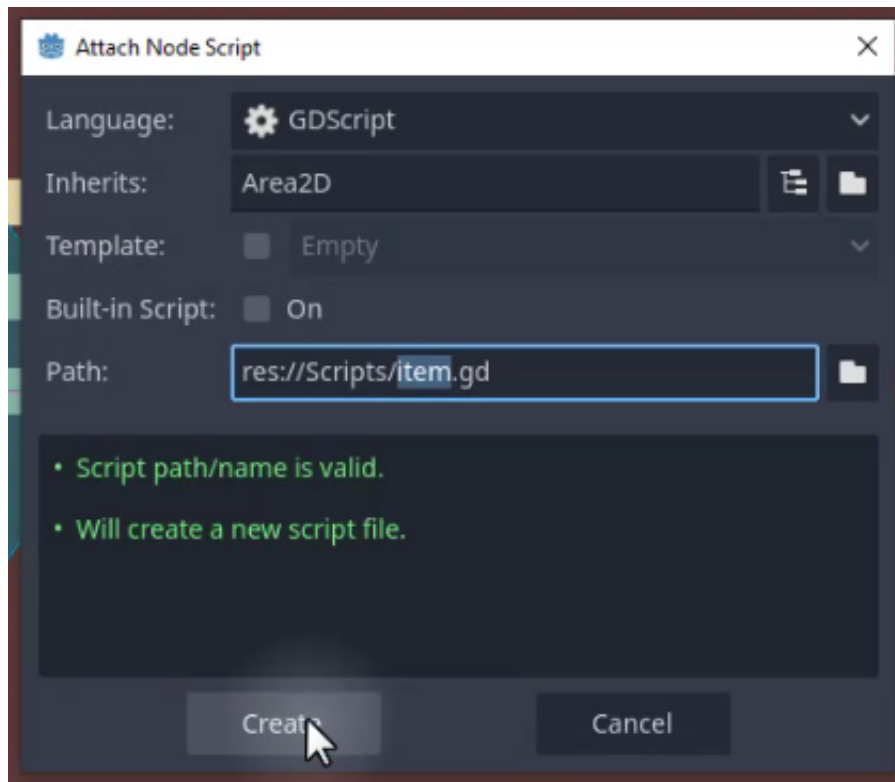


- Select the **Item** node, open the **Collision** section in the Inspector, and enable Mask **Layer 2** (Player) so the player can trigger it.





- Attach a new script called **item.gd** and save it in **Scripts**.



## Programming the Item Script

Open **Scripts/item.gd** so we can start scripting. First, we will add the necessary variables. In our game, we want three potion types:

- Health potions, which will heal the player
- Shooting potions, which will increase the shooting rate
- Move speed potions, which will increase the player's move speed

With our enum set up, we can then set up a variable to store what item type the potion is and the value it affects the stat by.

```
extends Area2D
```

```
enum ItemType
{
    HEALTH,
    SHOOT_RATE,
    MOVE_SPEED
}
```

```
@export var type : ItemType
@export var value : float
```

With our variables set up, we'll now connect up an **\_on\_body\_entered** signal to our script.



In the created function, we want to first make sure nobody but the player can trigger the activities. Then, using an else-if chain, we'll check the item type and update the states accordingly.

```
func _on_body_entered(body):  
    if not body.is_in_group("Player"):  
        return
```



```
if type == ItemType.HEALTH:
    body.heal(int(value))
elif type == ItemType.SHOOT_RATE:
    body.shoot_rate -= value
elif type == ItemType.MOVE_SPEED:
    body.move_speed += value
```

## Extending the Player Script

Next, we need to set up our player health system. In order for the player to be able to heal, they must possess health variables (similar to how we set up the enemy). Open **Scripts/player.gd** and add the following lines near the top:

```
@export var cur_hp : int = 8
@export var max_hp : int = 8
```

After this, we can add the new **heal** function after the existing **take\_damage** function. This function nearly needs to update the **cur\_hp** amount. We'll also do a check to clamp the value so it doesn't exceed **max\_hp**.

```
func heal(amount: int):
    cur_hp += amount
    if cur_hp > max_hp:
        cur_hp = max_hp
```

While we're here, let's also implement our **take\_damage** function. Similar to the enemy, this function just need to subtract from our **cur\_hp** and call the **die()** function if it is equal or less than 0.

```
func take_damage (amount : int):
    cur_hp -= amount

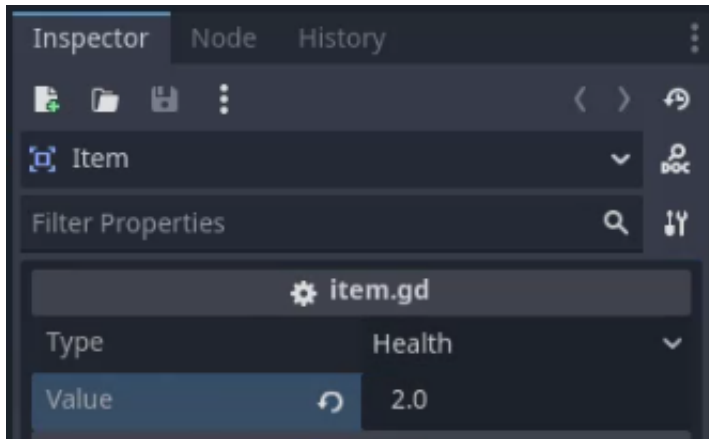
    if cur_hp <= 0:
        die()
```

## Creating Potion Scenes

We now create three inherited scenes, one for each potion type.

- **Health potion**

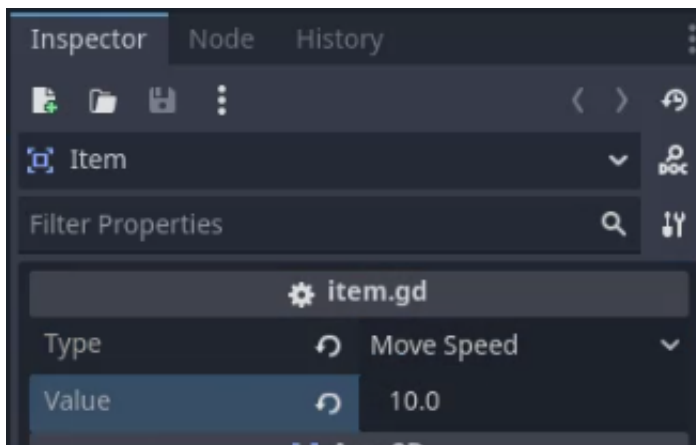
- Right-click **item.tscn** → **New Inherited Scene**.
- Save it as **item\_health.tscn**.
- Select the root **Item** node and set **Type** to **HEALTH** and **Value** to **2** in the Inspector.



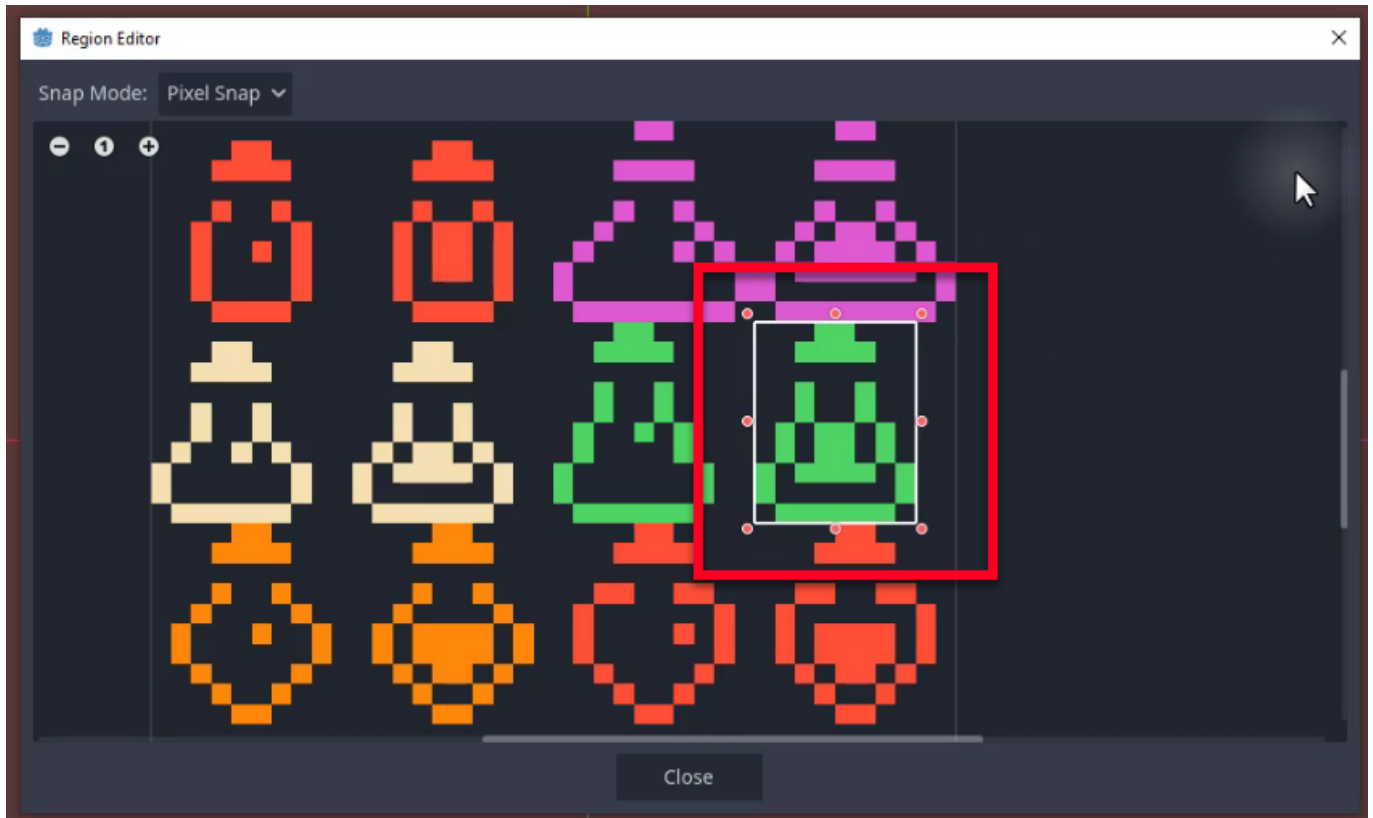
Optionally change the sprite region to a red vial (for us, we used this as the default sprite).

- **Speed potion**

- Repeat the steps above, saving as **item\_move\_speed.tscn**.
- Set **Type** to **MOVE\_SPEED** and **Value** to **10**.

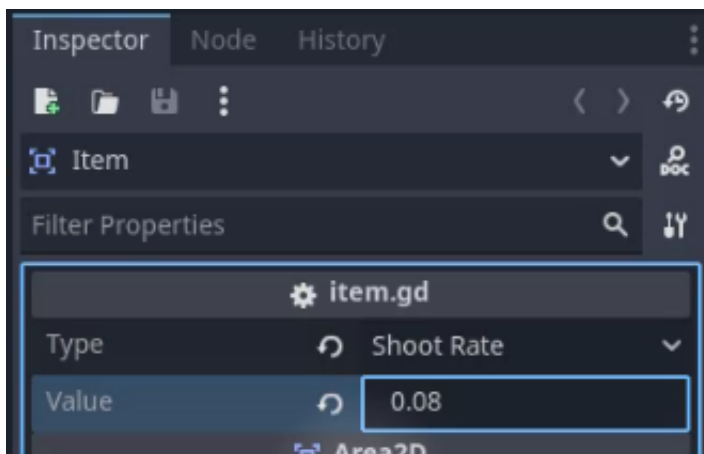


Choose a green vial sprite.

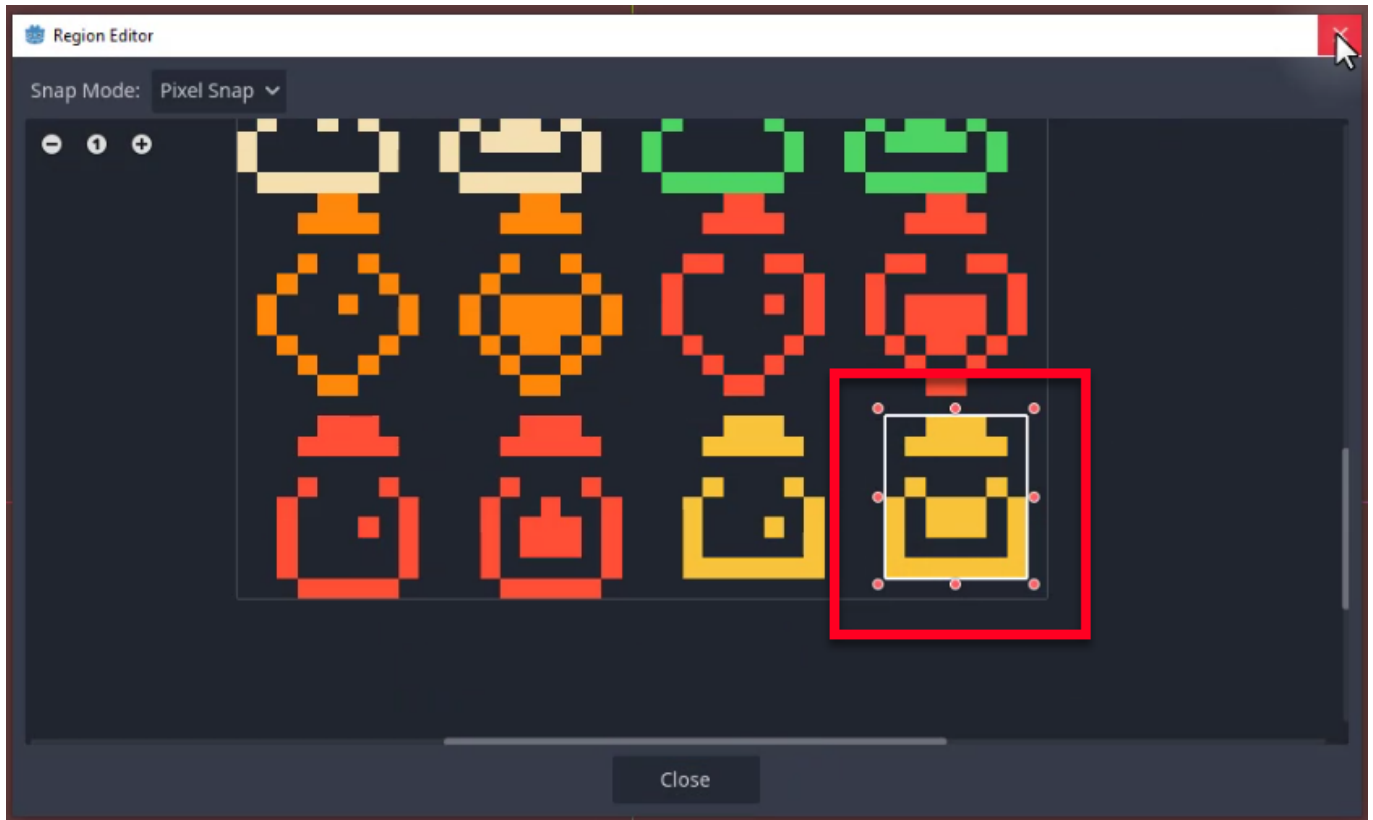


- **Fire-rate potion**

- Save as **item\_shoot\_rate.tscn**.
- Set **Type** to **SHOOT\_RATE** and **Value** to **0.08**.



Choose a yellow vial sprite.



### Challenge: Build an Item Room

As a practice exercise, create a new inherited scene from **room\_template.tscn** named **room\_item.tscn**. Place one of the potions in the center of the room, then add **room\_item.tscn** to the **room\_scenes** array in **RoomGeneration**.

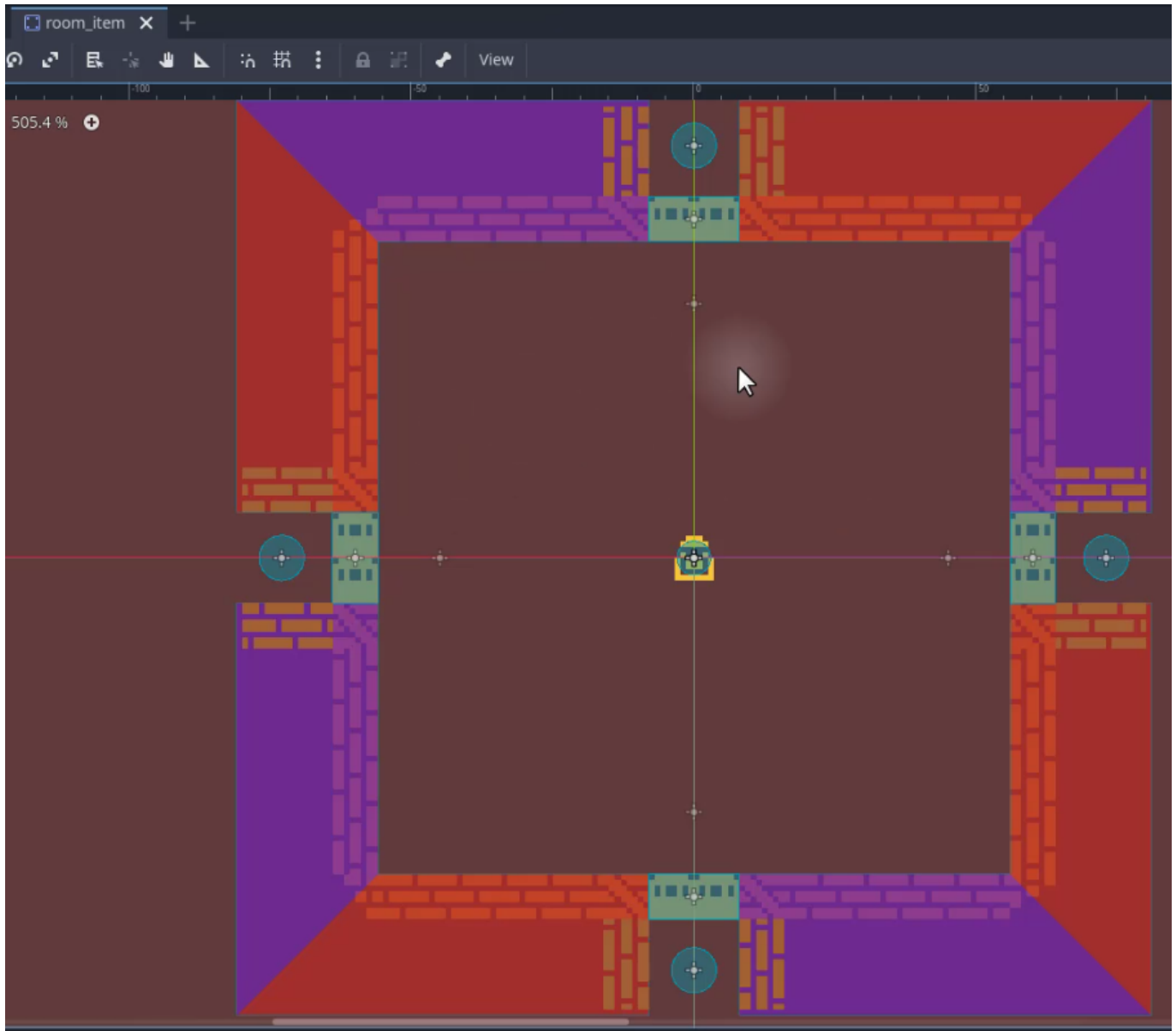
In the next lesson, we will automate the random selection of potions so that every item room spawns a different reward.

In this lesson, we will focus on creating the item room functionality in our game. We'll ensure that items are properly handled when picked up by the player and introduce a mechanism to spawn random items within the item room. By the end of this lesson, you'll have a dynamic item room that can spawn different types of items, adding variety and excitement to your game.

## Setting Up the Item Room

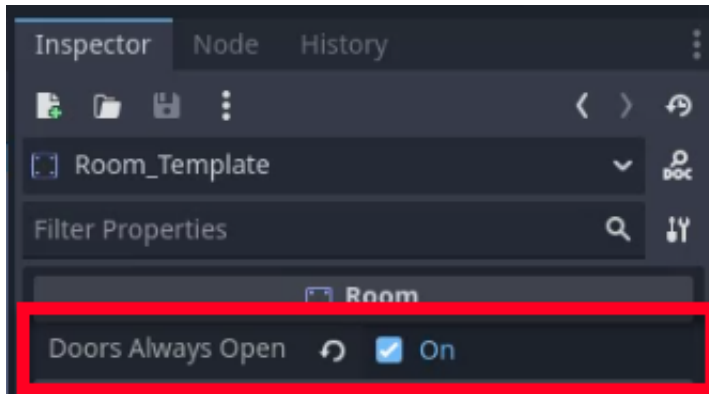
First, let's ensure our item room is correctly set up:

- Create an item room (as an Inherited Scene of the `room_template`) with an item placed in the center.

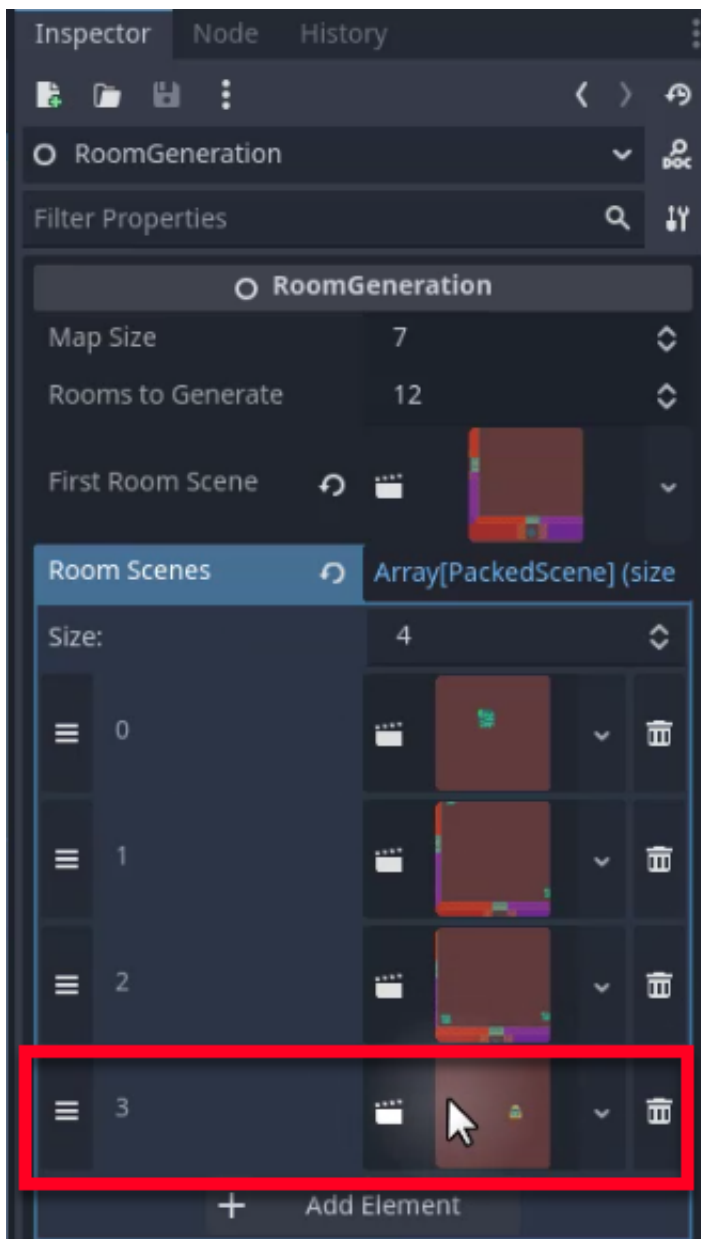


- Enable the “Doors Always Open” option on the root node to prevent doors from closing on the player.





- Add the item room to the room generation array to include it in the level generation.



## Handling Item Pickup



When the player picks up an item, we need to ensure the item is destroyed to prevent multiple pickups. Here's how to modify the item script to achieve this:

1. Open the item.gd script.
2. In the `_on_body_entered` function, after applying the item's effect to the player, add the following line to destroy the item:

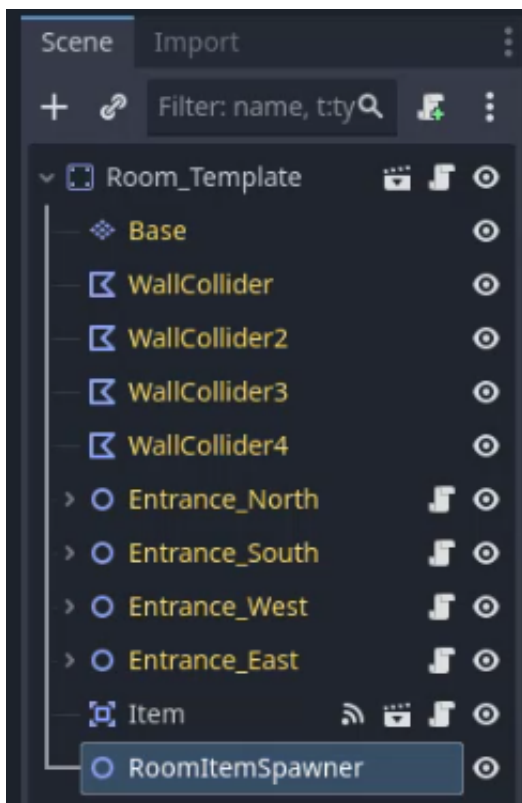
```
queue_free()
```

This ensures that the item is removed from the game world after being picked up, preventing the player from picking it up multiple times.

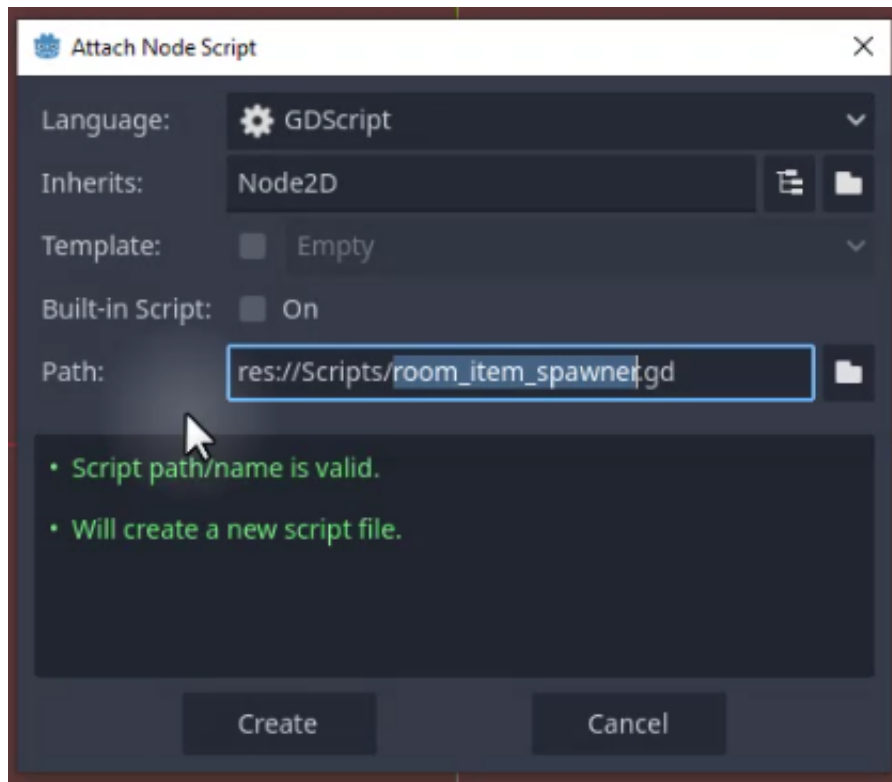
## Spawning Random Items

To add variety to our item room, we'll create a spawner that randomly selects an item to spawn. Follow these steps:

- Create a new node in the item room scene of type Node2D and name it RoomItemSpawner.



- Attach a new script to RoomItemSpawner and save it as `room_item_spawner.gd` in the scripts folder.



- In the room\_item\_spawner.gd script, define an exported variable to hold the item scenes:

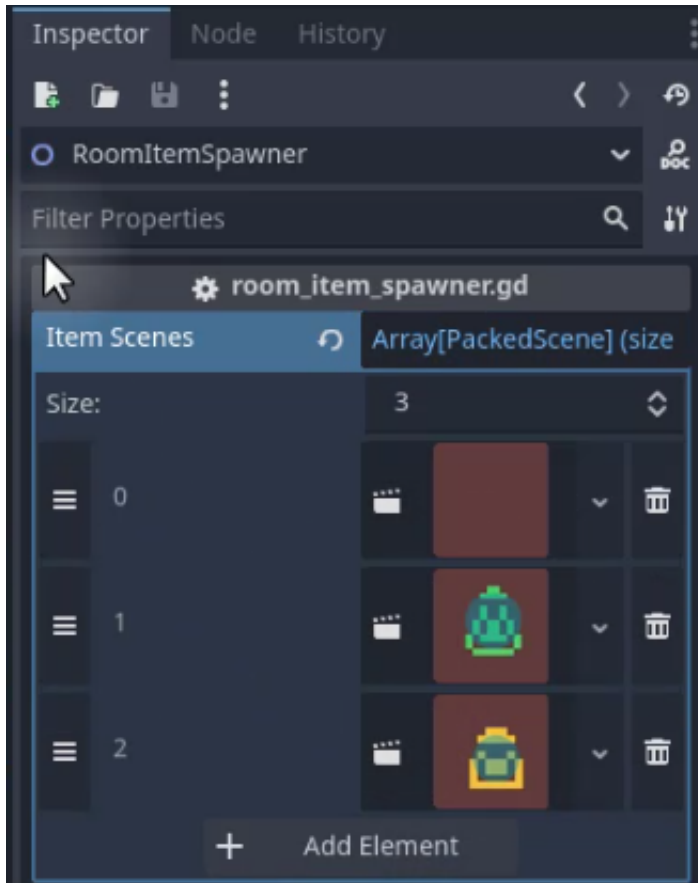
```
extends Node2D
```

```
@export var item_scenes : Array[PackedScene]
```

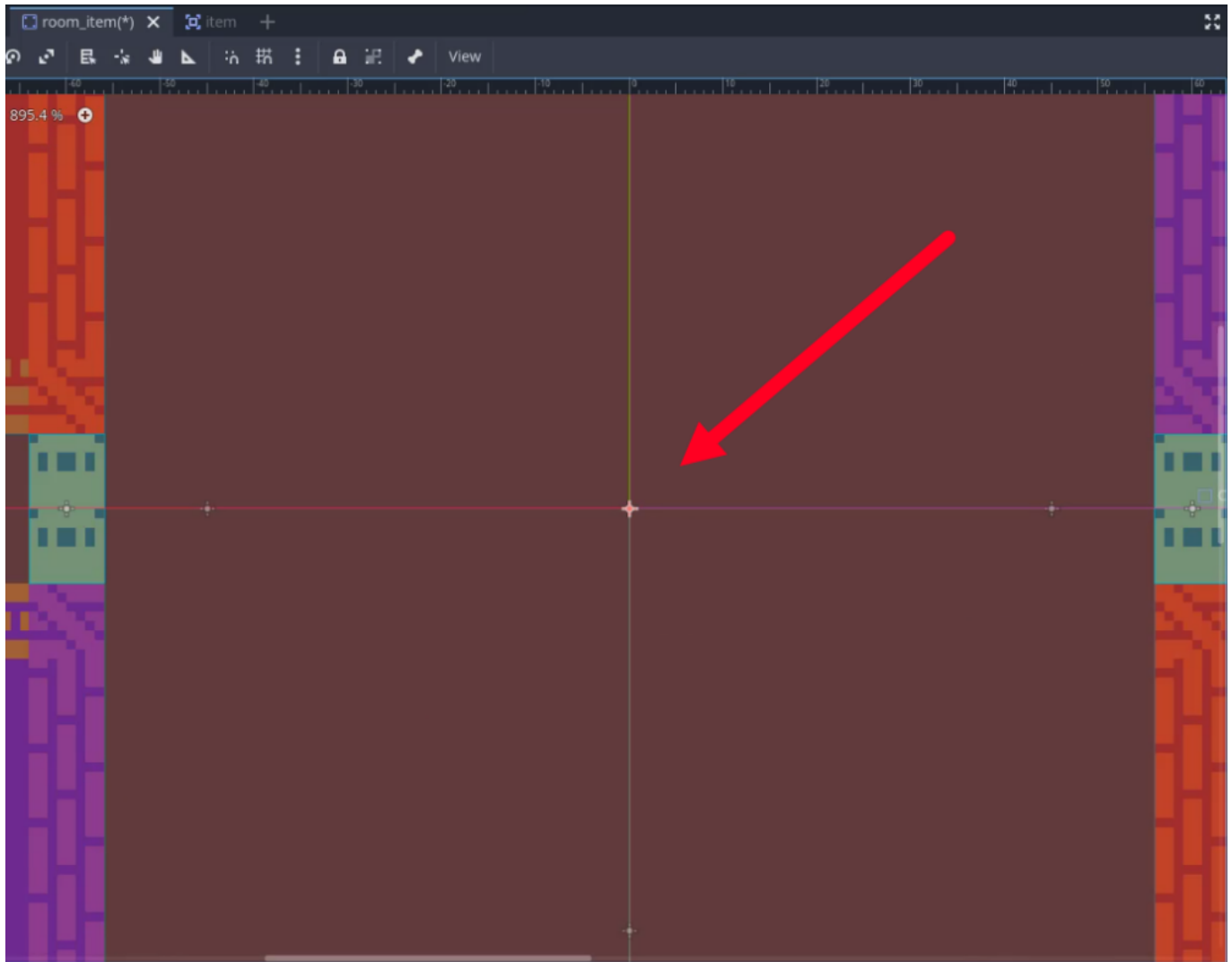
- In the \_ready function, add the following code to randomly select and instantiate an item scene:

```
func _ready():  
    var item = item_scenes[randi_range(0, item_scenes.size() - 1)].instantiate()  
    add_child(item)
```

- Back in the Godot editor, select the RoomItemSpawner node and populate the item\_scenes array with the item scenes you want to spawn.



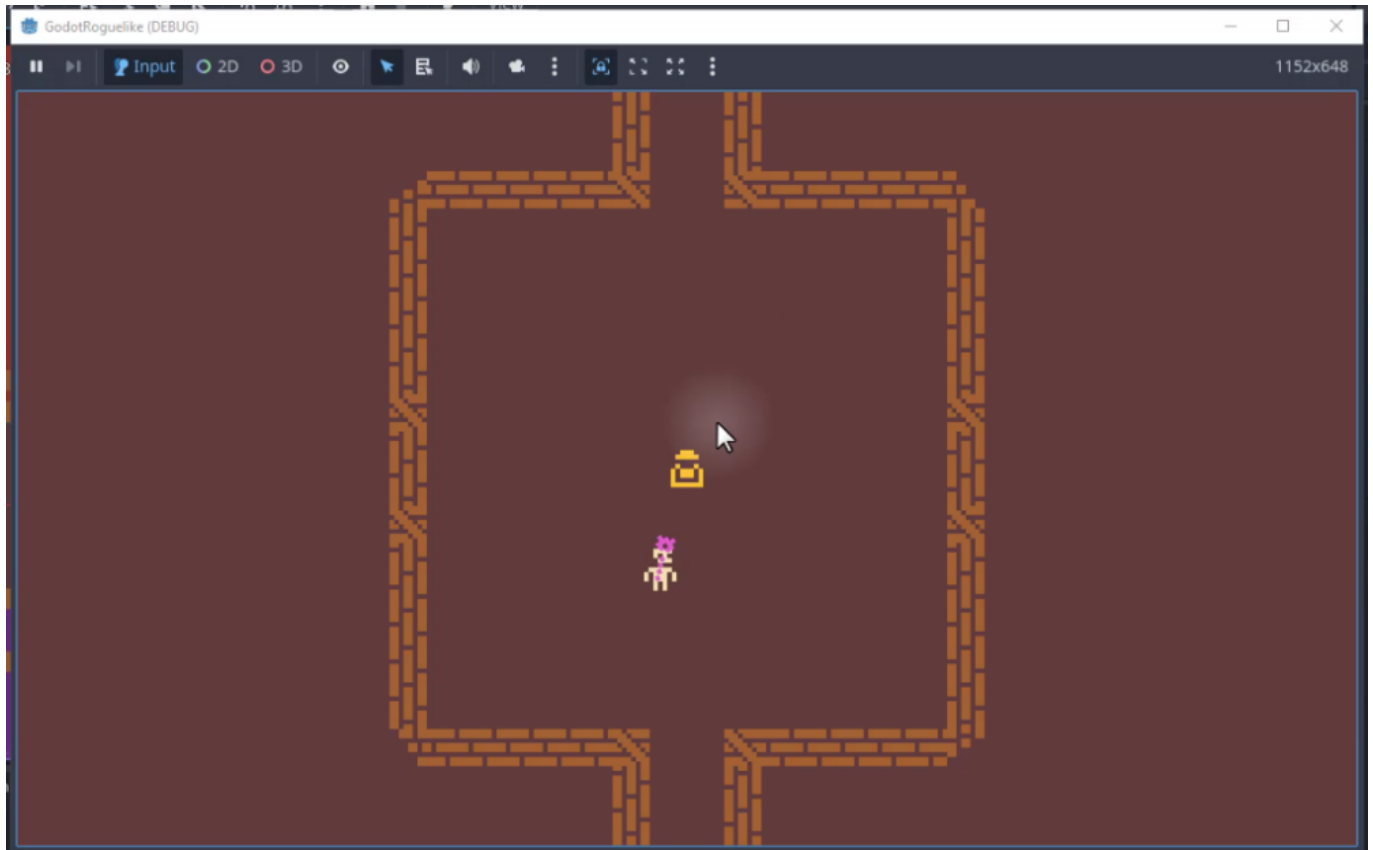
- Remove any pre-existing items from the item room and ensure the RoomItemSpawner is properly positioned to spawn items in the desired location.



## Testing the Item Room

With the setup complete, it's time to test the item room:

- Play the game and navigate to the item room.
- Verify that a random item is spawned in the room.
- Pick up the item and ensure it applies the correct effect to the player.
- Check that the item is destroyed after being picked up.



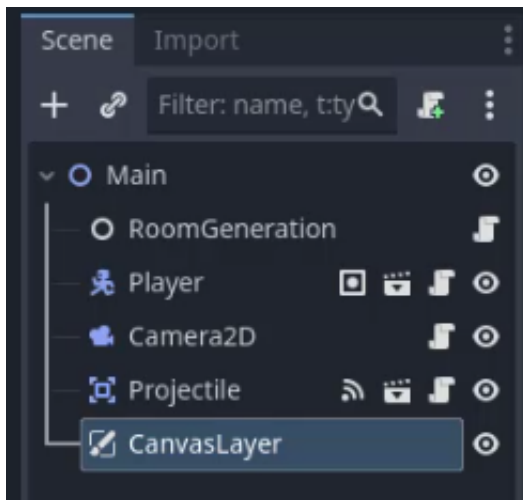
By following these steps, you'll have a functional item room that spawns random items, adding variety and excitement to your game. In the next lesson, we'll set up a mini-map to help players navigate the levels more easily.

In this lesson, we will begin setting up our UI elements, specifically focusing on creating a mini-map. The mini-map will help the player navigate through the game world by providing a visual representation of the rooms.

## Creating the Canvas Layer

To start with the mini-map, we need to create a **CanvasLayer** node. The CanvasLayer is essential because anything placed within it will be rendered to the camera, independent of world coordinates. This means the UI elements will stay fixed on the screen even as the camera moves.

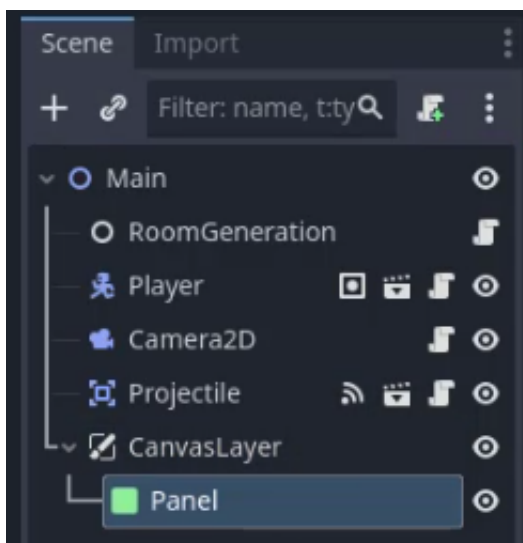
- Create a new **CanvasLayer** node.
- Anything within this CanvasLayer will be rendered to our camera, essentially pasted onto it.



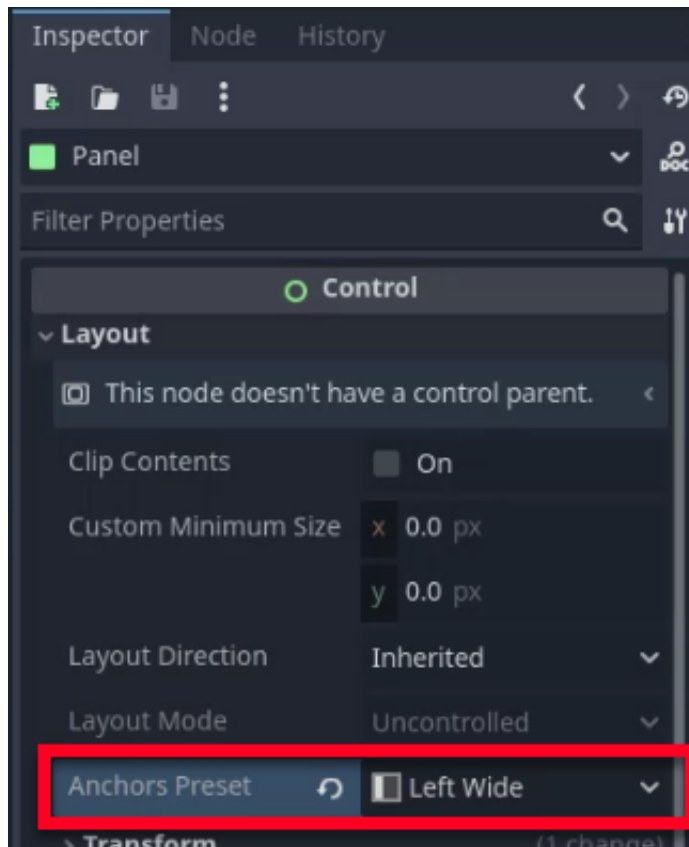
## Adding Borders for the Mini-Map

Next, we will add borders to the left and right sides of the screen. These borders will serve as containers for additional UI elements, such as the player's health and the mini-map itself.

- Add a new **Panel** node as a child of the CanvasLayer. This panel will act as the border for both the left and right sides.

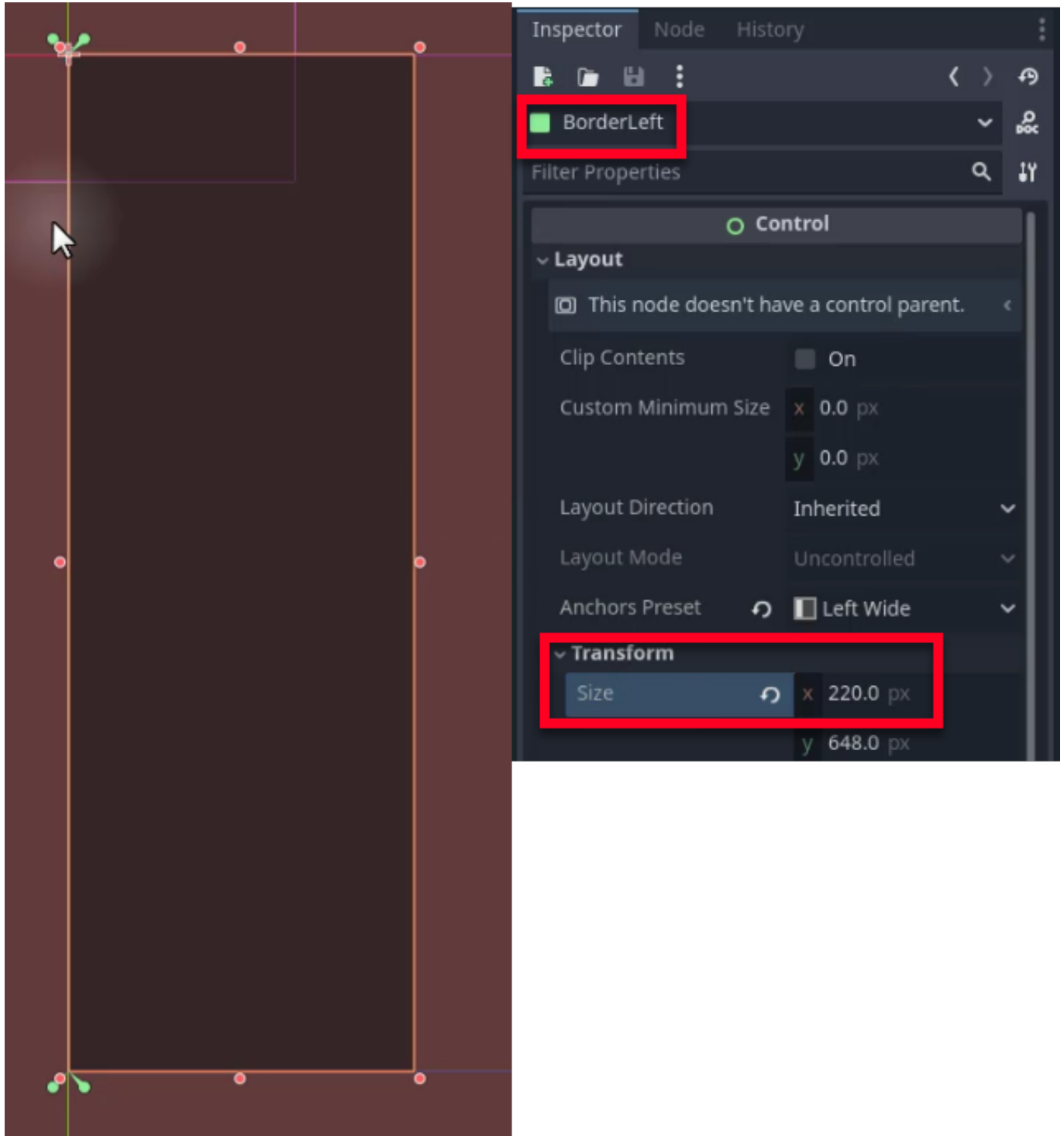


- Go to the **Layout** tab and set the **Container Sizing** to **Left Wide**.

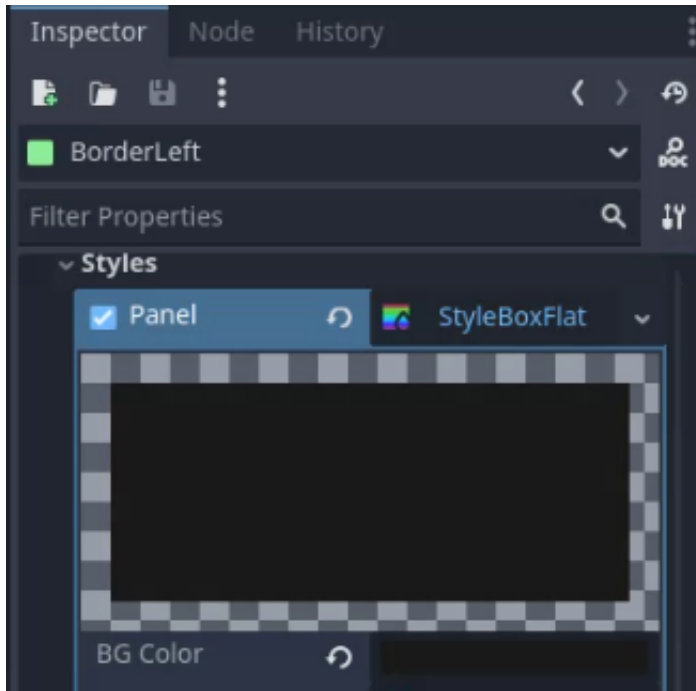


- Adjust the size of the panel by dragging it to the desired width, approximately 220 units.
- Rename the panel to **BorderLeft**.

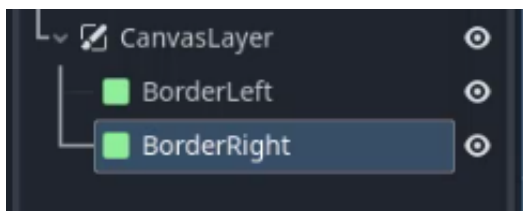




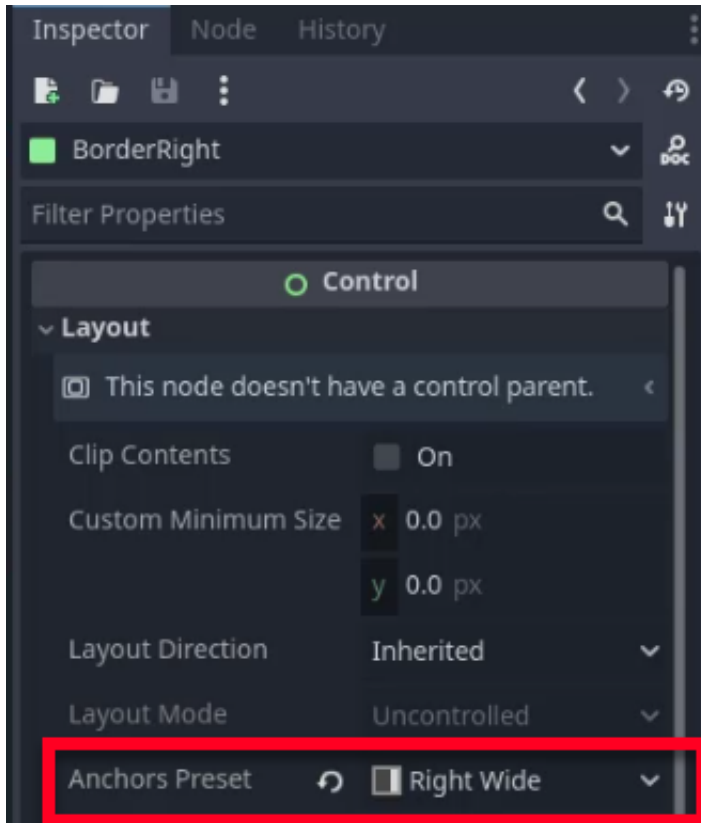
- Change the color of the border to a dark gray (#191919) by going to **Theme Overrides > Styles > Create a new StyleBoxFlat** and setting the color.



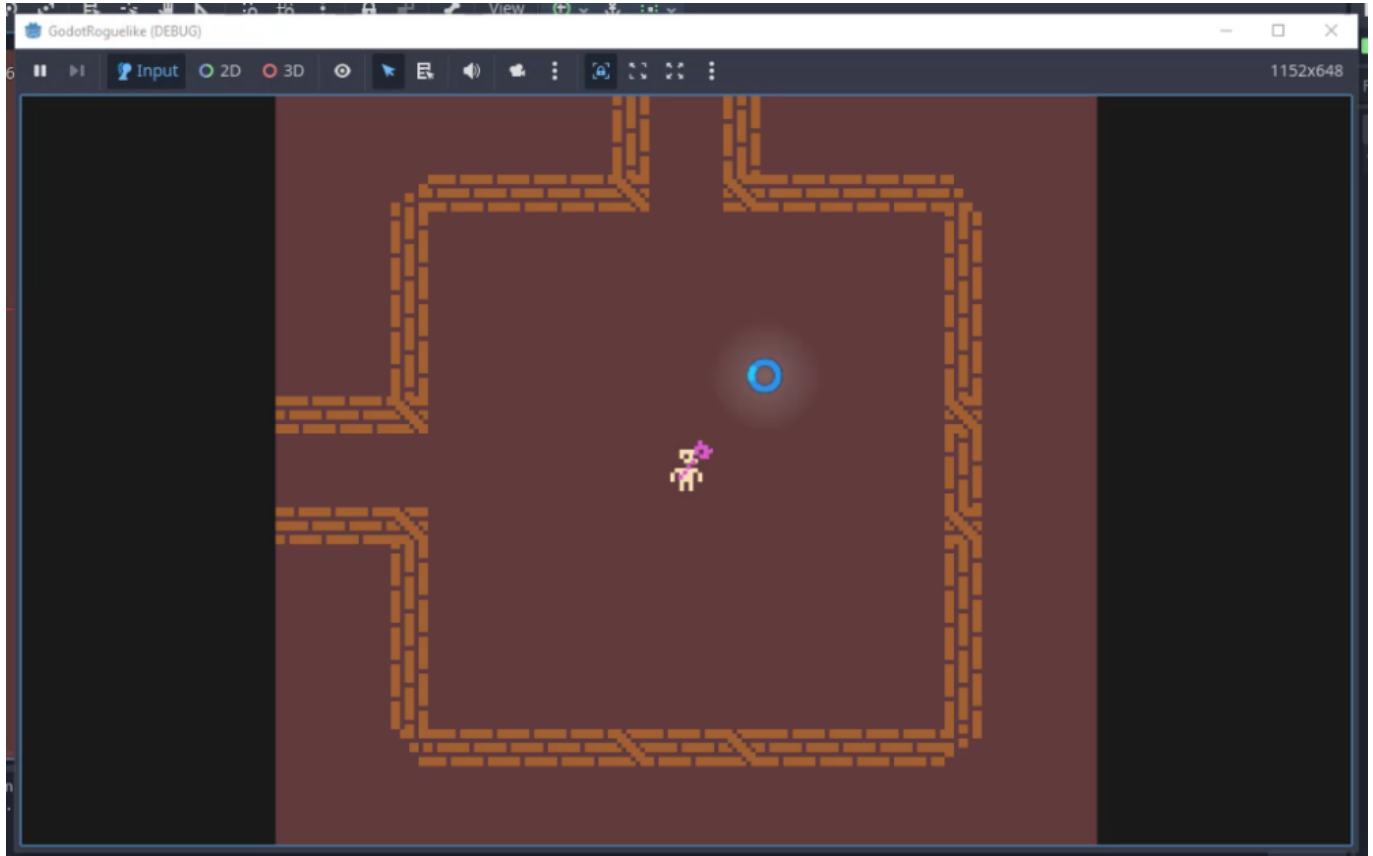
- Duplicate the **BorderLeft** panel to create a border for the right side and rename it to **BorderRight**.



- In the Inspector for BorderRight, change the Anchors Preset to **Right Wide**.



Now when you play the game, you should have a border on each side of the screen.



## Setting Up the Mini-Map Grid

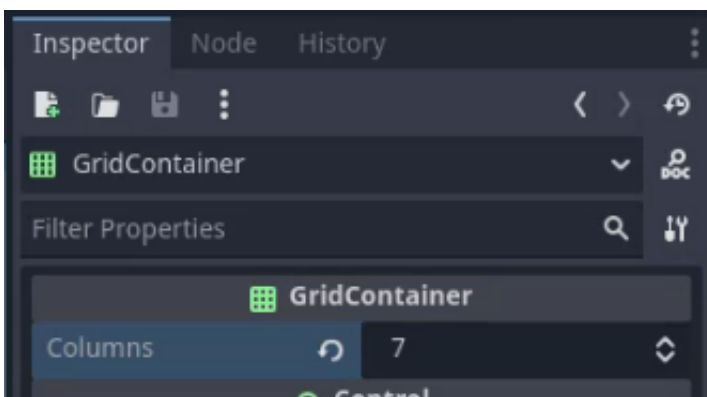


Now, let's set up the mini-map within the right border. The mini-map will be a grid of rooms, visually representing the game world.

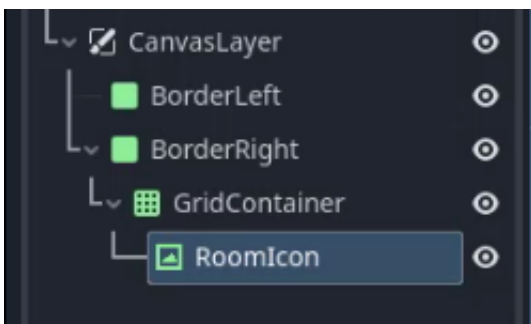
- Add a **GridContainer** node as a child of the **BorderRight** panel. The GridContainer will automatically arrange its children in a grid format.



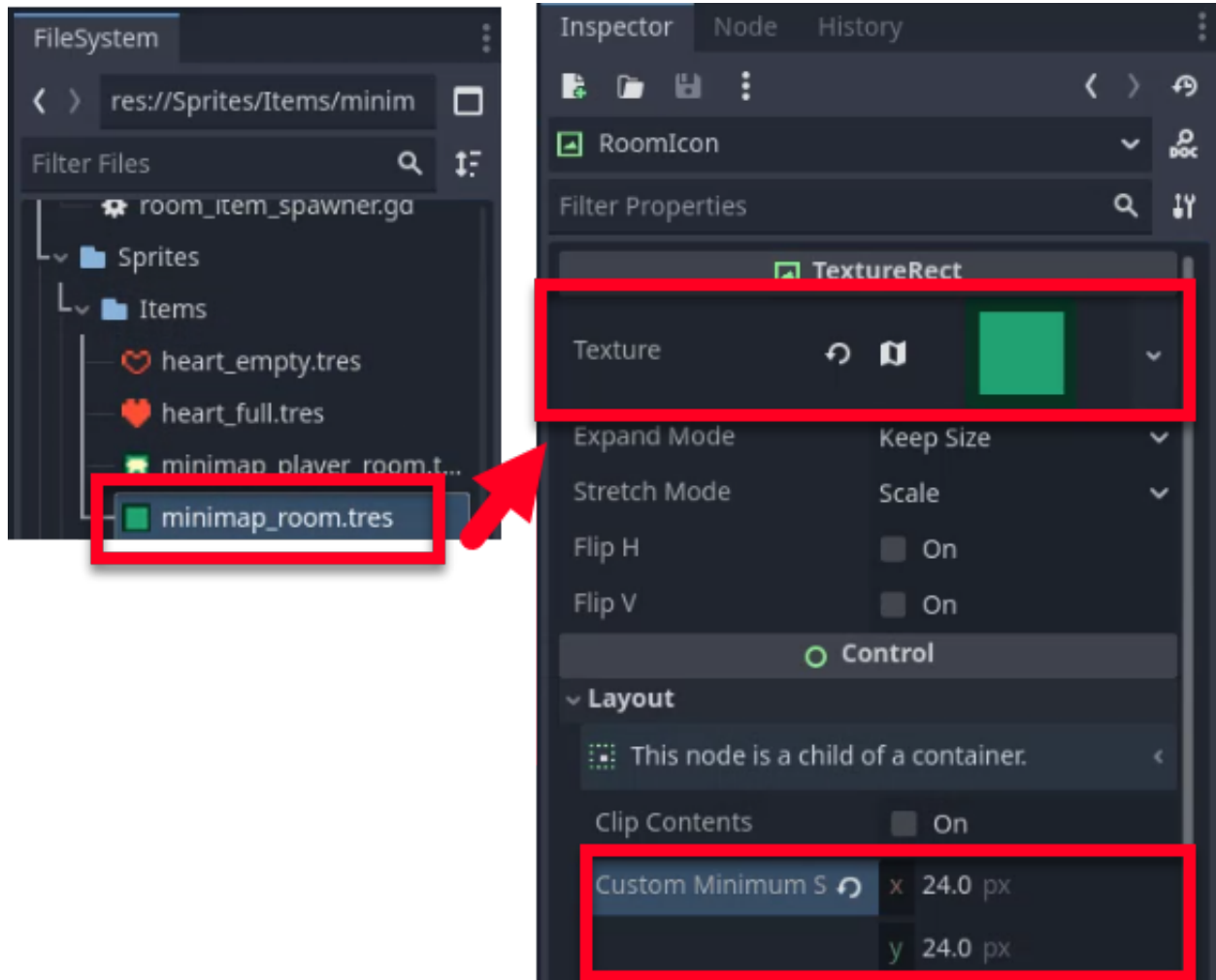
- Set the number of columns in the GridContainer to 7, matching the map size we set earlier.



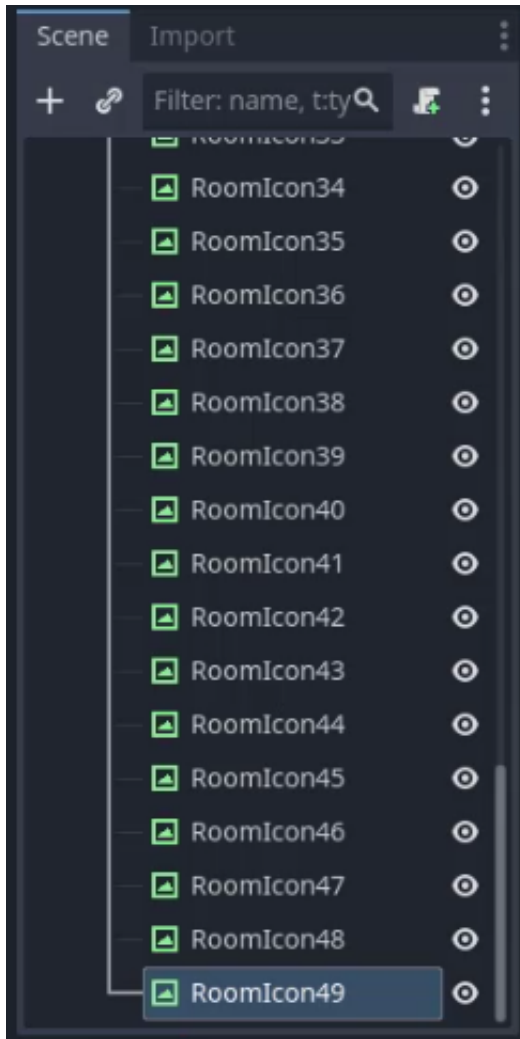
- Add a **TextureRect** node as a child of the GridContainer. This will represent a room icon.
- Rename the TextureRect to **RoomIcon**.



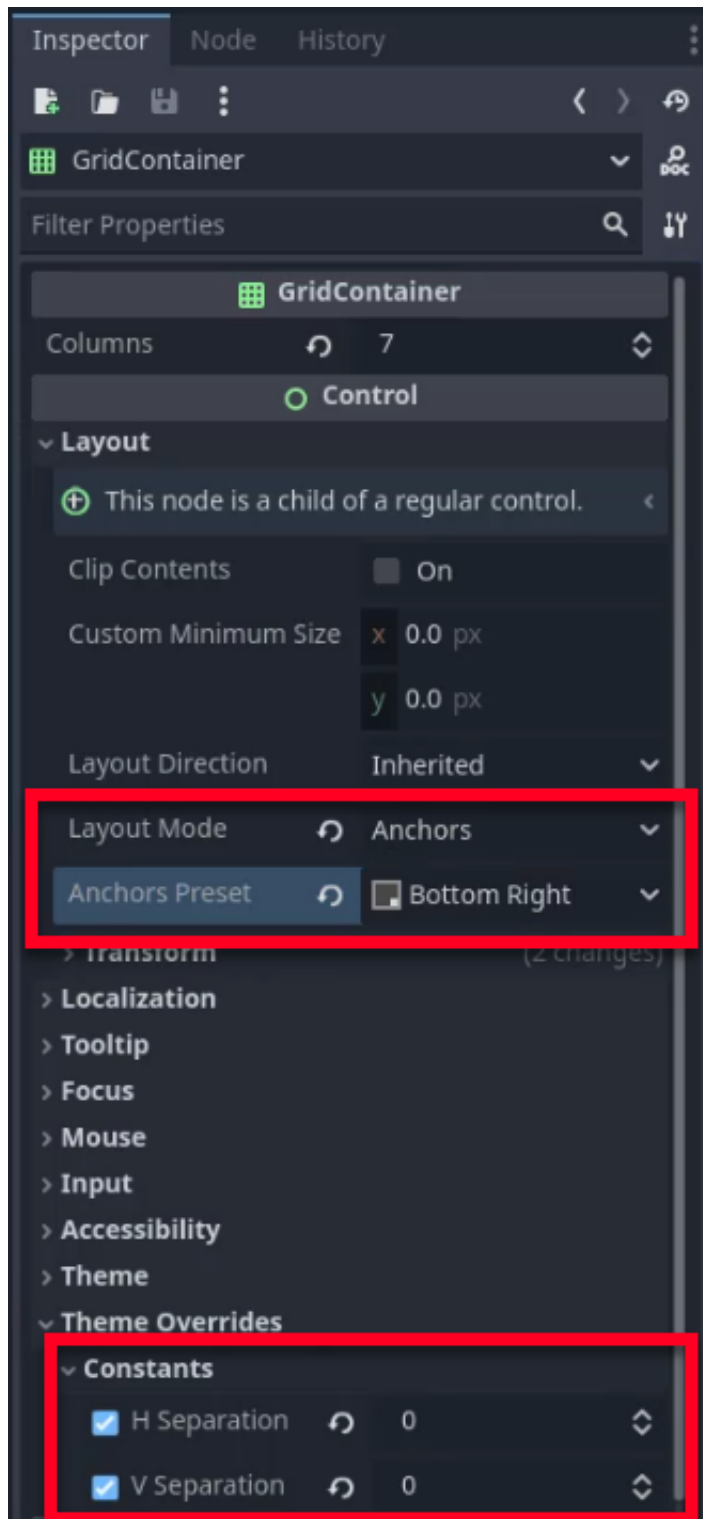
- Set the texture of the RoomIcon by dragging the **minimap\_room** texture from the **Sprites/Items** folder.
- Adjust the custom minimum size of the RoomIcon to 24×24 units to make it more visible.



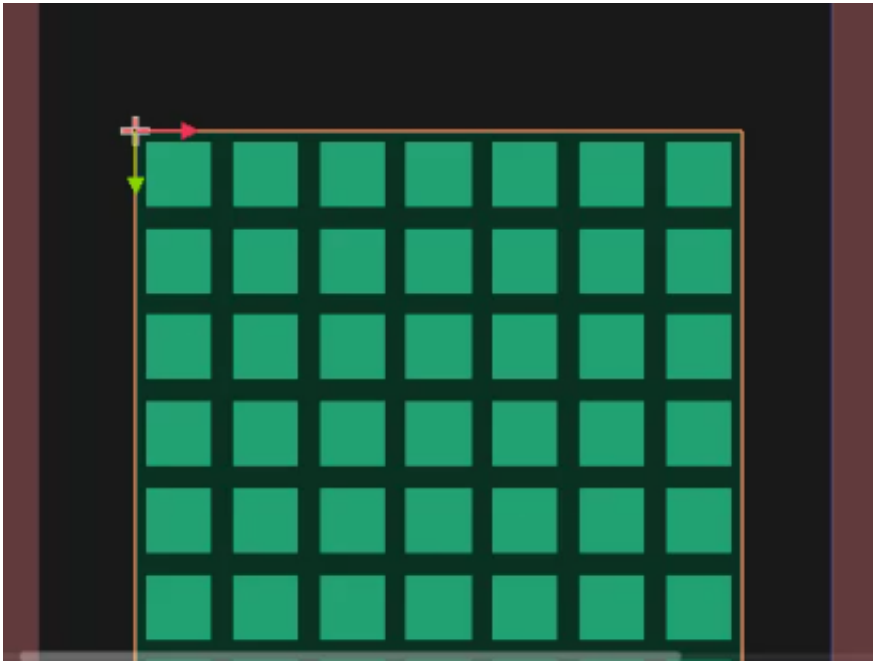
- Duplicate the RoomIcon until you have a total of 49 icons (7 rows and 7 columns).



- Remove any gaps between the room icons by setting the **H Separation** and **V Separation** to 0 pixels in the **Theme Overrides > Constants** section.
- Set the layout anchor to **Bottom Right** so the mini-map stays attached to the bottom right corner of the border.



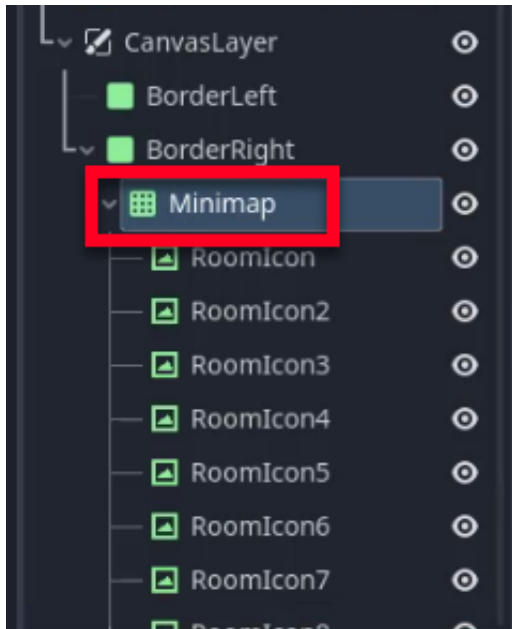
- Resize and adjust the position as you see fit so the mini-map appears nicely on the screen.



### Scripting the Mini-Map

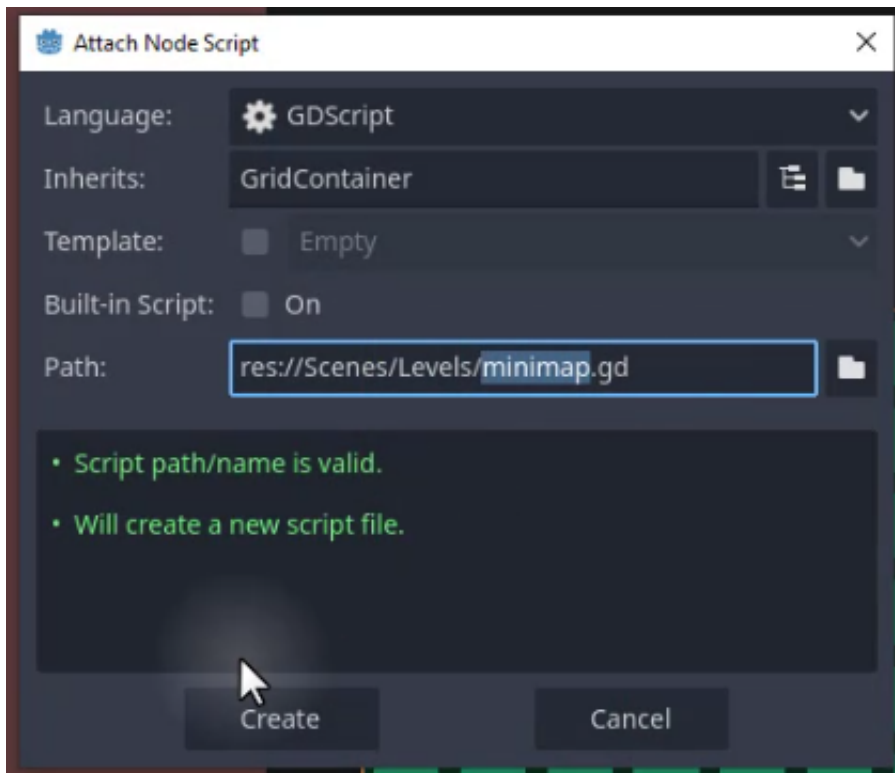
To make the mini-map functional, we need to attach a script to the GridContainer. This script will handle updating the mini-map based on the player's location.

- Rename the GridContainer to **Minimap**.



- Attach a new script to the Minimap node.





- In the script, create a variable to reference the room generation script:

```
var room_gen : RoomGeneration
```

- Create an array to keep track of all the room icons:

```
var icons : Array[TextureRect]
```

- Create variables to store the textures for the room and the player room:

```
var room_texture : Texture = preload("res://Sprites/Items/minimap_room.tres")
var player_room_texture : Texture = preload("res://Sprites/Items/minimap_player_room.tres")
```

- Create a function to update the mini-map when the player enters a new room. In this function, we want to first get a reference to the RoomGeneration and populate the icons array with all the child textures for the mini-map. We then loop through our map and check to see if there are any rooms in that location. If there aren't, we turn the icon off, if there are, we set the icon to one of our textures depending on if the player is in the room or not.

```
func _on_player_enter_room(room : Room):
    if not room_gen:
        room_gen = get_node("/root/Main/RoomGeneration")
    for child in get_children():
        if child is TextureRect:
            icons.append(child)

    for x in range(room_gen.map_size):
        for y in range(room_gen.map_size):
            var r = room_gen.get_room_from_map(x, y)
            var i = x + y * room_gen.map_size
```

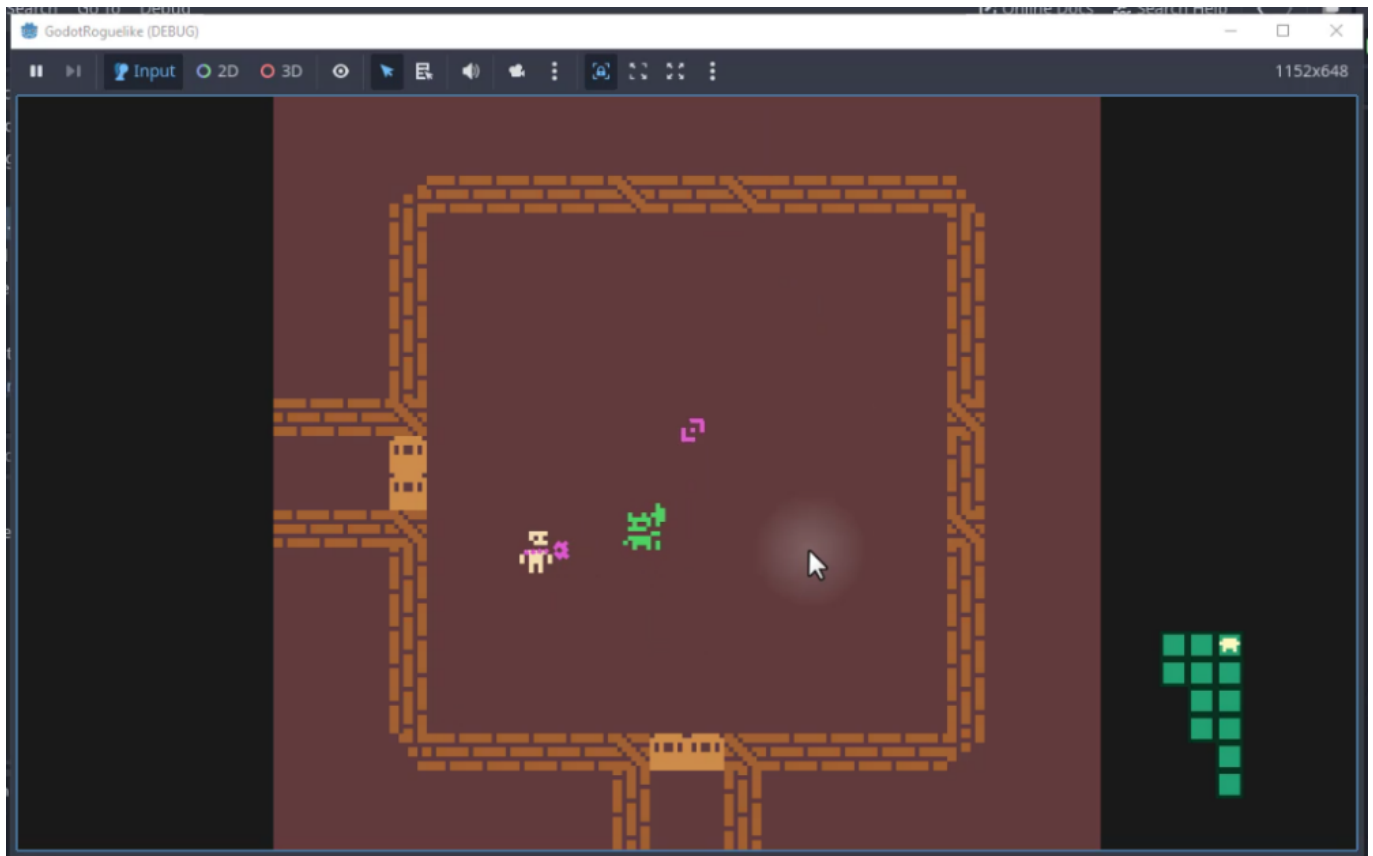


```
if i >= len(Icons):  
    continue  
if not r:  
    Icons[i].texture = null  
elif r == room:  
    Icons[i].texture = player_room_texture  
else:  
    Icons[i].texture = room_texture
```

- Connect the **\_on\_player\_enter\_room** function to the global signal **OnPlayerEnterRoom** in the **\_ready** function:

```
func _ready():  
    GlobalSignals.OnPlayerEnterRoom.connect(_on_player_enter_room)
```

With these steps, you have successfully set up the mini-map UI element. The mini-map will now update based on the player's location, providing a visual representation of the game world.



In the next lesson, we will set up the player's health system, including the ability to take damage and display the health on the left side of the screen.

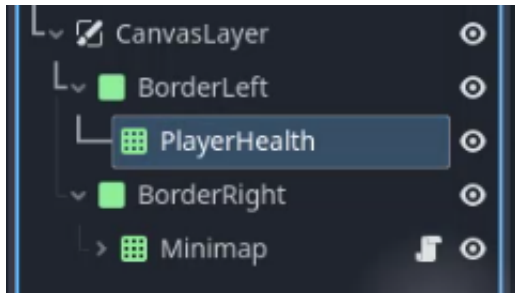


In this lesson, we will set up the player's health display in our game. The player's health will be represented by a series of heart icons in the top left corner of the screen. Each heart will be full if the player has that health point, and empty if the player has taken damage. Let's go through the steps to achieve this.

## Creating the Grid Container

First, we need to create a new GridContainer node to hold our heart icons. We'll name this node PlayerHealth.

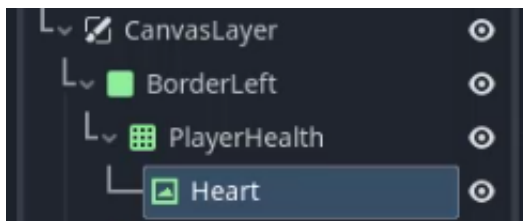
- Create a new GridContainer node as a child of the BorderLeft node.
- Name this node PlayerHealth.



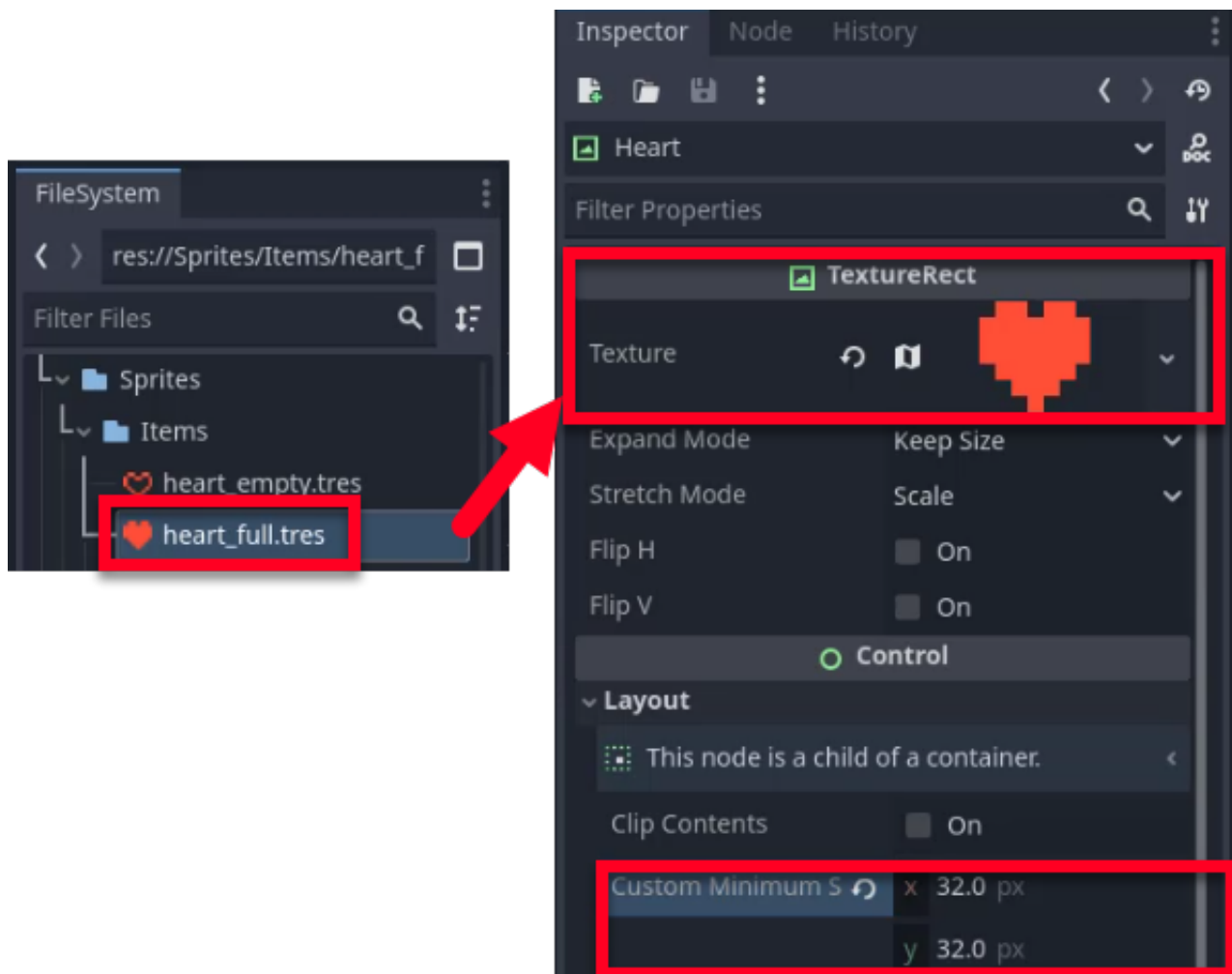
## Adding Heart Icons

Next, we'll add the heart icons as children of the PlayerHealth node.

- Create a new node of type TextureRect as a child of PlayerHealth.
- Name this node Heart.

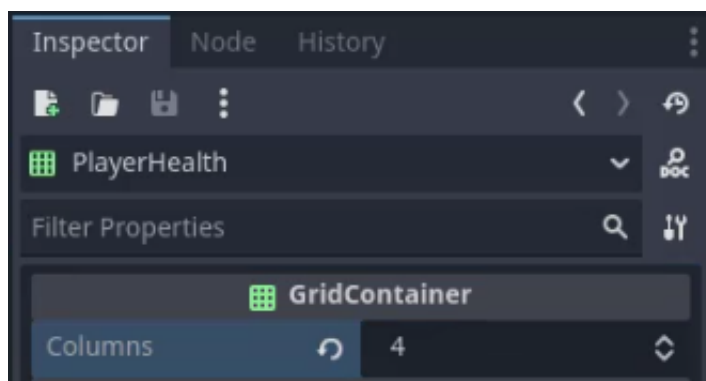


- Set the texture of this TextureRect to the heart\_full.tres texture.
- Adjust the size of the heart icon to 32×32 pixels using the Layout menu and setting a custom size.

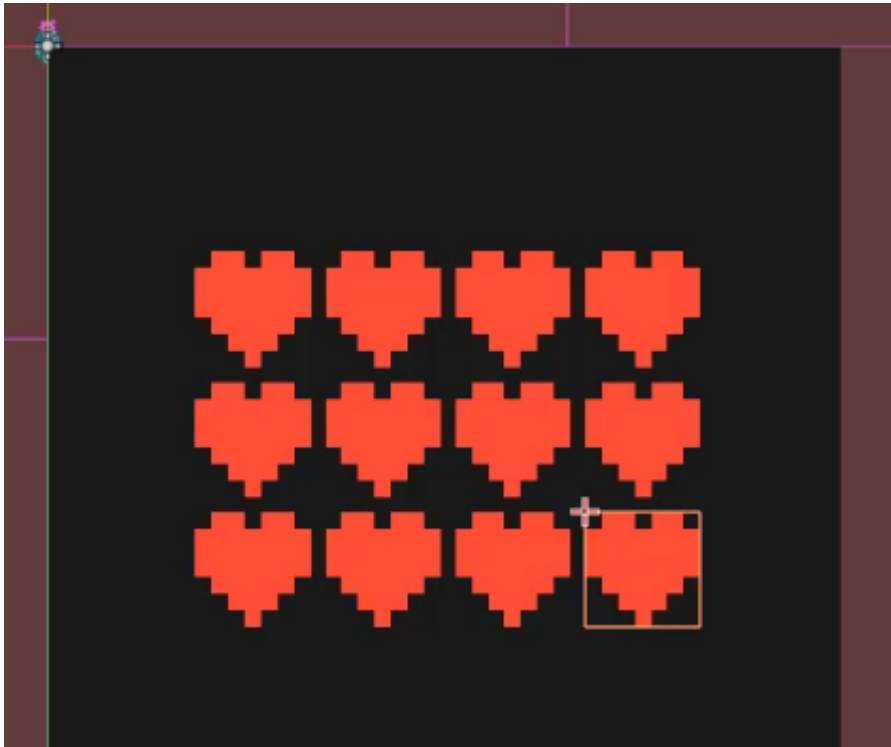


To create multiple heart icons, duplicate the Heart node. However, you may notice that the duplicated hearts are pasted in a single column. To fix this, change the number of columns in the GridContainer to 4.

- Select the PlayerHealth node.
- In the Inspector, set the Columns property to 4.

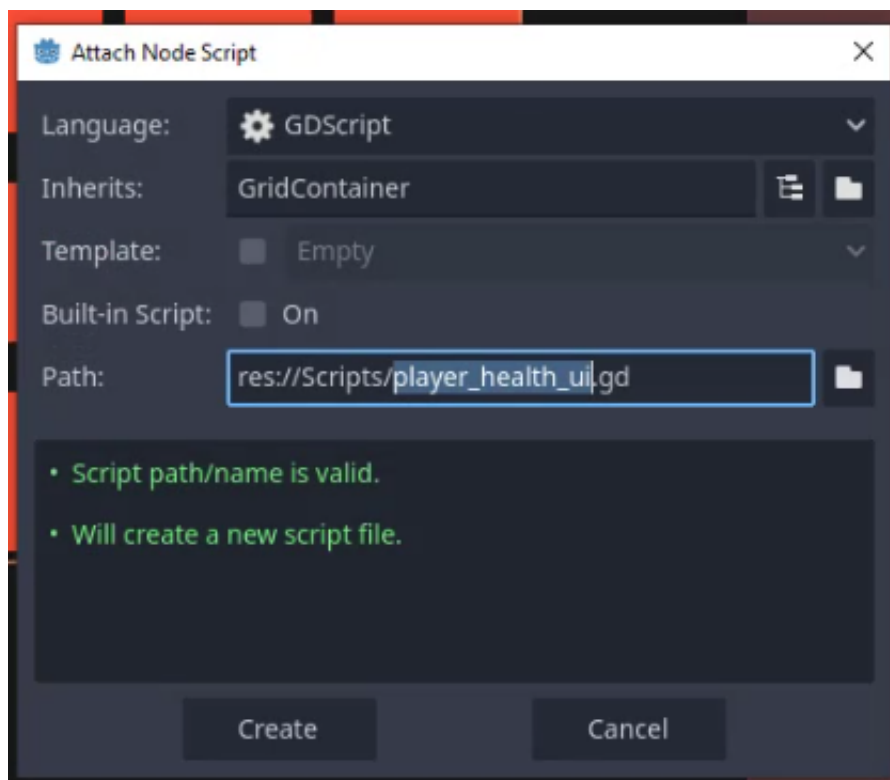


- Duplicate the Heart node until you have 12 heart icons in the grid. Resize and position the grid itself as needed.



### Creating the Player Health UI Script

Now, let's create a script to manage the player's health UI. This script will handle updating the heart icons based on the player's current and maximum health. First, attach a new script to the PlayerHealth node and name it `player_health_ui.gd`.



In this script, we will first set up our variables. In this case, we need references to our full and empty



heart textures, and an array to store all our heart icons.

```
extends GridContainer
```

```
var full_heart : Texture = preload("res://Sprites/Items/heart_full.tres")
var empty_heart : Texture = preload("res://Sprites/Items/heart_empty.tres")
```

```
var heart_icons : Array[TextureRect]
```

Next, we'll set up a **\_ready** function that loops through all the children and adds the number of appropriate heart icons as children in our array.

```
func _ready():
    for child in get_children():
        if child is TextureRect:
            heart_icons.append(child)
```

We also need a function to update our UI. In this function, we'll loop through our **heart\_icons** array, dynamically toggling the visibility of our icons based on our player's health, as well as selecting the correct texture.

```
func _update_ui (cur_hp : int, max_hp : int):
    for i in len(heart_icons):
        if i >= max_hp:
            heart_icons[i].visible = false
            continue

        heart_icons[i].visible = true

        if i < cur_hp:
            heart_icons[i].texture = full_heart
        else:
            heart_icons[i].texture = empty_heart
```

## Connecting the Signal

To ensure the UI updates when the player's health changes, we need to connect to a global signal. This signal will be emitted whenever the player's health is updated.

1. Open the `global_signals.gd` script.
2. Add a new signal for updating the player's health:

```
signal OnPlayerUpdateHealth(cur_hp : int, max_hp : int)
```

Next, we need to emit this signal from the player script whenever the player's health changes.



1. Open the player.gd script.
2. In the take\_damage and heal functions, add the following line to emit the signal:

```
GlobalSignals.OnPlayerUpdateHealth.emit(cur_hp, max_hp)
```

## Updating the UI at the Start of the Game

Finally, we need to ensure the UI is updated at the start of the game to reflect the player's initial health.

1. In the player.gd script, add the following line in the \_ready function to emit the signal at the start of the game:

```
GlobalSignals.OnPlayerUpdateHealth.emit.call_deferred(cur_hp, max_hp)
```

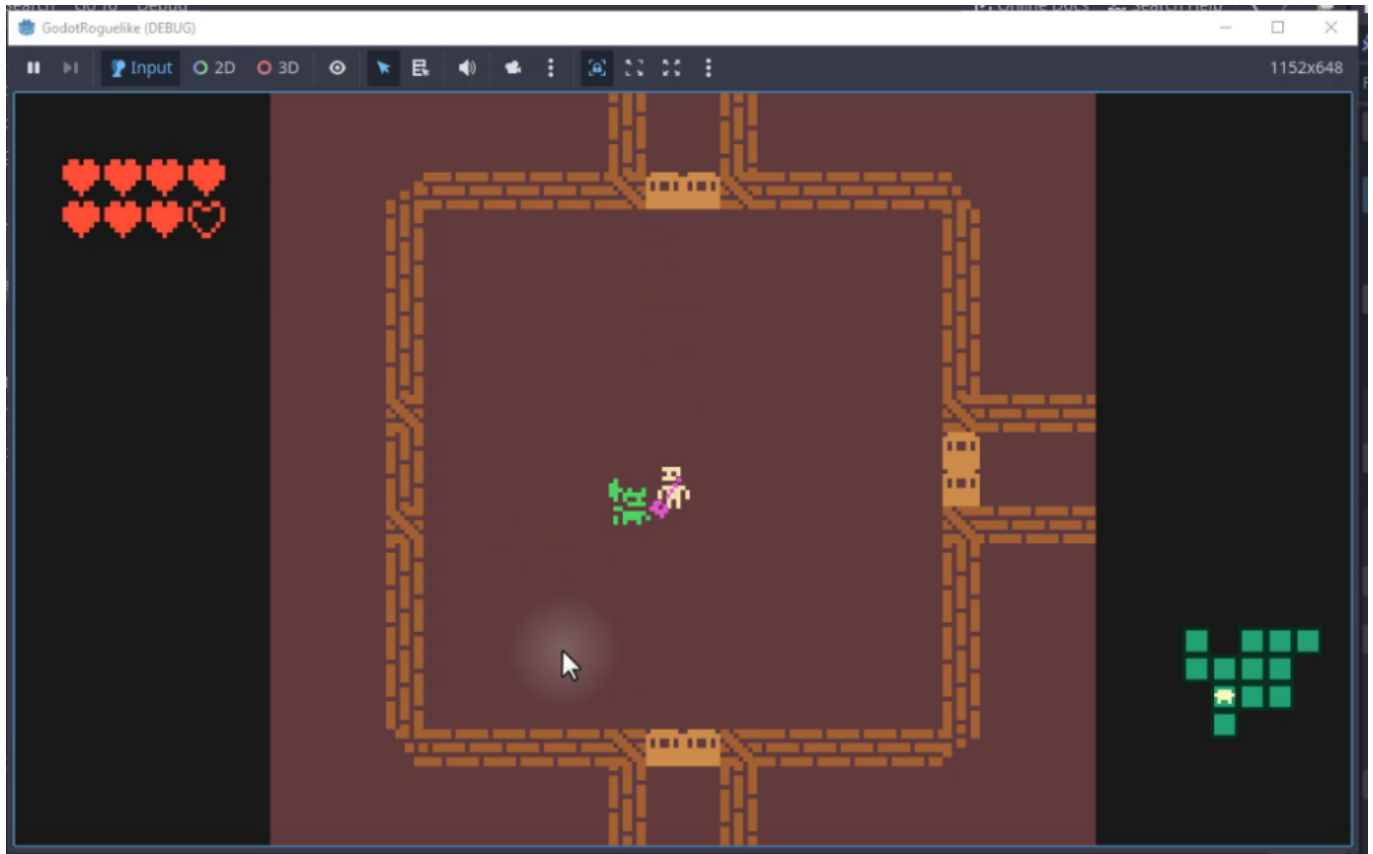
However, we need to ensure this signal is emitted after the UI has been initialized. To do this, we'll use the \_init function in the player\_health\_ui.gd script to connect the signal before the player script emits it.

1. In the player\_health\_ui.gd script, put the signal connection code to the \_init function:

```
func _init():  
    GlobalSignals.OnPlayerUpdateHealth.connect(_update_ui)
```

This ensures the signal is connected before the player script emits it, allowing the UI to update correctly at the start of the game.

With these steps, you have successfully set up the player's health display in your game. The heart icons will update based on the player's current and maximum health, providing a clear visual representation of the player's health status.



In the next lesson, we'll add some game juice by making a damage flash so we can more easily tell when someone has been hit.





In this lesson, we will be implementing a visual effect called **Damage Flash**. This effect will make the player or enemy sprite flash a certain color whenever they take damage. This adds a crucial visual feedback mechanism to your game, enhancing the overall gaming experience.

## Understanding Damage Flash

Damage Flash is a simple yet effective visual cue that indicates when a character (player or enemy) has taken damage. By briefly making the sprite invisible and then visible again, we create a flashing effect that simulates the impact of damage.

## Implementing Damage Flash for Enemies

Let's start by implementing the Damage Flash effect for enemies. Follow these steps:

1. Open the **enemy** script.
2. Create a new function called `_damage_flash`. This function will handle the flashing effect. In this function, we simply need to toggle the visibility, using a timer to pause the function in between the toggling.

```
func _damage_flash():  
    visible = false  
    await get_tree().create_timer(0.07).timeout  
    visible = true
```

3. To integrate this function into the enemy's behavior, call `_damage_flash` within the `take_damage` function. This ensures that the flash effect occurs whenever the enemy takes damage.

Here is how you can modify the `take_damage` function:

```
func take_damage(amount: int):  
    cur_hp -= amount  
    _damage_flash()  
  
    if cur_hp <= 0:  
        die()
```

## Implementing Damage Flash for the Player

Now, let's apply the same Damage Flash effect to the player. Follow these steps:

1. Open the **player** script.
2. Copy the `_damage_flash` function from the enemy script and paste it into the player script.

Here is the `_damage_flash` function for the player, the same as before:

```
func _damage_flash():  
    visible = false  
    await get_tree().create_timer(0.07).timeout  
    visible = true
```

3. Call the `_damage_flash` function within the `take_damage` function of the player script.



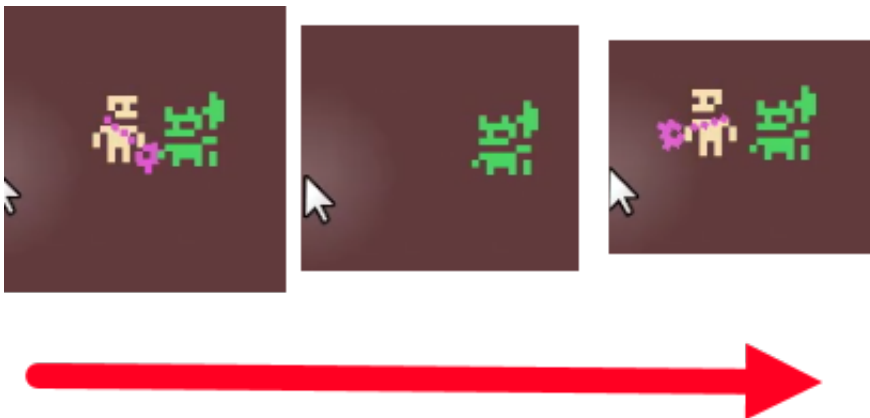
Here is how you can modify the take\_damage function for the player:

```
func take_damage(amount: int):  
    cur_hp -= amount  
    _damage_flash()  
    GlobalSignals.OnPlayerUpdateHealth.emit(cur_hp, max_hp)  
  
    if cur_hp <= 0:  
        die()
```

## Testing the Damage Flash Effect

To see the Damage Flash effect in action:

1. Run your game and attack an enemy. You should see the enemy sprite flash whenever it takes damage.
2. Similarly, find an enemy and let it attack the player. The player sprite should flash whenever it takes damage.



Congratulations! You have successfully implemented the Damage Flash effect for both enemies and the player. This visual feedback adds an extra layer of immersion to your game, making it more engaging for players.

In the next lesson, we will continue to enhance our game with sounds.



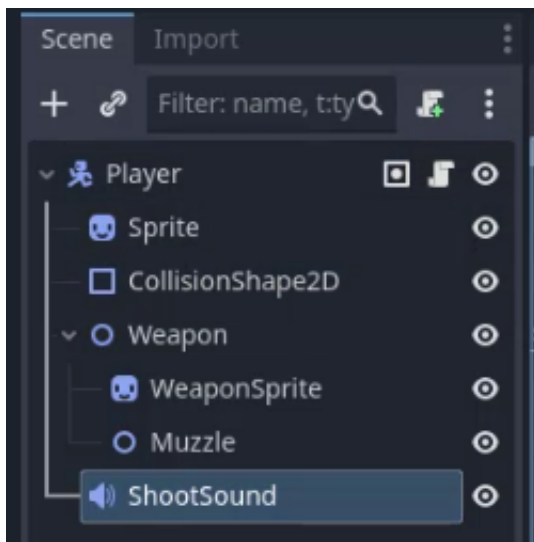
In this lesson, we will focus on implementing audio into our game. Audio is a crucial element that enhances the player's experience by providing feedback and immersion. We will add sound effects for various actions such as shooting, taking damage, and collecting items. Additionally, we will include background music to set the mood for our game.

## Adding Sound Effects

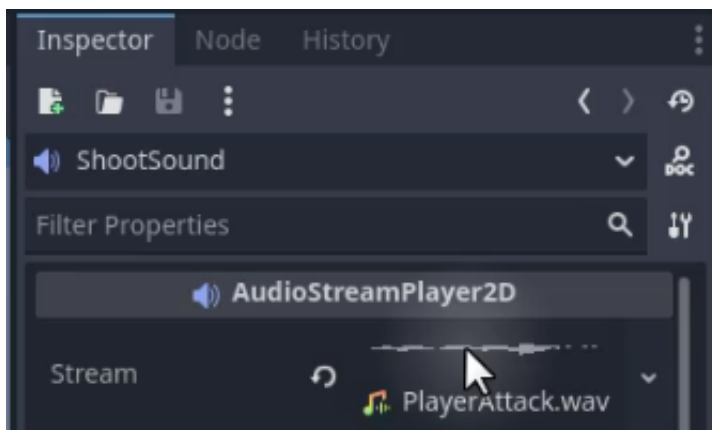
Let's start by adding sound effects to our player character. We will create sound effects for when the player shoots and when the player takes damage.

### Player Shoot Sound

- Open the player scene.
- Right-click on the player node and select Add Child.
- Choose AudioStreamPlayer2D to add an audio stream player node.
- Rename the node to ShootSound.



- In the Stream property, drag and drop the PlayerAttack audio file from the audio folder.



Next, we need to play this sound effect when the player shoots. Open the player script and locate the shooting function. Add the following line to play the sound:

```
func _shoot ():  
    # ... Existing code ...
```

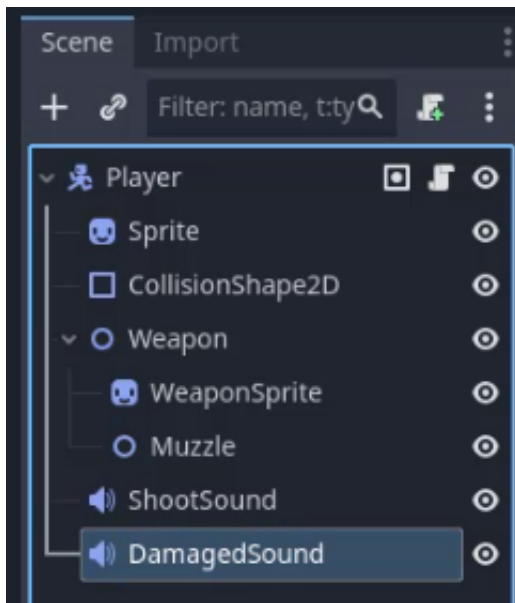
```
$ShootSound.play()
```

Now, when the player shoots, the sound effect will play, providing immediate feedback to the player.

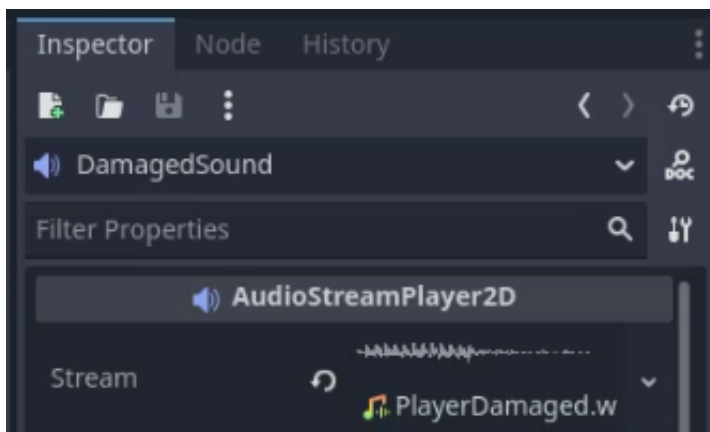
### Player Damage Sound

Similarly, we can add a sound effect for when the player takes damage.

- Copy the ShootSound node and rename it to DamagedSound.



- In the Stream property, drag and drop the PlayerDamaged audio file.



In the player script, locate the `take_damage` function and add the following to play the sound:

```
func take_damage (amount : int):  
    # ... Existing Code ...  
    GlobalSignals.OnPlayerUpdateHealth.emit(cur_hp, max_hp)  
    $DamagedSound.play()
```

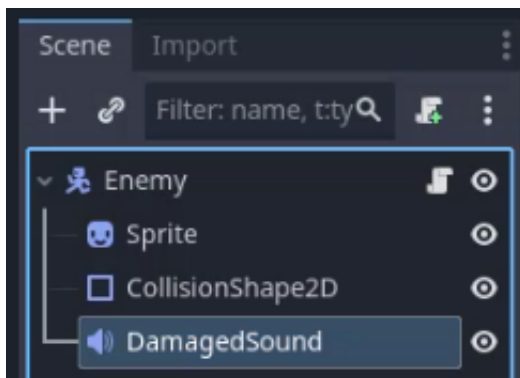
```
if cur_hp <= 0:  
    die()
```

Now, the player will hear a sound effect whenever they take damage, enhancing the immersion and providing immediate feedback.

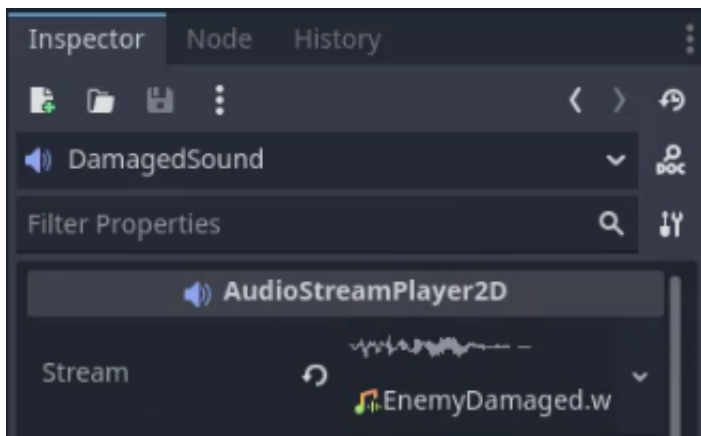
## Enemy Damage Sound

We can also add a sound effect for when the enemy takes damage. Open the enemy scene and follow these steps:

- Right-click on the enemy node and select Add Child.
- Choose AudioStreamPlayer2D to add an audio stream player node.
- Rename the node to DamagedSound.



- In the Stream property, drag and drop the EnemyDamaged audio file.



In the enemy script, locate the take\_damage function and add the following to play the sound:

```
func take_damage (amount : int):  
    cur_hp -= amount  
  
    _damage_flash()  
    $DamagedSound.play()  
  
if cur_hp <= 0:  
    die()
```

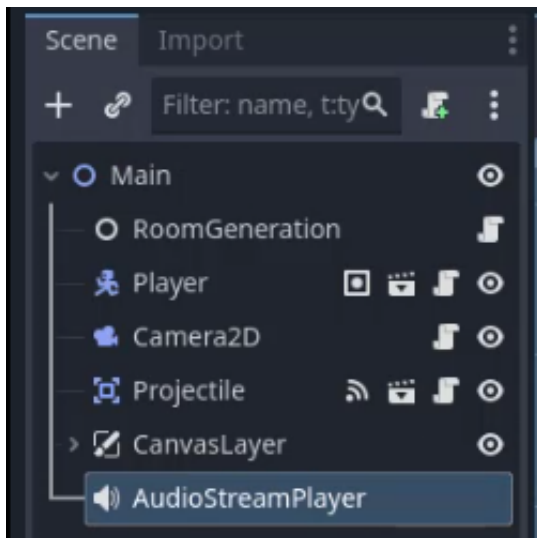


Now, the enemy will play a sound effect whenever it takes damage, providing feedback to the player.

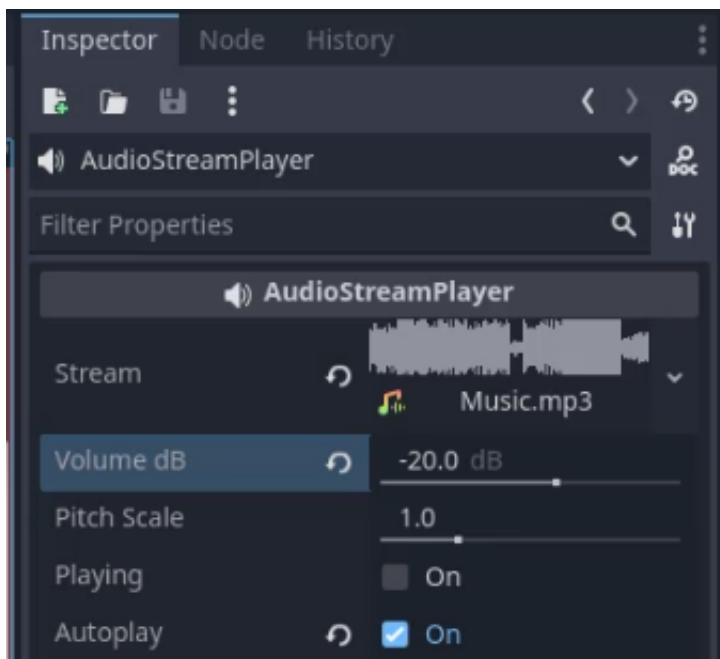
## Adding Background Music

Background music can set the mood and atmosphere for your game. Let's add background music to our level.

- Open level\_1.
- Create a new child node of type AudioStreamPlayer (not the 2D version).



- Drag and drop the Music audio file into the Stream property.
- Adjust the volume to a suitable level, such as -20.
- Enable the Autoplay option so the music starts at the beginning of the game.



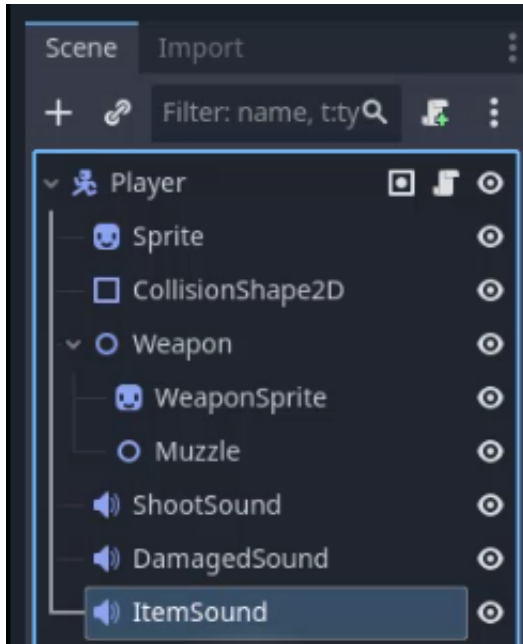


Now, when you play the game, background music will enhance the overall experience.

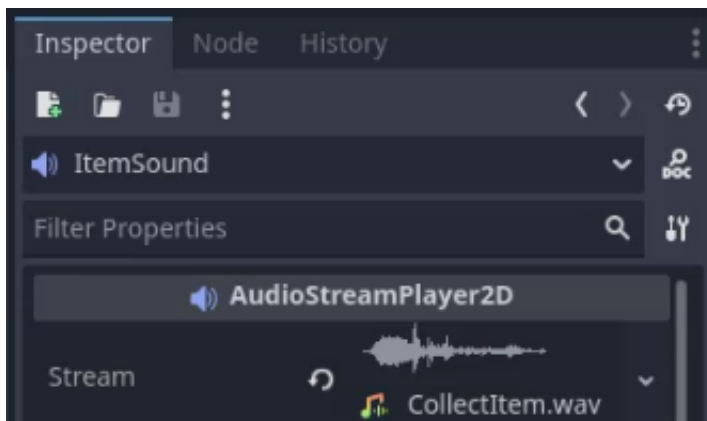
### Item Collection Sound

Finally, let's add a sound effect for when the player collects an item.

- Open the player scene.
- Copy the DamagedSound node and rename it to ItemSound.



- In the Stream property, drag and drop the CollectItem audio file.



In the item script, locate the `_on_body_entered` function and add the following line to play the sound before we remove the item:

```
func _on_body_entered(body):  
    # ... Existing code ...  
  
    body.get_node("ItemSound").play()  
  
    queue_free()
```



Now, when the player collects an item, a sound effect will play, providing feedback and enhancing the gameplay experience.

## **Conclusion**

In this lesson, we have successfully implemented various sound effects and background music into our game. These audio elements significantly enhance the player's experience by providing immediate feedback and setting the mood. Feel free to experiment with more sound effects, such as footsteps, door opening sounds, and other environmental sounds, to further enrich your game.

In the next lesson, we will make one more important room and enemy type: the boss.



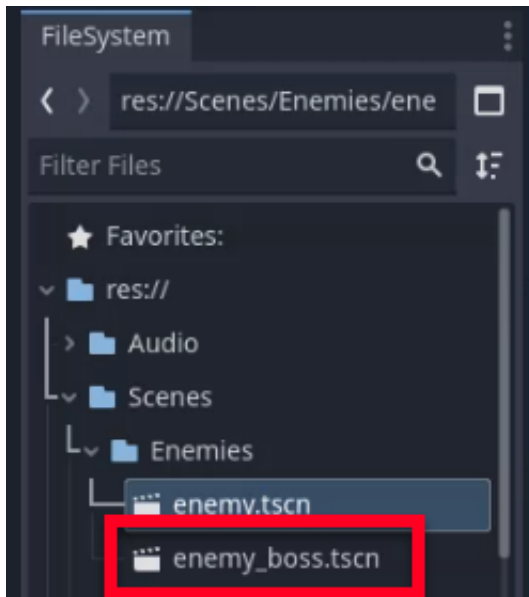


In this lesson, we will create a boss room with a boss enemy. The boss enemy will have enhanced attributes such as more health, speed, and damage. We will also ensure that the boss room is always included in the level and discuss how to spawn a portal to the next level upon defeating the boss.

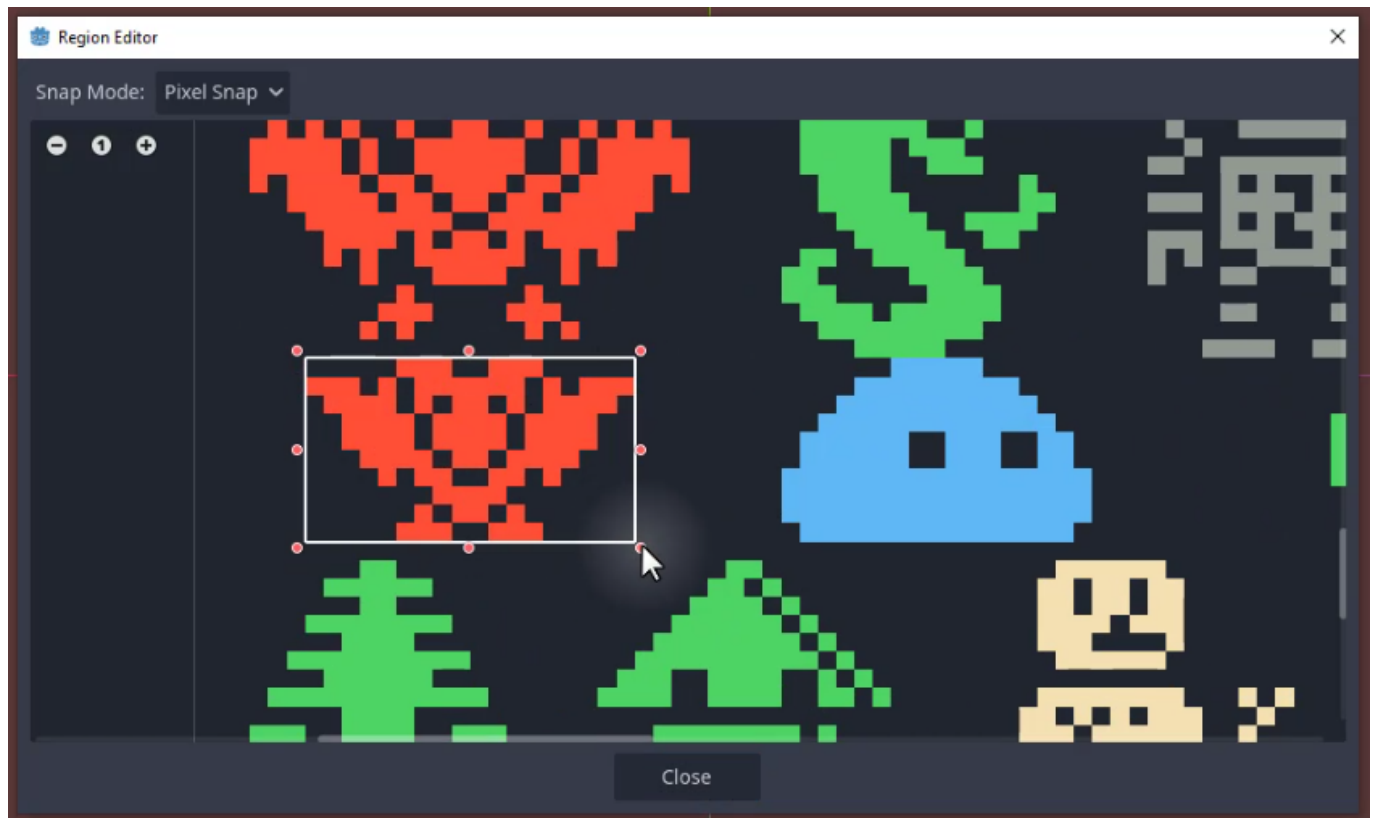
### Creating the Boss Enemy

To create the boss enemy, follow these steps:

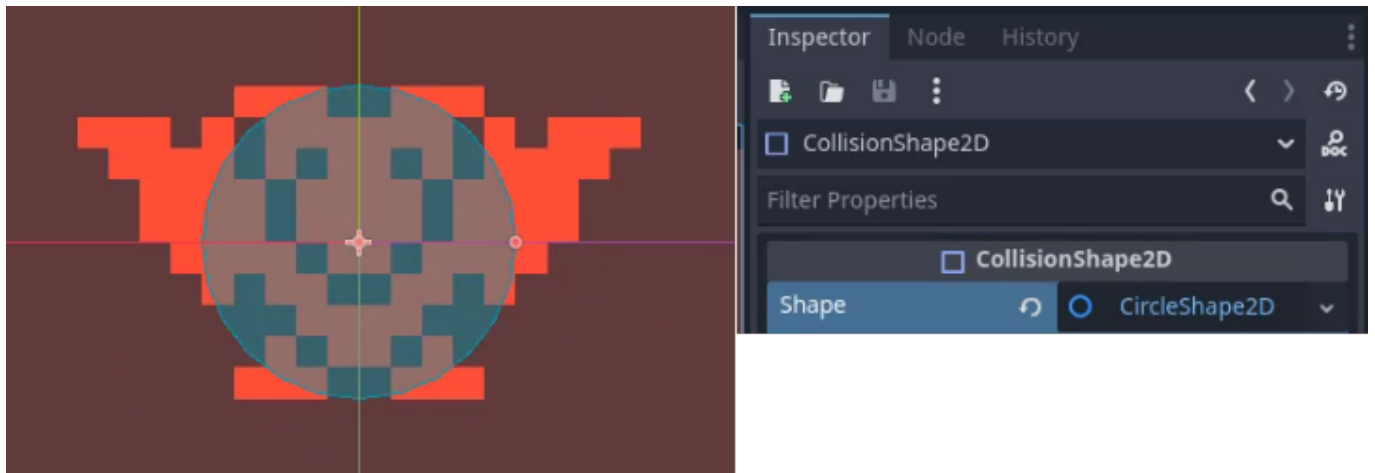
- Open the enemy scene.
- Create a new inherited scene from the enemy scene and save it as `enemy_boss`.



- Edit the Sprite's sprite region to change the sprite to a more menacing appearance and to differentiate the boss from regular enemies.



- Adjust the collision shape to better fit the new sprite. For example, you can change it to a circle.



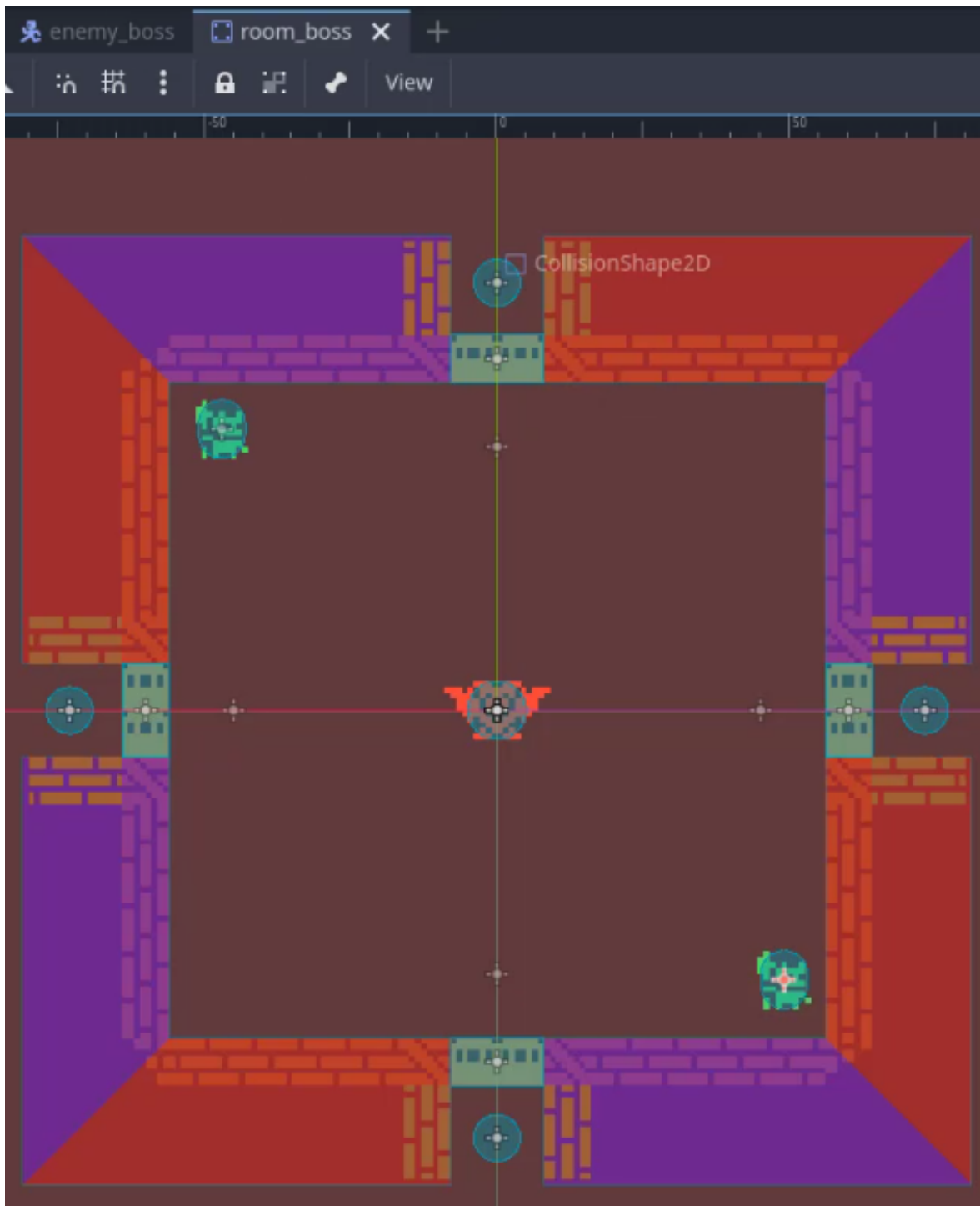
- Modify the boss enemy's stats in the Inspector:
  - Set max\_hp and cur\_hp to 20.
  - Increase move\_speed to 30 to make the boss faster.
  - Set attack\_damage to 1.
  - Set attack\_range to 15.
  - Set attack\_rate to 0.3 for faster attacks.



## Creating the Boss Room

Next, let's create the boss room:

- Create a new inherited scene from room\_template and name it room\_boss.
- Add the enemy\_boss and a few regular enemies to populate the room.



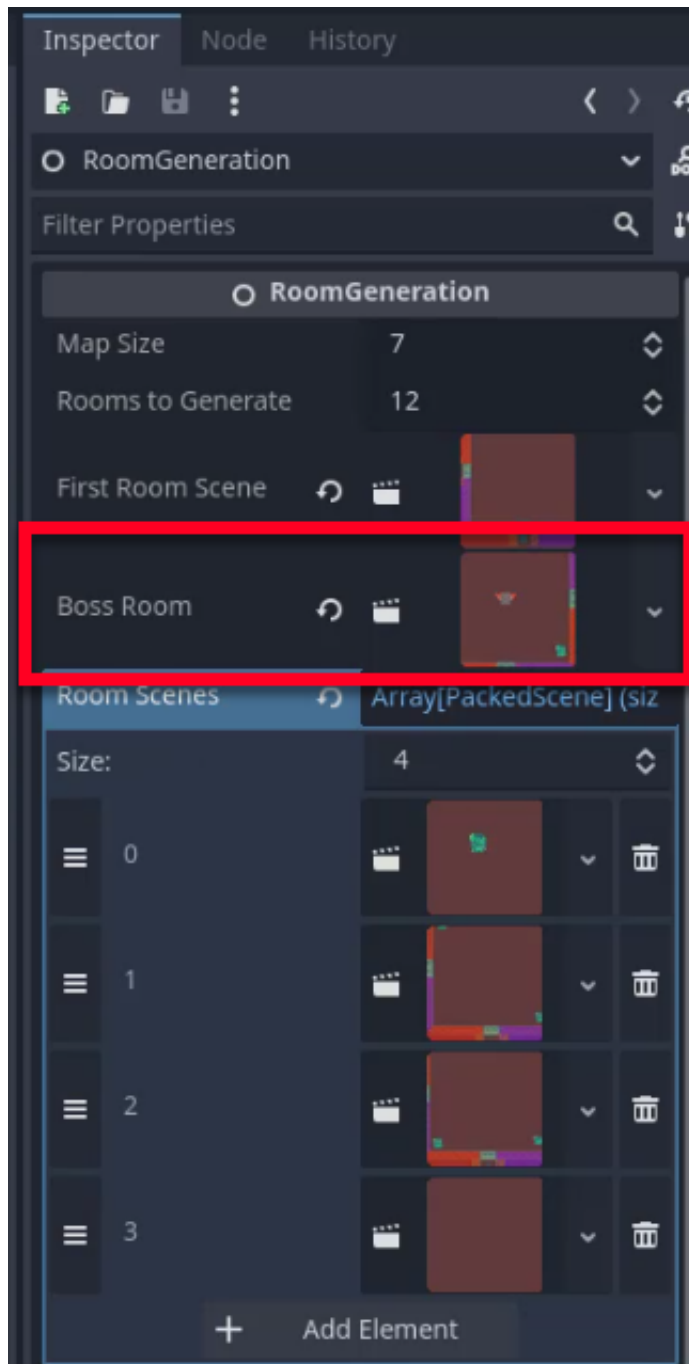
### Guaranteeing the Boss Room Spawn

To ensure the boss room is always included in the level, follow these steps:

- Open the room generation script (room\_generation.gd).
- Add a new export variable for the boss room:

```
@export var boss_room : PackedScene
```

- Assign the newly created boss room to this variable in the inspector.



- In the **instantiate\_rooms** function, create a new variable `boss_room_pos` to store the position of the boss room:

```
func _instantiate_rooms ():  
    var boss_room_pos : Vector2 = _decide_boss_room()  
  
    # rest of code
```

- Create a function `_decide_boss_room` to determine the position of the boss room. This function will use a loop to try 100 times to find one of the rooms generated that isn't the first room.



```
func _decide_boss_room() -> Vector2:
    var i = 0
    while i < 100:
        var x = randi_range(0, map_size - 1)
        var y = randi_range(0, map_size - 1)
        if first_room_x == x and first_room_y == y:
            continue
        if _get_map(x, y):
            return Vector2(x, y)
        i += 1
    return Vector2.ZERO
```

- Modify the room instantiation logic to check if the current position matches boss\_room\_pos and instantiate the boss room if it does:

```
func _instantiate_rooms():
    var boss_room_pos : Vector2 = _decide_boss_room()

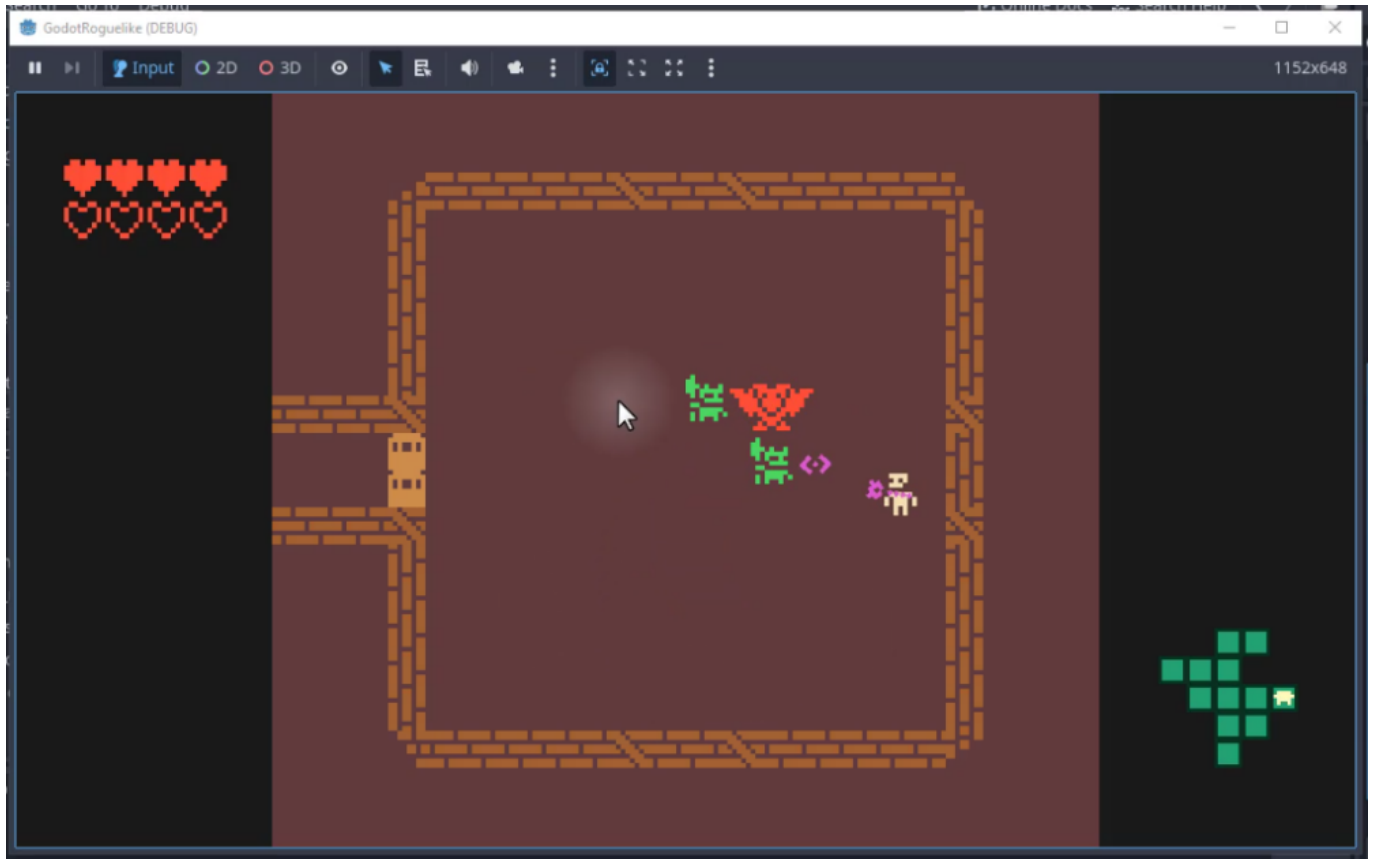
    for x in range(map_size):
        for y in range(map_size):
            if not _get_map(x, y):
                continue

            var room : Room
            var is_first_room = first_room_x == x and first_room_y == y

            if is_first_room:
                room = first_room_scene.instantiate()
            elif x == boss_room_pos.x and y == boss_room_pos.y:
                room = boss_room.instantiate()
            else:
                room = room_scenes[randi_range(0, len(room_scenes) - 1)].instantiate(
)

            #rest of code
```

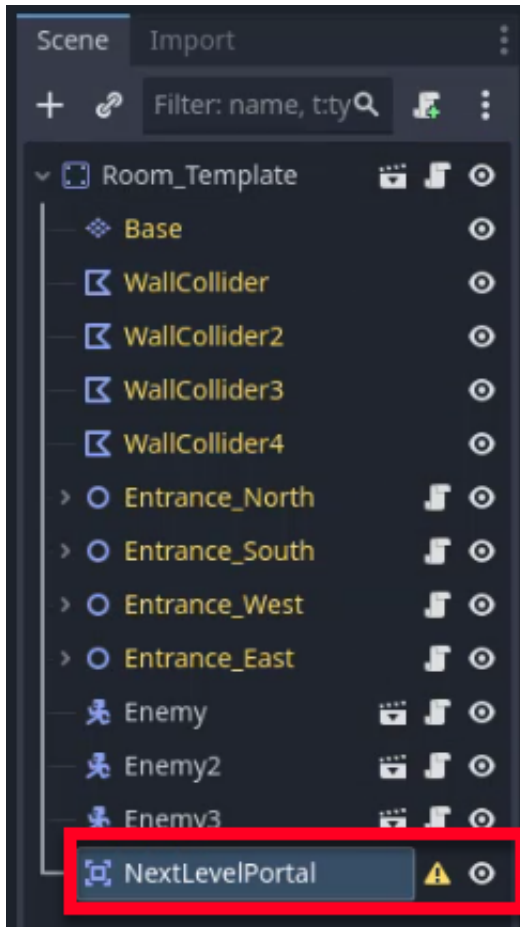
At this point, you should see that every level has at least one boss room that spawns.



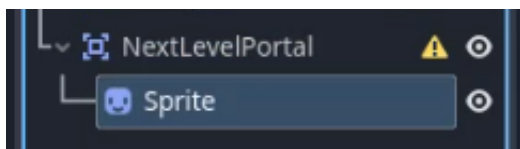
## Creating the Next Level Portal

To create a portal that appears after defeating the boss, follow these steps:

- In the `room_boss` scene, add a new `Area2D` node and name it `NextLevelPortal`.



- Add a Sprite2D node and rename it to Sprite.

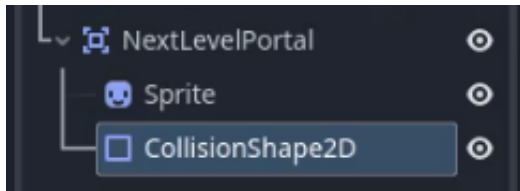


- Add Tiles.png to the Texture field of the Sprite and Edit the Region to a door-looking sprite.

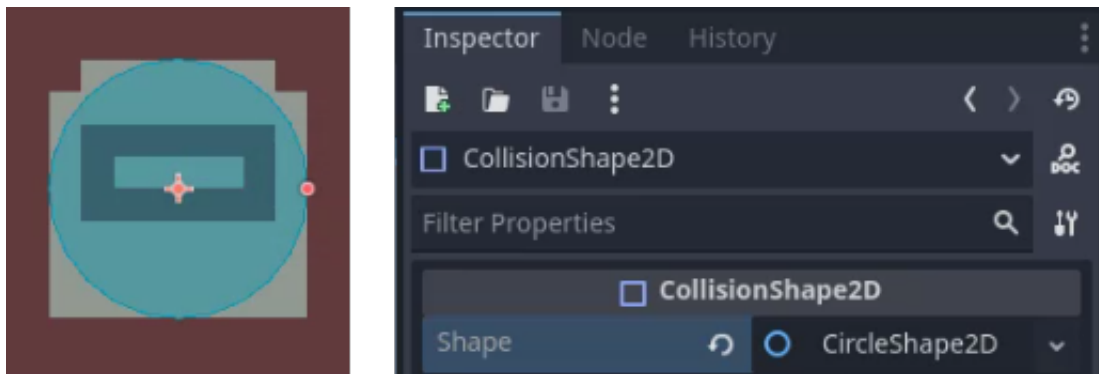




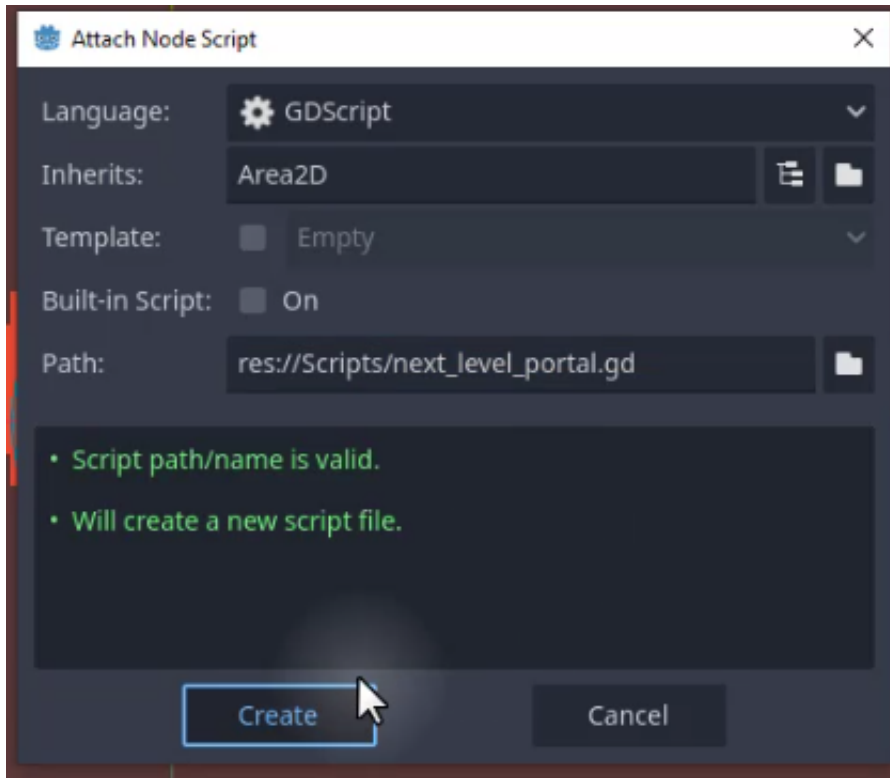
- Add a CollisionShape2D node as a child of the NextLevelPortal.



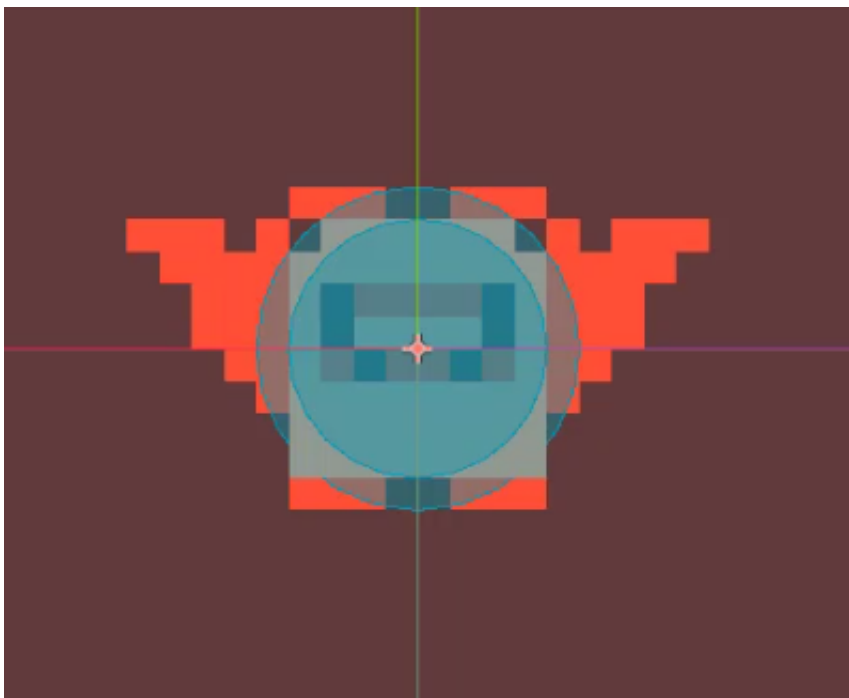
- Make the collision shape a circle and resize it to fit the door.



- Create a new script for the portal and name it next\_level\_portal.gd.



- Position the NextLevelPortal so it is in the direct center of the room.



- In the script, add variables for the boss and the next level:

```
@export var boss : Enemy  
@export var next_level : PackedScene
```

- In the `_ready` function, hide the portal and disable its collision shape:



```
func _ready():  
    visible = false  
    $CollisionShape2D.disabled = true
```

- Connect to the OnDefeatEnemy signal to a new function to enable the portal when the boss is defeated:

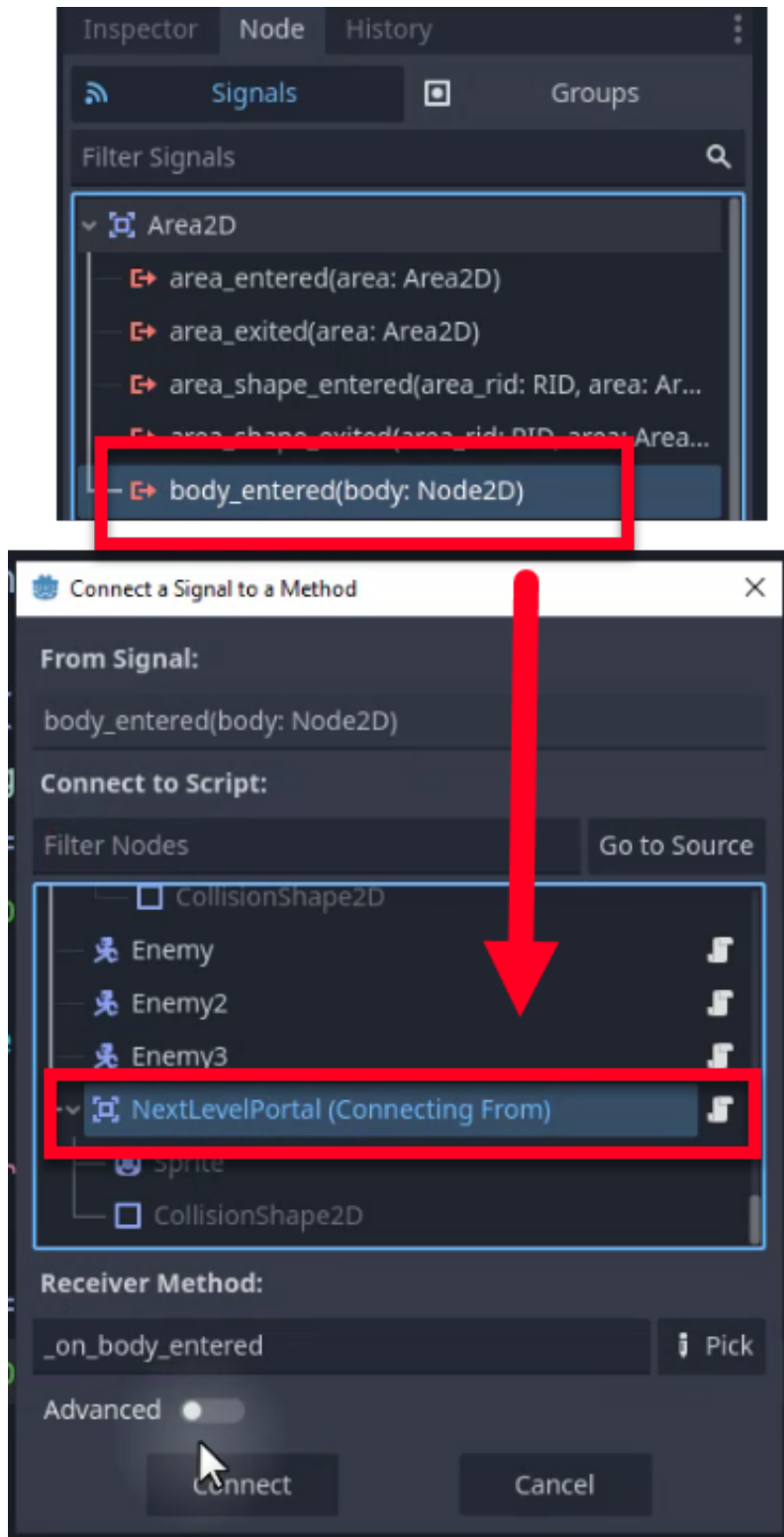
```
func _ready():  
    GlobalSignals.OnDefeatEnemy.connect(_on_defeat_enemy)  
    visible = false  
    $CollisionShape2D.disabled = true
```

```
func _on_defeat_enemy(enemy : Enemy):  
    pass
```

- In the **\_on\_defeat\_enemy** function, make sure the enemy is the boss, and then turn the visibility of the portal and its collision shape back on.

```
func _on_defeat_enemy (enemy : Enemy):  
    if enemy != boss:  
        return  
  
    visible = true  
    $CollisionShape2D.disabled = false
```

- Connect to the body\_entered signal to the script.

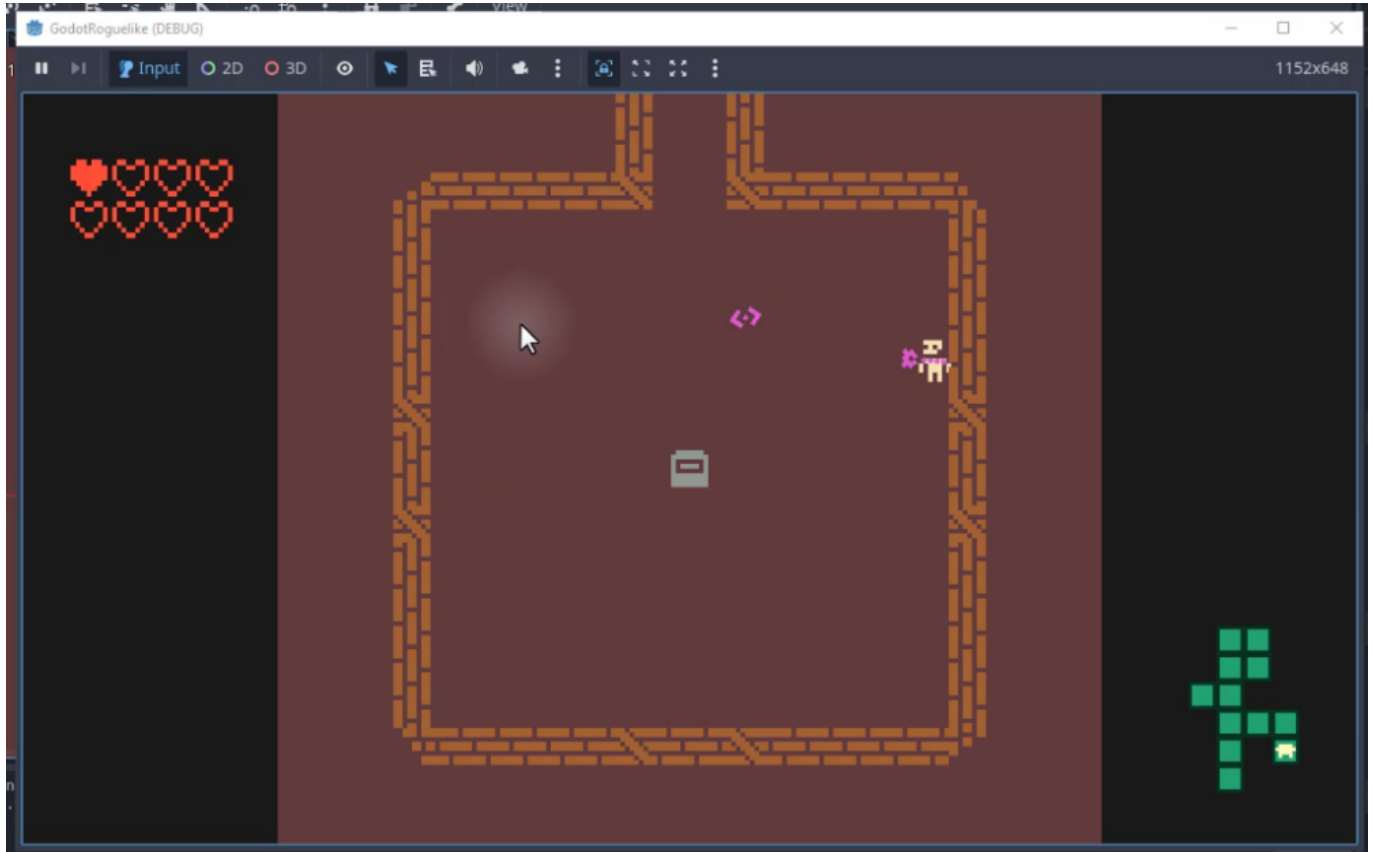


- Within the generated function, change the scene when the player enters the portal.

```
func _on_body_entered(body):  
    if not body.is_in_group("Player"):  
        return  
    get_tree().change_scene_to_packed(next_level)
```



By following these steps, you will have created a boss room with a boss enemy that has enhanced attributes. The boss room will always be included in the level, and a portal to the next level will appear upon defeating the boss.



In the next lesson, we will work on our main menu scene and add a few finishing touches to our game.

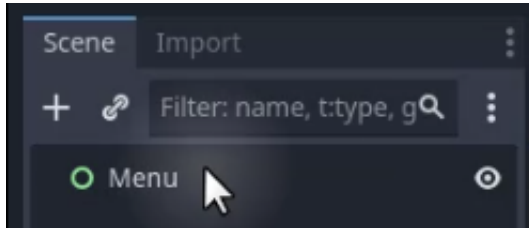


In this lesson, we will be focusing on creating a menu scene for our game. This menu will serve as the entry point for players, allowing them to start the game or quit. Let's dive right in and create this essential component step by step.

## Creating the Menu Scene

To begin, let's create a new scene for our menu. Follow these steps:

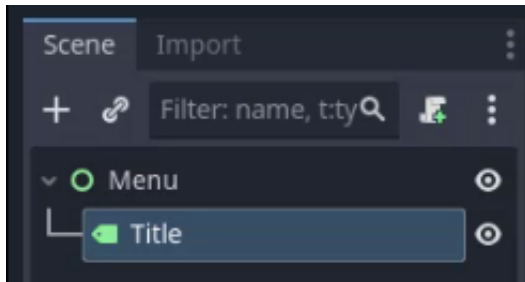
1. Navigate to the Scenes folder.
2. Create a new scene of type User Interface.
3. Name the scene Menu.



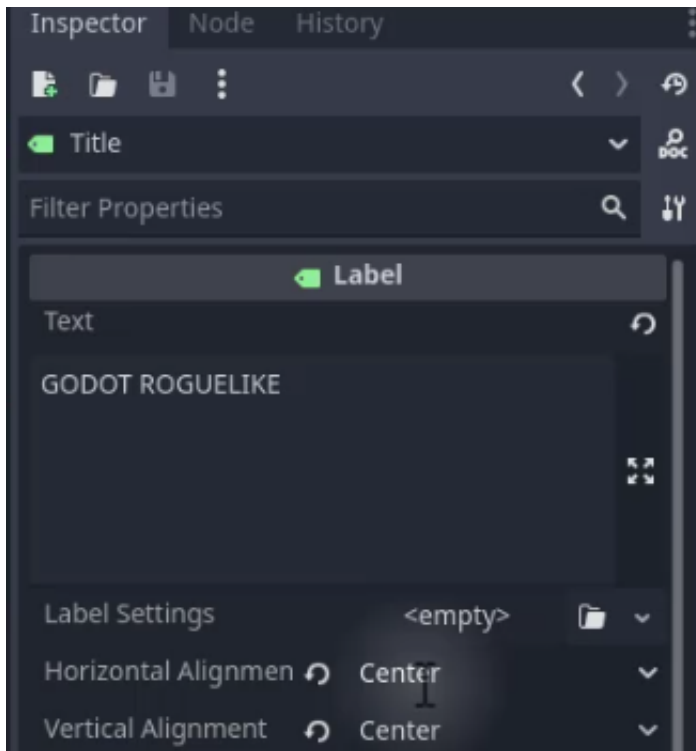
## Setting Up the Title

Inside the menu scene, we will set up a title and some buttons. Let's start with the title:

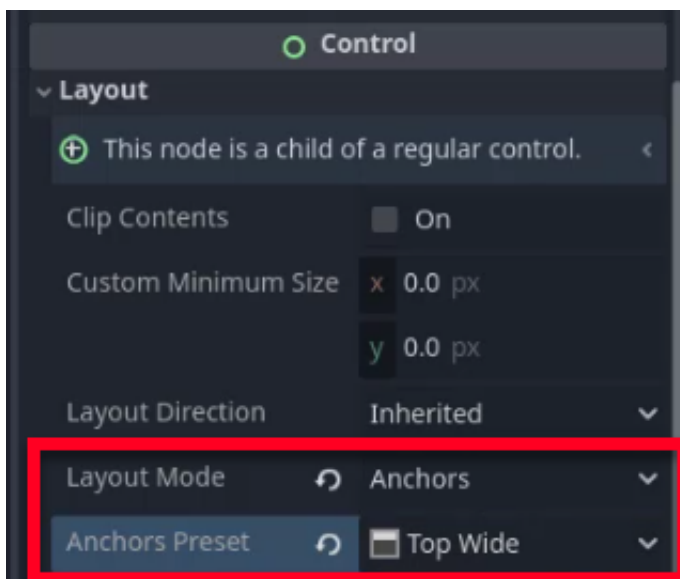
- Create a new Label node and name it Title.



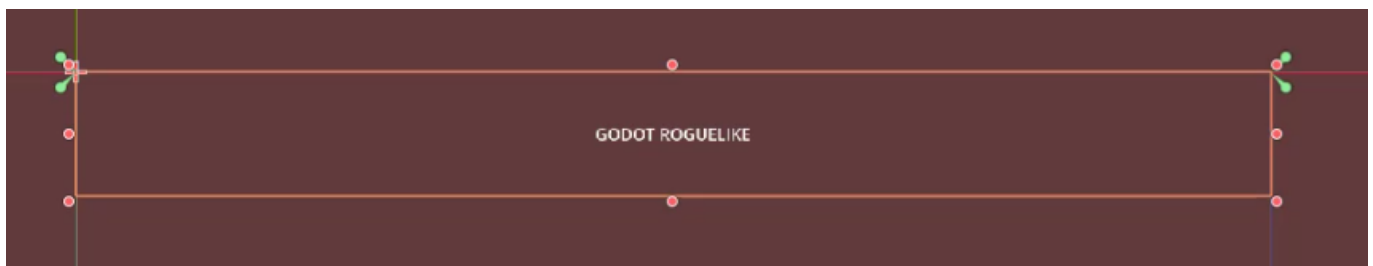
- Set the text of the label to GODOT ROGUELIKE.
- Center the horizontal and vertical alignment of the label.



- Change the layout mode to Anchors and set the anchor to Top Wide.

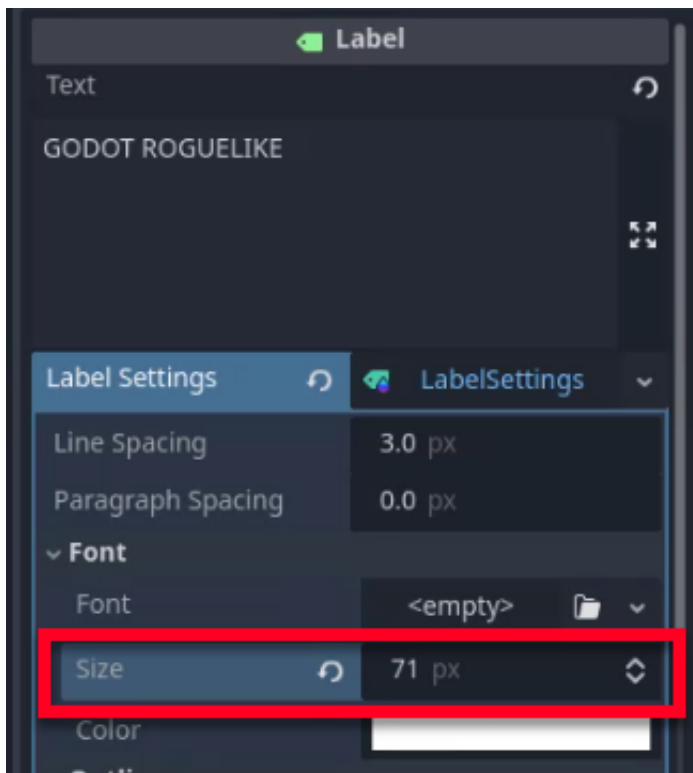


- Resize the label as desired.



- Add new LabelSettings to change the font size. Increase the font size to make it bigger. We

went with 71 px for our screen.



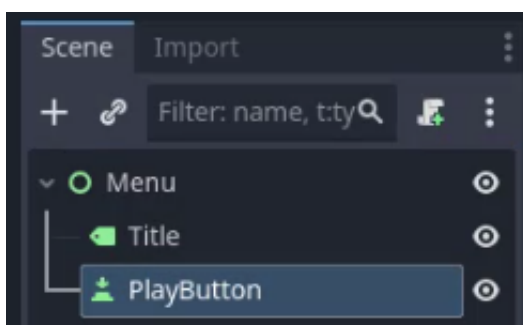
- After resizing the font, make any last-minute positional adjustments.



## Adding Buttons

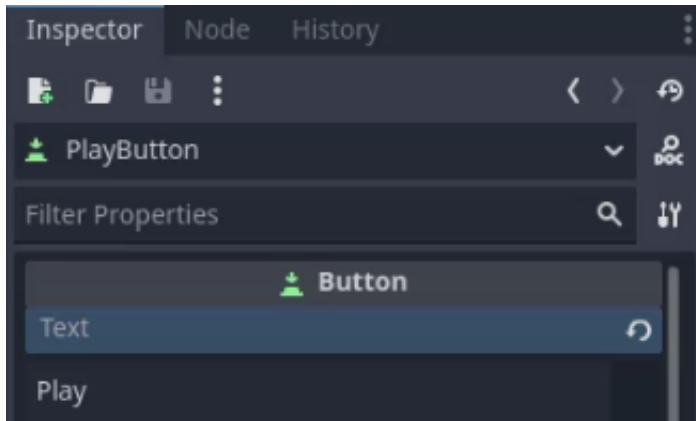
Next, we will add buttons for the player to interact with:

- Create a new Button node and name it PlayButton.

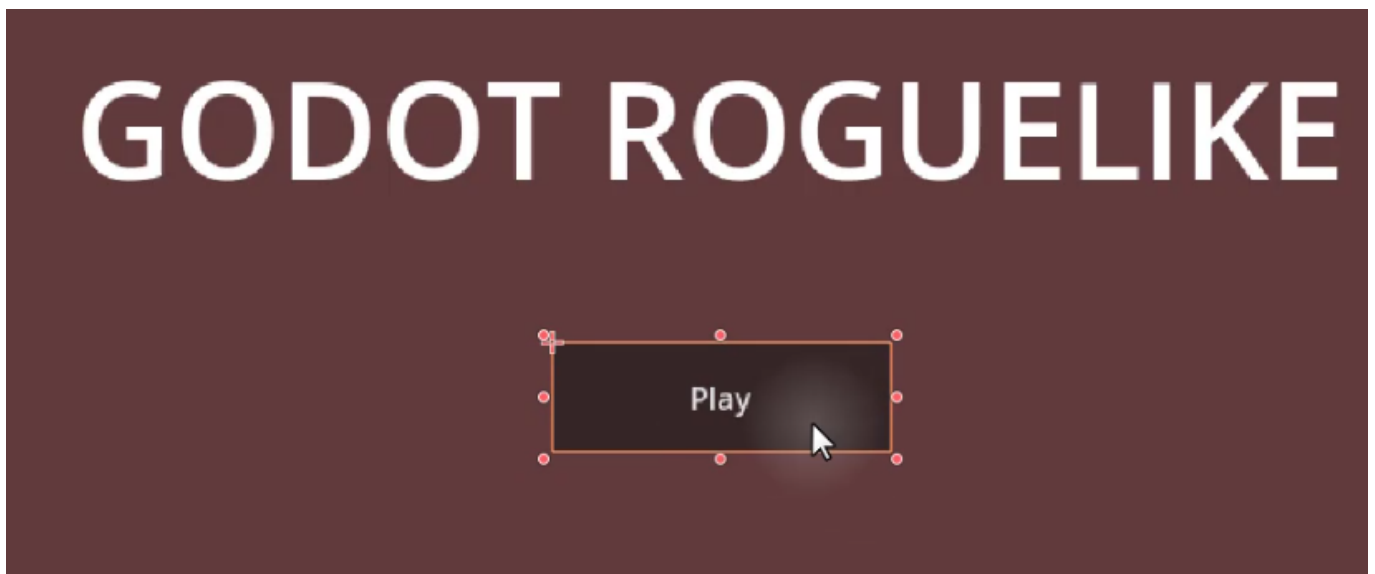


- Set the text of the button to Play.

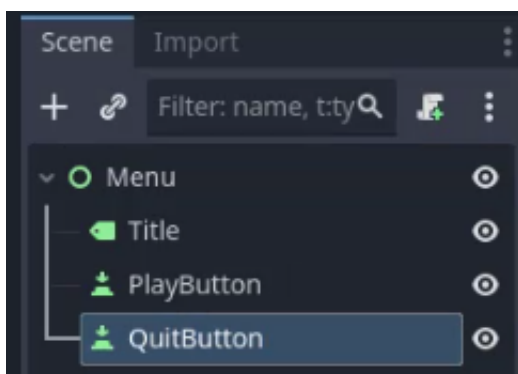




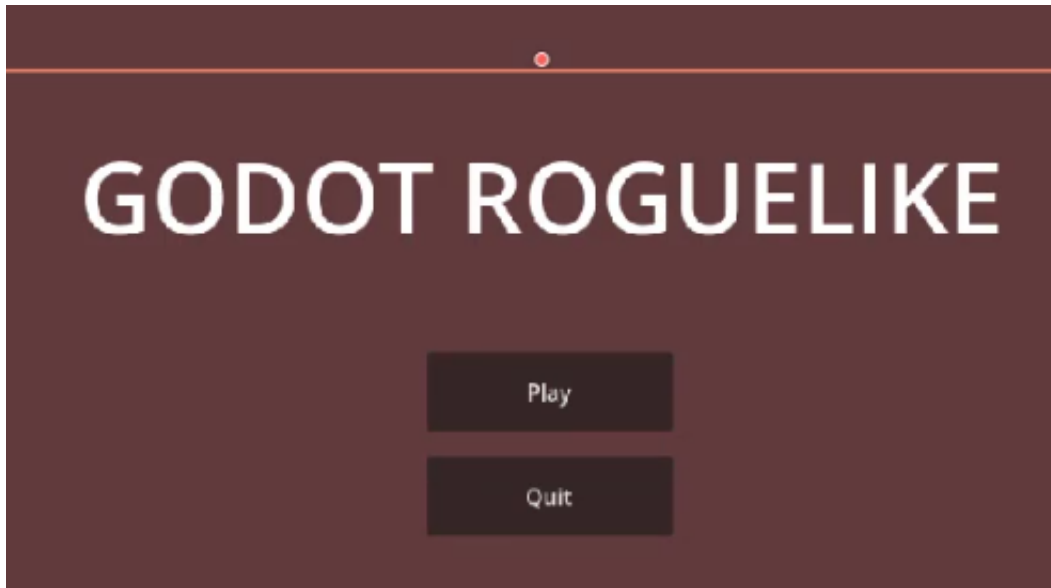
- Increase the size of the button and drag it to the center of the screen.



- Duplicate the Play button to create a QuitButton.



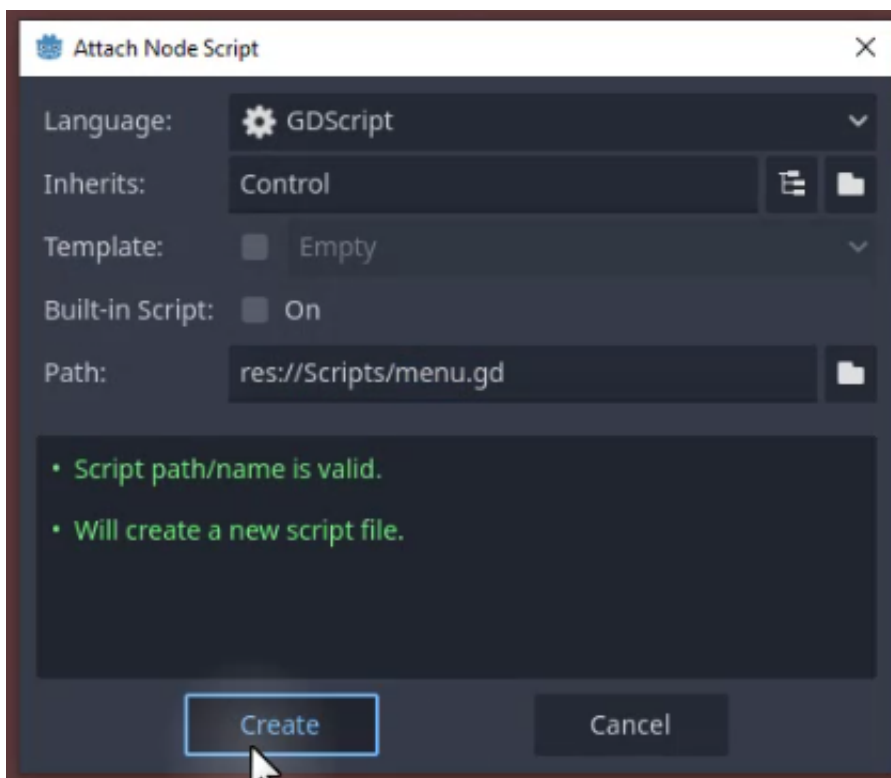
- Set the text of the duplicated button to Quit and position it below the Play button.



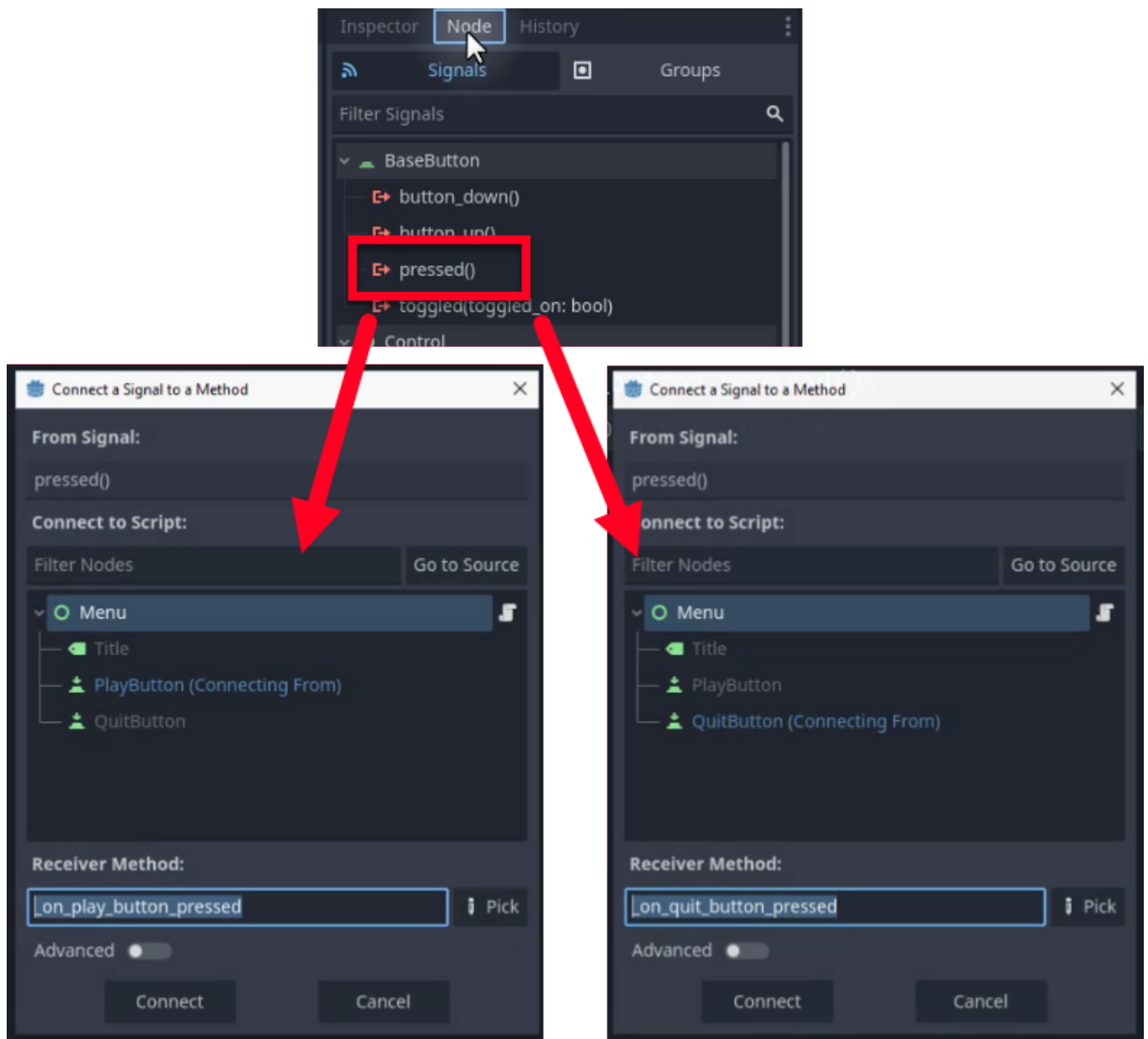
### Creating the Menu Script

Now, let's create a script to handle the functionality of our menu buttons. Follow these steps:

- Attach a new script to the menu root node and name it menu.gd.



- In the script, we will connect the pressed signals for the Play and Quit buttons.



Here is the code for the menu.gd script, which simply either loads our game scene or quits the game depending on which button is pressed:

```
extends Control

func _on_play_button_pressed():
    get_tree().change_scene_to_file("res://Scenes/Levels/level_1.tscn")

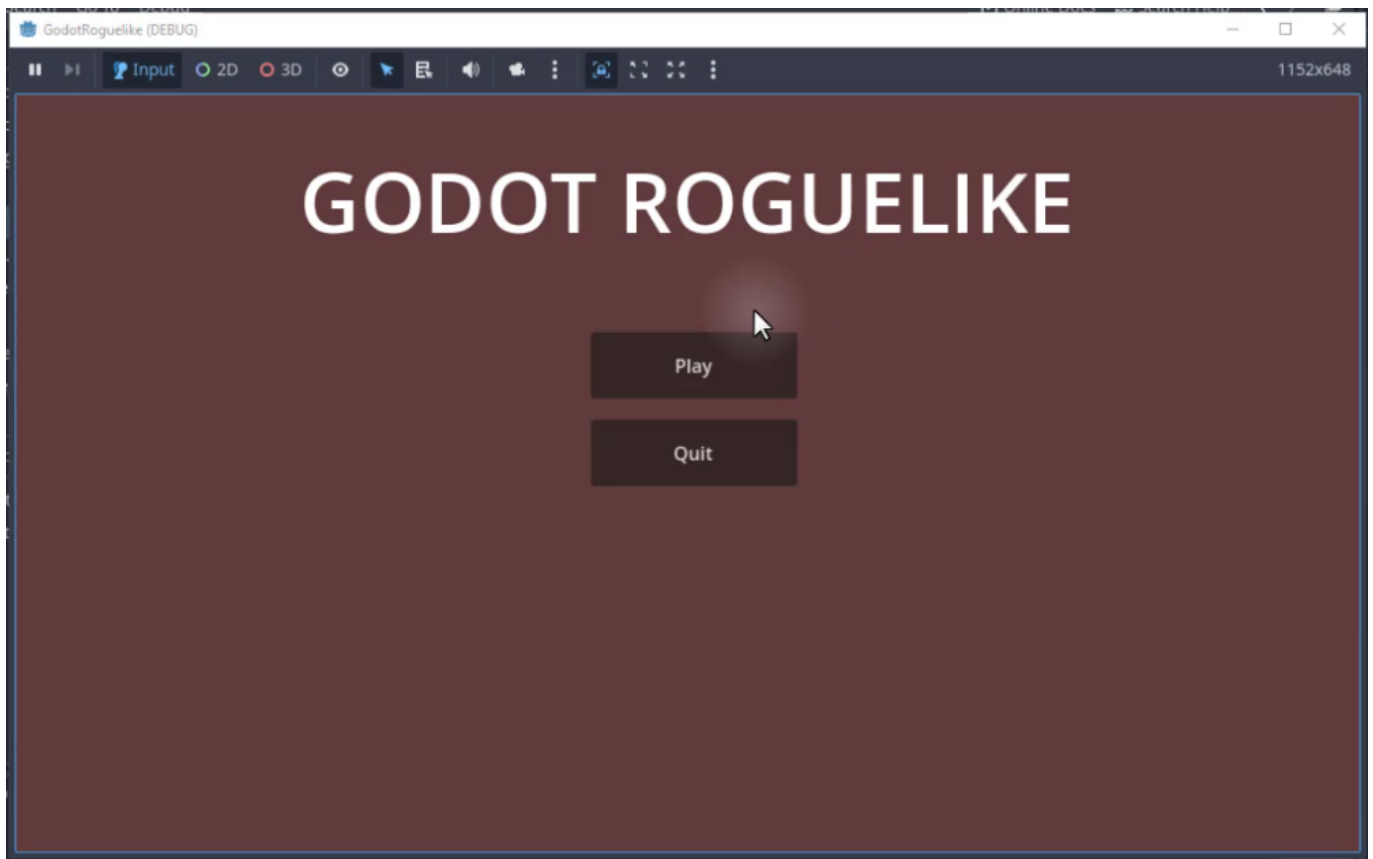
func _on_quit_button_pressed():
    get_tree().quit()
```

## Testing the Menu

Now that we have set up our menu scene, let's test it:



1. Play the menu scene.
2. Click the Play button to load level\_1.
3. Click the Quit button to close the game.



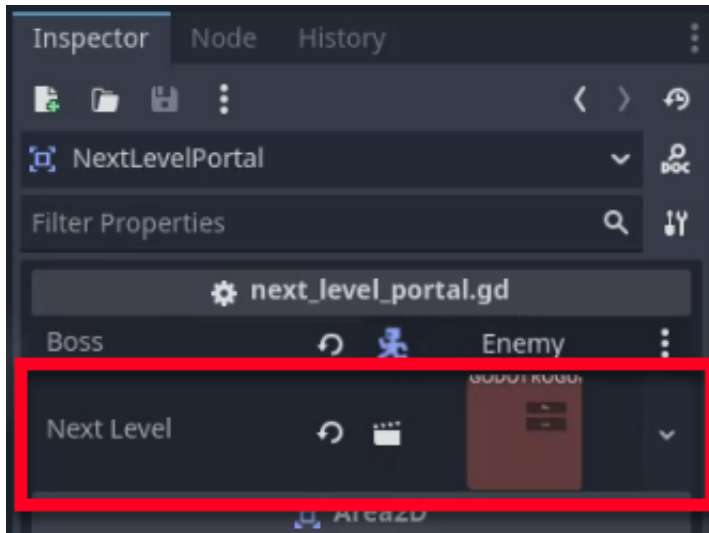
## Connecting the Menu to the Game

Finally, we need to connect our menu to the rest of the game. This involves loading the menu when the player is defeated and when the player goes through the portal to the next level. Follow these steps:

1. Open the player.gd script.
2. Inside the die function, add the following code to change the scene to the menu when the player is defeated:

```
func die():  
    get_tree().change_scene_to_file("res://Scenes/menu.tscn")
```

3. Open the boss\_room scene.
4. Select the next level portal node.
5. In the inspector, set the next level to be our menu scene.



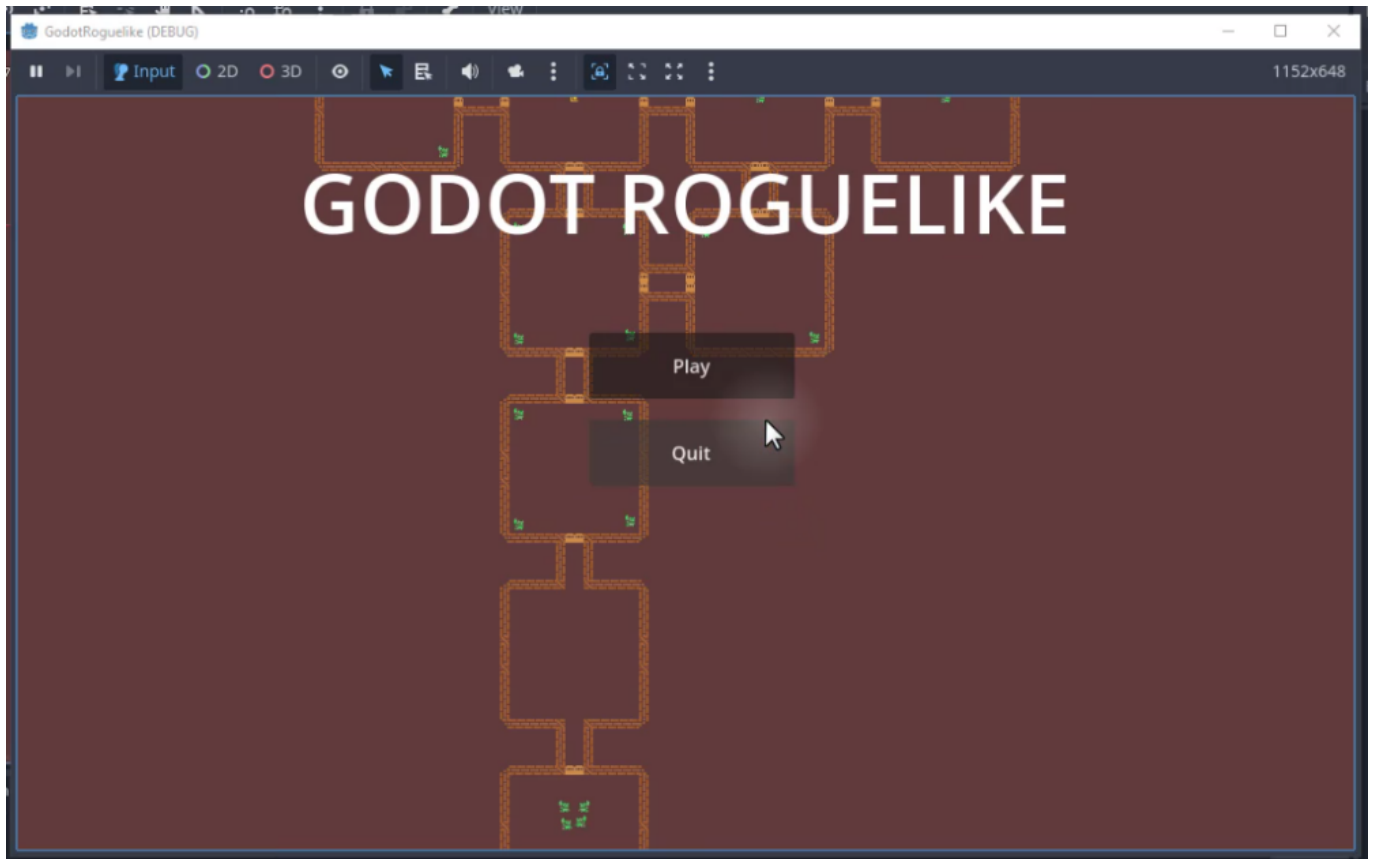
### Fixing Player Injury on Load

When you start the game, you may notice that the player gets hurt immediately at the start. This is because in our project, we've left the projectile scene attached to the player.

If you're having similar issues, make sure to delete it.

### Fixing Scene Persistence Issues

You might notice that when you go back to the menu, the previous level is still visible in the background.





This is because the rooms are being added to the root node, causing them to persist across scene changes. To fix this, follow these steps:

1. Open the `room_generation.gd` script.
2. In the `_instantiate_rooms` function, change the following line:

```
get_tree().root.add_child.call_deferred(room)
```

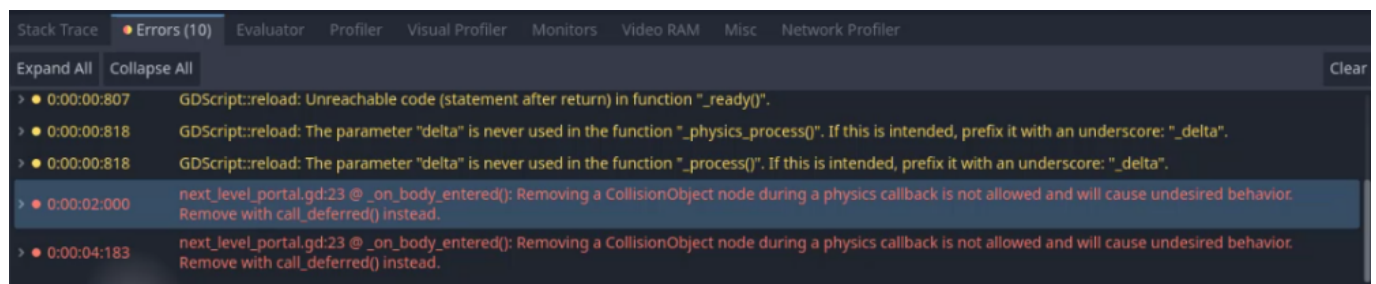
To:

```
get_tree().root.get_node("/root/Main").add_child.call_deferred(room)
```

This ensures that the rooms are added to the main scene node, preventing them from persisting across scene changes.

## Handling Physics Collision Errors

If you encounter an error related to setting the disabled property of a `CollisionShape2D`, you can fix it by using the `set_deferred` method.



Open the `next_level_portal.gd` script and modify the following line:

```
$CollisionShape2D.disabled = false
```

To:

```
$CollisionShape2D.set_deferred("disabled", false)
```

Additionally, ensure that the collision layer mask for the portal is set to detect overlaps with the player.



Another change we have to make is in our scene change. To prevent physics errors, we also need to defer that call.

```
func _on_body_entered(body):
    if not body.is_in_group("Player"):
        return

    get_tree().change_scene_to_packed.call_deferred(next_level)
```



With these changes, your menu should now correctly load and transition between scenes without any issues.





Congratulations on completing your course on creating a roguelike game using the Godot engine! Throughout this journey, you've explored a variety of concepts and features that make up a engaging roguelike experience. Let's recap what you've learned and discuss how you can apply this knowledge moving forward.

### Key Features of Your Roguelike Game

- **Player Character:** You created a player character that serves as the protagonist in your game.
- **Procedurally Generated Rooms:** You implemented procedural generation to create diverse and unpredictable levels.
- **Enemies and Bosses:** You populated your game with enemies and bosses to challenge the player.
- **Minimap:** You added a minimap to help players navigate through the game world.
- **Player Health:** You managed the player's health to create a sense of risk and consequence.
- **Items:** You included items to enhance gameplay and provide upgrades.
- **Sound and Damage Visuals:** You incorporated sound effects and visual feedback to create an immersive experience.

### Mastering Procedural Generation

Throughout the course, you gained insights into how procedural generation works. This knowledge is invaluable for creating replayable content and can be applied to various game genres beyond roguelikes. Here are some ways you can expand upon what you've learned:

1. **Experiment with Different Algorithms:** Try out different procedural generation algorithms to create unique levels and challenges.
2. **Enhance Existing Features:** Build upon the existing features in your game, such as adding more enemy types, creating complex boss battles, or introducing new items.
3. **Create New Game Mechanics:** Use your knowledge of procedural generation to develop new game mechanics that keep players engaged.

### Exploring Zenva's Learning Platform

Zenva is an online learning academy dedicated to helping students of all levels master new skills. With over a million students, Zenva offers a wide range of courses tailored to both beginners and those looking to expand their knowledge.

The courses at Zenva are designed to be versatile and accommodating to different learning styles. Here are some ways you can make the most of Zenva's learning platform:

- **Video Lessons:** Follow along with comprehensive video lessons that guide you through each concept step-by-step.
- **Lesson Summaries:** Read through included lesson summaries to reinforce your understanding of the material.
- **Course Files:** Utilize the provided course files to practice and apply what you've learned in a hands-on environment.

### Your Journey Ahead

Completing this course is just the beginning of your journey with the Godot engine. Armed with the knowledge and skills you've acquired, you are well-equipped to:

- Continue expanding and refining your roguelike game.
- Explore new game ideas and genres.



- Join the vibrant community of Godot developers to share your creations and learn from others.

Thank you for following along with the course. We wish you the best of luck in your future endeavors with Godot and game development. Keep learning, keep creating, and most importantly, keep having fun!



In this lesson, you can find the full source code for the project used within the course. Feel free to use this lesson as needed – whether to help you debug your code, use it as a reference later, or expand the project to practice your skills!

You can also download all project files from the **Course Files** tab via the project files or downloadable PDF summary.

## camera\_controller.gd

Found in folder: **/Scripts**

This code defines a class that extends the Camera2D class. In the `\_ready` function, it connects the `OnPlayerEnterRoom` signal from the `GlobalSignals` class to the `\_on\_player\_enter\_room` function. The `\_on\_player\_enter\_room` function takes a `Room` object as a parameter and sets the camera's global position to the room's global position.

```
extends Camera2D
```

```
func _ready ():  
    GlobalSignals.OnPlayerEnterRoom.connect(_on_player_enter_room)
```

```
func _on_player_enter_room (room : Room):  
    global_position = room.global_position
```

## enemy.gd

Found in folder: **/Scripts**

This code defines an Enemy class that extends the CharacterBody2D class. The enemy has properties such as current and maximum health points, movement speed, attack damage, range, and rate. The enemy becomes active when the player enters its room and moves towards the player, attacking if within range.

```
class_name Enemy  
extends CharacterBody2D
```

```
@export var cur_hp : int = 4  
@export var max_hp : int = 4  
@export var move_speed : float = 20
```

```
@export var attack_damage : int = 1  
@export var attack_range : float = 10  
@export var attack_rate : float = 0.5  
var last_attack_time : float
```

```
@onready var sprite : Sprite2D = $Sprite
```

```
var room : Room  
var is_active : bool = false  
var player : CharacterBody2D
```

```
var player_direction : Vector2  
var player_distance : float
```



```
func _ready():
    player = get_tree().get_first_node_in_group("Player")
    GlobalSignals.OnPlayerEnterRoom.connect(_on_player_enter_room)

func initialize (in_room : Room):
    is_active = false
    room = in_room

func _on_player_enter_room (player_room : Room):
    is_active = player_room == room

func _physics_process(delta):
    if not is_active or not player:
        return

    player_direction = global_position.direction_to(player.global_position)
    player_distance = global_position.distance_to(player.global_position)

    sprite.flip_h = player_direction.x < 0

    if player_distance < attack_range:
        _try_attack()
        return

    velocity = player_direction * move_speed
    move_and_slide()

func _try_attack():
    if Time.get_unix_time_from_system() - last_attack_time < attack_rate:
        return

    last_attack_time = Time.get_unix_time_from_system()

    player.take_damage(attack_damage)

func take_damage (amount : int):
    cur_hp -= amount

    _damage_flash()
    $DamagedSound.play()

    if cur_hp <= 0:
        die()

func _damage_flash():
    visible = false
    await get_tree().create_timer(0.07).timeout
    visible = true

func die():
    GlobalSignals.OnDefeatEnemy.emit(self)
    queue_free()
```



## global\_signals.gd

Found in folder: **/Scripts**

This code defines a class that extends the Node class. It declares three signals: OnPlayerEnterRoom, which is emitted when the player enters a room and passes the room object; OnDefeatEnemy, which is emitted when an enemy is defeated and passes the enemy object; and OnPlayerUpdateHealth, which is emitted when the player's health is updated and passes the current and maximum health values as integers.

```
extends Node
```

```
signal OnPlayerEnterRoom (room : Room)
signal OnDefeatEnemy (enemy : Enemy)
signal OnPlayerUpdateHealth (cur_hp : int, max_hp : int)
```

## item.gd

Found in folder: **/Scripts**

This code defines an enum called ItemType with three possible values: HEALTH, SHOOT\_RATE, and MOVE\_SPEED. It also defines two exported variables: type (of type ItemType) and value (of type float). When a body enters the area, if it is in the "Player" group, the code performs an action based on the type of the item (healing, increasing shoot rate, or increasing move speed) and plays a sound effect before freeing the item.

```
extends Area2D
```

```
enum ItemType
{
    HEALTH,
    SHOOT_RATE,
    MOVE_SPEED
}
```

```
@export var type : ItemType
@export var value : float
```

```
func _on_body_entered(body):
    if not body.is_in_group("Player"):
        return

    if type == ItemType.HEALTH:
        body.heal(int(value))
    elif type == ItemType.SHOOT_RATE:
        body.shoot_rate -= value
    elif type == ItemType.MOVE_SPEED:
        body.move_speed += value

    body.get_node("ItemSound").play()

    queue_free()
```

## menu.gd

Found in folder: **/Scripts**

This code defines a script that extends the Control class. It contains two functions: `_on_play_button_pressed()`, which changes the scene to the "level\_1.tscn" file when the play button is pressed, and `_on_quit_button_pressed()`, which quits the game when the quit button is pressed.

```
extends Control

func _on_play_button_pressed():
    get_tree().change_scene_to_file("res://Scenes/Levels/level_1.tscn")

func _on_quit_button_pressed():
    get_tree().quit()
```

## minimap.gd

Found in folder: **/Scripts**

This code defines a class that extends GridContainer and displays a minimap of rooms. It connects to the OnPlayerEnterRoom signal to update the minimap when the player enters a new room. The `_on_player_enter_room` function iterates through the rooms in the `room_gen` map and updates the `icons` array with the appropriate texture based on the room's state (null for no room, `player_room_texture` for the current room, and `room_texture` for other rooms).

```
extends GridContainer

var room_gen : RoomGeneration

var icons : Array[TextureRect]

var room_texture : Texture = preload("res://Sprites/Items/minimap_room.tres")
var player_room_texture : Texture = preload("res://Sprites/Items/minimap_player_room.tres")

func _ready():
    GlobalSignals.OnPlayerEnterRoom.connect(_on_player_enter_room)

func _on_player_enter_room (room : Room):
    if not room_gen:
        room_gen = get_node("/root/Main/RoomGeneration")

    for child in get_children():
        if child is TextureRect:
            icons.append(child)

    for x in range(room_gen.map_size):
        for y in range(room_gen.map_size):
            var r = room_gen.get_room_from_map(x, y)
            var i = x + y * room_gen.map_size

            if i >= len(icons):
```

```
continue

if not r:
    icons[i].texture = null
elif r == room:
    icons[i].texture = player_room_texture
else:
    icons[i].texture = room_texture
```

## next\_level\_portal.gd

Found in folder: **/Scripts**

This code defines a script for an Area2D node that triggers the transition to the next level when the player enters its area after defeating a specific boss enemy. The script connects to the OnDefeatEnemy signal to monitor when the boss is defeated, and once that happens, it enables the visibility and collision of the Area2D. When the player enters the area, the script changes the scene to the next level using the assigned PackedScene.

```
extends Area2D

@export var boss : Enemy
@export var next_level : PackedScene

func _ready ():
    GlobalSignals.OnDefeatEnemy.connect(_on_defeat_enemy)
    visible = false
    $CollisionShape2D.disabled = true

func _on_defeat_enemy (enemy : Enemy):
    if enemy != boss:
        return

    visible = true
    $CollisionShape2D.set_deferred("disabled", false)

func _on_body_entered(body):
    if not body.is_in_group("Player"):
        return

    get_tree().change_scene_to_packed.call_deferred(next_level)
```

## player.gd

Found in folder: **/Scripts**

This code defines a player character that can move, shoot projectiles, take damage, and heal. The player's movement is controlled by input actions, and the weapon's direction is determined by the mouse position. The player's health is tracked and updated through signals, and the character is removed from the scene when its health reaches zero.



```
extends CharacterBody2D

@export var cur_hp : int = 8
@export var max_hp : int = 8

@export var move_speed : float = 50

@export var shoot_rate : float = 0.4
var last_shoot_time : float

var projectile_scene : PackedScene = preload("res://Scenes/Projectiles/projectile.tscn")

@onready var sprite : Sprite2D = $Sprite
@onready var weapon_origin : Node2D = $Weapon
@onready var muzzle : Node2D = $Weapon/Muzzle

func _ready():
    GlobalSignals.OnPlayerUpdateHealth.emit.call_deferred(cur_hp, max_hp)

func _physics_process(delta):
    var move_input : Vector2 = Input.get_vector("move_left", "move_right", "move_up", "move_down")
    velocity = move_input * move_speed
    move_and_slide()

func _process (delta):
    var mouse_pos : Vector2 = get_global_mouse_position()
    var mouse_dir : Vector2 = (mouse_pos - global_position).normalized()
    weapon_origin.rotation_degrees = rad_to_deg(mouse_dir.angle()) + 90

    sprite.flip_h = mouse_dir.x < 0

    if Input.is_action_pressed("attack"):
        if Time.get_unix_time_from_system() - last_shoot_time > shoot_rate:
            _shoot()

func _shoot():
    last_shoot_time = Time.get_unix_time_from_system()

    var proj = projectile_scene.instantiate()
    get_tree().root.add_child(proj)
    proj.global_position = muzzle.global_position
    proj.rotation = weapon_origin.rotation
    proj.owner_character = self

    $ShootSound.play()

func take_damage (amount : int):
    cur_hp -= amount

    _damage_flash()
    GlobalSignals.OnPlayerUpdateHealth.emit(cur_hp, max_hp)
    $DamagedSound.play()
```





```
if cur_hp <= 0:
    die()

func _damage_flash ():
    visible = false
    await get_tree().create_timer(0.07).timeout
    visible = true

func die ():
    get_tree().change_scene_to_file("res://Scenes/menu.tscn")

func heal (amount : int):
    cur_hp += amount

    if cur_hp > max_hp:
        cur_hp = max_hp

GlobalSignals.OnPlayerUpdateHealth.emit(cur_hp, max_hp)
```

## player\_health\_ui.gd

Found in folder: **/Scripts**

This code defines a class that extends `GridContainer` and manages heart icons representing the player's health. It loads textures for full and empty hearts, connects to a signal for updating the player's health, and stores references to the heart icon `TextureRect` nodes in an array. When the player's health is updated, it updates the visibility and texture of each heart icon based on the current and maximum health values.

```
extends GridContainer

var full_heart : Texture = preload("res://Sprites/Items/heart_full.tres")
var empty_heart : Texture = preload("res://Sprites/Items/heart_empty.tres")

var heart_icons : Array[TextureRect]

func _init ():
    GlobalSignals.OnPlayerUpdateHealth.connect(_update_ui)

func _ready ():
    for child in get_children():
        if child is TextureRect:
            heart_icons.append(child)

func _update_ui (cur_hp : int, max_hp : int):
    for i in len(heart_icons):
        if i >= max_hp:
            heart_icons[i].visible = false
            continue

        heart_icons[i].visible = true

        if i < cur_hp:
```



```
heart_icons[i].texture = full_heart
else:
    heart_icons[i].texture = empty_heart
```

## projectile.gd

Found in folder: **/Scripts**

This code defines a projectile that moves in a straight line and damages any object it collides with, except for its owner. The projectile's speed is determined by the `base\_speed` variable, and it moves in the direction of the negative Y-axis of its local transform. If the projectile collides with an object that has a `take\_damage` method, it calls that method with an argument of 1 and then destroys itself.

```
extends Area2D

@export var base_speed : float = 100
var owner_character : CharacterBody2D

func _process (delta):
    translate(-transform.y * base_speed * delta)

func _on_body_entered(body):
    if body == owner_character:
        return

    if body.has_method("take_damage"):
        body.take_damage(1)

    queue_free()

func _on_timer_timeout():
    queue_free()
```

## room.gd

Found in folder: **/Scripts**

This code defines a Room class that extends StaticBody2D. It sets up entrances in four directions (north, south, east, west) and keeps track of enemies in the room. The room can open or close its doors based on the presence of enemies, and it handles player entry and enemy defeat events.

```
class_name Room
extends StaticBody2D

enum Direction
{
    NORTH,
    SOUTH,
    EAST,
    WEST
```

```
}

@export var doors_always_open : bool = false

@onready var entrance_north : RoomEntrance = $Entrance_North
@onready var entrance_south : RoomEntrance = $Entrance_South
@onready var entrance_east : RoomEntrance = $Entrance_East
@onready var entrance_west : RoomEntrance = $Entrance_West

var enemies_in_room : int

func _ready ():
    GlobalSignals.OnDefeatEnemy.connect(_on_enemy_defeated)

    for child in get_children():
        if child is Enemy:
            enemies_in_room += 1
            child.initialize(self)

func initialize ():
    pass

func set_neighbor (neighbor_direction : Direction, neighbor_room : Room):
    if neighbor_direction == Direction.NORTH:
        entrance_north.set_neighbor(neighbor_room)
    elif neighbor_direction == Direction.SOUTH:
        entrance_south.set_neighbor(neighbor_room)
    elif neighbor_direction == Direction.EAST:
        entrance_east.set_neighbor(neighbor_room)
    elif neighbor_direction == Direction.WEST:
        entrance_west.set_neighbor(neighbor_room)

# called when the player enters the room
func player_enter (entry_direction : Direction, player : CharacterBody2D, first_room
: bool = false):
    # set their position based on where they entered from
    if entry_direction == Direction.NORTH:
        player.global_position = entrance_north.player_spawn.global_position
    elif entry_direction == Direction.SOUTH:
        player.global_position = entrance_south.player_spawn.global_position
    elif entry_direction == Direction.EAST:
        player.global_position = entrance_east.player_spawn.global_position
    elif entry_direction == Direction.WEST:
        player.global_position = entrance_west.player_spawn.global_position

    # if this is the first room, spawn the player in the middle
    if first_room:
        player.global_position = global_position

    GlobalSignals.OnPlayerEnterRoom.emit(self)

    # close the doors if there are enemies to defeat
    if enemies_in_room > 0 and not doors_always_open:
        close_doors()
    else:
```

```
open_doors()

func _on_defeat_enemy (enemy):
    pass

func open_doors ():
    entrance_north.open_door.call_deferred()
    entrance_south.open_door.call_deferred()
    entrance_east.open_door.call_deferred()
    entrance_west.open_door.call_deferred()

func close_doors ():
    entrance_north.close_door.call_deferred()
    entrance_south.close_door.call_deferred()
    entrance_east.close_door.call_deferred()
    entrance_west.close_door.call_deferred()

func _on_enemy_defeated (enemy : Enemy):
    if enemy.get_parent() == self:
        enemies_in_room -= 1

    if enemies_in_room <= 0:
        open_doors()
```

## room\_entrance.gd

Found in folder: **/Scripts**

This code defines a RoomEntrance class that extends Node2D. It manages the entrance and exit of a room, handling the visibility and collision of barriers and doors. The class also handles player spawning and triggering events when the player enters the exit area, communicating with the neighboring room.

```
class_name RoomEntrance
extends Node2D

@export var direction : Room.Direction = Room.Direction.NORTH

@onready var barrier : StaticBody2D = $Barrier
@onready var barrier_collider : CollisionShape2D = $Barrier/CollisionShape2D

@onready var door : Node2D = $Door
@onready var door_shut : StaticBody2D = $Door/Door_Shut
@onready var door_shut_collider : CollisionShape2D = $Door/Door_Shut/CollisionShape2D

@onready var player_spawn : Node2D = $PlayerSpawn
@onready var exit_trigger : Area2D = $ExitTrigger

var neighbor : Room

func _ready ():
    exit_trigger.body_entered.connect(_on_body_entered_exit_trigger)
    toggle_barrier(true)
```



```
func set_neighbor (neighbor_room : Room):
    neighbor = neighbor_room
    toggle_barrier(false)

func toggle_barrier (toggle : bool):
    barrier.visible = toggle
    barrier_collider.disabled = !toggle

    door.visible = !toggle

func open_door ():
    if barrier.visible:
        return

    door_shut.visible = false
    door_shut_collider.disabled = true

func close_door ():
    if barrier.visible:
        return

    door_shut.visible = true
    door_shut_collider.disabled = false

func _on_body_entered_exit_trigger (body):
    if body.is_in_group("Player"):
        neighbor.player_enter(_get_neighbor_entry_direction(), body)

func _get_neighbor_entry_direction () -> Room.Direction:
    if direction == Room.Direction.NORTH:
        return Room.Direction.SOUTH
    elif direction == Room.Direction.SOUTH:
        return Room.Direction.NORTH
    elif direction == Room.Direction.EAST:
        return Room.Direction.WEST
    else:
        return Room.Direction.EAST
```

## room\_generation.gd

Found in folder: **/Scripts**

This code defines a RoomGeneration class that generates a grid of rooms for a game level. The `_generate()` function creates a map of rooms based on the `map_size` and `rooms_to_generate` variables, while the `_instantiate_rooms()` function instantiates the room scenes and sets up their neighbors. The code also includes functions for deciding the boss room location and retrieving rooms from the map.

```
class_name RoomGeneration
extends Node

@export var map_size : int = 7
```



```
@export var rooms_to_generate : int = 12
var room_count : int = 0
var map : Array[bool]
var rooms : Array[Room]
var room_pos_offset : float = 160

var first_room_x : int = 3
var first_room_y : int = 3
var first_room : Room

var room_scene : PackedScene = preload("res://Scenes/Rooms/room_template.tscn")

@export var first_room_scene : PackedScene
@export var boss_room : PackedScene
@export var room_scenes : Array[PackedScene]

@export var player : CharacterBody2D

func _ready ():
    _generate()

    # room layout preview
    #for x in range(map_size):
    #    var line : String
    #    for y in range(map_size):
    #        line += "# " if _get_map(x, y) else " "
    #    print(line)

func _generate ():
    room_count = 0
    map.resize(map_size * map_size)

    # start the check room chain at the center of the grid
    _check_room(first_room_x, first_room_y, Vector2.ZERO, true)

    # once we've defined our map, spawn in the room scenes
    _instantiate_rooms()

func _check_room (x : int, y : int, desired_direction : Vector2, is_first_room : bool
= false):
    # return if we've created enough rooms
    if room_count >= rooms_to_generate:
        return

    # return if out of bounds
    if x < 0 or x > map_size - 1 or y < 0 or y > map_size - 1:
        return

    # return if room already exists
    if _get_map(x, y) == true:
        return

    room_count += 1
    _set_map(x, y, true)
```



```
# define the directions we want to expand in
var go_north : bool = randf() > (0.2 if desired_direction == Vector2.UP else 0.8)
var go_south : bool = randf() > (0.2 if desired_direction == Vector2.DOWN else 0.8)
var go_east : bool = randf() > (0.2 if desired_direction == Vector2.RIGHT else 0.8)
var go_west : bool = randf() > (0.2 if desired_direction == Vector2.LEFT else 0.8)

# continue the check room chain in those directions
if go_north or is_first_room:
    _check_room(x, y - 1, Vector2.UP if is_first_room else desired_direction)
if go_south or is_first_room:
    _check_room(x, y + 1, Vector2.DOWN if is_first_room else desired_direction)
if go_east or is_first_room:
    _check_room(x + 1, y, Vector2.RIGHT if is_first_room else desired_direction)
if go_west or is_first_room:
    _check_room(x - 1, y, Vector2.LEFT if is_first_room else desired_direction)

func _instantiate_rooms ():
    var boss_room_pos : Vector2 = _decide_boss_room()

    # instantiate each room scene at their designated position
    for x in range(map_size):
        for y in range(map_size):
            if _get_map(x, y) == false:
                continue

            var room : Room
            var is_first_room : bool = first_room_x == x and first_room_y == y

            if is_first_room:
                room = first_room_scene.instantiate()
            elif x == boss_room_pos.x and y == boss_room_pos.y:
                room = boss_room.instantiate()
            else:
                room = room_scenes[randi_range(0, len(room_scenes) - 1)].instantiate()

            get_tree().root.get_node("/root/Main").add_child.call_deferred(room)
            rooms.append(room)

            room.global_position = Vector2(x, y) * room_pos_offset

            if is_first_room:
                first_room = room

            room.initialize()

    # set room neighbors
    for room in rooms:
        var map_pos : Vector2 = _get_map_index(room)
        var x = map_pos.x
        var y = map_pos.y

        if y > 0 and _get_map(x, y - 1):
            room.set_neighbor.call_deferred(Room.Direction.NORTH, get_room_from_map(x, y - 1))
        if y < map_size - 1 and _get_map(x, y + 1):
            room.set_neighbor.call_deferred(Room.Direction.SOUTH, get_room_from_map(x, y + 1))
```



```
if x < map_size - 1 and _get_map(x + 1, y):
    room.set_neighbor.call_deferred(Room.Direction.EAST, get_room_from_map(x + 1, y))
if x > 0 and _get_map(x - 1, y):
    room.set_neighbor.call_deferred(Room.Direction.WEST, get_room_from_map(x - 1, y))

# put the player in the first room
first_room.player_enter.call_deferred(Room.Direction.NORTH, player, true)

func _decide_boss_room () -> Vector2:
    var i = 0

    while i < 100:
        var x = randi_range(0, map_size - 1)
        var y = randi_range(0, map_size - 1)

        if first_room_x == x and first_room_y == y:
            continue

        if _get_map(x, y):
            return Vector2(x, y)

    return Vector2.ZERO

func get_room_from_map (x : int, y : int) -> Room:
    for room in rooms:
        var pos = _get_map_index(room)

        if pos.x != x or pos.y != y:
            continue

    return room

return null

func _get_map_index (room : Room) -> Vector2i:
    return Vector2i(room.global_position / room_pos_offset)

func _get_map (x : int, y : int) -> bool:
    return map[x + y * map_size]

func _set_map (x : int, y : int, value : bool):
    map[x + y * map_size] = value
```

## room\_item\_spawner.gd

Found in folder: **/Scripts**

This code defines a Node2D-based class that has an exported array of PackedScene objects called `item_scenes`. In the `_ready()` function, it randomly selects a PackedScene from the `item_scenes` array, instantiates it, and adds the instantiated scene as a child of the current node.

```
extends Node2D
```





```
@export var item_scenes : Array[PackedScene]

func _ready ():
    var item = item_scenes[randi_range(0, len(item_scenes) - 1)].instantiate()
    add_child(item)
```